CURSO DE CONFECCIÓN WEB 2026

JavaScript en el cliente

Manual de teoría · 13 bloques · 104 ejercicios explicados



Ramón Abramo

Profesor del curso

Índice

Antes de empezar 10

Presentación del curso 11

El temario 17

Bloque 01 · Introducción 25

1. ¿Qué es JavaScript? 25
2. JavaScript en el cliente 26
3. Cómo se añade JavaScript a una página 27
4. ¿Dónde coloco el <script>? 28
5. Las herramientas de desarrollo 30
6. Entradas y salidas 31
7. La plantilla de trabajo en este bloque 33
8. Ejercicios 33

🧪 Los ejercicios, explicados 35

Ejercicio 01 · ¡Hola, consola! 35
Ejercicio 02 · El script que llegó demasiado pronto 36
Ejercicio 03 · Del script interno al fichero externo 38
Ejercicio 04 · Los tres canales de salida 39
Ejercicio 05 · textContent frente a innerHTML 41
Ejercicio 06 · Mi primera entrada con prompt() 42
Ejercicio 07 · prompt() siempre devuelve texto 43
Ejercicio 08 · Caza de errores en la consola 44

Bloque 02 · Fundamentos del lenguaje 47

1. Variables y constantes 47
2. Cómo nombrar las variables 49
3. Tipos de datos primitivos 50
4. Operadores 51
5. Conversión de tipos 52
6. Comparaciones 53
7. El objeto Math y el redondeo 55
8. La plantilla de trabajo en este bloque 56
9. Ejercicios 57

🧪 Los ejercicios, explicados 58

Ejercicio 01 · Ficha de personaje: let y const 58
Ejercicio 02 · Detective de tipos 60
Ejercicio 03 · Calculadora de operadores 61
Ejercicio 04 · Marcador de videojuego 62
Ejercicio 05 · Ticket de compra con plantillas de texto 64
Ejercicio 06 · Laboratorio de conversiones 65
Ejercicio 07 · == frente a === 66
Ejercicio 08 · Dividir la cuenta del restaurante 68

Bloque 03 · Control de flujo 70

1. Tomar decisiones: if 70
2. Operadores lógicos 72
3. Valores truthy y falsy 73
4. El operador ternario 74

5. switch: muchos valores concretos 74
6. Repetir: los bucles 75
7. Contadores y acumuladores 78
8. break y continue 78
9. Cuidado con los bucles infinitos 79
10. La plantilla de trabajo en este bloque 79
11. Ejercicios 80
🧪 Los ejercicios, explicados 81
Ejercicio 01 · Calificador de notas 81
Ejercicio 02 · Control de acceso al concierto 82
Ejercicio 03 · Un saludo a prueba de Cancelar 84
Ejercicio 04 · ¿Qué día es? 85
Ejercicio 05 · Tabla de multiplicar con for 87
Ejercicio 06 · Adivina el número 88
Ejercicio 07 · Estadísticas hasta el cero 90
Ejercicio 08 · Fizz Buzz 92

Bloque 04 · Funciones 94

1. ¿Qué es una función? 94
2. Declarar y llamar 95
3. Parámetros y argumentos 95
4. return: devolver un resultado 96
5. Tres formas de escribir una función 97
6. El ámbito de las variables 99
7. Las funciones son valores: callbacks 100
8. Funciones que recuerdan: closures 101
9. Buenas prácticas con funciones 101
10. La plantilla de trabajo en este bloque 102
11. Ejercicios 103
🧪 Los ejercicios, explicados 104
Ejercicio 01 · Conversor de temperaturas 104
Ejercicio 02 · Precio de la entrada de cine 105
Ejercicio 03 · Saludos con valores por defecto 108
Ejercicio 04 · Tres formas de escribir una función 109
Ejercicio 05 · Detective del ámbito 111
Ejercicio 06 · Calculadora con callbacks 113
Ejercicio 07 · Fábrica de contadores 115
Ejercicio 08 · Planificador de estudio 117

Bloque 05 · Arrays y objetos 120

1. Arrays: listas ordenadas 120
2. Modificar un array 121
3. Buscar en un array 122
4. Recorrer un array 123
5. Transformar: map, filter y reduce 123
6. Objetos: fichas con nombre 125
7. Arrays de objetos 127
8. Desestructuración y spread 128
9. Valores y referencias 129
10. JSON: datos como texto 130
11. La plantilla de trabajo en este bloque 131
12. Ejercicios 132
🧪 Los ejercicios, explicados 133

Ejercicio 01 · La lista de la compra 133
Ejercicio 02 · Buscador de alumnos 134
Ejercicio 03 · Una semana de temperaturas 136
Ejercicio 04 · map, filter, reduce y sort 138
Ejercicio 05 · La ficha de un producto 140
Ejercicio 06 · Catálogo de una tienda 142
Ejercicio 07 · Desestructuración y spread 144
Ejercicio 08 · Guardar y cargar la partida 146

Bloque 06 · El DOM 149

1. Qué es el DOM 149
2. Seleccionar elementos 151
3. Moverse por el árbol 152
4. Cambiar el contenido 153
5. Atributos y datos propios 153
6. Cambiar el aspecto 154
7. Crear, mover y eliminar elementos 155
8. El patrón «de datos a HTML» 156
9. La plantilla de trabajo en este bloque 157
10. Ejercicios 158

🧪 Los ejercicios, explicados 159
Ejercicio 01 · Explorador del árbol DOM 159
Ejercicio 02 · Muchos elementos a la vez: NodeList 160
Ejercicio 03 · Retocar una tarjeta de perfil 162
Ejercicio 04 · Estilos: style frente a classList 164
Ejercicio 05 · La lista de la compra, ahora en HTML 166
Ejercicio 06 · Tarjetas de producto desde datos 167
Ejercicio 07 · Clasificación: mover y eliminar 169
Ejercicio 08 · Tabla de notas dinámica 171

Bloque 07 · Eventos 173

1. Programar con eventos 173
2. addEventListener 174
3. Los eventos más habituales 175
4. El objeto evento 176
5. Leer los campos de la página 177
6. preventDefault: «de esto me encargo yo» 177
7. Propagación: la burbuja 178
8. Delegación de eventos 178
9. El patrón estado → renderizar 179
10. La plantilla de trabajo, completa 180
11. Ejercicios 182

🧪 Los ejercicios, explicados 183
Ejercicio 01 · Contador de clics 183
Ejercicio 02 · Conversor en tiempo real 185
Ejercicio 03 · Mini-tuit con atajos de teclado 186
Ejercicio 04 · Galería con eventos de ratón 187
Ejercicio 05 · Campos que ayudan: focus, blur y preventDefault 189
Ejercicio 06 · Propagación: ¿quién se entera del clic? 190
Ejercicio 07 · Lista de tareas con delegación 192
Ejercicio 08 · Catálogo interactivo 194

Bloque 08 · Formularios 197

1. Un formulario bien construido 197

2. El envío: submit y preventDefault 198
3. Leer los datos 199
4. Validación nativa: atributos HTML 200
5. La API de validación 200
6. Errores que todo el mundo entiende 202
7. Formularios que se adaptan 203
8. La plantilla de trabajo con un formulario 204
9. Ejercicios 205
🧪 Los ejercicios, explicados 206
Ejercicio 01 · Pedido en la pizzería: leer cada control 206
Ejercicio 02 · FormData: el formulario entero de una vez 208
Ejercicio 03 · Validación nativa del navegador 209
Ejercicio 04 · Mensajes de error propios con la API de validación 211
Ejercicio 05 · Contraseñas: reglas propias con setCustomValidity 213
Ejercicio 06 · Formulario de contacto accesible 215
Ejercicio 07 · Formulario dinámico: campos que aparecen 217
Ejercicio 08 · Inscripciones a un taller 218

Bloque 09 · Almacenamiento en el navegador 221
1. Dos almacenes en el navegador 221
2. Solo guarda texto 223
3. El patrón: cargar al empezar, guardar en cada cambio 225
4. Cargar a la defensiva 226
5. Sincronizar pestañas: el evento storage 227
6. Qué guardar y qué no 228
7. La plantilla de trabajo con persistencia 229
8. Ejercicios 230
🧪 Los ejercicios, explicados 231
Ejercicio 01 · Mi primer almacén 231
Ejercicio 02 · Recuerda mi nombre 232
Ejercicio 03 · Preferencias de tema y tamaño de letra 234
Ejercicio 04 · localStorage frente a sessionStorage 235
Ejercicio 05 · El almacén solo guarda texto: JSON al rescate 237
Ejercicio 06 · Un borrador que no se pierde 239
Ejercicio 07 · Lista de tareas que persiste 240
Ejercicio 08 · Un carrito que sobrevive 242

Bloque 10 · Asincronía 245
1. Síncrono y asíncrono 245
2. Temporizadores 246
3. El bucle de eventos 248
4. Callbacks y la pirámide 249
5. Promesas 249
6. async y await 251
7. Una detrás de otra, o todas a la vez 252
8. La plantilla de trabajo con asincronía 253
9. Ejercicios 255
🧪 Los ejercicios, explicados 256
Ejercicio 01 · Un aviso con retraso: setTimeout 256
Ejercicio 02 · Cronómetro con setInterval 257
Ejercicio 03 · Cuenta atrás con barra de progreso 259
Ejercicio 04 · ¿En qué orden? El bucle de eventos 260
Ejercicio 05 · Del callback hell a las promesas 262
Ejercicio 06 · Promesas que pueden fallar: then, catch y finally 264

Ejercicio 07 · async y await: el mismo código, más claro 266
Ejercicio 08 · Juego de reflejos 268

Bloque 11 · Fetch y APIs 271

1. Qué es una API 271
2. fetch en cuatro pasos 273
3. Tres formas de fallar 274
4. Los cuatro estados de la interfaz 275
5. El origen, Live Server y CORS 276
6. Pedir y enviar: GET y POST 278
7. Varias peticiones a la vez 279
8. La plantilla de trabajo con fetch 279
9. Ejercicios 281

🧪 Los ejercicios, explicados 282
Ejercicio 01 · Mi primer fetch: leer un JSON 282
Ejercicio 02 · respuesta.ok y los códigos de estado 283
Ejercicio 03 · El catálogo llega de un fichero 284
Ejercicio 04 · Buscador de países 286
Ejercicio 05 · Estados de carga, errores y reintentos 287
Ejercicio 06 · Una API pública: usuarios y publicaciones 289
Ejercicio 07 · Enviar datos con POST 291
Ejercicio 08 · Tareas de la API con favoritos 293

Bloque 12 · Módulos, clases y buenas prácticas 295

1. Módulos: un fichero, una responsabilidad 295
2. Clases: el molde de los objetos 297
3. Herencia: extends y super 299
4. Encapsulación: privados, getters, setters y static 300
5. Separar responsabilidades: modelo, vista y controlador 301
6. Depuración: detener el programa y mirar 302
7. La plantilla de trabajo con módulos 303
8. Ejercicios 305

🧪 Los ejercicios, explicados 306
Ejercicio 01 · Mi primer módulo 306
Ejercicio 02 · Formas de exportar e importar 307
Ejercicio 03 · Mi primera clase: el carrito 309
Ejercicio 04 · Herencia: figuras geométricas 311
Ejercicio 05 · Encapsulación: la cuenta bancaria 312
Ejercicio 06 · Modelo, vista y controlador 315
Ejercicio 07 · Depuración: cuatro fallos escondidos 316
Ejercicio 08 · Proyecto: mi biblioteca con clases y módulos 318

Bloque 13 · Proyecto final 322

1. El enunciado 322
2. Los criterios de evaluación 323
3. La aplicación de referencia: un tablero de tareas 324
4. Las ocho fases 325
5. Cómo organizar vuestro propio proyecto 327
6. Qué hay que entregar 327
7. Lista de comprobación antes de entregar 327

🧪 Las fases del proyecto, explicadas 329
Fase 1 · El modelo: Tarea y Tablero 329
Fase 2 · La vista: pintar el tablero 330
Fase 3 · Eventos de ratón: completar, eliminar, editar y filtrar 332

Fase 4 · El formulario con validación 334
Fase 5 · Persistencia con localStorage 335
Fase 6 · Importar tareas de una API 337
Fase 7 · Teclado y búsqueda 339
Fase 8 · Pulido final: orden, progreso y tema 340

Fuentes y Documentación Oficial **343**

📊 Índice de infografías

Las 85 infografías del manual. Están pensadas para repasar de un vistazo: si solo tenéis cinco minutos, mirad estas.

Figura 1 · Mapa del curso 11
Figura 2 · Plantilla de trabajo 14
Figura 3 · Mapa del curso 17
Figura 4 · Tres capas, tres trabajos 26
Figura 5 · Dónde se ejecuta JavaScript 27
Figura 6 · Dónde colocar el script 28
Figura 7 · Cómo leer un error de la consola 30
Figura 8 · Entradas y salidas 31
Figura 9 · let, const y var 48
Figura 10 · Tipos primitivos 50
Figura 11 · Conversión de tipos 53
Figura 12 · == frente a === 54
Figura 13 · Redondear con Math y toFixed 55
Figura 14 · == frente a === 67
Figura 15 · La escalera if, else if, else 71
Figura 16 · Operadores lógicos 72
Figura 17 · Truthy y falsy 73
Figura 18 · Anatomía de un bucle for 75
Figura 19 · for, while y do...while 77
Figura 20 · Anatomía de una función 95
Figura 21 · Tres formas de escribir una función 97
Figura 22 · El ámbito 99
Figura 23 · Callback 100
Figura 24 · Tres formas de escribir una función 109
Figura 25 · El ámbito 112
Figura 26 · Callback 114
Figura 27 · Anatomía de un array 121
Figura 28 · map, filter y reduce 124
Figura 29 · Anatomía de un objeto 126
Figura 30 · Desestructuración, resto y spread 128
Figura 31 · Valores y referencias 129
Figura 32 · map, filter y reduce 138
Figura 33 · Un objeto 140
Figura 34 · Valores y referencias 147
Figura 35 · Del HTML al árbol del DOM 150
Figura 36 · Seleccionar uno o todos 151
Figura 37 · Moverse por el árbol del DOM 152
Figura 38 · style frente a classList 154
Figura 39 · Crear un elemento 155
Figura 40 · style frente a classList 165
Figura 41 · Crear un elemento 166
Figura 42 · El ciclo de un evento 174
Figura 43 · Los eventos más habituales 175
Figura 44 · La burbuja 178
Figura 45 · Delegación 179

Figura 46 · La burbuja 191
Figura 47 · Delegación 193
Figura 48 · Del botón «Enviar» al resultado 198
Figura 49 · Cómo leer cada control 199
Figura 50 · Cada regla tiene su chivato 201
Figura 51 · Errores accesibles 202
Figura 52 · Cómo leer cada control 207
Figura 53 · Errores accesibles 215
Figura 54 · localStorage frente a sessionStorage 222
Figura 55 · El almacén solo entiende texto 223
Figura 56 · Cargar al empezar, guardar en cada cambio 225
Figura 57 · El evento storage entre pestañas 227
Figura 58 · ¿Esto lo guardo en el navegador? 228
Figura 59 · localStorage frente a sessionStorage 236
Figura 60 · El almacén solo entiende texto 238
Figura 61 · Cargar al empezar, guardar en cada cambio 241
Figura 62 · Síncrono frente a asíncrono 246
Figura 63 · setTimeout y setInterval 246
Figura 64 · El bucle de eventos 248
Figura 65 · Los estados de una promesa 249
Figura 66 · De then a async/await 251
Figura 67 · Secuencial frente a Promise.all 252
Figura 68 · El bucle de eventos 261
Figura 69 · Los estados de una promesa 265
Figura 70 · De then a async/await 267
Figura 71 · Una petición HTTP a una API 272
Figura 72 · Los cuatro pasos de fetch 273
Figura 73 · Los cuatro estados de una interfaz que carga datos 275
Figura 74 · El origen y CORS 277
Figura 75 · GET frente a POST 278
Figura 76 · Los cuatro estados de la interfaz 288
Figura 77 · Módulos: export e import 296
Figura 78 · Una clase y sus instancias 298
Figura 79 · Herencia con extends y super 299
Figura 80 · Encapsulación: privados, getters y setters 300
Figura 81 · Depurar con un punto de interrupción 302
Figura 82 · Criterios de evaluación 323
Figura 83 · La arquitectura: cinco módulos 324
Figura 84 · El ciclo de cada acción del usuario 325
Figura 85 · Las ocho fases del proyecto 326

Antes de empezar



¡Hola! 👋 Soy **Ramón Abramo**, vuestro profesor, y este es el manual de teoría del curso de **JavaScript en el cliente**.

Aquí tenéis reunido, en un solo documento, todo lo que os explico en clase: la teoría de los trece bloques, con sus infografías, y la explicación paso a paso de cada uno de los ejercicios resueltos. Lo he preparado para que lo podáis leer de principio a fin, imprimir o consultar cuando os atasquéis.



Cómo está organizado

Cada bloque tiene dos partes:

1. 📖 **La teoría**: los conceptos, con ejemplos de código y dibujos que os ayudan a verlos.
2. 🧪 **Los ejercicios explicados**: qué pedía cada ejercicio, cómo lo he resuelto y por qué. Es la teoría puesta en práctica, con los detalles que solo aparecen cuando se escribe código de verdad.

Los enunciados y el código completo de cada ejercicio están en el repositorio del curso. En el manual, los nombres de ejercicio y los enlaces azules llevan directamente a GitHub.



Cómo sacarle partido

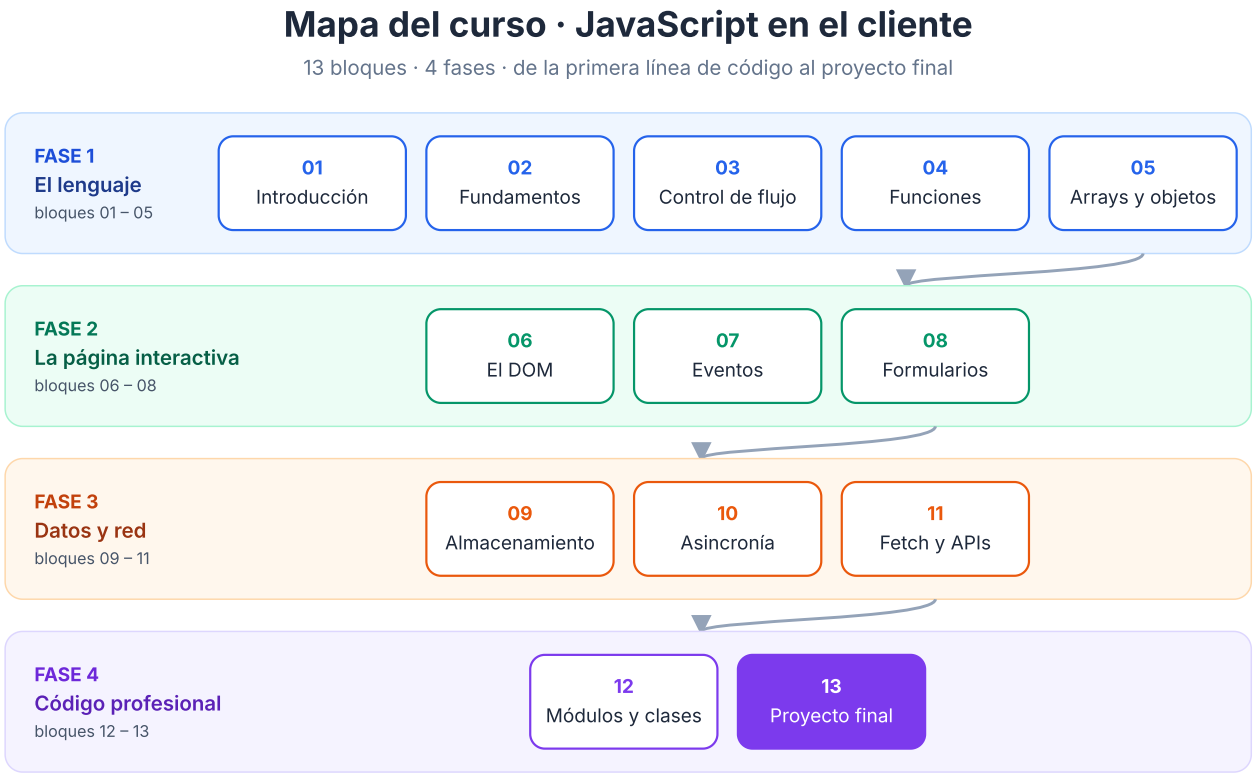
- 👩‍💻 **Leed con el editor abierto.** Cada fragmento de código se entiende mucho mejor si lo escribís y lo probáis.
- 🔁 **Primero el ejercicio, después la solución.** Intentad resolverlo antes de leer su explicación: equivocarse es la parte del aprendizaje que no se puede saltar.
- 🔢 **Los números entre corchetes**, como [1], remiten a la documentación oficial que tenéis al final del manual. Si queréis profundizar en algo, ahí está la fuente.

Y, como os digo siempre: preguntad todo lo que haga falta. 🙂

Presentación del curso

Buenas, soy Ramón Abramo, profesor con más de 25 años de experiencia docente. En este repositorio os voy a acompañar paso a paso en el aprendizaje de **JavaScript para páginas web**: el JavaScript que se ejecuta en el navegador, el que hace que una página deje de ser un cartel bonito y empiece a responder cuando alguien la toca [1].

Aquí tenéis la teoría de cada bloque, ejercicios para practicar y sus soluciones explicadas. Todo está pensado para que podáis estudiarlo a vuestro ritmo, pero también para usarlo en clase conmigo.



📊 Figura 1 · Mapa del curso



1. ¿Qué vais a aprender?

Una página web se apoya en tres capas, y cada una tiene su trabajo:

Capa	Lenguaje	Su trabajo	Analogía
Estructura	HTML	Decir qué hay en la página: títulos, párrafos, botones, formularios	El esqueleto
Presentación	CSS	Decidir cómo se ve: colores, tamaños, posiciones	La ropa
Comportamiento	JavaScript	Decidir qué pasa cuando el usuario hace algo	Los músculos

Con HTML y CSS ya sabéis montar una página. Con JavaScript vais a conseguir que esa página **reaccione**: que un botón calcule algo, que un formulario avise de un error antes de enviarse, que una lista recuerde lo que habéis apuntado aunque cerréis el navegador o que aparezcan datos traídos de un servidor.

Fijaos en un detalle: la analogía del cuerpo sirve para recordar el reparto de tareas, pero no la llevéis más lejos. JavaScript también puede cambiar la estructura y el aspecto de la página. Lo que no debéis hacer es usarlo para lo que HTML y CSS ya resuelven solos.

Al terminar el curso seréis capaces de:

- Escribir programas en JavaScript con variables, condiciones, bucles, funciones, arrays y objetos.
- Leer y modificar cualquier elemento de una página a través del DOM.
- Responder a las acciones del usuario con eventos.
- Validar formularios y dar mensajes claros cuando algo falla.
- Guardar información en el navegador.
- Pedir datos a una API y mostrarlos en la página.
- Organizar el código en módulos y clases para que no se convierta en un plato de espaguetis.



2. ¿Qué necesitáis antes de empezar?

Conocimientos previos

Necesitáis manejaros con **HTML y CSS básicos**: saber qué es una etiqueta, un atributo `id` o `class` y cómo se enlaza una hoja de estilos. No hace falta haber programado nunca. Empiezo desde cero.

Herramientas

1. **Un navegador moderno**: Chrome, Edge o Firefox. Os servirá para ver la página y, sobre todo, para usar las **herramientas de desarrollo** (las abrís con `F12`). La pestaña **Consola** va a ser vuestra mejor amiga durante todo el curso [2].
2. **Visual Studio Code** como editor.
3. La extensión **Live Server** de VS Code (opcional pero muy recomendable). Recarga la página sola cada vez que guardáis.

No tenéis que instalar nada más. Ni Node.js, ni frameworks, ni compiladores. Abrís el `index.html` en el navegador y listo.



📝 Nota

A partir del bloque 12, cuando trabajemos con módulos (`import` / `export`), el navegador ya no os dejará abrir el fichero con doble clic. Ahí sí necesitaréis Live Server. Os lo explico con calma cuando lleguemos [3].



3. Cómo está organizado el repositorio

```
cursoConfeccionWeb2026/
├── README.md                    ← Este documento: la introducción
├── TEMARIO.md                   ← El temario explicado bloque a bloque
├── img/                         ← Infografías generales del curso
│
├── 01-introduccion/
│   ├── README.md                ← Teoría del bloque, con infografías
│   ├── img/                     ← Infografías de ese bloque
│   ├── ejercicios/
│   │   ├── ejercicio_01/
│   │   │   ├── index.html
│   │   │   ├── style.css
│   │   │   └── app.js           ← Enunciado y plantilla para resolver
│   │   └── ejercicio_02/ …
│   └── soluciones/
│       ├── ejercicio_01/
│       │   ├── index.html
│       │   ├── style.css
│       │   ├── app.js           ← Código resuelto y comentado
│       │   └── README.md        ← La solución explicada paso a paso
│       └── ejercicio_02/ …
│
├── 02-fundamentos-del-lenguaje/
└── … un bloque por carpeta, hasta 13-proyecto-final/
```

Cada carpeta de ejercicio es **independiente**: tiene su propio HTML, su propio CSS y su propio JavaScript. No depende de ningún fichero de fuera. Podéis copiar una carpeta a un pendrive, abrir el `index.html` y funcionará igual.



4. Cómo trabajar cada bloque

Os propongo este orden. Sin saltaros pasos.

1. **Leed la teoría** del `README.md` del bloque. Mirad las infografías con calma. Están ahí para que veáis de un vistazo lo que el texto cuenta en varios párrafos.
2. **Abrid el ejercicio** en la carpeta `ejercicios/`. Leed el enunciado que aparece al principio del `app.js`.
3. **Resolvedlo vosotros**, siguiendo los pasos marcados y con la consola abierta.
4. **Solo después**, comparad con la carpeta `soluciones/`. Leed el `README.md` de la solución, que os explica el porqué de cada decisión, no solo el código.

Pero ojo aquí. Si abrís la solución a los dos minutos de atascaros, el ejercicio deja de serviros. Atascarse forma parte del aprendizaje. Dadle una vuelta, revisad la consola, releed la teoría... y si aun así no sale, entonces sí, mirad la solución y volved a intentarlo sin ella.



5. La plantilla de trabajo

Todos los `app.js` del curso siguen la misma estructura de comentarios. Así no empezáis nunca delante de una página en blanco: sabéis qué va primero, qué va después y dónde colocar cada cosa.



Figura 2 · Plantilla de trabajo

```
// estructura del programa principal
const IVA = 0.21;

// constantes al DOM que queremos escuchar
const inputPrecio = document.querySelector('#inputPrecio');
const btnCalcular = document.querySelector('#btnCalcular');
const salida = document.querySelector('#salida');

// colocar escuchadores de eventos
btnCalcular.addEventListener('click', () => {
    // entradas
    const precio = Number(inputPrecio.value);

    // procesamiento
    const total = precio + precio * IVA;

    // salidas
    salida.textContent = `Total con IVA: ${total.toFixed(2)} €`;
});
```

Fijaos en que hay dos momentos distintos:

- Lo que está **fuera** del escuchador se ejecuta una sola vez, cuando la página carga. Ahí preparo el terreno: constantes, referencias a los elementos y escuchadores.
- Lo que está **dentro** se ejecuta cada vez que el usuario pulsa el botón. Ahí va el esquema de siempre: **entradas → procesamiento → salidas**.

En los primeros ejercicios todavía no usamos eventos, así que el programa se ejecuta de arriba abajo y la plantilla se queda solo en entradas, procesamiento y salidas. En cuanto lleguemos a los eventos, la plantilla crece y ya no cambia en todo el curso.



6. Buenas prácticas que vais a ver en todo el curso

- Uso `const` por defecto y `let` solo cuando el valor tiene que cambiar. `var` no aparece en ningún ejercicio.
- Comparo siempre con `===` y `!==`, que comprueban también el tipo [4].
- Convierto lo que llega de un `input` con `Number()` antes de hacer cuentas. Un `input` siempre devuelve texto [5], y `"2" + "2"` da `"22"`. Ahí suele estar el lío.
- Pongo nombres que se entienden: `precioTotal`, no `pt`; `btnEnviar`, no `b1`.
- Coloco el `<script>` al final del `<body>` o con `defer`, para que el HTML exista cuando JavaScript lo busca [6].
- Separo la estructura (HTML), el estilo (CSS) y el comportamiento (JS) en sus propios ficheros.



7. El temario

El detalle de cada bloque, con objetivos y contenidos, lo tenéis en **TEMARIO.md**.

Bloque	Carpeta
01 · Introducción	`01-introduccion/`
02 · Fundamentos del lenguaje	`02-fundamentos-del-lenguaje/`
03 · Control de flujo	`03-control-de-flujo/`
04 · Funciones	`04-funciones/`
05 · Arrays y objetos	`05-arrays-y-objetos/`
06 · El DOM	`06-dom/`
07 · Eventos	`07-eventos/`
08 · Formularios	`08-formularios/`
09 · Almacenamiento en el navegador	`09-almacenamiento/`
10 · Asincronía	`10-asincronia/`
11 · Fetch y APIs	`11-fetch-y-apis/`
12 · Módulos, clases y buenas prácticas	`12-modulos-y-clases/`
13 · Proyecto final	`13-proyecto-final/`

El temario

En este documento os explico qué vamos a ver en cada bloque, en qué orden y por qué. Antes de abrir la primera carpeta, os recomiendo leerlo entero. Así sabréis en todo momento en qué punto del camino estáis y hacia dónde vamos.



Cómo está pensado el recorrido

El curso tiene **13 bloques** agrupados en **4 fases**. El orden no es caprichoso: cada bloque usa lo que habéis aprendido en los anteriores.

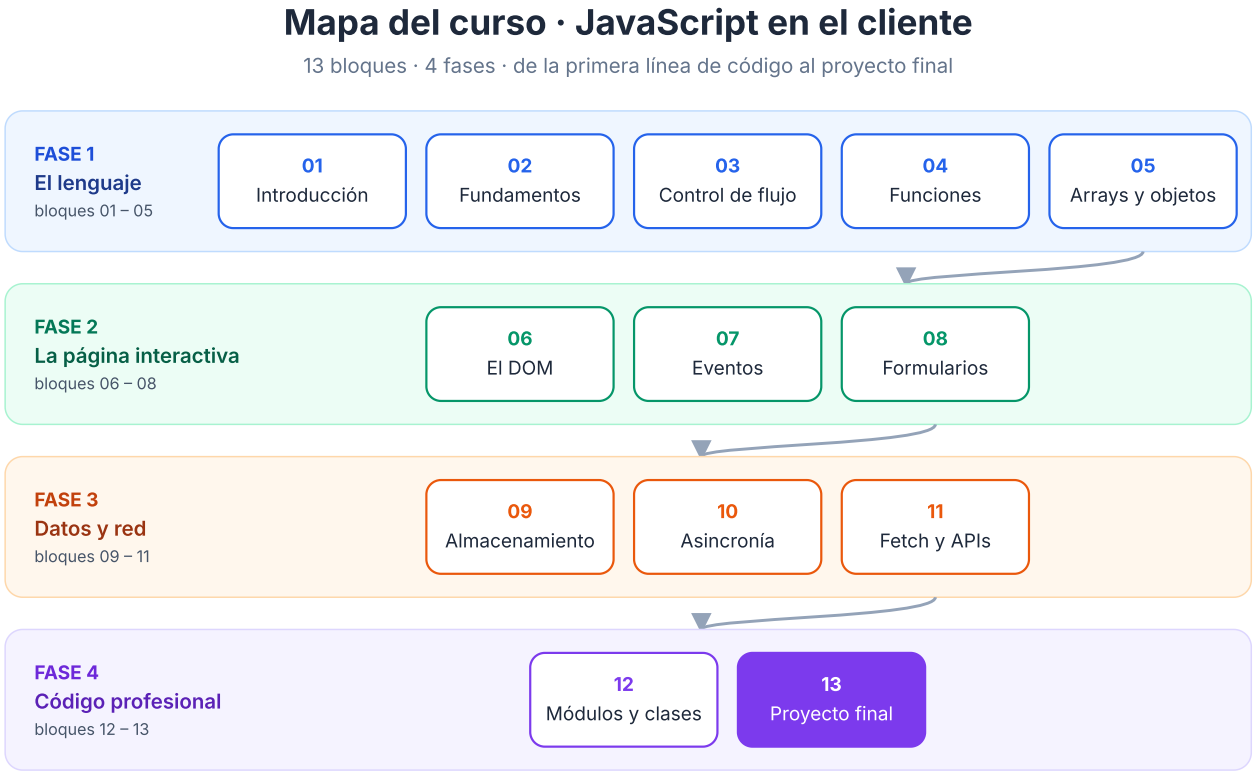
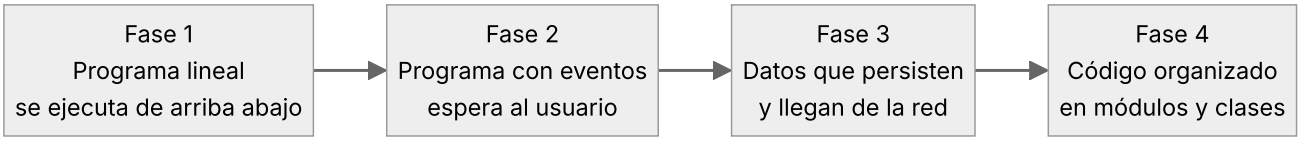


Figura 3 · Mapa del curso

Fase	Bloques	Qué conseguís
1 · El lenguaje	01 – 05	Pensar como un programador: datos, decisiones, repeticiones y funciones
2 · La página interactiva	06 – 08	Que la página responda a lo que hace el usuario
3 · Datos y red	09 – 11	Recordar información y traerla de fuera
4 · Código profesional	12 – 13	Ordenar el código y construir una aplicación completa

Fijaos en el salto entre la fase 1 y la fase 2. En la primera, el programa se ejecuta de arriba abajo, de golpe, y usamos la consola, `prompt()` y `alert()` para probar cosas rápido. En la segunda cambia la forma de pensar: el programa prepara la página y luego **se queda esperando** a que el usuario haga algo. A partir de ahí, `prompt()` y `alert()` desaparecen y trabajamos con interfaces de verdad.





Fase 1 · El lenguaje

Bloque 01 · Introducción

📁 `01-introduccion/`

Objetivo: entender dónde vive JavaScript, cómo se conecta con una página y cómo comprobar que está funcionando.

Contenidos:

- Qué es JavaScript y qué papel juega junto a HTML y CSS.
- JavaScript en el cliente frente a JavaScript en el servidor.
- Las tres formas de incluir código: dentro del HTML, en un fichero externo y en un atributo (y por qué esta última no la vamos a usar).
- Dónde colocar el `<script>`: en el `<head>`, al final del `<body>` o con `defer`.
- Las herramientas de desarrollo del navegador: la Consola, el inspector de elementos y los mensajes de error.
- Primeras salidas: `console.log()`, `alert()` y `textContent`.
- Primera entrada: `prompt()`.

📖 Documentación oficial: [1] [6] [2]

Al terminar: sabréis crear un `index.html` con su `app.js`, enlazarlos bien y leer un error en la consola sin entrar en pánico.

Bloque 02 · Fundamentos del lenguaje

📁 `02-fundamentos-del-lenguaje/`

Objetivo: aprender a guardar datos y a operar con ellos.

Contenidos:

- Variables y constantes: `let` y `const`. Por qué dejamos `var` fuera.
- Reglas para nombrar variables y la convención *camelCase*.
- Tipos primitivos: `string`, `number`, `boolean`, `undefined`, `null`, y el operador `typeof`.

- Operadores aritméticos, de asignación y de incremento.
- Concatenación con `+` y plantillas de texto con comillas invertidas: `` `Hola, ${nombre}` ``.
- Conversión de tipos: `Number()`, `String()`, `parseInt()`, `parseFloat()` y el caso de `NaN`.
- Comparaciones: `==` frente a `===`.
- El objeto `Math` y el redondeo con `toFixed()`.

📖 Documentación oficial: [7] [4]

Al terminar: sabréis pedir datos, convertirlos al tipo correcto, hacer cálculos y mostrar el resultado bien formateado.

Bloque 03 · Control de flujo

📁 `03-control-de-flujo/`

Objetivo: que el programa tome decisiones y repita tareas.

Contenidos:

- Condicionales: `if`, `else if`, `else`.
- Operadores lógicos: `&&`, `||`, `!`.
- Valores *truthy* y *falsy*.
- El operador ternario.
- La estructura `switch`.
- Bucles `for`, `while` y `do...while`.
- Contadores y acumuladores.
- Control del bucle con `break` y `continue`.
- Cómo evitar un bucle infinito (y qué hacer si se os cuela uno).

📖 Documentación oficial: [8] [9]

Al terminar: sabréis resolver problemas que dependen de condiciones y que exigen repetir operaciones, como calcular notas, tablas de multiplicar o sumar series de números.

Bloque 04 · Funciones

📁 `04-funciones/`

Objetivo: dividir el programa en piezas reutilizables.

Contenidos:

- Declarar y llamar a una función.
- Parámetros, argumentos y valores por defecto.
- `return`: devolver un resultado y salir antes de tiempo.
- Expresiones de función y funciones flecha.
- Ámbito (*scope*): global, de función y de bloque.
- *Hoisting*: qué se puede usar antes de declararse y qué no.

- Funciones como valores: pasar una función a otra (*callbacks*).
- Introducción a los *closures*.

📖 Documentación oficial: [10] [11]

Al terminar: sabréis escribir funciones pequeñas que hacen una sola cosa bien, y reutilizarlas en lugar de copiar y pegar código.

Bloque 05 · Arrays y objetos

📁 `05-arrays-y-objetos/`

Objetivo: trabajar con colecciones de datos y con datos que tienen varias propiedades.

Contenidos:

- Arrays: crear, leer por índice, `length`.
- Añadir y quitar elementos: `push`, `pop`, `unshift`, `shift`, `splice`.
- Buscar: `includes`, `indexOf`, `find`, `findIndex`.
- Recorrer: `for`, `for...of` y `forEach`.
- Transformar: `map`, `filter`, `reduce`, `sort`.
- Objetos literales: propiedades, métodos y acceso con punto o corchetes.
- Recorrer objetos con `for...in` y `Object.keys()`.
- Arrays de objetos, la estructura que más vais a usar en el mundo real.
- Desestructuración y operador *spread* (`...`).
- Valores frente a referencias: por qué copiar un array no es tan simple como parece.
- JSON: `JSON.stringify()` y `JSON.parse()`.

📖 Documentación oficial: [12] [13]

Al terminar: sabréis modelar datos reales (una lista de productos, un listado de alumnos) y filtrarlos, ordenarlos o calcular totales sobre ellos.

 Fase 2 · La página interactiva

Bloque 06 · El DOM

📁 `06-dom/`

Objetivo: leer y modificar cualquier parte de la página desde JavaScript.

Contenidos:

- Qué es el DOM y cómo el navegador convierte el HTML en un árbol de nodos.
- Seleccionar elementos: `querySelector` y `querySelectorAll`.
- `NodeList` frente a array, y cuándo usar `Array.from()`.

- Cambiar contenido: `textContent` frente a `innerHTML`, y el riesgo de meter HTML que viene del usuario.
- Cambiar atributos y los atributos `data-*` con `dataset`.
- Cambiar estilos: `style` frente a `classList` (`add`, `remove`, `toggle`, `contains`).
- Crear y eliminar elementos: `createElement`, `append`, `prepend`, `remove`.
- Pintar listas en la página a partir de un array.

📖 Documentación oficial: [14] [15]

Al terminar: sabréis construir partes de la página desde JavaScript, como una lista de tarjetas generada a partir de datos.

Bloque 07 · Eventos

📁 `07-eventos/`

Objetivo: que la página responda a lo que hace el usuario.

Contenidos:

- La programación dirigida por eventos: preparar y esperar.
- `addEventListener` y `removeEventListener`.
- Eventos de ratón: `click`, `dblclick`, `mouseover`, `mouseout`.
- Eventos de teclado: `keydown`, `keyup` y la propiedad `e.key`.
- Eventos de formulario: `input`, `change`, `focus`, `blur`.
- El objeto `event`: `e.target`, `e.currentTarget`, `e.type`.
- `e.preventDefault()`: cuándo y para qué.
- Propagación de eventos: fase de burbuja y `stopPropagation()`.
- Delegación de eventos: un solo escuchador para muchos elementos.

📖 Documentación oficial: [16] [17]

Al terminar: sabréis montar interfaces que reaccionan al ratón y al teclado, como pestañas, contadores, galerías o juegos sencillos.

Bloque 08 · Formularios

📁 `08-formularios/`

Objetivo: recoger datos del usuario y comprobarlos antes de darlos por buenos.

Contenidos:

- Leer valores de `input`, `select`, `textarea`, `checkbox` y `radio`.
- El evento `submit` y cómo evitar que recargue la página.
- `FormData` para leer el formulario completo de una vez.
- Validación nativa de HTML: `required`, `min`, `max`, `pattern`, `type="email"`.
- La API de validación de restricciones: `checkValidity()`, `validity` y `setCustomValidity()`.

- Validación personalizada con JavaScript.
- Mostrar errores al usuario de forma clara y accesible.
- Limpiar y reiniciar un formulario.

📖 Documentación oficial: [18] [19]

Al terminar: sabréis construir formularios que no dejan pasar datos incorrectos y que explican al usuario qué debe corregir.



Fase 3 · Datos y red

Bloque 09 · Almacenamiento en el navegador

📁 `09-almacenamiento/`

Objetivo: que la página recuerde información aunque se recargue o se cierre el navegador.

Contenidos:

- `localStorage` y `sessionStorage`: diferencias y cuándo usar cada uno.
- `setItem`, `getItem`, `removeItem` y `clear`.
- Por qué solo se guarda texto, y cómo guardar arrays y objetos con JSON.
- Patrón de trabajo: cargar al empezar, guardar en cada cambio.
- Qué no debéis guardar nunca en el navegador.
- Proyecto guiado: lista de tareas que persiste.

📖 Documentación oficial: [20] [21]

Al terminar: sabréis hacer aplicaciones pequeñas que conservan sus datos, como una lista de tareas, unas preferencias de tema o un carrito.

Bloque 10 · Asincronía

📁 `10-asincronia/`

Objetivo: entender que no todo ocurre al instante y trabajar con tareas que terminan más tarde.

Contenidos:

- Código síncrono frente a código asíncrono.
- Temporizadores: `setTimeout`, `setInterval`, `clearTimeout`, `clearInterval`.
- El *Event Loop*: pila de llamadas, APIs del navegador y cola de tareas.
- *Callbacks* y el problema de anidarlos demasiado.
- Promesas: estados, `then`, `catch` y `finally`.
- `async` y `await`.
- Gestión de errores con `try...catch`.

📖 Documentación oficial: [22] [23]

Al terminar: sabréis crear relojes, cuentas atrás y animaciones con temporizadores, y estaréis preparados para pedir datos a un servidor.

Bloque 11 · Fetch y APIs

📁 `11-fetch-y-apis/`

Objetivo: traer datos de un servidor y mostrarlos en la página.

Contenidos:

- Qué es una API y qué es una petición HTTP: métodos, URL, códigos de estado.
- `fetch()` con `async` / `await`.
- Leer la respuesta con `response.json()` y comprobar `response.ok`.
- Mostrar estados de carga y de error al usuario.
- Pintar en el DOM los datos recibidos.
- Enviar datos con `POST`.
- Qué es CORS y por qué a veces el navegador bloquea una petición.

📖 Documentación oficial: [24]

Al terminar: sabréis consumir una API pública y construir con ella un buscador o un listado dinámico.



Fase 4 · Código profesional

Bloque 12 · Módulos, clases y buenas prácticas

📁 `12-modulos-y-clases/`

Objetivo: ordenar el código para que se pueda leer, mantener y ampliar.

Contenidos:

- Módulos: `export`, `import` y `<script type="module">`.
- Por qué los módulos necesitan un servidor (Live Server).
- Clases: `class`, `constructor`, propiedades y métodos.
- Herencia con `extends` y `super`.
- Propiedades privadas con `#`, *getters* y *setters*.
- Separar responsabilidades: datos, lógica y vista.
- Depuración con *breakpoints* en las herramientas de desarrollo.

📖 Documentación oficial: [3] [25]

Al terminar: sabréis repartir una aplicación en varios ficheros con una responsabilidad clara cada uno.

Bloque 13 · Proyecto final

📁 `13-proyecto-final/`

Objetivo: construir una aplicación completa que reúna todo lo aprendido.

Qué tiene que incluir el proyecto:

- Una interfaz generada y actualizada con el DOM.
- Eventos de ratón y teclado.
- Al menos un formulario con validación.
- Datos guardados en `localStorage`.
- Datos obtenidos de una API con `fetch`.
- Código organizado en módulos.

En la carpeta encontraréis el enunciado completo, los criterios de evaluación y una solución de referencia explicada por partes.



Bloque 01

Bloque 01 · Introducción

Empezamos. En este primer bloque no os voy a pedir que programéis nada complicado. Lo que quiero es que entendáis **dónde vive JavaScript**, cómo se conecta con una página y cómo comprobar que está funcionando. Parece poca cosa, pero la mitad de los atascos del primer mes vienen de aquí: un fichero mal enlazado, un script que se ejecuta antes de tiempo o un error en la consola que nadie ha leído.

Al terminar este bloque sabréis:

- Explicar qué hace JavaScript en una página y en qué se diferencia de HTML y CSS.
- Enlazar un fichero `app.js` a un `index.html` y elegir bien dónde colocar el `<script>`.
- Usar la consola del navegador para ver mensajes y leer errores.
- Pedir un dato al usuario con `prompt()` y mostrar resultados con `console.log()`, `alert()` y `textContent`.



1. ¿Qué es JavaScript?

JavaScript es un lenguaje de programación que se usa para que las páginas web sean interactivas: botones que hacen algo al pulsarlos, menús que se despliegan, formularios que avisan de un error... [1]

Ya conocéis las otras dos piezas de una página. Os las pongo juntas con un mismo botón para que veáis qué aporta cada una:



📊 Figura 4 · Tres capas, tres trabajos

Capa	Pregunta que responde	Si falta...
HTML	¿Qué hay en la página?	No hay nada que mostrar
CSS	¿Cómo se ve?	Se ve, pero con el aspecto por defecto del navegador
JavaScript	¿Qué pasa cuando el usuario hace algo?	Se ve bien, pero no reacciona

Una aclaración para no liarnos con el nombre: JavaScript **no tiene nada que ver con Java**. Se parecen en el nombre y poco más, igual que un coche y un cochecito de bebé.



2. JavaScript en el cliente

JavaScript puede ejecutarse en dos sitios. MDN distingue entre JavaScript *de lado del cliente*, que trabaja con el navegador y con la página, y JavaScript *de lado del servidor*, que trabaja, por ejemplo, con bases de datos [1]. En este curso nos quedamos en el cliente.

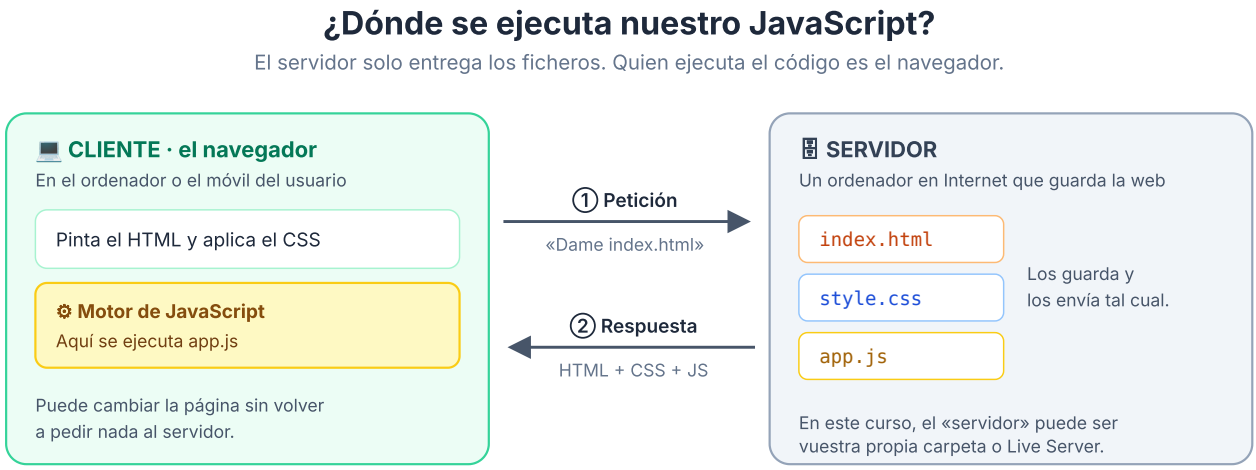


Figura 5 · Dónde se ejecuta JavaScript

Fijaos en el reparto de papeles. El servidor guarda los ficheros y los envía cuando el navegador los pide. Ya está. Quien **ejecuta** vuestro `app.js` es el navegador, en el ordenador o en el móvil de quien visita la web. Por eso JavaScript puede cambiar la página al instante, sin recargarla: todo ocurre en la máquina del usuario.



Nota

Durante el curso, ese «servidor» será vuestra propia carpeta: abrís `index.html` con doble clic y el navegador carga los ficheros desde el disco. Más adelante usaremos Live Server, que hace de servidor en vuestro ordenador.



3. Cómo se añade JavaScript a una página

Hay tres formas de meter JavaScript en una página. Os las enseño todas para que las reconozcáis cuando las veáis por ahí, pero en el curso solo vamos a usar una.

3.1. Script interno

El código va escrito dentro de una etiqueta `<script>` en el propio HTML:

```
<script>
    console.log('Hola desde el HTML');
</script>
```

Funciona, pero mezcla estructura y comportamiento en el mismo fichero.

3.2. Script externo ✔️

El código va en su propio fichero, y el HTML solo lo enlaza:

```
<script src="app.js"></script>
```

Esta es **la forma que vamos a usar en todo el curso**. Cada lenguaje en su fichero: `index.html`, `style.css` y `app.js`. Cuando la etiqueta tiene `src`, va vacía; lo que escribierais dentro se ignoraría [6].

3.3. Código dentro de atributos ✘

```
<button onclick="saludar()">Saludar</button>
```

Lo vais a encontrar en tutoriales antiguos. MDN lo marca como mala práctica: ensucia el HTML y obliga a repetir el atributo en cada elemento [26]. En el bloque 07 aprenderéis la forma buena, con `addEventListener`.



4. ¿Dónde coloco el `<script>`?

Aquí está la primera trampa del curso. El navegador lee el HTML **de arriba abajo**. Cuando encuentra un `<script>` normal, se para, lo descarga, lo ejecuta y solo entonces sigue leyendo [6]. Si vuestro script busca un botón que está más abajo en el HTML, todavía no existe.



📊 Figura 6 · Dónde colocar el script

Posición	¿Funciona?	Por qué
En el `<head>`, sin nada más	✘	Se ejecuta antes de que exista el `<body>`: `querySelector` devuelve `null`
Al final del `<body>`	✔	Cuando se ejecuta, todo el HTML ya está leído
En el `<head>` con `defer`	✔	Se descarga mientras se lee el HTML y se ejecuta al terminar [6]

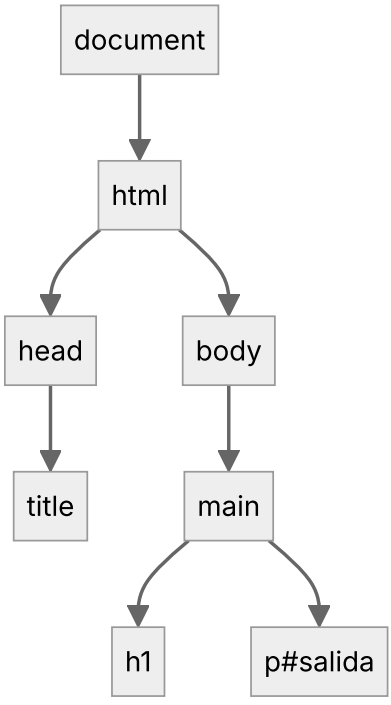
```
<!-- Opción 1: al final del body -->
<body>
    …
    <script src="app.js"></script>
</body>

<!-- Opción 2: en el head con defer -->
<head>
    <script src="app.js" defer></script>
</head>
```

Las dos opciones buenas aparecen en los ejercicios. Os toca reconocerlas y saber por qué funcionan. El ejercicio 02 está dedicado precisamente a provocar el error y arreglarlo.

Un adelanto: el DOM

Cuando el navegador lee el HTML, construye en memoria un árbol con todos los elementos de la página. Ese árbol se llama **DOM** (*Document Object Model*) y es lo que JavaScript puede leer y modificar [4]:



Para trabajar con un elemento primero hay que localizarlo. Lo haremos con `document.querySelector()`, que recibe un selector CSS y devuelve el primer elemento que lo cumple, o `null` si no encuentra ninguno [5]:

```
const salida = document.querySelector('#salida');
```

El DOM tiene su propio bloque, el 06. De momento os basta con esta línea.



5. Las herramientas de desarrollo

Todos los navegadores modernos traen unas herramientas para programadores. En Windows y Linux se abren con `F12` o con `Ctrl` + `Mayús` + `I` [2]. Desde hoy, **las tenéis abiertas siempre que programéis**. Sin excepciones.

Las dos pestañas que más vais a usar ahora mismo:

- **Consola**: muestra los mensajes de `console.log()` y, sobre todo, los errores en rojo.
- **Elementos** (o *Inspector*): muestra el DOM en vivo. Si JavaScript cambia un texto, lo veréis cambiar aquí.

Leer un error



Figura 7 · Cómo leer un error de la consola

Un mensaje de error tiene tres partes. Acostumbraos a buscarlas siempre:

```
Uncaught TypeError: Cannot set properties of null (setting 'textContent')     app.js:12
        ─────────  ──────────────────────────────────────────────────────     ─────────
          tipo                          mensaje                              fichero:línea
```

Cuando JavaScript encuentra un error, **detiene el script** en ese punto [27]. Lo que venga después no se ejecuta. Por eso un solo error puede hacer que «no funcione nada».

Los tres tipos que más vais a ver al principio:

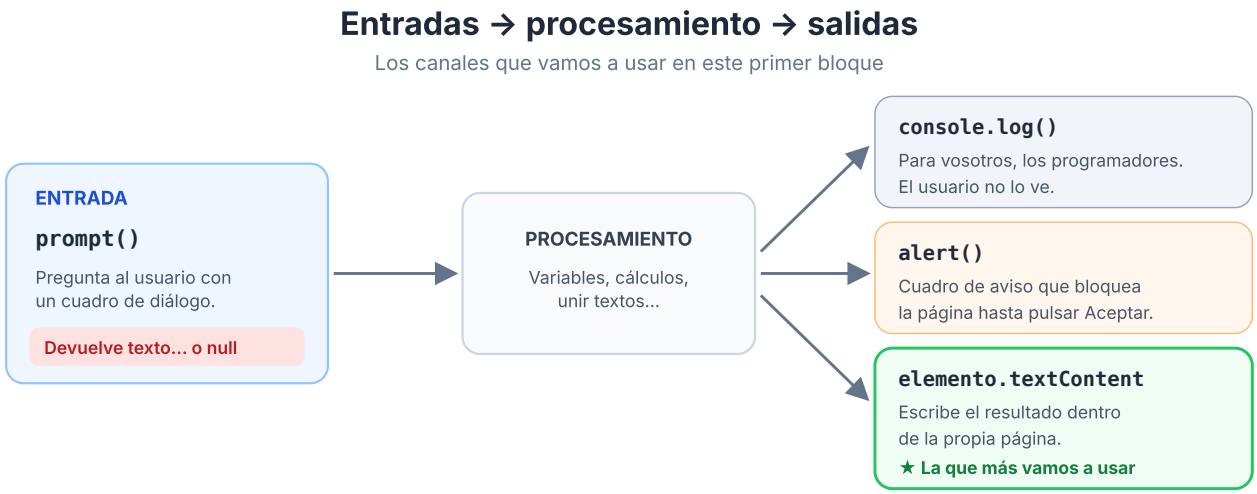
Error	Qué suele significar	Ejemplo típico
`SyntaxError`	Algo está mal escrito: falta un paréntesis, unas comillas…	`console.log('Hola);`
`ReferenceError`	Usáis un nombre que no existe	`nombrecurso` en lugar de `nombreCurso` [28]
`TypeError`	Usáis un valor de una forma que no admite	`null.textContent`, `console.lg()` [29]

El ejercicio 08 es una caza de errores para que practiquéis esto con calma.



6. Entradas y salidas

Todo programa, por pequeño que sea, sigue el mismo esquema: **recibe datos, los procesa y muestra un resultado**. En este bloque usamos estos canales:



📊 Figura 8 · Entradas y salidas

6.1. Entrada: `prompt()`

```
const nombre = prompt('¿Cómo te llamas?', 'invitado');
```

Muestra un cuadro con una pregunta y una caja de texto. El segundo argumento es el valor que aparece escrito por defecto. Y aquí viene lo importante: devuelve **siempre un texto**, o `null` si el usuario pulsa Cancelar [30].



⚠️ Importante

Aunque el usuario escriba `5`, `prompt()` devuelve `'5'`, un texto. Si vais a hacer cuentas, convertidlo antes con `Number()` [31]. Si no, `'2' + '2'` os dará `'22'`. Lo veréis en el ejercicio 07.

6.2. Salida para vosotros: `console.log()`

```
console.log('El valor de nombre es:', nombre);
```

Escribe en la consola [32]. El usuario no la ve; es vuestra herramienta para comprobar qué está pasando. También existen `console.warn()` y `console.error()`, que muestran el mensaje destacado como aviso o como error.

6.3. Salida que interrumpe: `alert()`

```
alert('¡Bienvenidos!');
```

Muestra un cuadro de aviso y **espera** a que el usuario lo cierre [33]. Mientras tanto, el programa está parado. Lo usaremos en las primeras pruebas y lo abandonaremos en cuanto lleguen los eventos.

6.4. Salida en la página: `textContent`

```
salida.textContent = `¡Hola, ${nombre}!`;
```

Sustituye el texto de un elemento [34]. Es la salida que más vais a usar en todo el curso, porque el resultado queda integrado en la página.

Existe otra propiedad parecida, `innerHTML`, que interpreta las etiquetas HTML que le pasáis. Tened cuidado con ella: si el texto viene del usuario, puede colar código en vuestra página. MDN la señala como una vía para ataques de *cross-site scripting* (XSS) [15]. Mi regla: **textos del usuario, siempre con `textContent`**. El ejercicio 05 os enseña el peligro en directo.

6.5. Plantillas de texto

En los ejemplos habéis visto textos entre comillas invertidas con `${…}` dentro. Son **plantillas de texto**: lo que va entre `${` y `}` se sustituye por su valor [35].

```
const saludo = `¡Hola, ${nombre}!`;          // plantilla
const saludo2 = '¡Hola, ' + nombre + '!';    // lo mismo, uniendo trozos con +
```

Las comillas invertidas están en la tecla que hay a la derecha de la `P` en el teclado español.



7. La plantilla de trabajo en este bloque

En este bloque los programas se ejecutan **de arriba abajo, una sola vez**, al cargar la página. Todavía no hay eventos, así que la plantilla de cada `app.js` es la versión corta:

```
// estructura del programa principal
const IVA = 0.21;

// constantes al DOM
const salida = document.querySelector('#salida');

// entradas
const precio = Number(prompt('Precio del producto:', '100'));

// procesamiento
const total = precio + precio * IVA;

// salidas
salida.textContent = `Total con IVA: ${total} €`;
```

En el bloque 07 aparecerán los escuchadores de eventos y la plantilla crecerá a su forma completa. Lo tenéis explicado en la introducción del curso.



8. Ejercicios

Cada ejercicio es una carpeta independiente con `index.html`, `style.css` y `app.js`. El enunciado está al principio de `app.js`. Abrid siempre la consola antes de empezar.

Nº	Ejercicio	Qué practicáis	Solución
01	¡Hola, consola!	Enlazar `app.js`, `console.log()`, `textContent`, número frente a texto	Ver
02	El script que llegó demasiado pronto	Script en el `<head>`, `null`, atributo `defer`	Ver
03	Del script interno al fichero externo	Separar el JavaScript del HTML	Ver
04	Los tres canales de salida	`console.log()`, `alert()`, `textContent`, `console.warn()`, `console.error()`	Ver
05	textContent frente a innerHTML	Diferencias y riesgo de XSS	Ver
06	Mi primera entrada con prompt()	`prompt()`, plantillas de texto, `document.title`, el caso `null`	Ver
07	prompt() siempre devuelve texto	`typeof`, `Number()`, concatenar frente a sumar, `NaN`	Ver
08	Caza de errores en la consola	Leer `TypeError` y `ReferenceError` y corregirlos	Ver

Recordad el método: primero lo intentáis vosotros, con la consola abierta. La solución, después. Cada carpeta de solución tiene un `README.md` donde os explico el porqué de cada línea.



🧪 Los ejercicios, explicados

Ejercicio 01 · ¡Hola, consola!

> 🎯 **Objetivo:** Comprobar que el fichero app.js está bien enlazado y conocer los dos primeros canales de salida: la consola del navegador (console.log) y la propia página (textContent).
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Comprobar que `app.js` está bien enlazado al HTML y usar los dos canales de salida que más vais a ver en el curso: la **consola** y la **página**.



La solución paso a paso

1. Un mensaje en la consola

```
console.log('¡Hola desde app.js!');
```

`console.log()` escribe en la consola de las herramientas de desarrollo [32]. El usuario normal no la abre nunca, así que este canal es para vosotros: sirve para comprobar que el código se ejecuta y para ver qué valor tiene una variable en un momento dado.

Si al abrir la consola **no** aparece el mensaje, el problema no está en esta línea. Está en el enlace: revisad que el `<script src="app.js">` está escrito bien y que el fichero se llama exactamente `app.js`.

2. Números frente a textos

```
console.log(10 + 5);     // 15
console.log('10' + '5'); // "105"
```

Aquí está la trampa del ejercicio. El símbolo `+` hace dos cosas distintas según lo que tenga a los lados:

Expresión	Tipo de los datos	Qué hace +	Resultado
`10 + 5`	dos números	suma	`15`
`'10' + '5'`	dos textos	une (concatena)	`'105'`

Con las comillas, `'10'` deja de ser un número y pasa a ser un texto que casualmente contiene dígitos. Guardad esta idea, porque en el ejercicio 07 os vais a encontrar con ella de frente.

3. Apuntar al párrafo

```
const salida = document.querySelector('#salida');
```

`document.querySelector()` busca en la página el primer elemento que cumple el selector CSS que le pasáis y lo devuelve [36]. Lo guardo en una constante para poder usarlo después. El `#` significa lo mismo que en CSS: «busca por `id`».

4. Escribir en la página

```
salida.textContent = 'JavaScript está funcionando ✔';
```

La propiedad `textContent` representa el texto que hay dentro de un elemento [34]. Si le asigno un valor, sustituye lo que hubiera. El texto «Esperando a JavaScript...» desaparece y aparece el nuevo.



Qué tenéis que observar

- En la consola salen tres líneas: el saludo, `15` y `105`.
- En la página, el párrafo azul cambia de texto.
- Fijaos en el orden del código: primero localizo el elemento y después escribo en él. Al revés no funcionaría.



Errores típicos

- Escribir `console.log` con mayúscula (`Console.log`). JavaScript distingue mayúsculas y minúsculas.
- Olvidar el `#` en el selector: `querySelector('salida')` busca una **etiqueta** `<salida>`, que no existe.

Ejercicio 02 · El script que llegó demasiado pronto

> 🎯 **Objetivo:** Entender por qué un script colocado en el <head> no encuentra los elementos de la página y arreglarlo con el atributo defer.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Descubrir por qué un script colocado en el `<head>` no consigue cambiar la página y arreglarlo con `defer`.



El síntoma

Al abrir la versión sin resolver, el título no cambia y la consola muestra algo parecido a esto (el texto exacto depende del navegador):

```
Uncaught TypeError: Cannot set properties of null (setting 'textContent')
```

Leedlo despacio, porque os lo vais a encontrar muchas veces: «no puedo poner la propiedad `textContent` a algo que es `null`». Es decir, la constante `titulo` no contiene ningún elemento.



La causa

Con el `console.log(titulo)` lo confirmáis: aparece `null`. `querySelector()` devuelve `null` cuando no encuentra ningún elemento que cumpla el selector [36].

¿Y por qué no lo encuentra, si el `<h1 id="titulo">` está en el HTML? Por el **momento** en que se ejecuta el script. El navegador lee el HTML de arriba abajo. Cuando llega a un `<script>` normal, se detiene, lo descarga, lo ejecuta y solo después sigue leyendo [6]. Como el script está en el `<head>`, se ejecuta antes de que el navegador haya leído el `<body>`. El `<h1>` todavía no existe.

Es como entrar a un cine antes de que abran la sala y quejarse de que no hay butacas.



La solución

He añadido `defer` a la etiqueta, sin moverla de sitio:

```
<script src="app.js" defer></script>
```

El atributo `defer` indica al navegador que ejecute el script cuando haya terminado de leer el documento [6]. Mientras tanto, lo va descargando en paralelo, así que no se pierde tiempo.

La otra solución válida es mover el `<script>` al final del `<body>`, justo antes de `</body>`. Las dos funcionan. En el curso uso las dos, y en la teoría del bloque tenéis una infografía que compara los tres casos.



⚠️ Importante

Lo que **no** debéis hacer es dejar el script en el `<head>` sin `defer` cuando ese script busca elementos del `<body>`.



Qué tenéis que observar

- Con `defer`, el `console.log(titulo)` ya muestra el elemento `<h1>` en lugar de `null`.
- El error desaparece y los dos textos cambian.

Ejercicio 03 · Del script interno al fichero externo

> 🎯 **Objetivo:** Separar el comportamiento de la estructura: llevar el código JavaScript que está dentro de index.html a un fichero externo app.js y enlazarlo correctamente.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Sacar el JavaScript que estaba escrito dentro de `index.html` y llevarlo a su propio fichero, `app.js`.



Por qué lo hacemos

La página funcionaba igual antes y después. Entonces, ¿para qué tanto lío? Por orden. Con el código dentro del HTML:

- No podéis reutilizar ese JavaScript en otra página sin copiarlo y pegarlo.
- El HTML se llena de cosas que no son estructura.
- Cuando el programa crezca, encontrar un fallo será como buscar las llaves en un bolso lleno.

Con un fichero externo, cada lenguaje vive en su casa: `index.html` para la estructura, `style.css` para el aspecto y `app.js` para el comportamiento. La documentación de MDN muestra el script interno y el externo como formas válidas de cargar JavaScript, y recomienda no mezclar código dentro de atributos HTML como `onclick` [26].



La solución paso a paso

1. El HTML queda limpio

Del final del `<body>` desaparece todo el bloque `<script> … </script>`, y en el `<head>` aparece una sola línea:

```
<script src="app.js" defer></script>
```

Fijaos en que el `<script>` va **vacío**. Cuando una etiqueta `<script>` tiene atributo `src`, el navegador carga el fichero indicado e ignora lo que haya escrito entre las etiquetas [6]. Y le pongo `defer` porque está en el `<head>`, como visteis en el ejercicio 02.

2. Cada instrucción en su sitio

He colocado cada línea en la sección que le corresponde de la plantilla:

Instrucción	Sección
`const curso = 'JavaScript en el cliente';`	estructura del programa principal
`const salida = document.querySelector('#salida');`	constantes al DOM
`console.log('Código ejecutado desde app.js');`	procesamiento
`salida.textContent = 'Bienvenidos al curso de ' + curso;`	salidas

Esto no lo exige JavaScript, que ejecutaría las líneas en cualquier orden lógico. Lo hago yo, y os pido que lo hagáis vosotros, para que cualquier `app.js` del curso se lea siempre de la misma manera.



Qué tenéis que observar

- La página se ve exactamente igual que antes.
- En la consola el mensaje ahora dice «desde app.js». Así comprobáis que se está ejecutando el fichero externo y no un resto olvidado en el HTML.



Errores típicos

- Copiar también las etiquetas `<script>` y `</script>` dentro de `app.js`. Un fichero `.js` solo lleva JavaScript; esas etiquetas son HTML y provocan un error de sintaxis.
- Dejar el bloque antiguo en el HTML. Entonces el código se ejecuta dos veces y la constante `salida` se declara dos veces, lo que también da error.

Ejercicio 04 · Los tres canales de salida

> 🎯 **Objetivo:** Usar los tres canales de salida del bloque (console.log, alert y textContent) con el mismo mensaje y comparar quién ve cada uno y cómo se comporta.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Enviar el mismo mensaje por los tres canales de salida del bloque y comparar quién lo ve y cómo se comporta cada uno.



La solución paso a paso

Canal 1 · La consola

```
console.log(mensaje);
```

Escribe en la consola de las herramientas de desarrollo [32]. El usuario no la ve. Es vuestra libreta de apuntes mientras programáis.

Las variantes `console.warn()` y `console.error()` funcionan igual, pero la consola las destaca con otro color y otro icono. Os vendrán bien para distinguir de un vistazo un aviso de un mensaje normal.

Canal 2 · El cuadro de aviso

```
alert(mensaje);
```

`alert()` pide al navegador que muestre un cuadro con el mensaje y que **espere** a que el usuario lo cierre [33]. Mientras está abierto, vuestro programa se queda parado en esa línea.

Canal 3 · La página

```
salida.textContent = mensaje;
```

Escribe el mensaje dentro del párrafo [34]. Es el canal más natural para el usuario: el resultado forma parte de la página, con su diseño, y no interrumpe nada.



La pregunta del final

¿El párrafo cambia antes o después de cerrar el `alert`? **Después.** El programa se ejecuta línea a línea. Como `alert()` espera a que el usuario pulse Aceptar, la línea de `textContent` no llega a ejecutarse hasta entonces.

Por eso os decía que `alert()` es útil para empezar, pero incómodo en una página de verdad. Imaginad una web que os saluda con tres cuadros seguidos antes de dejaros hacer nada. Pues eso.



Comparativa

Canal	¿Quién lo ve?	¿Bloquea la página?	¿Cuándo lo usaré?
`console.log()`	Solo el programador	No	Siempre, para comprobar valores
`alert()`	El usuario	Sí	Solo en los primeros bloques, para pruebas rápidas
`textContent`	El usuario	No	En casi todos los ejercicios del curso

Ejercicio 05 · textContent frente a innerHTML

> 🎯 **Objetivo:** Ver la diferencia entre textContent, que escribe texto tal cual, e innerHTML, que interpreta las etiquetas HTML, y entender por qué nunca debemos usar innerHTML con textos que escribe el usuario.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir el mismo texto con `textContent` y con `innerHTML`, ver la diferencia y descubrir por qué `innerHTML` es peligroso con textos que no controláis.



La diferencia

```
const frase = 'Esto es <strong>muy</strong> importante';

salidaTexto.textContent = frase;
salidaHtml.innerHTML = frase;
```

Propiedad	Qué hace con `<strong>`	Lo que veis
`textContent`	Lo trata como caracteres normales	`Esto es <strong>muy</strong> importante`
`innerHTML`	Lo interpreta como una etiqueta HTML	Esto es **muy** importante

`textContent` trabaja con texto plano [34]. `innerHTML`, en cambio, lee y escribe HTML: lo que le dais se convierte en elementos de la página [15].



El peligro

Ahora fijaos en el comentario:

```
const comentario = '<img src="x" onerror="alert(\'¡Te he colado código!\')">';
salidaComentario.textContent = comentario;
```

Con `textContent` se ve como texto y no pasa nada. Si hicisteis el experimento de cambiarlo por `innerHTML`, visteis saltar un `alert`. ¿Qué ha ocurrido? El navegador ha creado una imagen de verdad, no ha podido cargar `x`, ha disparado el atributo `onerror`... y ha ejecutado el código que alguien había escondido ahí.

Ahora sustituid ese `alert` por un código que roba datos y entenderéis el problema. Este tipo de ataque se llama *cross-site scripting* (XSS), y MDN avisa expresamente de que `innerHTML` es una vía para sufrirlo cuando el contenido viene de fuera [15]. Por eso también desaconseja usar `innerHTML` para poner simple texto [34].



⚠️ Importante

Mi regla para todo el curso: si el texto lo escribe el usuario o llega de fuera, **siempre** `textContent`. `innerHTML` solo con HTML que habéis escrito vosotros y que no mezcla nada externo.



Qué tenéis que observar

- El primer párrafo muestra las etiquetas `<strong>` como texto.
- El segundo muestra «muy» en negrita.
- El tercero muestra la etiqueta `<img …>` como texto y **no** salta ningún aviso.

Ejercicio 06 · Mi primera entrada con prompt()

🎯 **Objetivo:** Recoger el primer dato del usuario con prompt(), guardarlo en una constante y usarlo para construir un saludo que se muestra en la página y en el título de la pestaña.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Pedir el nombre al usuario y usarlo para saludarle en la página y en la pestaña del navegador.



La solución paso a paso

1. Entrada: pedir el dato

```
const nombre = prompt('¿Cómo te llamas?', 'invitado');
```

`prompt()` muestra un cuadro con una pregunta y una caja de texto. Recibe dos argumentos: el mensaje y el valor que aparece escrito por defecto [30]. Lo que devuelve lo guardo en `nombre`.

Fijaos en que el programa se **para** en esta línea hasta que el usuario responde, igual que pasaba con `alert()`.

2. Procesamiento: construir el saludo

```
const saludo = `¡Hola, ${nombre}! Bienvenido al curso`;
```

Las comillas invertidas (`` ` ``, la tecla a la derecha de la `P` en el teclado español) crean una **plantilla de texto**. Dentro, todo lo que pongáis entre `${` y `}` se sustituye por su valor [2]. Es mucho más cómodo que ir uniendo trozos con `+`:

```
'¡Hola, ' + nombre + '! Bienvenido al curso'   // funciona, pero se lee peor
`¡Hola, ${nombre}! Bienvenido al curso`         // lo mismo, más claro
```

3. Salidas: la página y la pestaña

```
salida.textContent = saludo;
document.title = `Hola, ${nombre}`;
```

La primera línea ya la conocéis. La segunda es nueva: `document.title` es el título del documento, el que veis en la pestaña del navegador, y se puede leer y cambiar desde JavaScript [37].

Uso `textContent` y no `innerHTML` porque el nombre lo escribe el usuario. Recordad el ejercicio 05.



Los tres casos que os pedí probar

Qué hace el usuario	Qué devuelve `prompt()`	Qué aparece
Escribe «Lucía» y acepta	`'Lucía'`	¡Hola, Lucía!
Acepta sin tocar nada	`'invitado'`	¡Hola, invitado!
Pulsa **Cancelar**	`null`	¡Hola, null!

Ese último caso es feo, lo sé. `prompt()` devuelve un texto o `null` [1]: cuando el usuario cancela, no hay texto. Para tratarlo necesitamos tomar una decisión («si es `null`, entonces...»), y eso es justo lo que aprenderéis en el bloque 03 con `if`. De momento me basta con que sepáis por qué ocurre.

Ejercicio 07 · prompt() siempre devuelve texto

> 🎯 **Objetivo:** Comprobar que prompt() devuelve siempre un texto aunque el usuario escriba números, y convertirlo con Number() para poder sumar.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Sumar dos números pedidos con `prompt()` y descubrir por qué, sin convertirlos, la suma sale mal.



El problema

Si escribís 2 y 2, la primera suma da **22**. No es un error de JavaScript: es exactamente lo que le hemos pedido.

```
const numero1 = prompt('Escribe el primer número:', '2');
console.log(typeof numero1);   // "string"
```

`prompt()` devuelve siempre un texto (o `null` si se cancela) [30]. Da igual que el usuario escriba cifras: lo que llega es `'2'`, con comillas, no `2`. Y ya visteis en el ejercicio 01 qué hace `+` con dos textos: los une.



La solución

```
const sumaBien = Number(numero1) + Number(numero2);
```

`Number()` convierte un valor en número [31]. Ahora `+` tiene dos números a los lados y suma.

Entrada	`numero1 + numero2`	`Number(numero1) + Number(numero2)`
2 y 2	`'22'`	`4`
10 y 5	`'105'`	`15`
hola y 5	`'hola5'`	`NaN`



¿Y ese `NaN`?

Si escribís una palabra, `Number('hola')` no sabe qué número es y devuelve `NaN`, que significa *Not a Number*, «no es un número» [31]. Al sumar cualquier cosa con `NaN`, el resultado sigue siendo `NaN`. Comprobar que el usuario ha escrito un número válido lo aprenderéis en los bloques 02 y 03.



⚠️ Importante

Esta regla os va a acompañar todo el curso: **lo que escribe el usuario llega como texto**. Pasa con `prompt()` y pasará igual con los `<input>` de los formularios. Si vais a hacer cuentas, convertid primero.

Ejercicio 08 · Caza de errores en la consola

🎯 **Objetivo:** Aprender a leer los mensajes de error de la consola: qué dicen, en qué línea están y cómo corregirlos uno a uno.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Encontrar y corregir tres errores leyendo los mensajes de la consola, uno detrás de otro.



Cómo se lee un error

Cada mensaje rojo de la consola tiene tres partes que debéis buscar siempre:

1. **El tipo de error**: `TypeError`, `ReferenceError`, `SyntaxError`…
2. **El mensaje**: la explicación, en inglés, de qué ha fallado.
3. **El fichero y la línea**: a la derecha, algo como `app.js:13`. Si hacéis clic, el navegador os lleva a esa línea.

Y una idea más: cuando JavaScript encuentra un error, **deja de ejecutar el resto del script** [27]. Por eso la consola solo os enseña un error cada vez. No es que haya uno; es que no llega a ver los demás.



Los tres errores, en el orden en que aparecen

Error 1 · `TypeError: console.lg is not a function`

```
console.lg('Empezando el programa…');   // ✘
console.log('Empezando el programa…');  // ✔
```

Una errata. `console.lg` no existe, así que JavaScript no puede ejecutarlo como función. Este tipo de error aparece cuando un valor no es del tipo que se esperaba [29].

Error 2 · `ReferenceError: nombrecurso is not defined`

```
${nombrecurso}   // ✘ c minúscula
${nombreCurso}   // ✔ como se declaró
```

JavaScript distingue entre mayúsculas y minúsculas. `nombreCurso` y `nombrecurso` son, para él, dos variables diferentes, y la segunda no se ha declarado nunca [28]. Es probablemente el error más frecuente del primer mes de curso.

Error 3 · `TypeError: Cannot set properties of null (setting 'textContent')`

```
document.querySelector('#resultado');  // ✘ no existe en el HTML
document.querySelector('#salida');     // ✔
```

Este ya lo conocéis del ejercicio 02. El `id` del párrafo es `salida`, no `resultado`. `querySelector()` no encuentra nada y devuelve `null` [36]. El error no salta en esa línea, sino más abajo, cuando se intenta escribir en `null`. Ojo con esto: **la línea del error no siempre es la línea del fallo**. A veces hay que mirar hacia arriba para ver de dónde viene el valor.

Fijaos en que esta línea está casi al principio del programa y, sin embargo, su error es el último en aparecer. La constante se crea sin problema; el fallo solo se manifiesta cuando por fin se usa.



Qué tenéis que observar

- Con los tres errores corregidos, el párrafo muestra el mensaje de enhorabuena.
- La consola termina con «Programa terminado sin errores».

Error	Tipo	Causa
`console.lg`	`TypeError`	Errata en el nombre del método
`nombrecurso`	`ReferenceError`	Mayúsculas y minúsculas distintas
`#resultado`	`TypeError`	El `id` no existe en el HTML → `null`



Bloque 02

Bloque 02 · Fundamentos del lenguaje

En el bloque anterior visteis dónde vive JavaScript. Ahora toca aprender a hablarlo. En este bloque os voy a enseñar las piezas con las que se construye cualquier programa: **guardar datos, saber de qué tipo son, operar con ellos y compararlos**. Es la gramática del lenguaje. Sin ella, todo lo que venga después se tambalea.

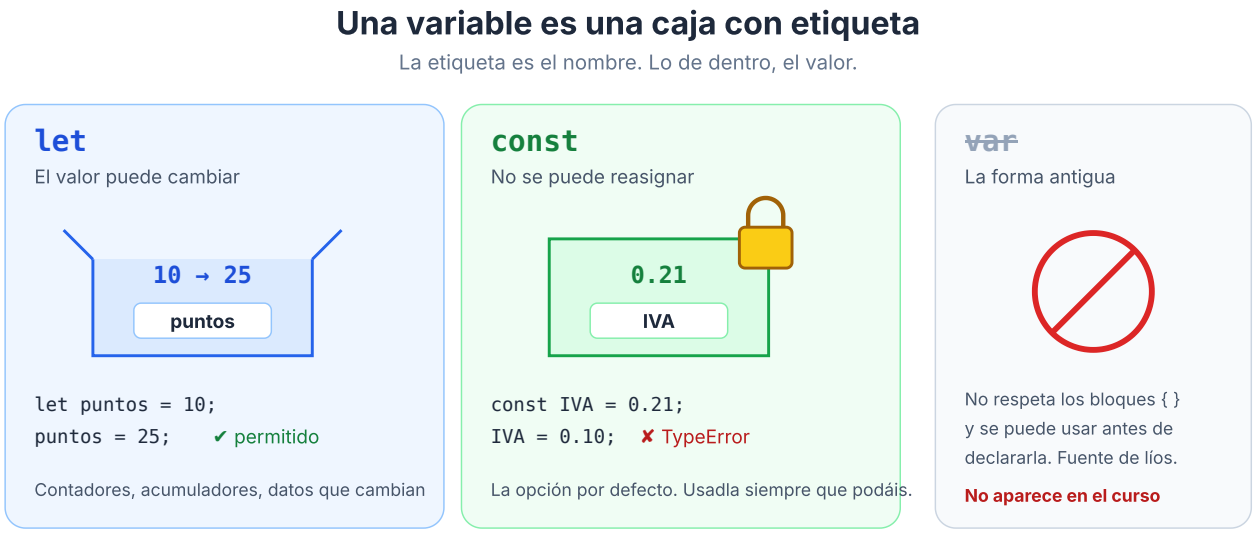
Al terminar este bloque sabréis:

- Declarar variables y constantes con `let` y `const`, y elegir bien entre las dos.
- Poner nombres que se entienden y que JavaScript acepta.
- Reconocer los tipos de datos primitivos y comprobarlos con `typeof`.
- Usar los operadores aritméticos y de asignación.
- Convertir textos en números sin llevaros sorpresas.
- Comparar valores con `===` y entender por qué no usamos `==`.
- Redondear y generar números aleatorios con `Math`.



1. Variables y constantes

Un programa necesita recordar cosas: el nombre del usuario, un precio, cuántos puntos lleva en un juego. Para eso están las variables. Pensad en ellas como **cajas con una etiqueta**: la etiqueta es el nombre y lo que hay dentro es el valor.



📊 Figura 9 · let, const y var

1.1. `let`: valores que cambian

```
let puntos = 0;
puntos = 10;      // ✔ se puede reasignar
puntos = 25;      // ✔ las veces que haga falta
```

`let` declara una variable con ámbito de bloque que se puede reasignar [38]. Fijaos en que `let` solo se escribe la primera vez. Para cambiar el valor, basta con el nombre y el `=`.

1.2. `const`: valores que no cambian

```
const IVA = 0.21;
IVA = 0.10;       // ✘ TypeError: Assignment to constant variable.
```

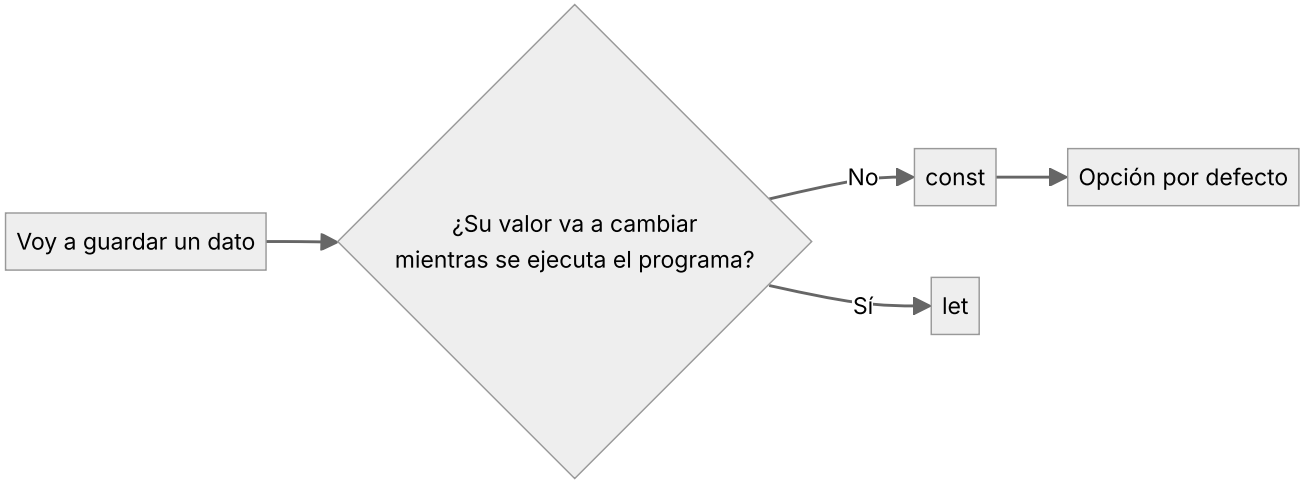
`const` crea una referencia de solo lectura: el identificador no se puede reasignar [39]. Si lo intentáis, JavaScript lanza un `TypeError` [40]. Y otra diferencia con `let`: una constante hay que inicializarla en el momento de declararla.

1.3. ¿Y `var`?

Lo vais a ver en código antiguo y en muchos tutoriales. `var` no respeta los bloques `{ }` como `let` y `const` [41] y se puede usar antes de la línea en la que se declara, lo que da lugar a errores difíciles de encontrar. En este curso **no lo usamos**.

1.4. ¿`let` o `const`?

Mi regla es muy sencilla:



Empezad siempre con `const`. Si después descubrís que el valor tiene que cambiar, lo cambiáis a `let`. Así, cuando alguien lea vuestro código y vea un `let`, sabrá que ese valor va a moverse.



2. Cómo nombrar las variables

2.1. Las reglas (obligatorias)

Un nombre de variable debe empezar por una letra, un guion bajo (`_`) o un dólar (`$`). Después puede llevar también dígitos [7]. Y JavaScript distingue entre mayúsculas y minúsculas: `total` y `Total` son dos variables distintas [7].

Nombre	¿Válido?	Motivo
`precioTotal`	✔	
`_temporal`	✔	Puede empezar por `_`
`nota2`	✔	Los dígitos pueden ir después
`2nota`	✘	No puede empezar por un dígito
`precio total`	✘	No puede llevar espacios
`precio-total`	✘	El guion se interpretaría como una resta
`let`	✘	Es una palabra reservada del lenguaje

2.2. Las convenciones (muy recomendables)

En JavaScript las variables se escriben en ***camelCase***: todo junto, la primera palabra en minúscula y la primera letra de cada palabra siguiente en mayúscula [6]: `precioConIva`, `numeroDeAlumnos`, `btnEnviar`.

Y lo más importante: **nombres que expliquen lo que guardan**.

Mal	Bien
`x`	`edad`
`pt`	`precioTotal`
`dato1`, `dato2`	`nombre`, `apellido`
`b`	`btnCalcular`

Dentro de seis meses no os acordaréis de qué era `pt`. Os lo garantizo.



3. Tipos de datos primitivos

Cada valor tiene un tipo, y el tipo decide qué podéis hacer con él. JavaScript tiene siete tipos primitivos [7]. En este curso vais a trabajar con cinco:



Figura 10 · Tipos primitivos

Tipo	Para qué	Ejemplos
`string`	Texto, siempre entre comillas	`'Hola'`, `"42"`, `` `Hola ${nombre}` ``
`number`	Números, enteros o decimales	`42`, `3.14`, `-7`, `NaN`
`boolean`	Sí o no	`true`, `false`
`undefined`	Variable que aún no tiene valor	`let x;`
`null`	«Aquí no hay nada», puesto a propósito	`null`

3.1. Preguntar el tipo: `typeof`

```
typeof 'Hola'    // "string"
typeof 42        // "number"
typeof true      // "boolean"
typeof undefined // "undefined"
typeof null      // "object"  ⚠
```

`typeof` devuelve un texto con el nombre del tipo [42]. Y sí, la última línea es correcta: `typeof null` devuelve `"object"` [42]. Es una rareza histórica del lenguaje. `null` no es un objeto; simplemente, no le preguntéis a `typeof` por él.

3.2. `NaN`, el número que no es un número

`NaN` significa *Not a Number*. Aparece cuando una operación numérica no puede dar un resultado válido, como `Number('hola')`. Tiene dos particularidades: es de tipo `number` y **no es igual a nada, ni siquiera a sí mismo** [43]. Para detectarlo se usa `Number.isNaN(valor)`.



4. Operadores

4.1. Aritméticos

Operador	Operación	Ejemplo	Resultado
`+`	Suma	`7 + 2`	`9`
`-`	Resta	`7 - 2`	`5`
`*`	Multiplicación	`7 * 2`	`14`
`/`	División	`7 / 2`	`3.5`
`%`	Resto de la división	`7 % 2`	`1`
`**`	Potencia	`7 ** 2`	`49`

Los dos últimos son los que menos conocéis. El **resto** (`%`) devuelve lo que sobra de una división entera [9]: es la forma clásica de saber si un número es par (`n % 2` da `0`). No tiene nada que ver con los porcentajes, aunque use el mismo símbolo. La **potencia** (`**`) eleva el primer número al segundo [44].

4.2. De asignación

Son atajos para modificar una variable a partir de su propio valor [11]:

Atajo	Equivale a
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x++`	`x = x + 1`
`x--`	`x = x - 1`

`++` suma uno y `--` resta uno [45]. Todos reasignan la variable, así que solo funcionan con `let`.

4.3. El `+` con textos

El `+` es el único operador aritmético con doble personalidad: si alguno de los dos lados es un texto, **une** en lugar de sumar.

```
2 + 2          // 4
'2' + '2'      // "22"
'Nota: ' + 7   // "Nota: 7"
```

Y `+=` también funciona con textos: es la técnica del **acumulador**, que consiste en ir añadiendo trozos a un texto que empieza vacío. La veréis en el ejercicio 04.

4.4. Plantillas de texto

Las conocéis del bloque 01. Os recuerdo dos ventajas frente a unir con `+` [13]:

```
// 1. Insertan valores con ${…}, incluso operaciones
const resumen = `Total: ${precio * cantidad} €`;

// 2. Admiten saltos de línea reales
const ticket = `Producto: ${producto}
Cantidad: ${cantidad}`;
```



5. Conversión de tipos

Este apartado os va a ahorrar muchas horas de «¿por qué me sale 22?». Todo lo que escribe el usuario, ya sea con `prompt()` o en un formulario, **llega como texto**. Si vais a operar con números, hay que convertir.

De texto a número: tres herramientas

El mismo texto de entrada, tres resultados distintos

Texto de entrada	`Number()`	`parseInt()`	`parseFloat()`
`'42'`	42	42	42
`'3.75'`	3.75	3	3.75
`'42px'`	NaN	42	42
`''`	0	NaN	NaN
`'hola'`	NaN	NaN	NaN
	Estricto: todo o nada	Lee lo que puede · enteros	Lee lo que puede · decimales

📊 Figura 11 · Conversión de tipos

Función	Cómo trabaja
`Number(texto)`	Estricta: el texto entero tiene que ser un número; si no, `NaN`. El texto vacío lo convierte en `0` [31]
`parseInt(texto, 10)`	Lee dígitos de izquierda a derecha hasta que encuentra algo que no lo es. Devuelve un entero, o `NaN` si el primer carácter no sirve [46]
`parseFloat(texto)`	Igual que `parseInt`, pero admite decimales [47]
`String(valor)`	El camino contrario: convierte cualquier valor en texto



💡 Consejo

Mi recomendación para los datos del usuario: `Number()`. Precisamente porque es estricta. Si alguien escribe `12abc`, prefiero un `NaN` que me avise a un `12` que lo deje pasar sin decir nada. Dejad `parseInt` y `parseFloat` para textos que sabéis que traen unidades, como `'42px'`.

Y una costumbre: convertid **en el momento de leer**.

```
const edad = Number(prompt('¿Cuántos años tienes?'));
```

A partir de esa línea, `edad` ya es un número y os olvidáis del problema.



6. Comparaciones

Comparar dos valores devuelve siempre un `boolean`: `true` o `false`. Estos resultados serán la base de las decisiones del bloque 03.

Operador	Significado	Ejemplo	Resultado
`===`	Igual (valor y tipo)	`5 === 5`	`true`
`!==`	Distinto (valor o tipo)	`5 !== '5'`	`true`
`>`	Mayor que	`7 > 3`	`true`
`<`	Menor que	`7 < 3`	`false`
`>=`	Mayor o igual	`5 >= 5`	`true`
`<=`	Menor o igual	`4 <= 5`	`true`

6.1. `==` frente a `===`

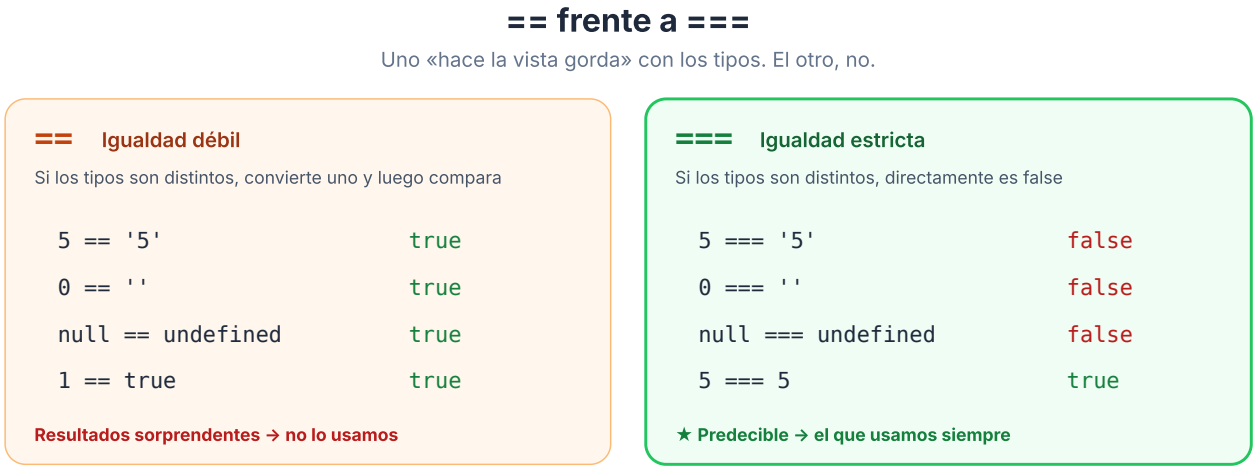


Figura 12 · == frente a ===

`==` intenta convertir los dos valores al mismo tipo antes de comparar [48]. `===` no convierte nada: si los tipos son distintos, el resultado es `false` [49]. ¿El problema de `==`? Que sus conversiones dan resultados poco intuitivos, como `0 == ''` → `true`.

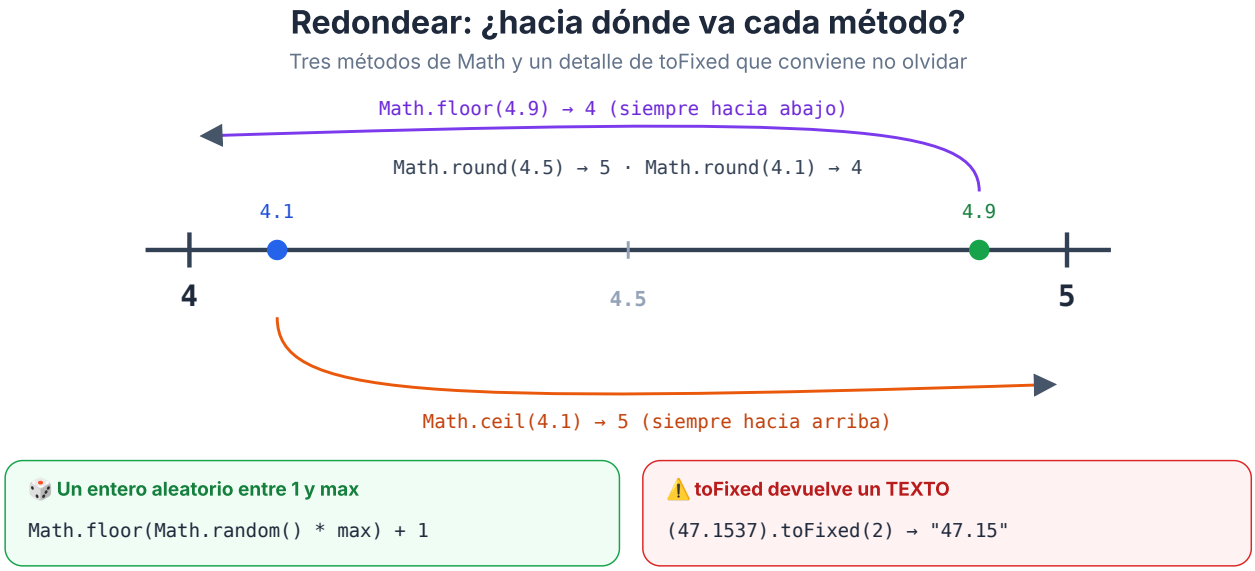


⚠️ Importante

En este curso comparamos **siempre** con `===` y `!==`. Si necesitáis comparar un texto con un número, convertid vosotros primero: `Number(texto) === 5`.



7. El objeto `Math` y el redondeo



📊 Figura 13 · Redondear con Math y toFixed

`Math` es un objeto que JavaScript trae de serie con constantes y funciones matemáticas [19]:

Método	Qué hace	Ejemplo
`Math.round(x)`	Redondea al entero más cercano	`Math.round(4.5)` → `5`
`Math.floor(x)`	Redondea hacia abajo	`Math.floor(4.9)` → `4`
`Math.ceil(x)`	Redondea hacia arriba	`Math.ceil(4.1)` → `5`
`Math.max(a, b, …)`	Devuelve el mayor	`Math.max(3, 8, 1)` → `8`
`Math.min(a, b, …)`	Devuelve el menor	`Math.min(3, 8, 1)` → `1`
`Math.random()`	Decimal aleatorio desde 0 (incluido) hasta 1 (excluido) [50]	`0.4821…`
`Math.PI`	El número π	`3.14159…`

7.1. Un número entero aleatorio

La fórmula que os vais a llevar a todas partes:

```
// entero aleatorio entre 1 y max, ambos incluidos
const numero = Math.floor(Math.random() * max) + 1;
```

Un dado de seis caras es `Math.floor(Math.random() * 6) + 1`. En el ejercicio 08 os la explico paso a paso.

7.2. Mostrar decimales: `toFixed()`

```
const total = 47.1537;
total.toFixed(2)   // "47.15"
```

`toFixed()` escribe el número con los decimales que le pidáis [51]. Perfecto para mostrar precios. Pero ojo aquí: **devuelve un texto** [51]. Usadlo solo en la salida, nunca para seguir calculando.



8. La plantilla de trabajo en este bloque

Seguimos con programas lineales, igual que en el bloque 01: se ejecutan una vez, de arriba abajo. La plantilla es la misma:

```
// estructura del programa principal
const IVA = 0.21;

// constantes al DOM
const salida = document.querySelector('#salida');

// entradas
const precio = Number(prompt('Precio:', '100'));

// procesamiento
const total = precio + precio * IVA;

// salidas
salida.textContent = `Total: ${total.toFixed(2)} €`;
```

Fijaos en cómo encaja todo lo de este bloque: `const` por defecto, nombres en *camelCase*, conversión al leer y `toFixed()` solo en la salida.



9. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Ficha de personaje	`let` frente a `const`, reasignar, error al cambiar una constante	Ver
02	Detective de tipos	Tipos primitivos, `typeof`, las rarezas de `null` y `NaN`	Ver
03	Calculadora de operadores	`+ - * / % **`, dividir entre cero	Ver
04	Marcador de videojuego	`+= -= *= /= ++ --`, acumulador de texto, `\n`	Ver
05	Ticket de compra	Plantillas de varias líneas, IVA, `toFixed()`	Ver
06	Laboratorio de conversiones	`Number()`, `parseInt()`, `parseFloat()`, `String()`	Ver
07	== frente a ===	Igualdad débil y estricta, el caso de `NaN`	Ver
08	Dividir la cuenta del restaurante	`Math.ceil`, redondeo al céntimo, `Math.random()`	Ver

Lo de siempre: consola abierta, primero vosotros y la solución después.



🧪 Los ejercicios, explicados

Ejercicio 01 · Ficha de personaje: let y const

> 🎯 **Objetivo:** Decidir cuándo usar const y cuándo let, reasignar una variable declarada con let y comprobar qué pasa si intentamos reasignar una constante.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Repartir los datos de un personaje entre `const` y `let`, cambiar los que deben cambiar y comprobar qué pasa si intentáis cambiar una constante.



La decisión: ¿`const` o `let`?

La pregunta que os tenéis que hacer con cada dato es muy simple: **¿va a cambiar mientras se ejecuta el programa?**

Dato	¿Cambia?	Declaración
`nombre`	No	`const`
`clase`	No	`const`
`nivel`	Sí, al subir de nivel	`let`
`vida`	Sí, al recibir daño	`let`

`let` declara una variable que se puede reasignar [38]. `const` declara una constante: el identificador no se puede reasignar [39].

Mi recomendación, y la que vais a ver en todo el curso: **empezad siempre con `const`**. Si más adelante descubrís que ese valor tiene que cambiar, lo pasáis a `let`. Así, cuando alguien lea vuestro código y vea un `let`, sabrá que ese valor va a cambiar en algún momento.



La solución paso a paso

1. Declarar

```
const nombre = 'Aria';
const clase = 'Maga';

let nivel = 1;
let vida = 100;
```

2. Mostrar, cambiar y volver a mostrar

```
fichaInicial.textContent = `${nombre} · ${clase} · Nivel ${nivel} · Vida ${vida}`;

nivel = 2;
vida = 65;

fichaFinal.textContent = `${nombre} · ${clase} · Nivel ${nivel} · Vida ${vida}`;
```

Fijaos en un detalle: la plantilla de texto es **idéntica** en las dos líneas, y sin embargo el resultado es distinto. ¿Por qué? Porque cada vez que JavaScript llega a esa línea lee el valor que tienen las variables **en ese momento**. Esa es la gracia de una variable.

Y fijaos también en que al reasignar no se vuelve a escribir `let`. `let` se escribe una sola vez, al declarar. Si escribís `let nivel = 2;` otra vez, JavaScript os dirá que esa variable ya está declarada.

3. El error a propósito

```
nombre = 'Otra';
```

En Chrome y Edge, la consola muestra:

```
Uncaught TypeError: Assignment to constant variable.
```

Firefox lo dice con otras palabras (`invalid assignment to const 'nombre'`), pero significa lo mismo: estáis intentando reasignar una constante [40]. Y recordad lo que visteis en el bloque 01: tras el error, el script se detiene. Por eso hay que comentar la línea.



Qué tenéis que observar

- Los dos párrafos muestran el mismo nombre y la misma clase, pero distinto nivel y distinta vida.
- Con la línea `nombre = 'Otra'` activa, la consola muestra un `TypeError`.

Ejercicio 02 · Detective de tipos

> 🎯 **Objetivo:** Conocer los tipos primitivos de JavaScript y usar el operador typeof para averiguar el tipo de cualquier valor.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Preguntar a `typeof` por el tipo de ocho valores distintos y detectar los resultados que no son lo que parecen.



Los resultados

Variable	Valor	`typeof`	Comentario
`texto`	`'Hola'`	`"string"`	
`numero`	`42`	`"number"`	
`decimal`	`3.14`	`"number"`	Enteros y decimales comparten tipo
`numeroEnTexto`	`'42'`	`"string"`	Las comillas mandan
`verdadero`	`true`	`"boolean"`	
`sinValor`	(nada)	`"undefined"`	Declarada sin valor
`vacio`	`null`	`"object"`	⚠ Sorpresa 1
`noEsNumero`	`NaN`	`"number"`	⚠ Sorpresa 2

`typeof` devuelve un texto con el nombre del tipo del valor que le pasáis [42].



Lo que os tenéis que llevar

Enteros y decimales: el mismo tipo

En otros lenguajes hay tipos distintos para enteros y decimales. En JavaScript, `42` y `3.14` son los dos `number` [7]. Uno menos que recordar.

Las comillas convierten cualquier cosa en texto

`42` es un número. `'42'` es un texto que contiene los caracteres «4» y «2». Esto ya os mordió en el bloque 01 con `prompt()`, y os va a seguir mordiendo hasta que lo tengáis tatuado.

`undefined` frente a `null`

Parecen lo mismo, pero no lo son:

- `undefined` es el valor que tiene una variable que **todavía no ha recibido** valor [7].
- `null` es un valor que **vosotros ponéis** a propósito para decir «aquí no hay nada» [7].

Una analogía: `undefined` es un cajón que nadie ha abierto todavía. `null` es un cajón que alguien ha vaciado y ha dejado así a propósito.

Sorpresa 1: `typeof null` es `"object"`

MDN lo recoge tal cual: `typeof null === "object"` [42]. Es una rareza que el lenguaje arrastra desde sus inicios y que no se puede corregir sin romper páginas antiguas. `null` **no** es un objeto. Simplemente, `typeof` no os sirve para detectarlo.

Sorpresa 2: `typeof NaN` es `"number"`

`NaN` significa *Not a Number*, y sin embargo es del tipo `number`. Tiene su lógica: es el valor que devuelve una operación numérica que no ha podido dar un número válido, como `Number('hola')` [43]. Es un número «roto», pero número al fin.

Ejercicio 03 · Calculadora de operadores

> 🎯 **Objetivo:** Usar los seis operadores aritméticos de JavaScript (+, -, *, /, % y **) con dos números introducidos por el usuario.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Aplicar los seis operadores aritméticos a dos números introducidos por el usuario.



La solución paso a paso

1. Convertir al leer

```
const a = Number(prompt('Primer número (a):', '17'));
const b = Number(prompt('Segundo número (b):', '5'));
```

Os acordáis del bloque 01: `prompt()` devuelve texto. Aquí hago la conversión **en la misma línea** en la que leo el dato. Así, a partir de ese momento, `a` y `b` ya son números y me olvido del problema. Es la costumbre que os recomiendo.

2. Los seis operadores

Operador	Nombre	Con 17 y 5
`+`	Suma	`22`
`-`	Resta	`12`
`*`	Multiplicación	`85`
`/`	División	`3.4`
`%`	Resto	`2`
`**`	Potencia	`1419857`

Los cuatro primeros los conocéis de toda la vida [52]. Los otros dos merecen un momento.

El resto (`%`) devuelve lo que sobra al dividir [53]. 17 entre 5 cabe 3 veces (15) y sobran 2. No es un porcentaje, aunque use el mismo símbolo. Os aseguro que lo vais a usar mucho: para saber si un número es par (`n % 2` da `0`), para repartir cosas en grupos o para hacer que algo «dé la vuelta».

La potencia (`**`) eleva el primer número al segundo [44]. `2 ** 3` es 2 × 2 × 2 = 8.

3. Escribir números con `textContent`

```
celdaSuma.textContent = suma;
```

`textContent` espera un texto, pero si le dais un número lo convierte solo. No hace falta hacer nada más.



La pregunta del final: dividir entre cero

Operación	Resultado
`10 / 0`	`Infinity`
`10 % 0`	`NaN`

En matemáticas dividir entre cero no está definido, pero JavaScript no da error: devuelve el valor especial `Infinity`. Y el resto entre cero da `NaN`. Para impedir que el usuario divida entre cero necesitaréis una condición, y eso llega en el bloque 03.

Ejercicio 04 · Marcador de videojuego

> 🎯 **Objetivo:** Modificar una variable con los operadores de asignación compuesta (+=, -=, *=, /=) y con los de incremento y decremento (++ y --), y usar += también para ir acumulando texto.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Ir modificando una puntuación con los operadores de asignación compuesta y de incremento, y construir a la vez un historial de texto.



Los operadores de asignación compuesta

Todos son atajos. Hacen una operación con la variable y guardan el resultado en la misma variable [1]:

Atajo	Equivale a	Jugada	Puntos
`puntos += 50`	`puntos = puntos + 50`	Moneda	50
`puntos -= 20`	`puntos = puntos - 20`	Enemigo	30
`puntos *= 3`	`puntos = puntos * 3`	Cofre	90
`puntos /= 2`	`puntos = puntos / 2`	Trampa	45
`puntos++`	`puntos = puntos + 1`	Vida extra	46
`puntos--`	`puntos = puntos - 1`	Penalización	45

`++` suma uno a la variable y `--` le resta uno [45]. Los vais a ver muchísimo en los bucles del bloque 03.

Fijaos en que `puntos` se declara con `let`. Con `const` ninguna de estas líneas funcionaría: todas reasignan la variable.



`+=` con textos: el acumulador

```
let historial = '';
historial += `Moneda        +50  → ${puntos}\n`;
```

El operador `+=` también funciona con textos. Como `+` une textos, `historial += 'algo'` añade «algo» al final de lo que ya había [54]. Empiezo con un texto vacío y, jugada a jugada, lo voy llenando. A esta técnica se le llama **acumulador** y os acompañará todo el curso.

¿Y el `\n`?

Es el carácter de salto de línea. Dentro de un texto, `\n` significa «aquí empieza una línea nueva». Para que el navegador respete esos saltos, el historial se muestra en un elemento `<pre>`, que conserva los espacios y los saltos de línea tal cual.



Qué tenéis que observar

- El historial muestra seis líneas, una por jugada, con los puntos acumulados.
- La puntuación final es **45**.

Ejercicio 05 · Ticket de compra con plantillas de texto

> 🎯 **Objetivo:** Construir un texto de varias líneas con una plantilla de texto, calcular un subtotal, el IVA y el total, y mostrar los importes siempre con dos decimales usando toFixed().
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Calcular un ticket con IVA y mostrarlo como un texto de varias líneas, con los importes siempre a dos decimales.



La solución paso a paso

1. Los cálculos

```
const subtotal = precio * cantidad;
const importeIva = subtotal * IVA;
const total = subtotal + importeIva;
```

Nada nuevo: tres constantes, cada una con un cálculo. Os recomiendo guardar cada resultado intermedio con su propio nombre en vez de meterlo todo en una línea larga. Se lee mejor y, si algo sale mal, podéis comprobar cada paso con `console.log`.

2. Los decimales «raros»

Con 12.99 € y 3 unidades, el `console.log(total)` muestra algo como `47.1537`. Con otros valores puede salir incluso `47.15370000000001`. El ordenador guarda los decimales en binario y algunos no tienen una representación exacta, así que aparecen esas colas. No es un fallo de vuestro programa.

3. `toFixed(2)`

```
total.toFixed(2)   // "47.15"
```

`toFixed()` devuelve el número escrito con exactamente los decimales que le pidáis [51]. Perfecto para importes en euros.



> ⚠️ **Importante**
> `toFixed()` devuelve un **texto**, no un número [51]. Usadlo solo para **mostrar**. Si después hacéis cuentas con ese resultado, volveréis a tener el problema de `'47.15' + 1 = '47.151'`. Por eso en la solución lo aplico solo dentro de la plantilla, en el último momento.

4. La plantilla de varias líneas

```
const ticket = `========= TIENDA JS =========
Producto:   ${producto}
…`;
```

Las comillas invertidas tienen otra ventaja sobre las normales: admiten **saltos de línea reales** dentro del texto [35]. No hace falta `\n`. Lo que escribís es lo que sale.

Para que el navegador respete esos saltos y los espacios de alineación, el ticket se muestra en un `<pre>`.



Qué tenéis que observar

- Con 12.99 € × 3: subtotal 38.97 €, IVA 8.18 €, total 47.15 €.
- Todos los importes del ticket tienen dos decimales; el de la consola, no.

Ejercicio 06 · Laboratorio de conversiones

> 🎯 **Objetivo:** Comparar Number(), parseInt() y parseFloat() con el mismo texto de entrada, y usar String() para convertir en sentido contrario: de número a texto.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Pasar el mismo texto por `Number()`, `parseInt()` y `parseFloat()`, comparar los resultados y ver la conversión contraria con `String()`.



Los resultados con cada texto

Texto	`Number()`	`parseInt(…, 10)`	`parseFloat()`
`42`	`42`	`42`	`42`
`3.75`	`3.75`	`3`	`3.75`
`42.5px`	`NaN`	`42`	`42.5`
`px42`	`NaN`	`NaN`	`NaN`
(vacío)	`0`	`NaN`	`NaN`



Cómo piensa cada función

`Number()` es estricta: o el texto entero es un número, o devuelve `NaN` [31]. Con una excepción curiosa: el texto vacío lo convierte en `0` [31]. Tenedlo en cuenta, porque un campo que el usuario

deja en blanco puede colarse como un cero.

`parseInt()` lee el texto de izquierda a derecha y se queda con los dígitos que encuentra **hasta que aparece algo que no es un dígito**. Si el primer carácter ya no se puede convertir, devuelve `NaN` [46]. Por eso `'42.5px'` da `42` (el punto la detiene) y `'px42'` da `NaN` (empieza por una letra). Y siempre devuelve un entero.

El segundo argumento, `10`, es la base de numeración: le estoy diciendo que lea en decimal. MDN recomienda indicarlo siempre [46].

`parseFloat()` funciona igual que `parseInt()`, pero acepta el punto decimal [47].



La conversión contraria: `String()`

```
String(42) + 1   // "421"
```

`String()` convierte un valor en texto. Y como ya sabéis, texto `+` lo que sea, se une. Lo he puesto aquí para que veáis el problema del bloque 01 desde el otro lado.



La pregunta del final

Para leer `'42.5px'`, la herramienta es `parseFloat()`: es la única que conserva el `42.5`. Un caso real: leer el tamaño de un elemento que CSS os da en píxeles.

¿Y para los datos que escribe el usuario en un formulario? Mi recomendación es `Number()`. Precisamente porque es estricta: si alguien escribe `12abc`, prefiero un `NaN` que me avise del problema a un `12` que se lo trague sin decir nada.

Ejercicio 07 · == frente a ===

> 🎯 **Objetivo:** Comparar la igualdad débil (==) con la igualdad estricta (===) y entender por qué en el curso usamos siempre ===.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Comparar seis parejas de valores con `==` y con `===` y entender de dónde salen las diferencias.



Los resultados

Pareja	`==`	`===`
`5` y `'5'`	`true`	`false`
`0` y `''`	`true`	`false`
`0` y `false`	`true`	`false`
`null` y `undefined`	`true`	`false`
`'1'` y `true`	`true`	`false`
`NaN` y `NaN`	`false`	`false`

¿Cuántos acertasteis con `==`? Si fallasteis alguno, no os preocupéis. Ese es precisamente el problema de `==`.



Qué hace cada operador

`==` **(igualdad débil):** si los dos valores son de tipos distintos, intenta convertirlos a un mismo tipo y **después** compara [48]. Al comparar un número con un texto, convierte el texto en número [48]. Por eso:

- `5 == '5'` → convierte `'5'` en `5` → `true`.
- `0 == ''` → convierte `''` en `0` (lo visteis en el ejercicio 06) → `true`.

Y tiene una regla especial: `null == undefined` da `true` [48].

`===` **(igualdad estricta):** si los tipos son distintos, el resultado es `false`, sin convertir nada [49]. Solo da `true` cuando el tipo y el valor coinciden.

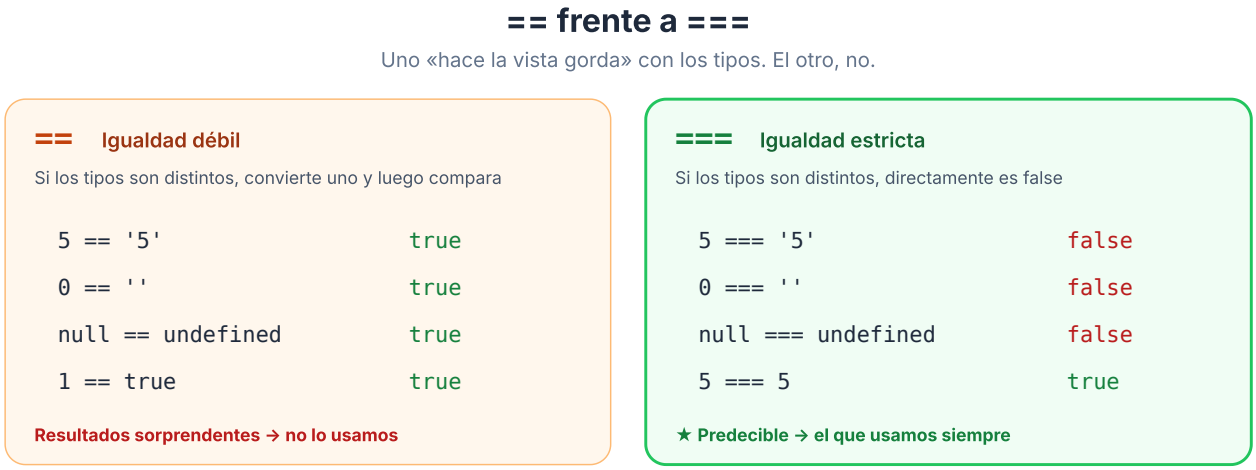


Figura 14 · == frente a ===



¿Por qué usamos siempre `===`?

Porque es **predecible**. Con `===` sabéis el resultado sin tener que recordar las reglas de conversión de `==`, que son muchas y algunas bastante retorcidas. Si necesitáis comparar un número con un texto, convertid primero vosotros, de forma explícita, y comparad después con `===`:

```
Number(textoUsuario) === 5
```



El caso de `NaN`

`NaN` es el único valor de JavaScript que **no es igual a sí mismo**, ni con `==` ni con `===` [43]. Así que nunca podéis comprobar `valor === NaN`: siempre dará `false`. Para eso existe `Number.isNaN(valor)` [43], que usaremos en el bloque 03 para validar lo que escribe el usuario.

Ejercicio 08 · Dividir la cuenta del restaurante

> 🎯 **Objetivo:** Usar el objeto Math (round, ceil, floor, max, min y random) junto con toFixed() para resolver un problema real con dinero y un sorteo.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Calcular cuánto paga cada persona en una cena de grupo, con propina y redondeos, y sortear quién invita a los cafés.



Los cálculos con 87.40 €, 4 personas y 10 % de propina

Concepto	Cálculo	Resultado
Propina	87.40 × 10 / 100	8.74 €
Total con propina	87.40 + 8.74	96.14 €
Cada uno (exacto)	96.14 / 4	24.035 €
Cada uno (céntimo ↑)	`Math.ceil(2403.5) / 100`	24.04 €
Cada uno (euro ↑)	`Math.ceil(24.035)`	25.00 €
Bote	25 × 4 − 96.14	3.86 €



Las herramientas de `Math`

`Math` es un objeto que viene con JavaScript y agrupa constantes y funciones matemáticas [55]. Estas son las que usamos:

Método	Qué hace	Ejemplo
`Math.round(x)`	Redondea al entero más cercano	`Math.round(2.5)` → `3`
`Math.floor(x)`	Redondea hacia abajo	`Math.floor(2.9)` → `2`
`Math.ceil(x)`	Redondea hacia arriba	`Math.ceil(2.1)` → `3`
`Math.max(a, b, …)`	El mayor	`Math.max(4, 9, 2)` → `9`
`Math.min(a, b, …)`	El menor	`Math.min(4, 9, 2)` → `2`
`Math.random()`	Número aleatorio desde 0 (incluido) hasta 1 (sin incluir)	`0.7341…`



Redondear al céntimo

`Math.ceil()` redondea a **enteros**. ¿Cómo lo uso para redondear a dos decimales? Con un truco:

```
Math.ceil(porPersona * 100) / 100
```

1. Multiplico por 100: `24.035` → `2403.5` (paso de euros a céntimos).
2. Redondeo hacia arriba: `2403.5` → `2404`.
3. Divido entre 100: `2404` → `24.04` (vuelvo a euros).

¿Por qué hacia arriba y no al más cercano? Porque si redondeo hacia abajo, al camarero le faltarían unos céntimos. Hacia arriba, como mucho sobra algo. Ojo a la diferencia con `toFixed(2)`: aquel solo **muestra** dos decimales; este **calcula** un número nuevo.



El sorteo

```
Math.floor(Math.random() * personas) + 1
```

`Math.random()` devuelve un decimal entre 0 y 1, sin llegar nunca a 1 [50]. Para convertirlo en «una persona entre 1 y 4»:

Paso	Operación	Rango posible
1	`Math.random()`	de 0 a 0.999…
2	`× personas` (4)	de 0 a 3.999…
3	`Math.floor(…)`	0, 1, 2 o 3
4	`+ 1`	1, 2, 3 o 4

Guardaos esta fórmula: la vais a usar cada vez que necesitéis un dado, una carta al azar o un color aleatorio.



Bloque 03

Bloque 03 · Control de flujo

Hasta ahora vuestros programas eran una lista de instrucciones que se ejecutaban de arriba abajo, todas, siempre en el mismo orden. Eso se acaba hoy. En este bloque os voy a enseñar a hacer que el programa **tome decisiones** («si pasa esto, haz aquello») y **repita tareas** («haz esto diez veces» o «sigue preguntando hasta que acierte»). Con estas dos herramientas se resuelve una cantidad enorme de problemas.

Al terminar este bloque sabréis:

- Tomar decisiones con `if`, `else if` y `else`, y elegir el orden correcto de las condiciones.
- Combinar condiciones con `&&`, `||` y `!`.
- Usar valores *truthy* y *falsy* y el operador ternario.
- Elegir entre muchos valores con `switch`.
- Repetir tareas con `for`, `while` y `do...while`, y saber cuál usar en cada caso.
- Construir contadores y acumuladores dentro de un bucle.
- Controlar un bucle con `break` y `continue` sin caer en un bucle infinito.



1. Tomar decisiones: `if`

1.1. `if` y `else`

```
if (edad >= 18) {
    mensaje = 'Puedes votar';
} else {
    mensaje = 'Todavía no puedes votar';
}
```

La instrucción `if` ejecuta un bloque si la condición es verdadera. Si no lo es, se ejecuta el bloque del `else`, que es opcional [56]. La condición va entre paréntesis y cada bloque, entre llaves.

Dos costumbres que os pido desde el primer día:

- **Poned siempre las llaves**, aunque el bloque tenga una sola línea. JavaScript permite omitirlas, pero es una fuente clásica de errores cuando después alguien añade una segunda línea.

- **Indentad** el contenido de cada bloque. A JavaScript le da igual; a quien lee el código, no.

1.2. Varias opciones: `else if`

Cuando hay más de dos caminos, se encadenan condiciones con `else if` [1]:



Figura 15 · La escalera if, else if, else

```
if (nota < 5) {
    calificacion = 'Suspenso';
} else if (nota < 7) {
    calificacion = 'Aprobado';
} else if (nota < 9) {
    calificacion = 'Notable';
} else {
    calificacion = 'Sobresaliente';
}
```

Fijaos bien en el dibujo: se comprueban las condiciones de arriba abajo y **solo se ejecuta el primer bloque cuya condición se cumple**. Las demás ni se miran. Eso tiene dos consecuencias:

1. **El orden importa.** Si ponéis `nota < 9` la primera, un 3 entraría ahí.
2. **No hace falta repetir lo que ya sabéis.** En el escalón de `Notable` no escribo `nota >= 7 && nota < 9`: si he llegado hasta ahí, la nota ya no es menor que 7.

1.3. Primero, descartar lo que no vale

Ahora que podéis decidir, ya podéis **validar** lo que escribe el usuario. Mi recomendación es hacerlo siempre al principio:

```
if (Number.isNaN(nota) || nota < 0 || nota > 10) {
    mensaje = 'La nota debe estar entre 0 y 10';
} else if (…) {
    // a partir de aquí, la nota es válida
}
```

`Number.isNaN()` es la forma correcta de detectar un `NaN` [57]. Recordad que `nota === NaN` siempre da `false`.



2. Operadores lógicos

Sirven para combinar varias condiciones en una sola.



📊 Figura 16 · Operadores lógicos

Operador	Nombre	Devuelve `true` si...
`&&`	Y (AND)	los dos lados son verdaderos [58]
`||`	O (OR)	al menos uno de los dos lo es [59]
`!`	NO (NOT)	el valor es falso: le da la vuelta [60]

```
const puedeEntrar = tieneEntrada && (edad >= 18 || (edad >= 16 && vaAcompanado));
```



💡 Consejo

`&&` tiene más prioridad que `||`, igual que la multiplicación va antes que la suma. En lugar de memorizarlo, **poned paréntesis** siempre que mezcléis los dos. El código se lee mejor y no hay sorpresas.

Un detalle más: los dos operadores dejan de evaluar en cuanto conocen el resultado. Si el lado izquierdo de un `&&` es falso, el derecho ni se mira, porque el resultado ya será `false`. Es lo que MDN llama evaluación de cortocircuito [58].

2.1. Una entrada nueva: `confirm()`

```
const tieneEntrada = confirm('¿Tienes entrada?');
```

Muestra un cuadro con Aceptar y Cancelar y devuelve directamente un booleano: `true` o `false` [61]. Viene de perlas para practicar con condiciones.



3. Valores *truthy* y *falsy*

La condición de un `if` no tiene por qué ser una comparación. Podéis poner cualquier valor, y JavaScript decide si cuenta como verdadero o como falso.

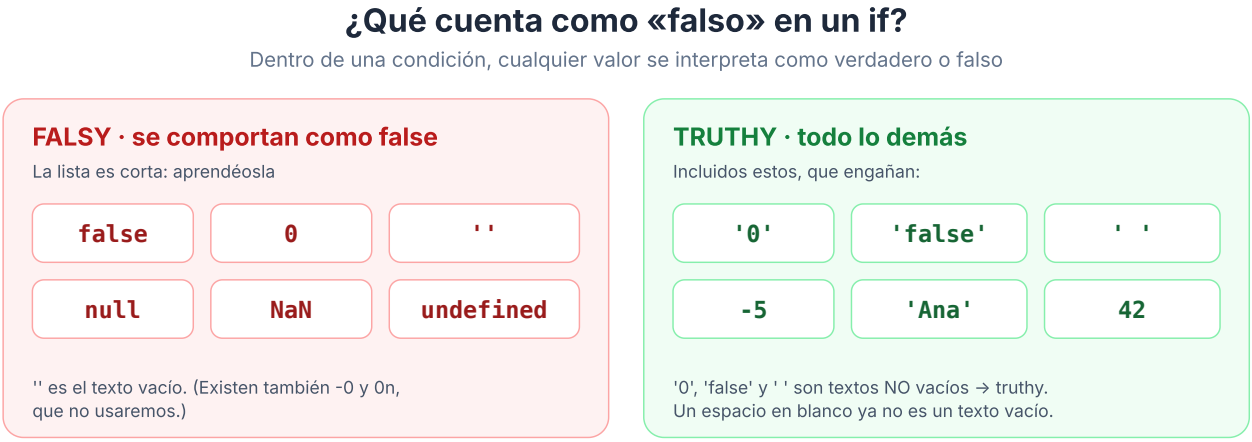


Figura 17 · Truthy y falsy

- **Falsy:** `false`, `0`, `''` (texto vacío), `null`, `undefined` y `NaN`, entre otros [62].
- **Truthy:** todo lo demás [63]. Incluidos `'0'`, `'false'` y `' '`, que son textos **no vacíos**.

Donde más lo vais a usar es para comprobar si el usuario ha escrito algo:

```
let nombre = prompt('¿Cómo te llamas?');
if (nombre === null) {
    nombre = '';          // ha pulsado Cancelar
}
nombre = nombre.trim();   // quita espacios al principio y al final [9]
if (!nombre) {
    nombre = 'invitado';  // '' es falsy → !'' es true
}
```



4. El operador ternario

Es un `if…else` en miniatura que **devuelve un valor** [10]:

```
condicion ? valorSiTrue : valorSiFalse
```

```
const etiqueta = stock > 0 ? 'Disponible' : 'Agotado';
```

Usadlo para **elegir entre dos valores**. Si en cada caso hay que ejecutar varias instrucciones, volved al `if…else`. Un ternario dentro de otro ternario es la forma más rápida de que nadie entienda vuestro código, vosotros incluidos dentro de un mes.



5. `switch`: muchos valores concretos

Cuando comparáis **una misma variable** con muchos **valores concretos**, `switch` queda más limpio que una escalera de `else if`:

```
switch (numeroDia) {
    case 1:
        nombre = 'lunes';
        break;
    case 2:
        nombre = 'martes';
        break;
    // …
    default:
        nombre = 'día no válido';
}
```

Lo que tenéis que saber [11]:

- Compara con **igualdad estricta** (`===`). El texto `'1'` no encaja con `case 1`.

- `default` se ejecuta si ningún `case` coincide. Es el `else` del `switch`.
- `break` hace salir del `switch`. **Si lo olvidáis, la ejecución sigue en el `case` siguiente.**

Ese último punto, que suele ser un error, también se puede aprovechar para agrupar casos:

```
switch (numeroDia) {
    case 6:
    case 7:
        tipo = 'fin de semana';
        break;
    default:
        tipo = 'laborable';
}
```

Situación	Mejor opción
Una variable comparada con valores concretos	`switch`
Rangos (`< 5`), varias variables o condiciones combinadas	`if…else`



6. Repetir: los bucles

Un bucle repite un bloque de código mientras se cumpla una condición [9]. JavaScript tiene varios; en este bloque vemos los tres básicos. En los bloques 05 y 06 aparecerán `for...of` y `forEach`, pensados para recorrer listas.

6.1. `for`: cuando sabéis cuántas vueltas



Figura 18 · Anatomía de un bucle for

```
for (let i = 1; i <= 10; i++) {
    console.log(i);
}
```

La cabecera tiene tres partes separadas por punto y coma: la inicialización, que se ejecuta una vez; la condición, que se comprueba antes de cada vuelta; y la actualización, que se ejecuta al final de cada vuelta [64].

El contador se suele llamar `i` por tradición, pero podéis llamarlo como queráis. Y no solo sirve para contar: dentro del bucle lo podéis usar como dato.

6.2. `while`: mientras se cumpla algo

```
let intentos = 0;
let acertado = false;
while (!acertado) {
    // pedir un número, comprobar, actualizar acertado…
}
```

`while` comprueba la condición **antes** de cada vuelta y repite mientras sea verdadera [65]. Usadlo cuando **no sabéis cuántas vueltas** hará falta dar: hasta que el usuario acierte, hasta que escriba algo válido...

6.3. `do...while`: al menos una vez

```
let numero;
do {
    numero = Number(prompt('Escribe un número (0 para terminar)'));
    // procesar…
} while (numero !== 0);
```

Igual que `while`, pero comprueba la condición **después** de cada vuelta. Por eso el cuerpo se ejecuta siempre al menos una vez [66]. Es la opción natural cuando primero tenéis que pedir el dato para poder decidir si seguís.

6.4. ¿Cuál uso?

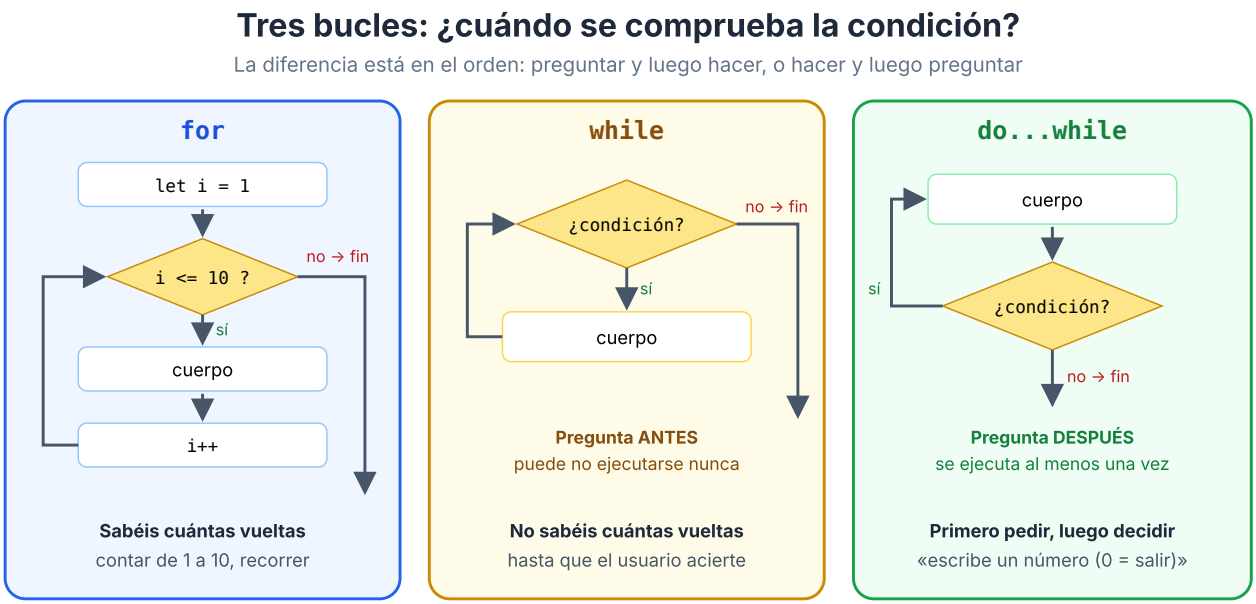
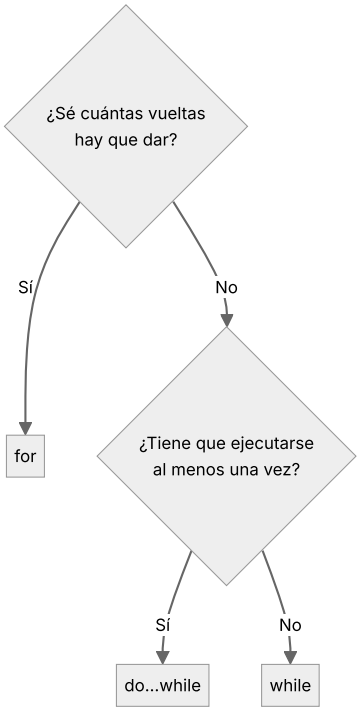


Figura 19 · for, while y do...while



Bucle	Comprueba la condición	Se ejecuta como mínimo	Uso típico
`for`	Antes de cada vuelta	0 veces	Repetir N veces, contar de A a B
`while`	Antes de cada vuelta	0 veces	Repetir hasta que pase algo
`do...while`	Después de cada vuelta	1 vez	Pedir datos hasta un valor de parada



7. Contadores y acumuladores

Casi todos los bucles útiles llevan alguno de estos dos patrones. Los dos se declaran **antes** del bucle, se actualizan **dentro** y se usan **después**.

Patrón	Valor inicial	Dentro del bucle	Para qué
Contador	`let cantidad = 0;`	`cantidad++;`	Contar cuántas veces pasa algo
Acumulador numérico	`let suma = 0;`	`suma += valor;`	Sumar valores
Acumulador de texto	`let texto = '';`	`texto += linea;`	Construir un texto por partes
Máximo	`let mayor = -Infinity;`	`mayor = Math.max(mayor, valor);`	Quedarse con el mayor



🚨 Atención

Si declaráis el acumulador **dentro** del bucle, se reinicia en cada vuelta. Al terminar, solo tendréis el último valor.



8. `break` y `continue`

Instrucción	Qué hace	Analogía
`break`	Termina el bucle y sigue en la línea de después [67]	Abandonar la carrera
`continue`	Abandona la vuelta actual y pasa a la siguiente [68]	Saltarse una vuelta y seguir corriendo

```
while (!acertado) {
    const respuesta = prompt('¿Qué número es?');
    if (respuesta === null) {
        break;        // se rinde: salgo del bucle
    }
    if (Number.isNaN(Number(respuesta))) {
        continue;     // no vale: vuelvo a preguntar sin contar el intento
    }
    // …
}
```

`break` también lo habéis visto en el `switch`: allí sirve para salir del `switch`, no de un bucle.



9. Cuidado con los bucles infinitos

Un bucle cuya condición nunca llega a ser falsa no termina jamás. MDN lo dice sin rodeos: aseguraos de que la condición acabe siendo `false` [9].

```
let i = 1;
while (i <= 10) {
    console.log(i);
    // ✘ falta i++: i vale 1 para siempre
}
```

Si os pasa, la pestaña del navegador se quedará congelada. No es grave: cerrad la pestaña, corregid el código y volved a abrir la página. Antes de ejecutar un bucle, haceos siempre esta pregunta: **¿qué hay dentro del bucle que acabará haciendo falsa la condición?**



10. La plantilla de trabajo en este bloque

Seguimos con programas lineales. La novedad es que ahora la sección de procesamiento incluye decisiones y bucles, y que **validamos los datos antes de usarlos**:

```
// estructura del programa principal
let mensaje = '';

// constantes al DOM
const salida = document.querySelector('#salida');

// entradas
const limite = Number(prompt('¿Hasta qué número cuento?', '10'));

// procesamiento
if (Number.isNaN(limite) || limite < 1) {
    mensaje = 'El límite debe ser un número positivo';
} else {
    let suma = 0;
    for (let i = 1; i <= limite; i++) {
        suma += i;
    }
    mensaje = `La suma del 1 al ${limite} es ${suma}`;
}

// salidas
salida.textContent = mensaje;
```



11. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Calificador de notas	`if`, `else if`, `else`, validación, orden de las condiciones	Ver
02	Control de acceso al concierto	`&&`, `||`, `!`, paréntesis, `confirm()`	Ver
03	Un saludo a prueba de Cancelar	*Truthy* y *falsy*, `trim()`, operador ternario	Ver
04	¿Qué día es?	`switch`, `break`, `default`, agrupar `case`	Ver
05	Tabla de multiplicar con for	`for`, contador como dato, acumuladores	Ver
06	Adivina el número	`while`, `break`, `continue`, búsqueda binaria	Ver
07	Estadísticas hasta el cero	`do...while`, valor de parada, media y máximo	Ver
08	Fizz Buzz	`for` + `%` + escalera de `if` + contadores	Ver

Con bucles, más que nunca: consola abierta y la pregunta de antes en la cabeza. ¿Qué hará que este bucle termine?



🧪 Los ejercicios, explicados

Ejercicio 01 · Calificador de notas

> 🎯 **Objetivo:** Tomar decisiones con if, else if y else, encadenando varias condiciones en el orden correcto, y rechazar antes los datos que no son válidos.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Convertir una nota numérica en su calificación, descartando antes las notas que no tienen sentido.



La solución paso a paso

1. Primero, lo que no vale

```
if (Number.isNaN(nota) || nota < 0 || nota > 10) {
    mensaje = 'La nota debe ser un número entre 0 y 10.';
}
```

Esta es una costumbre que os quiero inculcar desde ya: **antes de calcular nada, descartad los datos imposibles**. Si alguien escribe «hola», `Number()` devuelve `NaN`, y `Number.isNaN()` lo detecta [57]. Recordad del bloque 02 que `nota === NaN` no sirve: `NaN` no es igual a nada.

El operador `||` hace que basta con que **una** de las tres condiciones se cumpla para entrar [59].

2. La escalera

```
} else if (nota < 5) {
    mensaje = `Un ${nota} es un Suspenso.`;
} else if (nota < 6) {
    mensaje = `Un ${nota} es un Aprobado.`;
} …
} else {
    mensaje = `Un ${nota} es un Sobresaliente.`;
}
```

`if…else` ejecuta un bloque si la condición es verdadera y otro si no lo es; encadenando `else if` se comprueban varias condiciones seguidas [56]. Y aquí está la clave: **solo se ejecuta el primer bloque cuya condición se cumple**. Los demás ni se miran.

Por eso, en el escalón de `Notable` me basta con `nota < 9`. ¿Hace falta comprobar también que `nota >= 7`? No. Si la nota fuese menor que 7, habría entrado en un escalón anterior y no habría llegado hasta aquí. Tenéis el recorrido dibujado en la infografía de la teoría.

3. El `else` final

El último caso no necesita condición. Si la nota es válida y no es menor que 9, solo puede ser un sobresaliente. El `else` recoge «todo lo demás».



Probad estos casos

Entrada	Resultado	Por qué
7.5	Notable	Falla `< 5`, `< 6` y `< 7`; cumple `< 9`
5	Aprobado	`5 < 5` es `false`; `5 < 6` es `true`
9	Sobresaliente	Falla todos los escalones; va al `else`
10	Sobresaliente	Igual que el 9
12, -1 o «hola»	Mensaje de error	Lo para la primera condición



Error típico

Escribir la escalera al revés, empezando por `if (nota < 9)`. Un 3 cumpliría esa condición y saldría «Notable». En una escalera de `else if`, **el orden importa**.

Ejercicio 02 · Control de acceso al concierto

> 🎯 **Objetivo:** Combinar condiciones con los operadores lógicos && (y), || (o) y ! (no), y usar confirm() para obtener directamente un valor booleano del usuario.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Traducir a JavaScript las normas de la puerta de un concierto combinando condiciones con `&&`, `||` y `!`.



Una entrada nueva: `confirm()`

```
const tieneEntrada = confirm('¿Tienes entrada?');
```

`confirm()` muestra un cuadro con los botones Aceptar y Cancelar, y devuelve `true` si se pulsa Aceptar y `false` si se pulsa Cancelar [61]. A diferencia de `prompt()`, aquí no hay que convertir nada: ya recibís un booleano.



La condición, pieza a pieza

Las normas dicen: «hace falta entrada. Además, hay que tener 18 años, o al menos 16 si vienes con una persona adulta». Os lo traduzco por partes:

Norma	Código
tiene 16 o más **y** va acompañado	`edad >= 16 && vaAcompanado`
es mayor **o** lo anterior	`esMayor || (edad >= 16 && vaAcompanado)`
tiene entrada **y** lo anterior	`tieneEntrada && (esMayor || (…))`

```
const puedeEntrar = tieneEntrada && (esMayor || (edad >= 16 && vaAcompanado));
```

- `&&` da `true` solo si los dos lados son verdaderos [58].
- `||` da `true` si al menos uno de los dos lo es [59].

Los paréntesis no son decoración. Sin ellos, `&&` se evalúa antes que `||`, igual que en matemáticas la multiplicación va antes que la suma, y la condición diría otra cosa. Mi consejo: cuando mezcléis `&&` y `||`, poned siempre paréntesis, aunque creáis que no hacen falta. Quien lea vuestro código os lo agradecerá.



El `!`

```
if (!tieneEntrada) {
```

`!` invierte un valor: convierte `true` en `false` y al revés [60]. La línea se lee tal cual: «si no tiene entrada». Por eso compruebo ese caso el primero. Si no hay entrada, me da igual la edad.



Probad estos casos

Edad	Entrada	Acompañado	Resultado
17	Sí	Sí	Adelante
17	Sí	No	No cumple la edad
15	Sí	Sí	No cumple la edad
25	No	—	Pasa por taquilla
25	Sí	No	Adelante

Ejercicio 03 · Un saludo a prueba de Cancelar

> 🎯 **Objetivo:** Usar los valores truthy y falsy en una condición para detectar respuestas vacías o canceladas, limpiar espacios con trim() y elegir entre dos mensajes con el operador ternario.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Arreglar el saludo del bloque 01 para que no muestre `null` ni un nombre vacío, y elegir el mensaje con el operador ternario.



La solución paso a paso

1. Cancelar

```
if (nombre === null) {
    nombre = '';
}
```

Al pulsar Cancelar, `prompt()` devuelve `null` [30]. Lo convierto en un texto vacío para tratar los dos casos, «canceló» y «no escribió nada», de la misma forma. Fijaos en que `nombre` se declara con `let`: lo voy a ir corrigiendo.

2. Los espacios

```
nombre = nombre.trim();
```

`trim()` devuelve el texto sin los espacios en blanco del principio y del final [69]. Sin este paso, alguien que escribe tres espacios conseguiría un saludo con un nombre invisible, porque `'  '` **no** es un texto vacío.

Y ojo: esto solo funciona porque antes he quitado el `null`. Si hacéis `null.trim()`, salta un `TypeError`, igual que el `null.textContent` del bloque 01. El orden de los pasos importa.

3. Truthy y falsy

```
if (!nombre) {
    nombre = 'invitado';
}
```

Aquí está lo nuevo. Una condición no tiene por qué ser una comparación. Podéis poner cualquier valor, y JavaScript lo interpreta como verdadero o falso. Los valores que cuentan como falso se llaman *falsy*: `false`, `0`, `''`, `null`, `undefined` y `NaN`, entre otros [62]. Todos los demás son *truthy* [63].

Como `''` es falsy, `!''` es `true`, y la línea se lee como suena: «si no hay nombre».

4. El operador ternario

```
const saludo = esPrimeraVisita
    ? `Te damos la bienvenida, ${nombre}.`
    : `¡Qué alegría verte de nuevo, ${nombre}!`;
```

El ternario es un atajo de `if…else` que **devuelve un valor** [5]: si la condición es verdadera, el primero; si no, el segundo. Es perfecto para elegir entre dos valores. Si tenéis que ejecutar varias instrucciones en cada caso, volved al `if…else` de siempre: el ternario se vuelve ilegible en cuanto le pedís demasiado.



Los cuatro casos

Lo que hace el usuario	`prompt()` devuelve	Tras limpiar	Saludo
Escribe «Lucía»	`'Lucía'`	`'Lucía'`	…, Lucía
Pulsa Cancelar	`null`	`''` → invitado	…, invitado
Deja la caja vacía	`''`	`''` → invitado	…, invitado
Escribe solo espacios	`' '`	`''` → invitado	…, invitado

Ejercicio 04 · ¿Qué día es?

> 🎯 **Objetivo:** Elegir entre muchos valores concretos con switch, usar break para salir de cada caso, default para lo que no encaja y agrupar varios case que comparten resultado.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Traducir un número del 1 al 7 en un día de la semana con `switch` y clasificarlo como laborable o fin de semana.



Cuándo usar `switch`

Cuando comparáis **una misma variable** con muchos **valores concretos**, una escalera de `else if` se hace larga y repetitiva:

```
if (numero === 1) { … } else if (numero === 2) { … } else if (numero === 3) { … }
```

`switch` lo expresa de forma más limpia: evalúa una expresión una vez y salta al `case` cuyo valor coincide [70].



La solución paso a paso

1. Un `case` por valor

```
switch (numero) {
    case 1:
        nombreDia = 'lunes';
        break;
    …
    default:
        nombreDia = '';
}
```

- La comparación se hace con **igualdad estricta** (`===`) [70]. Por eso convierto con `Number()` al leer: el texto `'6'` no encajaría con `case 6` .
- `default` se ejecuta cuando ningún `case` coincide [70]. Es el `else` del `switch` .
- `break` hace que el programa salga del `switch` [67].

2. Lo que pasa sin `break`

Este es el error más típico con `switch` . Si se omite el `break` , la ejecución **continúa en la instrucción siguiente** del `switch` , aunque pertenezca a otro `case` [70]. Si quitáis el `break` del `case 6` , con un 6 se asigna `'sábado'` … y acto seguido el `case 7` lo machaca con `'domingo'` .

3. Aprovechar la caída: agrupar casos

Lo que en el punto anterior era un error, aquí es una herramienta:

```
switch (numero) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        tipoDia = 'Es laborable';
        break;
    case 6:
    case 7:
        tipoDia = 'Es fin de semana';
        break;
    …
}
```

Los `case` 1 a 4 no tienen instrucciones ni `break` , así que caen hasta el `case 5` y comparten su bloque. Es la forma de decir «del 1 al 5, lo mismo».



¿`switch` o `if`?

Situación	Mejor opción
Una variable comparada con valores concretos (1, 2, 3… o 'rojo', 'verde'…)	`switch`
Rangos (`nota < 5`), varias variables o condiciones con `&&` y `||`	`if…else`

Ejercicio 05 · Tabla de multiplicar con for

> 🎯 **Objetivo:** Repetir una instrucción un número conocido de veces con un bucle for, usando el contador dentro del cuerpo y dos acumuladores: uno de texto y otro numérico.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Generar la tabla de multiplicar de un número con un bucle `for` y calcular la suma de todos sus resultados.



La solución paso a paso

1. La cabecera del `for`

```
for (let i = 1; i <= 10; i++) {
```

Un `for` tiene tres partes separadas por punto y coma [1]:

Parte	Código	Cuándo se ejecuta
Inicialización	`let i = 1`	Una vez, al empezar
Condición	`i <= 10`	Antes de cada vuelta. Si es `false` , el bucle termina
Actualización	`i++`	Al final de cada vuelta

Lo tenéis dibujado, vuelta a vuelta, en la infografía de la teoría. Usamos `for` cuando **sabemos cuántas vueltas** hay que dar: aquí, diez [9].

2. El cuerpo: usar el contador

```
const resultado = numero * i;
tabla += `${numero} × ${i} = ${resultado}\n`;
suma += resultado;
```

Dentro del bucle, `i` vale 1 en la primera vuelta, 2 en la segunda… hasta 10. Ese contador no solo sirve para contar vueltas: lo uso como dato.

Fijaos también en que `resultado` se declara con `const` **dentro** del bucle. En cada vuelta se crea una constante nueva, así que no hay problema por reasignarla.

3. Dos acumuladores

```
let tabla = '';
let suma = 0;
```

Los dos se declaran **antes** del bucle y se muestran **después**. Es el patrón del acumulador que visteis en el bloque 02, ahora dentro de un bucle:

- El de texto empieza en `''` y en cada vuelta se le añade una línea.
- El numérico empieza en `0` y en cada vuelta se le suma el resultado.



🚨 Atención

Si declaráis el acumulador **dentro** del bucle, se reinicia en cada vuelta y al final solo tendréis la última línea. Error clásico.

4. Comprobar antes de repetir

Todo el bucle va dentro del `else`. Si el número no es válido, no tiene sentido generar diez líneas de `NaN`.



Qué tenéis que observar

- Con el 7 aparecen diez líneas, de `7 × 1 = 7` a `7 × 10 = 70`.
- La suma es 385 (7 × 55).

Ejercicio 06 · Adivina el número

🎯 **Objetivo:** Repetir mientras se cumpla una condición con un bucle while, sin saber de antemano cuántas vueltas dará, y salir antes de tiempo con break o saltar una vuelta con continue.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Programar el juego de adivinar un número con pistas de «mayor» y «menor», usando un bucle `while` y las instrucciones `break` y `continue`.



¿Por qué `while` y no `for`?

En la tabla de multiplicar sabíamos que eran diez vueltas. Aquí **no tenemos ni idea**: puede acertar a la primera o tardar quince intentos. `while` repite su cuerpo mientras la condición sea verdadera, y la comprueba antes de cada vuelta [65].

```
while (!acertado) {
```

Se lee «mientras no haya acertado». La variable `acertado` empieza en `false`, y solo cuando el usuario adivina el número la pongo a `true`. En la siguiente comprobación, la condición deja de cumplirse y el bucle termina.



🚨 Atención

Algo dentro del bucle **tiene** que poder hacer falsa la condición. Si no, tenéis un bucle infinito: la página se queda colgada. MDN lo resume así: asegúraos de que la condición acabe siendo `false` [9]. Aquí hay dos salidas: acertar o cancelar.



`break`: salir ya

```
if (respuesta === null) {
    break;
}
```

`break` termina el bucle en el acto y el programa sigue en la primera línea **después** del bucle [67]. Es la salida de emergencia: el usuario se rinde y no tiene sentido seguir preguntando.



`continue`: saltar esta vuelta

```
if (Number.isNaN(numero) || numero < 1 || numero > 100) {
    pista = 'Eso no es un número del 1 al 100.';
    continue;
}
```

`continue` no sale del bucle: abandona **la vuelta actual** y, en un `while`, vuelve a comprobar la condición [68]. Lo uso para no contar como intento una respuesta que no vale. Las líneas que hay debajo, `intentos++` incluida, se saltan.

Instrucción	¿Qué hace?	Analogía
`break`	Sale del bucle	Abandonar la carrera
`continue`	Salta a la siguiente vuelta	Saltarse una vuelta del circuito y seguir corriendo



La estrategia ganadora

Si siempre probáis el número central del rango que os queda (50, luego 25 o 75…), cada intento descarta la mitad de los números. Con 100 posibilidades no necesitáis nunca más de 7 intentos. Esta idea se llama **búsqueda binaria** y la vais a volver a encontrar.

Ejercicio 07 · Estadísticas hasta el cero

> 🎯 **Objetivo:** Usar un bucle do...while, que se ejecuta siempre al menos una vez, para leer datos hasta que llegue un valor de parada, calculando por el camino contador, suma, media y máximo.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Leer números hasta que el usuario escriba un 0 y calcular cuántos ha introducido, su suma, su media y el mayor.

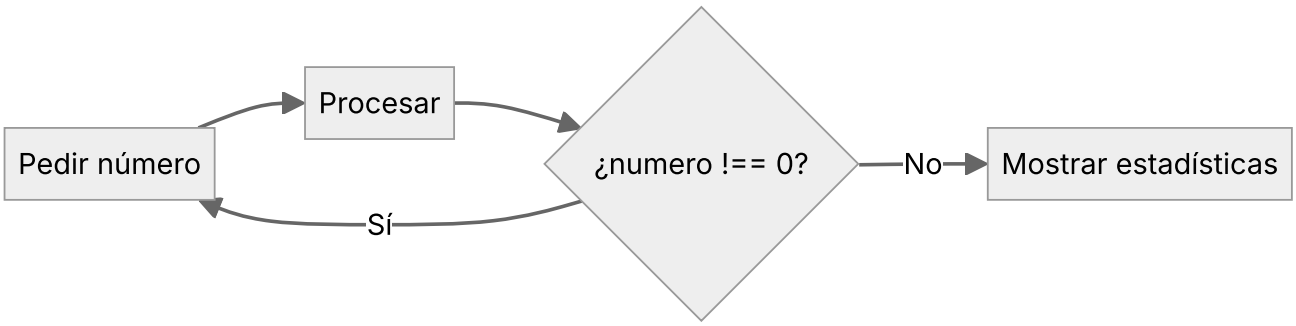


¿Por qué `do...while`?

`do...while` ejecuta el cuerpo y **después** comprueba la condición. Por eso el cuerpo se ejecuta siempre al menos una vez [66].

```
do {
    numero = Number(prompt('Escribe un número (0 para terminar):', '0'));
    …
} while (numero !== 0);
```

Es exactamente lo que necesito. No puedo saber si el número es 0 hasta que lo haya pedido, así que tengo que preguntar primero y comprobar después. Con un `while` normal tendría que inventarme un valor inicial para `numero` solo para entrar en el bucle.



Un detalle: `numero` se declara **fuera** del bucle porque la condición del `while` está fuera de las llaves y necesita verlo.



Los cuatro acumuladores

Variable	Valor inicial	En cada número válido
`cantidad`	`0`	`cantidad++`
`suma`	`0`	`suma += numero`
`mayor`	`-Infinity`	`mayor = Math.max(mayor, numero)`
`invalidos`	`0`	`invalidos++` (solo en los no válidos)

¿Por qué `mayor` empieza en `-Infinity` y no en `0`? Porque si el usuario introduce solo números negativos, `-2` y `-8`, el mayor es `-2`. Empezando en `0`, el programa diría que el mayor es `0`, que ni siquiera se ha introducido. `-Infinity` es menor que cualquier número, así que el primero que llegue siempre lo sustituye. `Math.max()` devuelve el mayor de los valores que le paséis [55].



`continue` en un `do...while`

Si el usuario escribe «hola», `Number()` da `NaN`. Lo cuento como no válido y hago `continue`, que salta directamente a la comprobación de la condición [68]. Como `NaN !== 0` es `true`, el bucle sigue y vuelve a preguntar.



La media y el caso del cero

```
if (cantidad === 0) {
    salida.textContent = 'No has introducido ningún número.';
} else {
    const media = suma / cantidad;
```

Si el usuario escribe 0 a la primera, `cantidad` vale 0 y la media sería `0 / 0`, es decir, `NaN`. Por eso compruebo antes de dividir. Y como el bloque 02 os enseñó, `toFixed(2)` solo al mostrar.



Con 4, hola, 10, -2 y 0

Estadística	Valor
Números introducidos	3
Suma	12
Media	4.00
Mayor	10
No válidos	1

Ejercicio 08 · Fizz Buzz

> 🎯 **Objetivo:** Combinar todo lo del bloque: un bucle for, el operador resto para detectar múltiplos, una escalera de if en el orden correcto, cuatro contadores y un acumulador de texto con separadores.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Contar del 1 al límite sustituyendo los múltiplos de 3 por «Fizz», los de 5 por «Buzz» y los de ambos por «FizzBuzz», y contar cuántas veces aparece cada palabra. Es un clásico de las entrevistas de programación, y junta todo lo del bloque en pocas líneas: un bucle `for` que recorre del 1 al límite [64], el operador resto y una escalera de condiciones.



La solución paso a paso

1. Detectar múltiplos con `%`

Un número es múltiplo de otro si al dividirlos **no sobra nada**. El operador resto, que conocéis del bloque 02, devuelve justo lo que sobra [1]:

```
9 % 3 === 0    // true:  9 es múltiplo de 3
10 % 3 === 0   // false: sobra 1
```

2. El orden de las condiciones

```
if (i % 3 === 0 && i % 5 === 0) {
    resultado += 'FizzBuzz';
} else if (i % 3 === 0) {
    resultado += 'Fizz';
} else if (i % 5 === 0) {
    resultado += 'Buzz';
} else {
    resultado += i;
}
```

Aquí está la trampa del ejercicio. ¿Qué pasa con el 15 si empiezo por `i % 3 === 0`? Se cumple, entra, escribe «Fizz»... y como en una escalera de `else if` solo se ejecuta el primer bloque que encaja [56], nunca llega a comprobar el caso «FizzBuzz».

La regla general: **en una escalera, el caso más específico va primero**. «Múltiplo de 3 y de 5» es más específico que «múltiplo de 3», así que va antes. Es lo mismo que pasaba con las notas del ejercicio 01.

3. Los separadores

```
if (i < limite) {
    resultado += ', ';
}
```

Si añado `', '` en todas las vueltas, el texto termina en una coma colgando: `…14, FizzBuzz, `. Solo añado el separador si **no** es el último número.

4. Los contadores

Cada rama del `if` actualiza su propio contador. Como solo se ejecuta una rama por vuelta, cada número se cuenta una sola vez, y la suma de los cuatro contadores coincide siempre con el límite.



Con límite 30

Palabra	Veces	Cuáles
Fizz	8	3, 6, 9, 12, 18, 21, 24, 27
Buzz	4	5, 10, 20, 25
FizzBuzz	2	15, 30
Números	16	El resto

8 + 4 + 2 + 16 = 30. Cuadra.



Bloque 04

Bloque 04 · Funciones

Hasta ahora, cada programa era un único bloque de instrucciones. Funciona mientras el programa es pequeño, pero en cuanto crece aparecen dos problemas: código repetido y código imposible de leer. Las funciones resuelven los dos. En este bloque os voy a enseñar a **empaquetar un trozo de código, ponerle un nombre y reutilizarlo** tantas veces como queráis. Es probablemente el salto más importante de la fase 1.

Al terminar este bloque sabréis:

- Declarar funciones y llamarlas con argumentos.
- Devolver resultados con `return` y salir antes de tiempo.
- Dar valores por defecto a los parámetros.
- Escribir funciones de tres formas: declaración, expresión y flecha.
- Entender el ámbito de las variables: global, de función y de bloque.
- Pasar funciones como argumento (*callbacks*) y crear funciones que recuerdan datos (*closures*).
- Dividir un programa en funciones pequeñas, cada una con una sola tarea.



1. ¿Qué es una función?

Una función es un bloque de código con nombre que realiza una tarea y que podéis ejecutar cuando queráis [10]. Pensad en una **receta**: se escribe una vez y se cocina tantas veces como haga falta, con los ingredientes que toque cada día.

¿Por qué merece la pena?

- **No repetís código.** Si una fórmula aparece cinco veces y tiene un fallo, lo corregís en un solo sitio.
- **El programa se lee mejor.** `calcularIva(precio)` se entiende sin mirar cómo está hecho por dentro.
- **Podéis probar cada pieza por separado.** Una función pequeña es fácil de comprobar con varios casos.



2. Declarar y llamar

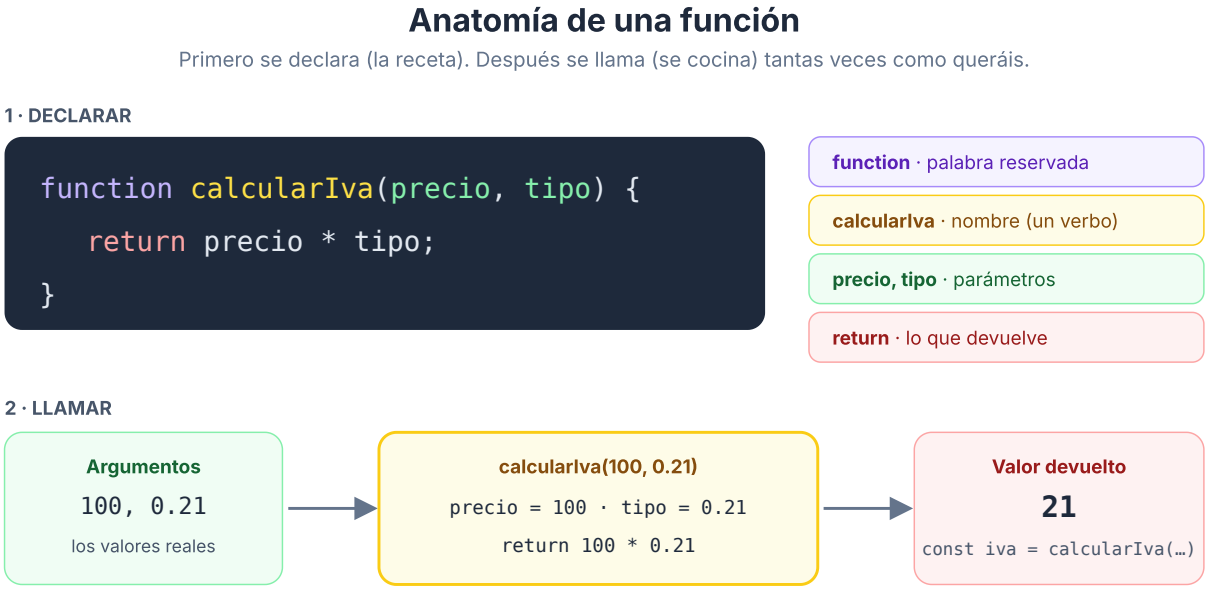


Figura 20 · Anatomía de una función

```
// 1. declarar: escribir la receta
function calcularIva(precio, tipo) {
    return precio * tipo;
}

// 2. llamar: cocinarla
const iva = calcularIva(100, 0.21);   // 21
```

Una declaración de función lleva la palabra `function`, un nombre, la lista de parámetros entre paréntesis y el cuerpo entre llaves [71]. Declarar una función **no la ejecuta**. Solo se ejecuta cuando la llamáis, escribiendo su nombre seguido de paréntesis.

2.1. Ponerles nombre

Las funciones **hacen** cosas, así que su nombre suele empezar por un verbo: `calcularTotal`, `mostrarMensaje`, `validarEmail`. Las que responden a una pregunta de sí o no suelen empezar por `es`, `esta` o `tiene`: `esPar`, `estaEntre`, `tieneStock`.



3. Parámetros y argumentos

Los dos términos se confunden mucho, así que os los separo:

Término	Dónde aparece	Qué es	Ejemplo
Parámetro	En la declaración	El nombre del hueco	`precio`, `tipo`
Argumento	En la llamada	El valor real que rellena el hueco	`100`, `0.21`

Los argumentos se asignan a los parámetros **por orden**: el primero al primero, el segundo al segundo. Si os falta alguno, ese parámetro vale `undefined`.

3.1. Valores por defecto

```
function saludar(nombre = 'invitado', idioma = 'es') {
    // …
}

saludar();                  // nombre = 'invitado', idioma = 'es'
saludar('Lucía');           // idioma = 'es'
saludar(undefined, 'fr');   // nombre = 'invitado'
saludar(null, 'it');        // nombre = null  ⚠
```

El valor por defecto se usa cuando el argumento falta o vale `undefined`, pero **no** cuando vale `null` u otro valor *falsy* [72]. La idea es que `null` es un valor que alguien ha pasado a propósito.

Colocad los parámetros con valor por defecto **al final**. Así, los opcionales se pueden omitir sin tener que escribir `undefined`.



4. `return`: devolver un resultado

```
function celsiusAFahrenheit(celsius) {
    return celsius * 9 / 5 + 32;
}
const f = celsiusAFahrenheit(25);   // 77
```

`return` hace dos cosas: **termina la función** y **devuelve un valor** a quien la llamó. Si no hay `return`, o no lleva valor, la función devuelve `undefined` [73].

Hay, por tanto, dos tipos de funciones:

Tipo	Lleva `return`	Ejemplo
Calcula y devuelve	Sí	`celsiusAFahrenheit(25)` → `77`
Hace una acción	No (devuelve `undefined`)	`mostrarMensaje('Hola')`, que escribe en la página

4.1. Salir antes de tiempo

Como `return` termina la función, podéis usarlo para salir en cuanto conozcáis la respuesta:

```
function precioEntrada(edad, dia) {
    if (Number.isNaN(edad) || edad < 0) {
        return null;          // datos no válidos: salgo
    }
    if (edad < 4) {
        return 0;
    }
    if (dia === 'miércoles') {
        return 5;
    }
    return 8;                 // ninguna excepción: precio normal
}
```

Esta técnica, el *early return*, evita las escaleras de `else` y deja cada caso a la vista.



5. Tres formas de escribir una función



Figura 21 · Tres formas de escribir una función

```
// declaración
function doble(n) {
    return n * 2;
}

// expresión: la función se guarda en una constante
const doble = function (n) {
    return n * 2;
};

// flecha
const doble = (n) => {
    return n * 2;
};

// flecha con retorno implícito
const doble = (n) => n * 2;
```

Las tres se llaman igual: `doble(5)`. La expresión de función crea una función y la guarda en una constante [74]. La función flecha es una forma más corta de escribir una expresión; si el cuerpo es una sola expresión, se pueden quitar las llaves y el `return`, y el resultado se devuelve solo [75].

5.1. *Hoisting*

Las **declaraciones** de función se pueden llamar antes de la línea donde están escritas: JavaScript las registra antes de ejecutar el código. Con las expresiones y las flechas no pasa [7]:

```
doble(5);                  // ✔ si doble es una declaración
triple(5);                 // ✘ ReferenceError si triple es una expresión o una flecha
const triple = (n) => n * 3;
```

Mi recomendación: aunque el *hoisting* lo permita, **declarad las funciones al principio** del programa y llamadlas después. Se lee de arriba abajo, sin sorpresas.

Forma	Cuándo la uso en el curso
Declaración	Funciones principales del programa
Flecha	Funciones cortas y *callbacks*
Expresión	Sobre todo para entender las flechas



6. El ámbito de las variables

El ámbito (*scope*) es la zona del código donde una variable existe. En JavaScript hay tres que os interesan: el **global**, el **de función** y el **de bloque**, que crean unas llaves `{ }` [76].

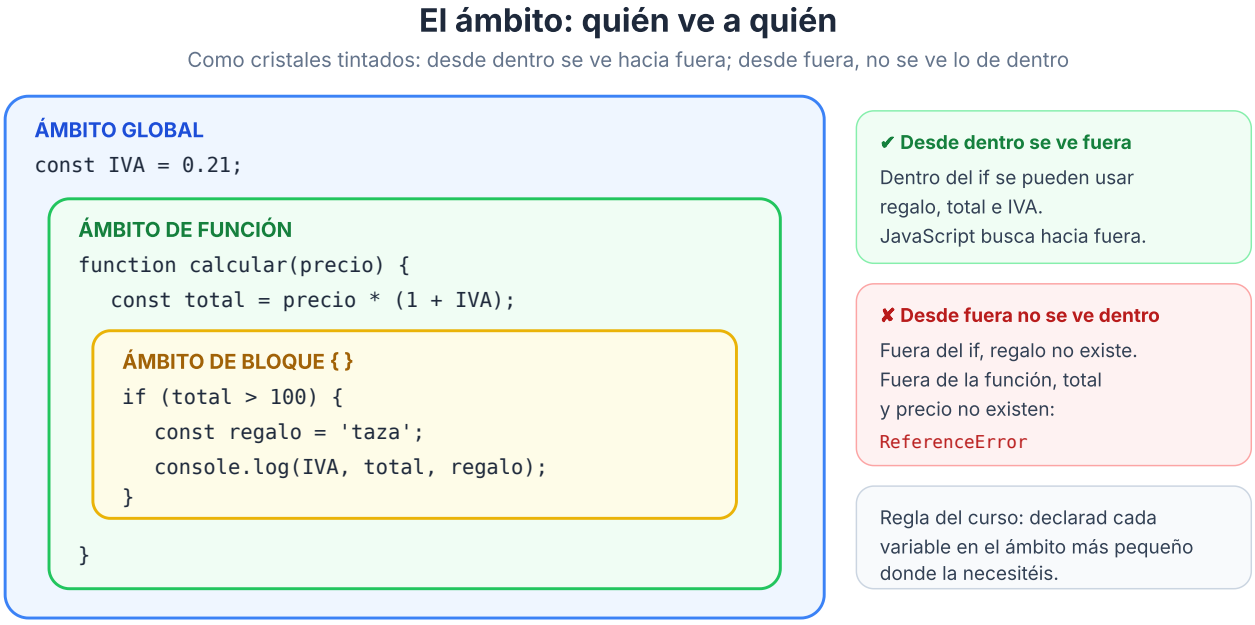


Figura 22 · El ámbito

La regla se resume en una frase: **desde dentro se ve hacia fuera, pero desde fuera no se ve hacia dentro**.

```
const IVA = 0.21;                       // global

function calcular(precio) {
    const total = precio * (1 + IVA);   // ✔ ve IVA, que está fuera
    return total;
}

console.log(total);                     // ✘ ReferenceError: total no existe aquí
```

Si dentro declaráis una variable con el mismo nombre que otra de fuera, la de dentro **tapa** a la de fuera mientras estéis dentro. A esto se le llama *shadowing*. Funciona, pero es una fuente de despistes: evitadlo.



💡 Consejo

Declarad cada variable en el **ámbito más pequeño** donde la necesitéis. Una variable que solo usa una función, dentro de la función. Cuantas menos variables globales, menos sitios desde los que alguien puede cambiarlas por error.



7. Las funciones son valores: *callbacks*

En JavaScript, una función es un valor más, como un número o un texto. Se puede guardar en una constante… y también **pasar como argumento a otra función**. Una función que se pasa a otra para que esta la ejecute se llama *callback* [77].



Figura 23 · Callback

```
const sumar = (a, b) => a + b;

function operar(a, b, operacion) {
    return operacion(a, b);       // llama al callback
}

operar(8, 2, sumar);              // 10
operar(8, 2, (x, y) => x * y);    // 16: callback anónimo escrito en la llamada
```



🚨 Atención

Pasad el *callback* **sin paréntesis**: `operar(8, 2, sumar)`. Si escribís `sumar()`, la estáis ejecutando en ese momento y pasando su resultado, no la función.

Ahora mismo puede parecer un rodeo, pero esta idea está en todas partes: `addEventListener` en el bloque 07, `map` y `filter` en el bloque 05, `setTimeout` en el bloque 10… En todos esos casos vosotros entregáis la función y otro decide **cuándo** llamarla.



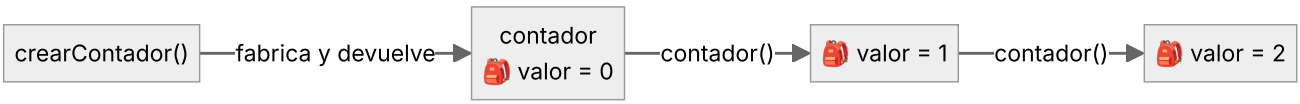
8. Funciones que recuerdan: *closures*

Una función también puede **devolver** otra función. Y la función devuelta recuerda las variables del lugar donde se creó, aunque ese lugar ya haya terminado de ejecutarse:

```
function crearContador() {
    let valor = 0;
    return () => {
        valor++;
        return valor;
    };
}

const contador = crearContador();
contador();   // 1
contador();   // 2
contador();   // 3
```

Una función junto con las referencias a su entorno forma un *closure* [11]. Pensad en una mochila: la función se lleva consigo las variables que tenía a mano al nacer. Cada llamada a `crearContador()` fabrica una mochila nueva, así que cada contador lleva su propia cuenta. Y nadie de fuera puede tocar `valor` directamente: es un dato privado.



No os agobiéis si no lo veis del todo a la primera. Con quedaros con la idea de que la función recuerda dónde nació, por ahora es suficiente.



9. Buenas prácticas con funciones

- **Una función, una tarea.** Si para describir lo que hace necesitáis la palabra «y», probablemente son dos funciones.
- **Separad el cálculo de la presentación.** Una función calcula el precio y otra lo convierte en texto. Si cambia el formato, el cálculo no se toca.
- **Preferid funciones puras** cuando podáis: que con los mismos argumentos devuelvan siempre lo mismo y no toquen nada de fuera (ni el DOM, ni variables globales). Son las más fáciles de probar y de reutilizar.
- **Que reciban lo que necesitan como parámetros** en lugar de leer variables globales.
- **Nombres que cuenten lo que hacen.** Si el nombre es bueno, no hace falta leer el cuerpo.



10. La plantilla de trabajo en este bloque

La sección de estructura del programa principal estrena un apartado: **las funciones**, declaradas al principio.

```
// estructura del programa principal
// funciones
function calcularTotal(precio, cantidad, iva = 0.21) {
    const subtotal = precio * cantidad;
    return subtotal + subtotal * iva;
}

function formatearEuros(importe) {
    return `${importe.toFixed(2)} €`;
}

// constantes al DOM
const salida = document.querySelector('#salida');

// entradas
const precio = Number(prompt('Precio:', '10'));
const cantidad = Number(prompt('Cantidad:', '3'));

// procesamiento
const total = calcularTotal(precio, cantidad);

// salidas
salida.textContent = `Total: ${formatearEuros(total)}`;
```



🚨 Atención

Una trampa que aparece en el ejercicio 02: si el usuario pulsa Cancelar, `prompt()` devuelve `null` … y `Number(null)` vale `0`, no `NaN`. Lo mismo ocurre con un texto vacío [78]. Si un 0 puede colarse como dato válido, leed primero el texto y comprobad si es `null` o está vacío antes de convertir.

En el bloque 07, cuando lleguen los eventos, las funciones serán las que se ejecuten al pulsar un botón. Todo lo que aprendáis aquí se aprovecha allí directamente.



11. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Conversor de temperaturas	Declarar, llamar, `return`, funciones sin `return`	Ver
02	Precio de la entrada de cine	Varios parámetros, *early return*, devolver `null`, `Number(null)`	Ver
03	Saludos con valores por defecto	Parámetros por defecto, `undefined` frente a `null`	Ver
04	Tres formas de escribir una función	Declaración, expresión, flecha, retorno implícito, *hoisting*	Ver
05	Detective del ámbito	Ámbito global, de función y de bloque, *shadowing*	Ver
06	Calculadora con callbacks	Funciones como argumento, *callbacks* anónimos	Ver
07	Fábrica de contadores	Funciones que devuelven funciones, *closures*, datos privados	Ver
08	Planificador de estudio	Dividir un programa en funciones pequeñas, validar con bucle, funciones puras	Ver

El ejercicio 08 cierra la fase 1 del curso: junta variables, condiciones, bucles y funciones en un programa completo. Si os sale sin mirar la solución, estáis preparados para el bloque 05.



🧪 Los ejercicios, explicados

Ejercicio 01 · Conversor de temperaturas

> 🎯 **Objetivo:** Declarar funciones con parámetros que devuelven un valor con return, llamarlas varias veces con argumentos distintos y comprobar qué devuelve una función que no tiene return.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir dos funciones de conversión de temperatura, reutilizarlas con varios valores y comprobar qué devuelve una función sin `return`.



La solución paso a paso

1. Declarar la función

```
function celsiusAFahrenheit(celsius) {
    return celsius * 9 / 5 + 32;
}
```

Una declaración de función tiene cuatro piezas [1]: la palabra `function`, el **nombre**, los **parámetros** entre paréntesis y el **cuerpo** entre llaves. Tenéis cada pieza señalada en la infografía de la teoría.

Fijaos en el nombre: `celsiusAFahrenheit`. Dice exactamente lo que hace. Os recomiendo nombres con verbo o con la forma «de A a B», para que al leer `celsiusAFahrenheit(25)` sepáis qué esperar sin mirar el cuerpo.

2. `return`: devolver el resultado

`return` termina la función y devuelve el valor indicado a quien la llamó [73]. Ese valor «sustituye» a la llamada:

```
const fahrenheit = celsiusAFahrenheit(25);   // es como escribir: const fahrenheit = 77;
```

3. Llamar muchas veces

Lo bueno de una función es que **la escribo una vez y la uso todas las que quiera**. En la tabla la llamo seis veces con argumentos distintos, y en ningún momento repito la fórmula.

Incluso puedo pasar el resultado de una función como argumento de otra:

```
fahrenheitACelsius(celsiusAFahrenheit(0))   // 0 → 32 → 0
```

Se lee de dentro hacia fuera: primero se ejecuta `celsiusAFahrenheit(0)`, que devuelve `32`, y ese `32` se pasa a `fahrenheitACelsius`.

4. Una función sin `return`

```
function mostrarEnConsola(texto) {
    console.log(texto);
}
const devuelto = mostrarEnConsola('Hola');
console.log(devuelto);   // undefined
```

La función hace su trabajo, escribir en la consola, pero no devuelve nada. Si una función no ejecuta ningún `return`, o lo ejecuta sin valor, devuelve `undefined` [73].

Tipo de función	¿Devuelve algo?	Ejemplo
Calcula un resultado	Sí, con `return`	`celsiusAFahrenheit(25)` → `77`
Hace una acción	No (`undefined`)	`mostrarEnConsola('Hola')`



Qué tenéis que observar

- 0 °C son 32 °F y 100 °C son 212 °F.
- A −40 grados, las dos escalas coinciden. Un dato curioso para la próxima cena.
- La última columna recupera siempre la temperatura de partida.

Ejercicio 02 · Precio de la entrada de cine

> 🎯 **Objetivo:** Escribir una función con varios parámetros que usa return anticipados (early return) para salir en cuanto conoce la respuesta, y devolver null cuando los datos no son válidos.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Calcular el precio de una entrada con una función que recibe dos datos, usando `return` anticipados, y probarla con varios casos.



La solución paso a paso

1. Varios parámetros

```
function precioEntrada(edad, dia) {
```

Una función puede recibir tantos parámetros como necesite, separados por comas. Al llamarla, los argumentos se asignan **por orden**: el primero a `edad`, el segundo a `dia`. Si los cambiáis de orden, `precioEntrada('lunes', 30)`, la función recibirá un día como edad y el resultado no tendrá sentido.

2. `return` anticipado

```
if (edad < 4) {
    return 0;
}
if (edad >= 65) {
    return 4;
}
…
return 8;
```

Cuando se ejecuta un `return`, la función **se detiene** en ese punto [73]. Lo aprovecho para salir en cuanto conozco la respuesta. Cada `if` es una excepción: si se cumple, devuelvo el precio y se acabó. Si llego al final, es que no se ha cumplido ninguna y devuelvo el precio normal.

Comparadlo con la versión con `else if`:

```
let precio;
if (edad < 4) {
    precio = 0;
} else if (edad >= 65) {
    precio = 4;
} else if …
return precio;
```

Funcionan igual, pero la primera se lee mejor: no hace falta una variable auxiliar y cada caso se entiende por separado. A esta técnica se la conoce como *early return*, o cláusula de guarda, y es muy habitual.

3. El orden de las comprobaciones

Primero van los datos no válidos, comprobados con `Number.isNaN()` igual que en el bloque 03 [57]. Y después, ¿por qué la edad va antes que el miércoles? Porque un jubilado que va el miércoles paga 4 € y no 5 €: las rebajas por edad ganan. El orden de los `return` es el orden de prioridad de las normas.

4. Devolver `null`

Si los datos no son válidos, devuelvo `null`: «no hay precio». Quien llama a la función decide qué hacer con eso. Aquí, `formatearPrecio` lo convierte en un mensaje.

5. Separar el cálculo de la presentación

Fijaos en que `precioEntrada` devuelve un **número** y `formatearPrecio` lo convierte en **texto**. Dos funciones, dos responsabilidades. Así, si mañana el precio hay que mostrarlo de otra forma, solo cambio `formatearPrecio`, y el cálculo ni se toca.

6. Una trampa de la entrada: `Number(null)` es `0`

```
const textoEdad = prompt('¿Qué edad tienes?', '30');
const edad = textoEdad === null || textoEdad.trim() === '' ? NaN : Number(textoEdad);
```

Si escribís `Number(prompt(…))` como hasta ahora y el usuario pulsa Cancelar, `prompt` devuelve `null`… y `Number(null)` vale `0`, no `NaN` [78]. Lo mismo pasa con un texto vacío: `Number('')` es `0`. En este ejercicio sería grave, porque una edad 0 significa entrada gratis. Por eso leo primero el texto y, si es `null` o está vacío, lo convierto en `NaN` a propósito, para que la validación de la función lo rechace.

7. El día

```
const dia = (prompt('¿Qué día vas al cine?', 'miércoles') || '').trim();
```

Si el usuario cancela, `prompt` devuelve `null`, y `null.trim()` daría error. `prompt(…) || ''` aprovecha que `null` es *falsy*: `||` devuelve el primer operando si es *truthy* y, si no, el segundo [59]. Así, si `prompt` devuelve `null`, se usa `''`. Es la versión corta de lo que hicisteis con un `if` en el bloque 03.



Las pruebas

Edad	Día	Precio	Regla que se aplica
3	lunes	Gratis	Menor de 4
70	miércoles	4.00 €	65 o más (gana al miércoles)
30	miércoles	5.00 €	Día del espectador
30	sábado	8.00 €	Precio normal
-5	lunes	Datos no válidos	Edad negativa

Ejercicio 03 · Saludos con valores por defecto

> 🎯 **Objetivo:** Dar valores por defecto a los parámetros de una función y descubrir cuándo se aplican: si falta el argumento o vale undefined, pero no si vale null.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir una función con parámetros por defecto y averiguar en qué casos se usan esos valores.



Los parámetros por defecto

```
function saludar(nombre = 'invitado', idioma = 'es') {
```

Si al llamar a la función falta un argumento, el parámetro vale `undefined`. Con `= valor` le dais un valor alternativo, que se usa cuando no llega ningún valor o llega `undefined` [72].



Los resultados

Llamada	Resultado	Qué ha pasado
`saludar('Lucía', 'en')`	Hello, Lucía	Llegan los dos argumentos
`saludar('Lucía')`	Hola, Lucía	Falta el idioma → `'es'`
`saludar()`	Hola, invitado	Faltan los dos
`saludar(undefined, 'fr')`	Bonjour, invitado	`undefined` activa el valor por defecto
`saludar(null, 'it')`	Ciao, null	⚠ `null` **no** lo activa
`saludar('Lucía', 'de')`	Hola, Lucía	Idioma desconocido → `default` del `switch`



La sorpresa: `null`

MDN lo especifica con precisión: el valor por defecto se aplica con `undefined`, pero **no** con `null` ni con otros valores *falsy* [72]. La idea es que `undefined` significa «no me han dado nada», mientras que `null` es un valor que alguien ha pasado a propósito, y la función lo respeta.

¿Y si quisierais que `null` también diese «invitado»? Entonces no os sirve el valor por defecto: tendríais que comprobarlo dentro de la función con un `if`, como hicisteis en el bloque 03.



Dos detalles de diseño

Los parámetros con valor por defecto, al final. Si `nombre` tuviera valor por defecto pero `idioma` no, para usar el nombre por defecto tendríais que escribir `saludar(undefined, 'fr')`, que es feo. Por eso el orden habitual es: primero los obligatorios, después los opcionales.

El `default` del `switch` se encarga de los idiomas que no conocemos [70]. Así la función nunca devuelve algo como `undefined, Lucía`.

Ejercicio 04 · Tres formas de escribir una función

> 🎯 **Objetivo:** Escribir la misma idea con las tres sintaxis de función de JavaScript (declaración, expresión y flecha), usar el retorno implícito de las funciones flecha y comprobar qué es el hoisting.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir tres funciones de área con las tres sintaxis de JavaScript y descubrir qué es el *hoisting*.



Las tres formas



📊 Figura 24 · Tres formas de escribir una función

1. Declaración de función

```
function areaRectangulo(base, altura) {
    return base * altura;
}
```

La que conocéis desde el ejercicio 01 [71].

2. Expresión de función

```
const areaTriangulo = function (base, altura) {
    return base * altura / 2;
};
```

Aquí la función no tiene nombre propio: se crea y se **guarda en una constante**, igual que guardaríais un número o un texto [74]. Se llama exactamente igual: `areaTriangulo(4, 3)`. Fijaos en el punto y coma del final: es una asignación, como `const x = 5;`.

3. Función flecha

```
const areaCirculo = (radio) => Math.PI * radio ** 2;
```

Una forma más corta de escribir una expresión de función [75]. Si el cuerpo es una sola expresión, podéis quitar las llaves y el `return`: el resultado de la expresión se devuelve solo. Es el **retorno implícito** [75].

Sintaxis	Cuándo la uso en el curso
Declaración	Funciones principales del programa
Flecha	Funciones cortas y, sobre todo, callbacks (ejercicio 06)
Expresión	Para entender las flechas; en código moderno se ve menos



El *hoisting*

La primera línea del programa llama a `areaRectangulo` **antes** de que esté escrita... y funciona. ¿Magia? No. Antes de ejecutar el código, JavaScript registra todas las **declaraciones** de función, así que ya existen desde el principio. A esto se le llama elevación o *hoisting*, y solo afecta a las declaraciones, no a las expresiones [7].

Con `areaTriangulo` no pasa. La constante no tiene valor hasta que se ejecuta su línea, y si intentáis usarla antes, Chrome os dice:

```
ReferenceError: Cannot access 'areaTriangulo' before initialization
```



💡 Consejo

Aunque el *hoisting* lo permita, mi recomendación es **declarar las funciones al principio del programa**, en la sección de estructura, y llamarlas después. Es lo que hace la plantilla del curso y se lee de arriba abajo sin sorpresas.



Qué tenéis que observar

- Con 4, 3 y 2: rectángulo 12.00, triángulo 6.00 y círculo 12.57.

- La figura mayor es el círculo, por poco.

Ejercicio 05 · Detective del ámbito

> 🎯 **Objetivo:** Entender los tres ámbitos de JavaScript (global, de función y de bloque), qué ocurre cuando una variable interior tiene el mismo nombre que una exterior (shadowing) y por qué una variable local no existe fuera de su función.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Averiguar qué variable ve cada parte del programa cuando hay tres variables con el mismo nombre en tres ámbitos distintos.



Los resultados

Dónde se lee	Valor de `lugar`
Dentro del bloque `if`	`'bloque'`
Dentro de la función, fuera del `if`	`'función'`
Fuera de todo	`'global'`

Pregunta	Respuesta
`llamadas` después de 3 llamadas	`3`
`typeof secreto` fuera de la función	`'undefined'`



Los tres ámbitos

El ámbito (*scope*) es la zona del código donde una variable existe y se puede usar. JavaScript tiene tres que os interesan ahora [1]:

Ámbito	Se crea con	En el ejercicio
Global	El propio programa	`lugar = 'global'`, `llamadas` y las constantes al DOM
De función	Una función	`lugar = 'función'` y `secreto`
De bloque	Unas llaves `{ }` (un `if`, un bucle…)	`lugar = 'bloque'`

El ámbito: quién ve a quién

Como cristales tintados: desde dentro se ve hacia fuera; desde fuera, no se ve lo de dentro

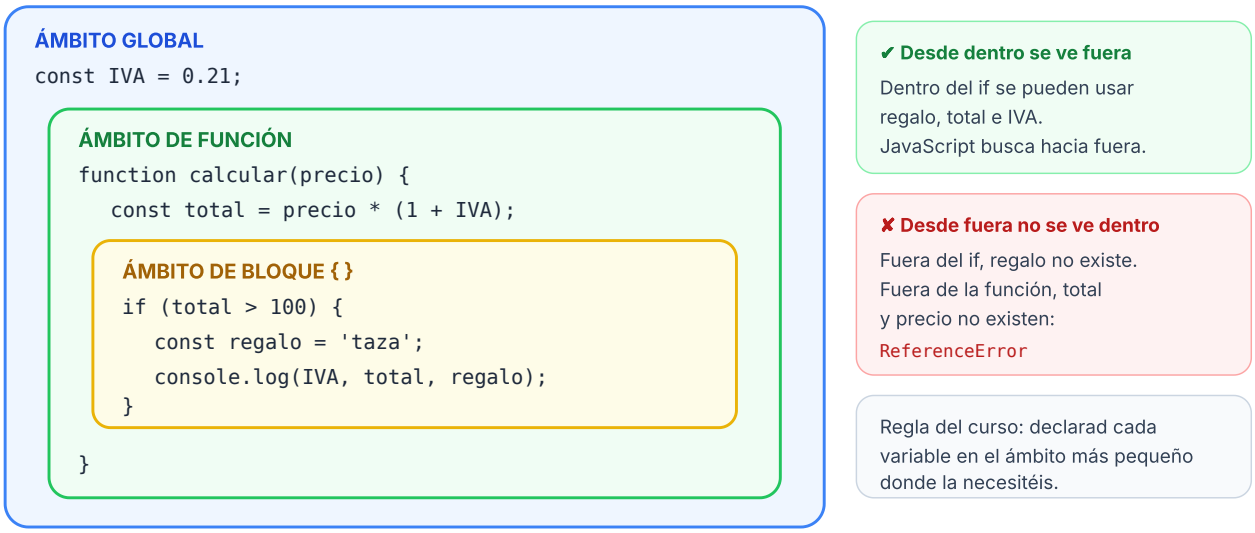


Figura 25 · El ámbito



La regla: desde dentro se ve hacia fuera

Cuando JavaScript encuentra un nombre de variable, lo busca primero en el ámbito actual. Si no está, sube al ámbito que lo contiene, y así hasta el global.

- `llamadas++` **dentro de la función** modifica la variable global, porque dentro de la función no hay ninguna `llamadas`: JavaScript sube y la encuentra fuera. Por eso, tras tres llamadas, vale 3.
- **Las constantes al DOM** son globales, así que la función puede escribir en ellas sin recibirlas como parámetro.



Lo que no funciona: desde fuera hacia dentro

`secreto` es local de la función. Fuera de ella **no existe**:

```
console.log(secreto);   // ReferenceError: secreto is not defined
```

¿Y por qué `typeof secreto` no da error? Es una excepción de `typeof`: con una variable que no existe devuelve `'undefined'` en lugar de lanzar el error. Por eso es útil para comprobar si algo existe.



Shadowing: tapar una variable

Cuando declaráis una variable con el mismo nombre que otra de un ámbito exterior, la interior **tapa** a la exterior mientras estáis dentro. Las dos existen, pero desde dentro solo veis la más cercana. Al salir del bloque, la interior desaparece y la exterior vuelve a verse.



💡 Consejo

Este ejercicio usa el mismo nombre tres veces para que veáis el mecanismo. En vuestro código, **evitadlo**: tres variables `lugar` distintas son una invitación al despiste. Y declarad cada variable en el ámbito más pequeño donde la necesitéis.

Ejercicio 06 · Calculadora con callbacks

🎯 **Objetivo:** Pasar una función como argumento de otra (callback), tanto una función guardada en una constante como una función flecha anónima escrita en la propia llamada.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir una función `operar()` que no sabe qué operación hará, y decírselo pasándole otra función como argumento.



Las funciones son valores

```
const sumar = (a, b) => a + b;
```

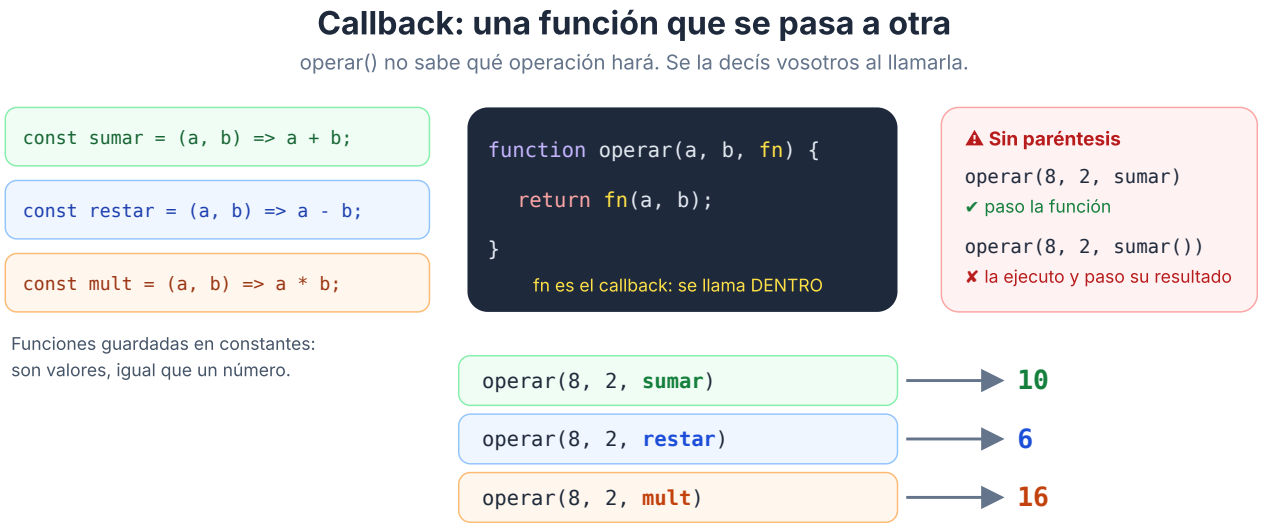
Esta línea guarda una función en una constante, igual que `const precio = 10;` guarda un número. Y si una función es un valor, se puede hacer con ella lo mismo que con cualquier valor... incluido **pasarla como argumento** a otra función.



El callback

```
function operar(a, b, operacion) {
    return operacion(a, b);
}
```

El tercer parámetro, `operacion`, no es un número: es una función. Y `operar` la llama por dentro con `operacion(a, b)`. Una función que se pasa a otra como argumento para que esta la ejecute se llama **callback** [77].



📊 Figura 26 · Callback



Con paréntesis o sin paréntesis

Este es el error número uno con los callbacks:

```
operar(a, b, sumar)     // ✔ entrego la función
operar(a, b, sumar())   // ✘ la ejecuto ahora (sin argumentos) y entrego su resultado: NaN
```

Sin paréntesis estáis **entregando la receta**. Con paréntesis, la estáis **cocinando** en ese momento y entregando el plato. `operar` necesita la receta para cocinarla ella misma con `a` y `b`.



Callbacks anónimos

```
operar(a, b, (x, y) => x ** y);
```

Si una operación solo la necesitáis una vez, no hace falta guardarla en una constante: se escribe directamente en la llamada. Es una función **anónima**, sin nombre. Aquí es donde las funciones flecha de una línea brillan [75].



¿Para qué sirve esto?

Ahora mismo parece un rodeo para sumar dos números, lo reconozco. Pero esta idea es la base de casi todo lo que viene:

- En el bloque 05, `map`, `filter` y `forEach` reciben un callback que dice qué hacer con cada elemento de una lista.
- En el bloque 07, `addEventListener('click', callback)` recibe la función que se ejecutará cuando el usuario haga clic.
- En el bloque 10, `setTimeout(callback, 1000)` recibe la función que se ejecutará al cabo de un segundo.

En todos esos casos, vosotros entregáis la función y es otro quien decide **cuándo** llamarla.



Con 8 y 2

Callback	Resultado
`sumar`	10
`restar`	6
`multiplicar`	16
`dividir`	4
`(x, y) => x ** y`	64
media	5

Ejercicio 07 · Fábrica de contadores

> 🎯 **Objetivo:** Crear funciones que devuelven otras funciones y comprobar que la función devuelta recuerda las variables del lugar donde se creó (closure), de forma independiente para cada copia.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir funciones que fabrican y devuelven otras funciones, y comprobar que cada función fabricada recuerda sus propios datos.



Una función que devuelve una función

```
function crearContador(inicio = 0, paso = 1) {
    let valor = inicio;
    return () => {
        valor += paso;
        return valor;
    };
}
```

`crearContador` no cuenta nada. Lo que hace es **fabricar** un contador y devolverlo. Sus parámetros tienen valores por defecto, como en el ejercicio 03 [2]: sin argumentos, empieza en 0 y sube de 1 en 1. Como visteis en el ejercicio 06, las funciones son valores, así que se pueden devolver con `return` igual que un número.

```
const visitas = crearContador();
```

Después de esta línea, `visitas` es una función. Cada vez que la llamáis, `visitas()`, suma 1 y devuelve la cuenta.



El *closure*: la función recuerda

Aquí viene lo interesante. Cuando `crearContador` termina, su variable local `valor` debería desaparecer, como cualquier variable local. Pero la función flecha que hemos devuelto **sigue usándola**. Y JavaScript la conserva mientras alguien la necesite.

Una función junto con las variables del lugar donde se creó forma lo que se llama un *closure* [11]. Dicho de forma más llana: la función se lleva consigo una mochila con las variables que tenía a mano al nacer.



Cada contador, su propia mochila

Llamada	Resultado
`visitas()` × 3	1, 2, 3
`puntos()` × 2	110, 120
`visitas()` otra vez	**4**

Cada llamada a `crearContador` crea un `valor` **nuevo**. `visitas` y `puntos` tienen cada uno el suyo y no se pisan: aunque entre medias hayáis usado `puntos`, `visitas` sigue por donde iba.



Datos privados

```
console.log(valor);   // ReferenceError: valor is not defined
```

Desde fuera nadie puede tocar `valor` directamente. La única forma de cambiarlo es llamar al contador, que solo sabe sumar `paso`. Nadie puede poner las visitas a −500 por error. Esa protección es una de las grandes utilidades de los *closures*.



Con un parámetro también vale

```
function crearMultiplicador(factor) {
    return (n) => n * factor;
}
const doble = crearMultiplicador(2);
doble(7);   // 14
```

Aquí la mochila contiene el parámetro `factor`. `doble` recuerda un 2; `triple`, un 3.



📝 Nota

Los *closures* son de los conceptos que cuesta ver a la primera. Si os ha quedado la idea de que «la función devuelta recuerda las variables de donde se creó», tenéis lo necesario. Los volveréis a ver en acción en los bloques 07 y 10.

Ejercicio 08 · Planificador de estudio

🎯 **Objetivo:** Organizar un programa en funciones pequeñas y reutilizables, cada una con una sola tarea, y combinarlas: validar, pedir datos con repetición hasta que sean válidos y dar formato a un resultado.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Construir un programa completo repartiendo el trabajo en tres funciones pequeñas, cada una con una sola tarea, y combinarlas.



Tres funciones, tres tareas

Función	Tarea	Devuelve
`estaEntre(valor, min, max)`	Comprobar	`true` o `false`
`pedirNumero(mensaje, min, max, porDefecto)`	Preguntar hasta que la respuesta valga	Un número o `null`
`formatearTiempo(horas)`	Dar formato	Un texto como `"2 h 30 min"`

Con esto, el programa principal queda cortísimo y se lee casi como una frase: «pide las horas, pide los días, si alguien ha cancelado avisa y si no calcula y muestra».



La solución paso a paso

1. `estaEntre`: una función que devuelve un booleano

```
function estaEntre(valor, minimo, maximo) {
    return !Number.isNaN(valor) && valor >= minimo && valor <= maximo;
}
```

`Number.isNaN()` descarta lo que no es un número [57]. Y fijaos: no hace falta un `if` que devuelva `true` o `false`, porque la propia condición **ya es** un booleano, así que la devuelvo directamente. Las funciones que responden a una pregunta de sí o no suelen empezar por `es`, `esta` o `tiene`.

2. `pedirNumero`: un bucle dentro de una función

```
while (true) {
    const respuesta = prompt(…);
    if (respuesta === null) {
        return null;
    }
    const numero = Number(respuesta);
    if (estaEntre(numero, minimo, maximo)) {
        return numero;
    }
    aviso = '⚠ Valor no válido. ';
}
```

`while (true)` parece un bucle infinito, y lo sería... si no fuera porque `return` termina la función, y con ella el bucle [73]. Hay dos salidas: cancelar o escribir un valor válido. Mientras no pase ninguna de las dos, la función vuelve a preguntar, ahora con un aviso.

Fijaos en que `pedirNumero` **usa** `estaEntre`. Las funciones se apoyan unas en otras.

3. `formatearTiempo`: de decimales a horas y minutos

```
const horas = Math.floor(horasDecimales);                  // 1.75 → 1
const minutos = Math.round((horasDecimales - horas) * 60); // 0.75 × 60 → 45
```

`Math.floor()` redondea hacia abajo y `Math.round()`, al entero más cercano [55]. Y con tres `return` anticipados, como en el ejercicio 02, evito textos como `0 h 45 min` o `2 h 0 min`.

4. La reutilización

`pedirNumero` se llama dos veces con datos distintos, y `formatearTiempo`, tres. Sin funciones, ese bucle de validación estaría escrito dos veces y la conversión a horas y minutos, tres. Y si encontrarais un fallo, tendríais que corregirlo en cinco sitios.



Funciones puras

`estaEntre` y `formatearTiempo` tienen dos propiedades muy valiosas:

- Con los mismos argumentos, devuelven **siempre** el mismo resultado.
- No tocan nada de fuera: ni el DOM, ni variables globales, ni la consola.

A las funciones así se las llama **puras**. Son las más fáciles de probar y de reutilizar, porque no dependen de nada más. `pedirNumero` no es pura, ya que interactúa con el usuario, y no pasa nada: lo importante es que la parte impura esté localizada en un sitio concreto.



Con 10 horas y 4 días

Dato	Valor
Horas a la semana	10 h
Días	4
Cada día	2 h 30 min
Bloques de 25 min	6



Bloque 05

Bloque 05 · Arrays y objetos

Hasta ahora, cada variable guardaba un único dato: un número, un texto, un booleano. Pero los programas reales trabajan con **colecciones**: la lista de la compra, las notas de una clase, el catálogo de una tienda. En este bloque os voy a enseñar las dos estructuras con las que JavaScript organiza los datos: los **arrays**, listas ordenadas por posición, y los **objetos**, fichas con datos que tienen nombre. Y, sobre todo, cómo combinarlas.

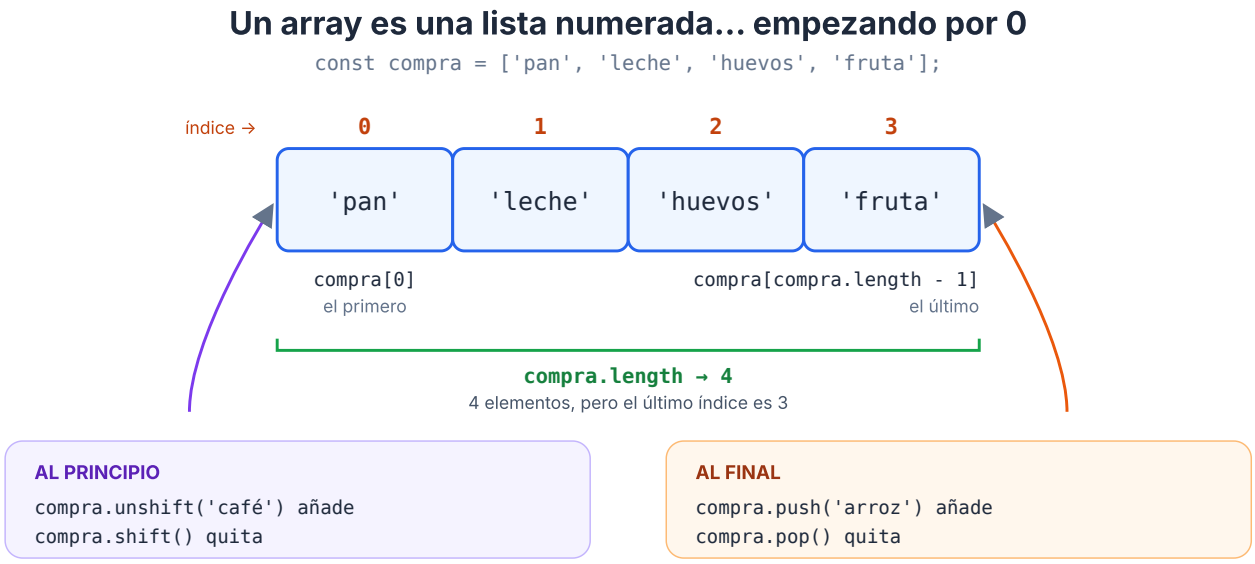
Al terminar este bloque sabréis:

- Crear arrays, leerlos por índice y modificarlos por los extremos y por el medio.
- Buscar elementos por valor y por condición.
- Recorrer un array con `for`, `for...of` y `forEach`.
- Transformar, filtrar, resumir y ordenar con `map`, `filter`, `reduce` y `sort`.
- Crear objetos con propiedades y métodos, y recorrerlos.
- Trabajar con arrays de objetos, la estructura más habitual en el mundo real.
- Usar desestructuración y *spread*.
- Distinguir entre copiar un valor y copiar una referencia, y convertir datos a JSON y de vuelta.



1. Arrays: listas ordenadas

Un array es una lista ordenada de valores. Cada valor ocupa una posición numerada, su **índice**, que **empieza en 0** [79].



📊 Figura 27 · Anatomía de un array

```
const compra = ['pan', 'leche', 'huevos'];

compra[0]                   // 'pan'      el primero
compra[2]                   // 'huevos'
compra.length               // 3          cuántos hay
compra[compra.length - 1]   // 'huevos'   el último, sea cual sea la longitud
compra[5]                   // undefined  no existe
```

`length` indica el número de elementos [80]. Como los índices empiezan en 0, el último siempre está en `length - 1`. Esta resta os va a acompañar mucho tiempo.

Un array puede guardar cualquier tipo de valor, incluso mezclados, aunque lo normal es que todos sus elementos sean del mismo tipo.



2. Modificar un array

Método	Dónde	Qué hace	Devuelve
`push(x)`	Al final	Añade	La nueva longitud [81]
`pop()`	Al final	Quita	El elemento quitado [82]
`unshift(x)`	Al principio	Añade	La nueva longitud [83]
`shift()`	Al principio	Quita	El elemento quitado [84]
`splice(i, n)`	En la posición `i`	Quita `n` elementos	Un array con los quitados [85]

Para quitar un elemento concreto, primero hay que encontrar su posición:

```
const posicion = compra.indexOf('leche');
if (posicion !== -1) {
    compra.splice(posicion, 1);
}
```

Comprobad siempre que `indexOf` no ha devuelto `-1`. Con un índice negativo, `splice` cuenta desde el final [85], así que `splice(-1, 1)` borraría el último elemento sin avisar.



3. Buscar en un array

Quiero saber...	Método	Devuelve
Si un valor está	`includes(valor)`	`true` / `false` [86]
Dónde está un valor	`indexOf(valor)`	Índice o `-1` [87]
El primero que cumple algo	`find(callback)`	El elemento o `undefined` [88]
Dónde está el primero que cumple algo	`findIndex(callback)`	Índice o `-1` [89]
Si todos cumplen algo	`every(callback)`	`true` / `false` [90]

```
const notas = [7.5, 4, 9, 3.5];
notas.includes(9);                // true
notas.find((n) => n < 5);         // 4
notas.every((n) => n >= 0);       // true
```

Los métodos que buscan **por condición** reciben un *callback*, como los del bloque 04: una función que se ejecuta con cada elemento y devuelve `true` o `false`.



4. Recorrer un array

```
const temperaturas = [18, 21, 25];

// for clásico: control total, con índice
for (let i = 0; i < temperaturas.length; i++) {
    console.log(i, temperaturas[i]);
}

// for...of: solo los valores
for (const t of temperaturas) {
    console.log(t);
}

// forEach: un callback por elemento, con valor e índice
temperaturas.forEach((t, i) => {
    console.log(i, t);
});
```

Necesito...	Uso
Solo los valores	`for...of` [91]
Valor e índice, de forma cómoda	`forEach` [92]
Parar a mitad con `break`, saltar elementos, ir al revés	`for` clásico



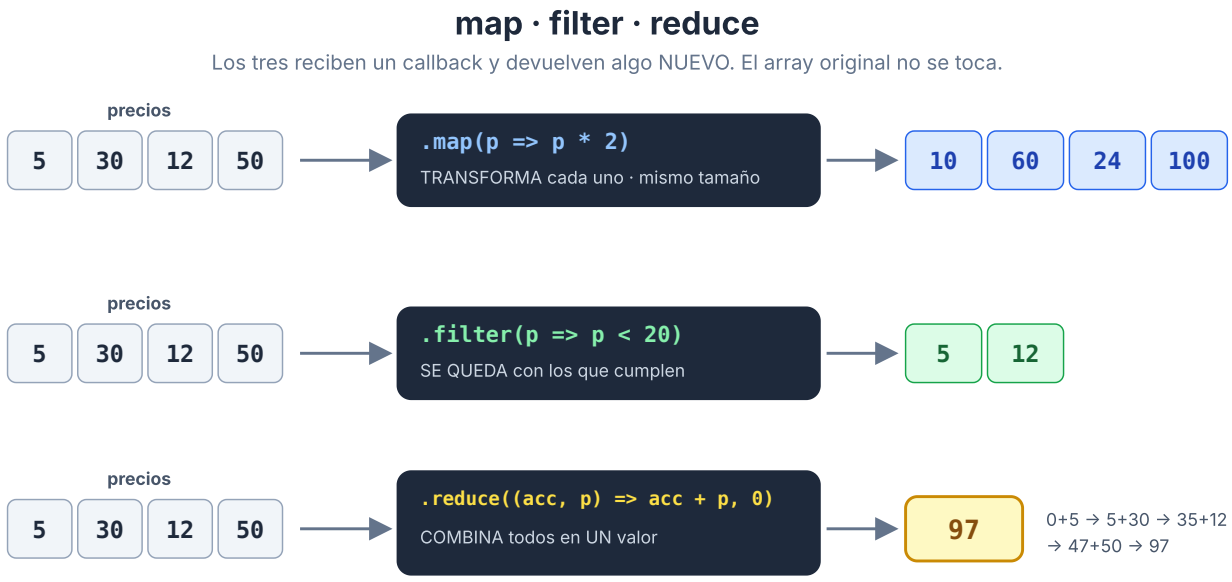
🚨 Atención

En el `for` clásico, la condición es `i < array.length`, con `<`. Con `<=`, la última vuelta lee una posición que no existe y obtenéis `undefined`.



5. Transformar: `map`, `filter` y `reduce`

Estos tres métodos son el corazón del trabajo con datos en JavaScript. Los tres reciben un *callback* y **devuelven algo nuevo sin modificar el array original**.



📊 Figura 28 · map, filter y reduce

```
const precios = [5, 30, 12, 50];

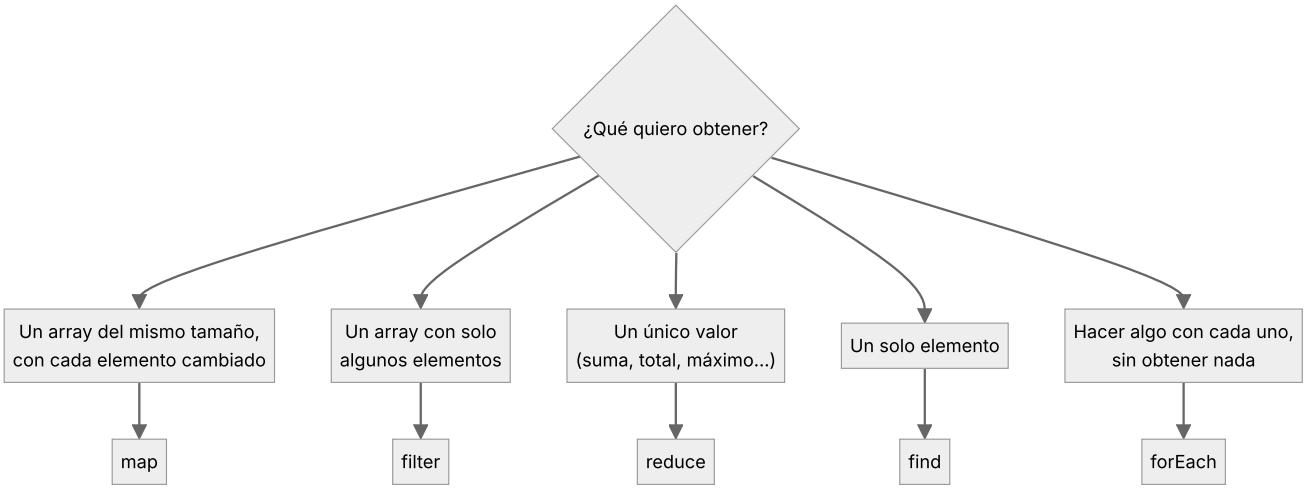
precios.map((p) => p * 2);                // [10, 60, 24, 100]  transforma cada uno
precios.filter((p) => p < 20);            // [5, 12]            se queda con algunos
precios.reduce((acc, p) => acc + p, 0);   // 97                 los combina en uno
```

- `map()` crea un array nuevo con el resultado de aplicar la función a cada elemento [93]. Mismo tamaño que el original.
- `filter()` crea un array nuevo con los elementos que cumplen la condición [94].
- `reduce()` recorre el array arrastrando un acumulador y devuelve un único valor [95]. El segundo argumento, aquí `0`, es el valor inicial del acumulador: ponedlo siempre.

Como `map` y `filter` devuelven arrays, se pueden **encadenar**:

```
const totalBaratosConIva = precios
    .filter((p) => p < 20)
    .map((p) => p * 1.21)
    .reduce((acc, p) => acc + p, 0);
```

5.1. ¿Qué método uso?



5.2. Ordenar con `sort`

```
[10, 9, 100].sort();                 // [10, 100, 9]   ✘ ordena como texto
[10, 9, 100].sort((a, b) => a - b);  // [9, 10, 100]   ✔ de menor a mayor
[10, 9, 100].sort((a, b) => b - a);  // [100, 10, 9]   ✔ de mayor a menor
```

`sort()` tiene dos trampas. La primera: sin argumentos, ordena los elementos como **texto**, según sus caracteres Unicode [96]. Para números hay que pasarle una función de comparación. La segunda: ordena el array **en el sitio**, modificando el original [97]. Si queréis conservarlo, ordenad una copia: `[...array].sort(…)`.

5.3. Otros dos útiles

- `join(separador)` une los elementos en un texto [20]: `['a', 'b'].join(', ')` → `'a, b'`.
- `slice(inicio, fin)` devuelve una copia de un trozo del array, sin incluir `fin`, y no modifica el original [98]. No lo confundáis con `splice`, que sí lo modifica.

5.4. ¿Modifica el original o no?

Modifican el array original	Devuelven algo nuevo
`push`, `pop`, `unshift`, `shift`, `splice`, `sort`	`map`, `filter`, `reduce`, `slice`, `join`, `find`, `includes`…



6. Objetos: fichas con nombre

En un array, los datos se identifican por su **posición**. En un objeto, por su **nombre**. Cuando los datos son distintos entre sí y cada uno tiene un significado, lo natural es un objeto [13].

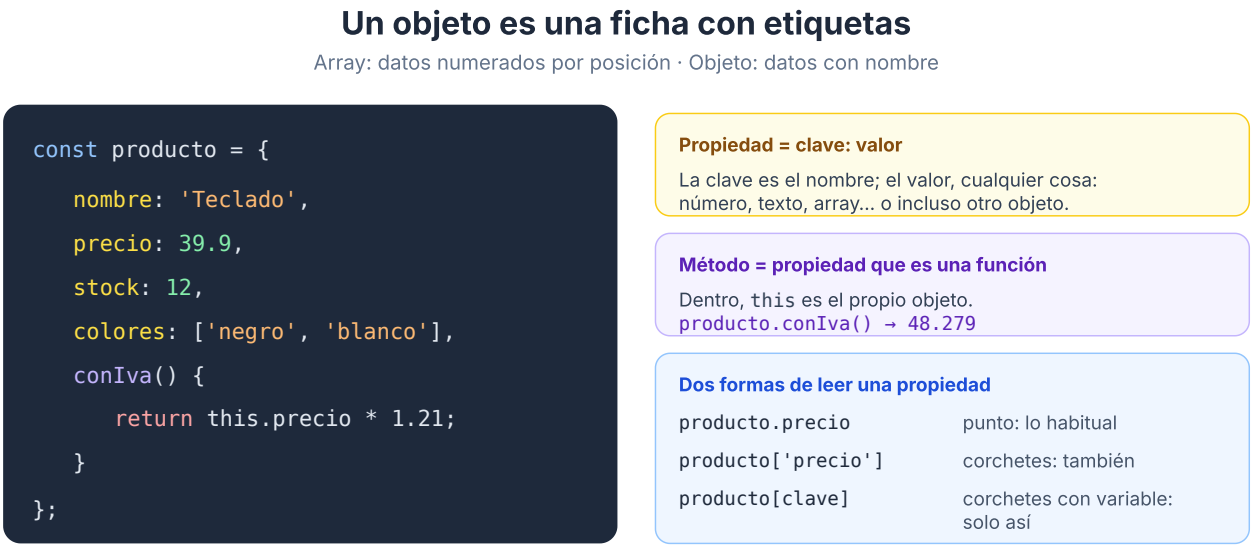


Figura 29 · Anatomía de un objeto

```
const producto = {
    nombre: 'Teclado',
    precio: 39.9,
    colores: ['negro', 'blanco'],
    conIva() {
        return this.precio * 1.21;
    },
};
```

Cada propiedad es un par `clave: valor`. Una propiedad cuyo valor es una función se llama **método** [13]. Dentro de un método, `this` se refiere al objeto desde el que se llama [99].

6.1. Leer, añadir, modificar y borrar

```
producto.precio          // leer con punto
producto['precio']       // leer con corchetes
producto[clave]          // corchetes con una variable: la única forma
producto.stock = 12;     // añadir (si no existía) o modificar
delete producto.colores; // borrar [24]
producto.peso            // undefined: no existe, pero no da error
```

Usad el punto siempre que podáis. Los corchetes, cuando el nombre de la propiedad está dentro de una variable.

6.2. Recorrer un objeto

```
for (const clave in producto) {
    console.log(clave, producto[clave]);
}

Object.keys(producto);   // ['nombre', 'precio', 'colores', 'conIva']
```

`for...in` recorre los nombres de las propiedades [100]. `Object.keys()` los devuelve en un array [101], con lo que podéis usar `length` o cualquier método de arrays.



💡 Consejo

`for...of` para arrays, `for...in` para objetos. Se parecen en el nombre, pero no son intercambiables.



7. Arrays de objetos

Combinad las dos estructuras y tendréis la forma en que se guardan casi todos los datos del mundo real: un array donde cada elemento es un objeto con las mismas propiedades. Es como una tabla: cada objeto es una fila y cada propiedad, una columna.

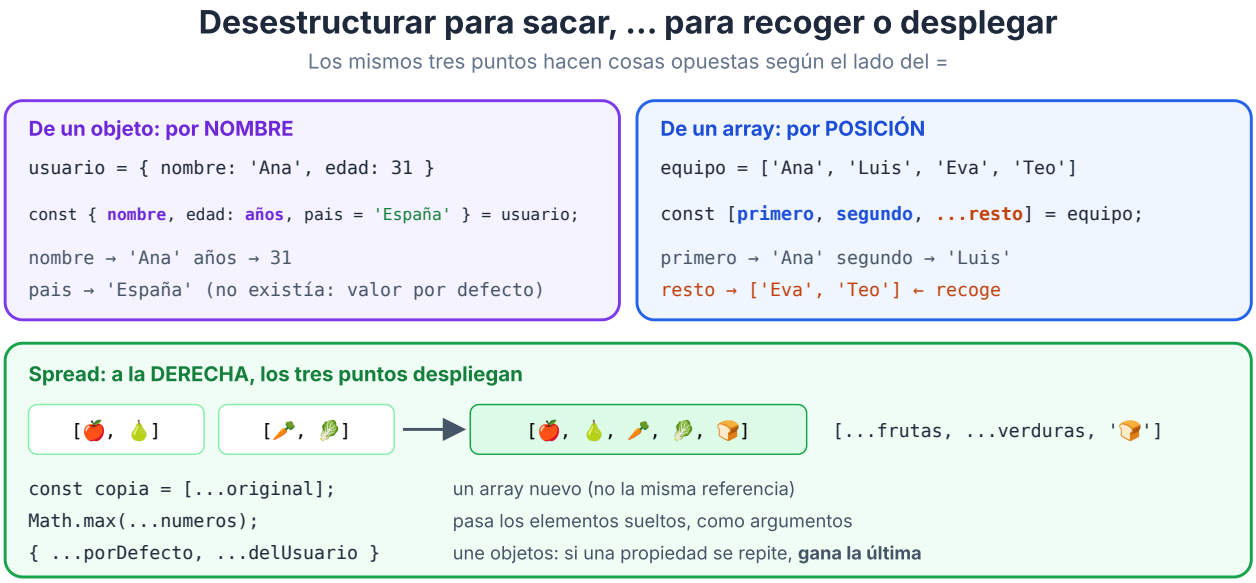
```
const productos = [
    { nombre: 'Teclado', categoria: 'periféricos', precio: 59.9, stock: 12 },
    { nombre: 'Monitor', categoria: 'monitores', precio: 219, stock: 0 },
    { nombre: 'Ratón', categoria: 'periféricos', precio: 24.5, stock: 8 },
];

productos.filter((p) => p.categoria === 'periféricos');     // los de una categoría
productos.find((p) => p.nombre === 'Monitor');              // uno concreto
productos.map((p) => p.nombre);                             // ['Teclado', 'Monitor',
'Ratón']
productos.reduce((acc, p) => acc + p.precio * p.stock, 0);  // valor del inventario
[...productos].sort((a, b) => a.precio - b.precio);         // ordenados por precio
```

Todo lo de los apartados anteriores funciona igual. La única diferencia es que, dentro del *callback*, cada elemento es un objeto y accedéis a sus propiedades con el punto. Esta es la estructura que os devolverán las API del bloque 11.



8. Desestructuración y *spread*



📊 Figura 30 · Desestructuración, resto y spread

8.1. Desestructuración: sacar datos

Permite extraer valores de arrays o propiedades de objetos en variables, en una sola línea [27]:

```
// de un objeto: por NOMBRE
const { nombre, ciudad, edad: edadUsuario, pais = 'España' } = usuario;

// de un array: por POSICIÓN
const [primero, segundo, ...resto] = participantes;

// en los parámetros de una función
function presentar({ nombre, ciudad }) {
    return `${nombre} vive en ${ciudad}`;
}
```

Con `edad: edadUsuario` renombráis la variable; con `pais = 'España'` le dais un valor por defecto, que se usa si la propiedad no existe o vale `undefined` [102].

8.2. *Spread*: desplegar

Los tres puntos delante de un array u objeto lo despliegan en sus elementos [28]:

```
const cesta = [...frutas, ...verduras, 'pan'];          // unir arrays
const copia = [...original];                            // copiar un array
Math.max(...numeros);                                   // pasar elementos sueltos
const config = { ...porDefecto, ...delUsuario };        // unir objetos: gana el último
```



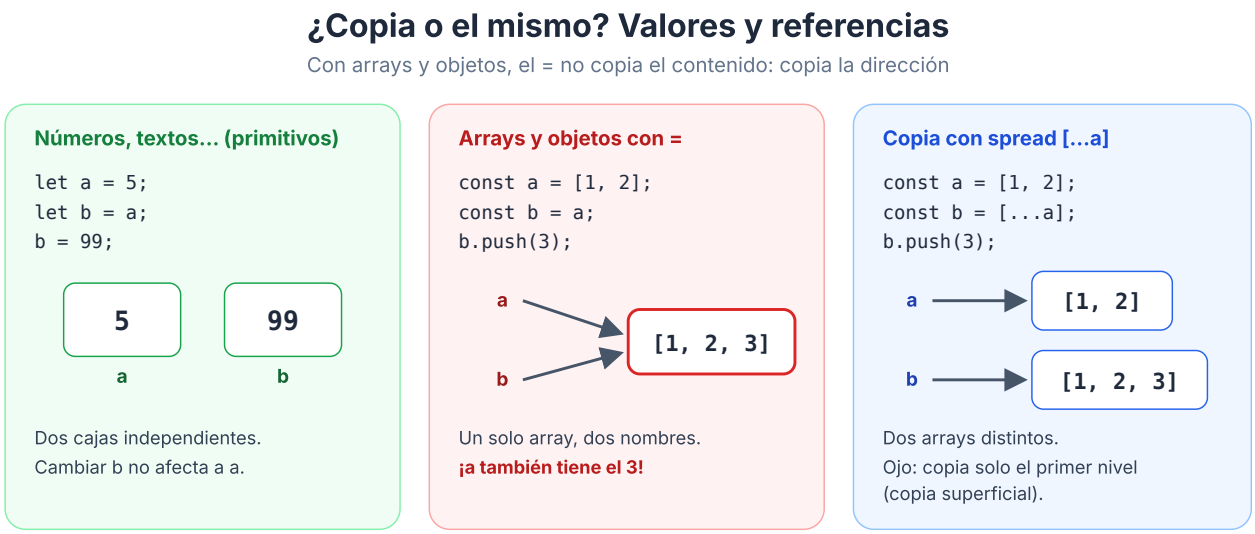
📝 **Nota**

Los mismos tres puntos hacen cosas opuestas según el lado. **A la izquierda** del `=` o en los parámetros, **recogen** lo que sobra: son el *resto*. **A la derecha**, **despliegan**: son el *spread*.



9. Valores y referencias

Este apartado explica muchos errores misteriosos, así que leedlo con calma.



📊 Figura 31 · Valores y referencias

Con los tipos primitivos, `=` copia el valor: tenéis dos cajas independientes. Con arrays y objetos no. La variable no guarda el contenido, sino una **referencia** que indica dónde está. Al hacer `=`, se copia la referencia, y las dos variables apuntan al **mismo** array.

```
const a = [1, 2];
const b = a;        // b es otro nombre para el MISMO array
b.push(3);
console.log(a);     // [1, 2, 3]  ¡a también ha cambiado!
```

Tres consecuencias:

1. **Para copiar, usad *spread*:** `const b = [...a];` o `const copia = { ...objeto };`.

2. `const` **no impide cambiar el contenido.** Impide que la variable apunte a otro array u objeto, pero no que modifiquéis el que ya tiene. Por eso `const compra = []` admite `push`.
3. `===` **compara referencias**, no contenido: `[1, 2] === [1, 2]` es `false`, porque son dos arrays distintos.

9.1. Copia superficial y profunda

El *spread* hace una **copia superficial**: copia el primer nivel, pero si una propiedad es a su vez un array u objeto, solo copia su referencia, y el original y la copia la comparten [103]. Para una copia completamente independiente, nivel a nivel, usad `structuredClone()`, que hace una **copia profunda** [104].

```
const partida = { nivel: 3, inventario: ['espada'] };

const superficial = { ...partida };
superficial.inventario.push('llave');   // ⚠ también cambia partida.inventario

const profunda = structuredClone(partida);
profunda.inventario.push('gema');       // ✔ partida no se entera
```



10. JSON: datos como texto

JSON es un formato de texto para representar datos. Es el idioma en el que las aplicaciones web se intercambian información, y lo vais a usar para guardar datos en el navegador (bloque 09) y para recibirlos de un servidor (bloque 11).

```
const texto = JSON.stringify(partida, null, 2);   // objeto → texto
const objeto = JSON.parse(texto);                 // texto → objeto nuevo
```

`JSON.stringify()` convierte un valor de JavaScript en un texto JSON [105]; los argumentos `null, 2` lo formatean con sangría para que se lea mejor. `JSON.parse()` hace el camino contrario y construye un objeto nuevo [106].

```
{
  "jugador": "Aria",
  "nivel": 3,
  "inventario": ["espada", "poción"]
}
```

En JSON las claves van **siempre entre comillas dobles** y no puede haber métodos, solo datos.



11. La plantilla de trabajo en este bloque

La estructura del programa principal se llena ahora con los **datos** con los que va a trabajar el programa. El procesamiento se apoya en los métodos de arrays:

```
// estructura del programa principal
const productos = [
    { nombre: 'Teclado', precio: 59.9, stock: 12 },
    { nombre: 'Ratón', precio: 24.5, stock: 0 },
];

// funciones
const formatearEuros = (n) => `${n.toFixed(2)} €`;

// constantes al DOM
const salida = document.querySelector('#salida');

// entradas
const presupuesto = Number(prompt('¿Cuánto quieres gastar?', '50'));

// procesamiento
const asequibles = productos
    .filter((p) => p.stock > 0 && p.precio <= presupuesto)
    .map((p) => `${p.nombre} (${formatearEuros(p.precio)})`);

// salidas
salida.textContent = asequibles.join(', ') || 'Nada que se ajuste a tu presupuesto';
```

En el bloque 06 dejaréis de mostrar los resultados como un texto unido con `join`: aprenderéis a **crear elementos HTML** a partir de un array. Todo lo de este bloque es la base para eso.



12. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	La lista de la compra	Índices, `length`, `push`, `pop`, `unshift`, `shift`, `indexOf` + `splice`	Ver
02	Buscador de alumnos	`includes`, `indexOf`, `find`, `findIndex`, `every`, arrays paralelos	Ver
03	Una semana de temperaturas	`for`, `for...of` y `forEach`: cuándo usar cada uno	Ver
04	map, filter, reduce y sort	Transformar, seleccionar, resumir, encadenar y ordenar números	Ver
05	La ficha de un producto	Objetos, métodos, `this`, punto y corchetes, `delete`, `for...in`, `Object.keys`	Ver
06	Catálogo de una tienda	Arrays de objetos: filtrar, buscar, totalizar y ordenar por propiedad	Ver
07	Desestructuración y spread	Sacar datos de objetos y arrays, unir arrays y objetos	Ver
08	Guardar y cargar la partida	Referencias, copias superficiales y profundas, JSON	Ver

El ejercicio 06 es el más importante del bloque. Si lo resolvéis sin mirar la solución, tenéis lo necesario para trabajar con datos reales.



🧪 Los ejercicios, explicados

Ejercicio 01 · La lista de la compra

> 🎯 **Objetivo:** Crear un array, leer sus elementos por índice, conocer su longitud y modificarlo por los extremos (push, pop, unshift, shift) y por el medio (indexOf + splice).
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Crear un array y modificarlo por el principio, por el final y por el medio, registrando su estado después de cada cambio.



La solución paso a paso

1. Crear el array y leerlo

```
const compra = ['pan', 'leche', 'huevos'];
compra[0]                    // 'pan'
compra[compra.length - 1]    // 'huevos'
```

Un array es una lista ordenada de valores a la que se accede por su **índice**, que empieza en 0 [79]. `length` indica cuántos elementos tiene [80]. Por eso el último está en `length - 1`: con tres elementos, los índices son 0, 1 y 2. Tenéis el dibujo en la infografía de la teoría.

¿Y por qué `const`, si el array va a cambiar? Porque `const` impide **reasignar** la variable (`compra = otraCosa`), pero no impide cambiar el **contenido** del array. Lo veréis con más detalle en el ejercicio 08.

2. Los cuatro métodos de los extremos

Método	Dónde	Qué hace	Devuelve
`push(x)`	Final	Añade	La nueva longitud [81]
`pop()`	Final	Quita	El elemento quitado [82]
`unshift(x)`	Principio	Añade	La nueva longitud [83]
`shift()`	Principio	Quita	El elemento quitado [84]

Fijaos en que `pop()` y `shift()` **devuelven** lo que quitan. Por eso puedo guardarlo en una constante y decir en el registro qué ha salido.

3. Quitar por el medio: `indexOf` + `splice`

```
const posicion = compra.indexOf('leche');
if (posicion !== -1) {
    compra.splice(posicion, 1);
}
```

Para quitar un elemento concreto necesito dos pasos:

1. `indexOf()` busca el elemento y devuelve su posición, o `-1` si no está [87].
2. `splice(posicion, 1)` quita, a partir de esa posición, un elemento [85].

¿Por qué el `if`? Porque `splice(-1, 1)` no daría error: quitaría el **último** elemento, ya que un índice negativo cuenta desde el final [85]. Si «leche» no estuviera, borraríais otra cosa sin daros cuenta.

4. ¿Está o no está?

```
compra.includes('pan')   // true o false
```

`includes()` responde directamente sí o no [86]. Si solo necesitáis saber **si** está, usad `includes`; si necesitáis saber **dónde**, `indexOf`.



Con «fruta»

```
Lista inicial                        → [pan, leche, huevos]
push('fruta')                        → [pan, leche, huevos, fruta]
unshift('café')                      → [café, pan, leche, huevos, fruta]
pop() → ha quitado 'fruta'           → [café, pan, leche, huevos]
shift() → ha quitado 'café'          → [pan, leche, huevos]
splice(1, 1) → ha quitado 'leche'    → [pan, huevos]
```

Si añadís «leche», `indexOf` encuentra **la primera** que aparece y solo quita esa. La otra sigue en la lista.

Ejercicio 02 · Buscador de alumnos

> 🎯 **Objetivo:** Buscar dentro de un array: saber si un valor está (includes), en qué posición (indexOf), encontrar el primer elemento que cumple una condición (find) y su posición (findIndex), y comprobar si todos la cumplen (every).
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Buscar información en dos arrays paralelos usando los métodos de búsqueda.



Buscar un valor concreto

```
alumnos.includes(nombre)   // ¿está? → true / false
alumnos.indexOf(nombre)    // ¿dónde? → posición o -1
```

`includes()` responde sí o no [86]; `indexOf()` devuelve la posición del primer elemento igual al que buscáis, o `-1` si no está [87]. Los dos comparan con igualdad estricta, como `===`.

Por eso **«marta» no encuentra a «Marta»**: para JavaScript son textos distintos. Si quisierais una búsqueda que ignore mayúsculas, tendríais que convertir los dos lados a minúsculas antes de comparar. Lo dejo como reto.

Arrays paralelos

```
const posicion = alumnos.indexOf(nombre);
const nota = notas[posicion];
```

Como Marta está en la posición 2 de `alumnos`, su nota está en la posición 2 de `notas`. Funciona... pero es frágil: si alguien añade un alumno y se olvida de añadir su nota, todo se descuadra. En el ejercicio 06 lo resolveréis de la forma buena, con un **array de objetos**.



Buscar por condición

```
notas.find((n) => n < 5)        // 4
notas.findIndex((n) => n < 5)   // 1
```

Aquí ya no buscáis un valor exacto, sino **el primero que cumpla algo**. Por eso reciben un *callback*, como los del bloque 04: una función que se ejecuta con cada elemento y devuelve `true` o `false`.

Método	Devuelve	Si nadie cumple
`find(callback)`	El **valor** del primer elemento que cumple [88]	`undefined` [88]
`findIndex(callback)`	La **posición** del primer elemento que cumple [89]	`-1` [89]

Las dos se detienen en cuanto encuentran uno. Pablo también suspende, pero `find` ni lo mira: ya había encontrado a Luis.



Preguntar por todos

```
notas.every((n) => n >= 0 && n <= 10)
```

`every()` devuelve `true` si **todos** los elementos cumplen la condición [90]. Muy útil para validar una lista entera de una vez.



Resumen de búsquedas

Quiero saber...	Método
Si un valor está	`includes(valor)`
Dónde está un valor	`indexOf(valor)`
El primero que cumple algo	`find(callback)`
Dónde está el primero que cumple algo	`findIndex(callback)`
Si todos cumplen algo	`every(callback)`

Ejercicio 03 · Una semana de temperaturas

> 🎯 **Objetivo:** Recorrer un array con las tres formas habituales (for clásico, for...of y forEach) y entender cuándo conviene cada una según necesitemos el índice o solo el valor.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Recorrer el mismo array de tres formas y elegir cada una según lo que hace falta.



1. `for` clásico: cuando necesitáis el índice

```
for (let i = 0; i < temperaturas.length; i++) {
    parte += `${dias[i]}: ${temperaturas[i]} °C\n`;
}
```

Es el `for` del bloque 03, ahora con `i` como índice del array. Empieza en `0` y la condición es `i < temperaturas.length`, con `<` y no con `<=`: el último índice válido es `length - 1` [80].

Aquí necesito el índice porque leo **dos** arrays a la vez, `dias[i]` y `temperaturas[i]`.



> 🚨 **Atención**
>
> Si ponéis `i <= temperaturas.length`, la última vuelta lee `temperaturas[7]`, que no existe, y obtenéis `undefined`. Es el error clásico de «uno de más».



2. `for...of`: cuando solo necesitáis el valor

```
for (const temperatura of temperaturas) {
    suma += temperatura;
}
```

`for...of` recorre directamente los **valores** del array [91]. No hay índice, ni condición, ni `i++` que vigilar: menos sitios donde equivocarse. Fijaos en que puedo usar `const`, porque en cada vuelta se crea una constante nueva.



3. `forEach`: un *callback* por cada elemento

```
temperaturas.forEach((temperatura, indice) => {
    …
});
```

`forEach()` ejecuta la función que le pasáis una vez por cada elemento, y le entrega el valor y el índice [92]. Es un *callback*, igual que en el bloque 04: vosotros escribís qué hacer con cada elemento y `forEach` se encarga de recorrer.

Dos detalles:

- `forEach` devuelve `undefined` [92]. Sirve para **hacer** algo con cada elemento, no para obtener un resultado. Para eso están `map`, `filter` y `reduce`, que veréis en el ejercicio 04.
- Dentro de `forEach` no podéis usar `break`. Si necesitáis parar a mitad, usad `for` o `for...of`.



¿Cuál uso?

Necesito…	Uso
Solo los valores	`for...of`
Valor e índice, de forma cómoda	`forEach((valor, indice) => …)`
Control total: saltos, recorrer al revés, parar con `break`	`for` clásico



Los resultados

- Media: 20.3 °C.
- Máxima: 25 °C el miércoles.
- Días de más de 22 °C: 2 (miércoles y jueves).

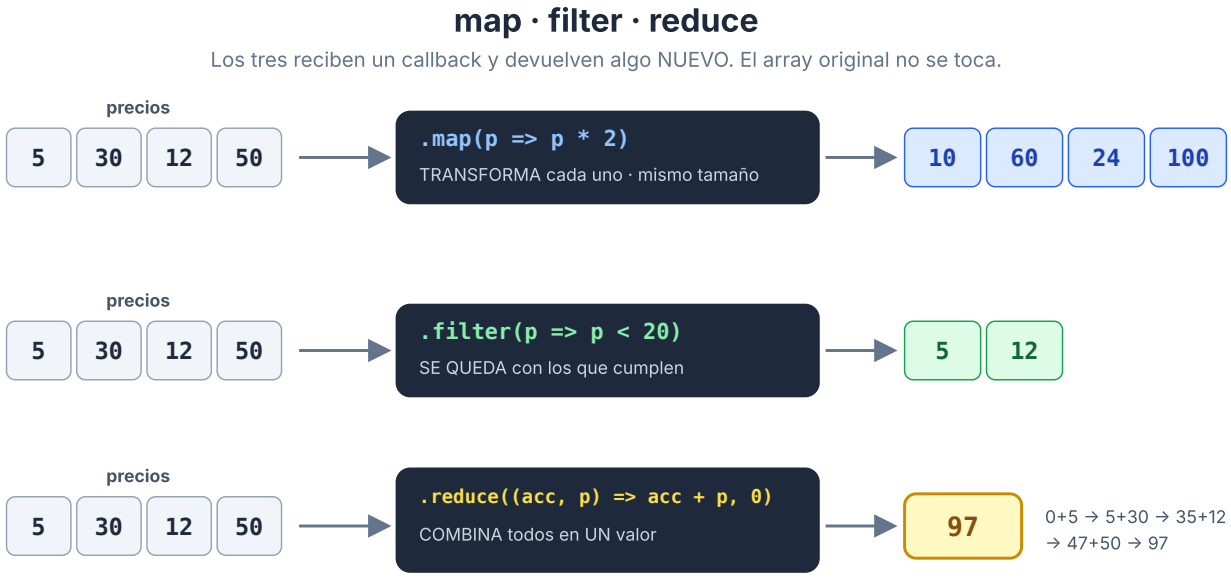
Ejercicio 04 · map, filter, reduce y sort

> 🎯 **Objetivo:** Usar los métodos que devuelven un resultado nuevo a partir de un array: map para transformar, filter para seleccionar, reduce para resumir, encadenarlos, y ordenar números correctamente con sort sin modificar el original.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Transformar, seleccionar, resumir y ordenar un array de precios sin modificar el original.



📊 Figura 32 · map, filter y reduce



Los tres grandes

`map` : transformar

```
const conIva = precios.map((p) => p * (1 + IVA));
```

`map()` crea un **array nuevo** con el resultado de aplicar la función a cada elemento [93]. Siempre tiene el mismo tamaño que el original: entran seis precios, salen seis precios con IVA.

`filter` : seleccionar

```
const baratos = precios.filter((p) => p < 20);
```

`filter()` crea un **array nuevo** con los elementos para los que la función devuelve `true` [94]. Si ninguno cumple, devuelve un array vacío [94].

`reduce`: resumir

```
const total = precios.reduce((acc, p) => acc + p, 0);
```

`reduce()` recorre el array arrastrando un **acumulador**, y al final devuelve un único valor [95]. Es el patrón del acumulador del bloque 03, empaquetado:

Vuelta	`acc`	`p`	Devuelve
1	0	12.5	12.5
2	12.5	30	42.5
3	42.5	8	50.5
…	…	…	…
6	115.49	5	**120.49**

El `0` del final es el valor inicial del acumulador. Ponedlo siempre.



Encadenar

```
precios
    .filter((p) => p < 20)
    .map((p) => p * (1 + IVA))
    .reduce((acc, p) => acc + p, 0);
```

Como `filter` y `map` devuelven arrays, podéis llamar al siguiente método directamente sobre el resultado. Se lee como una receta: «de los precios, quédate con los menores de 20, súmales el IVA y súmalos todos». Escribid un método por línea; así se lee de un vistazo.



`sort`: dos trampas

Trampa 1: ordena como texto

```
[...precios].sort()                 // 12.50 · 19.99 · 30.00 · 45.00 · 5.00 · 8.00  ✘
[...precios].sort((a, b) => a - b)  // 5.00 · 8.00 · 12.50 · 19.99 · 30.00 · 45.00  ✔
```

Sin argumentos, `sort()` convierte los elementos en texto y los ordena según sus caracteres Unicode [96]. Igual que en un diccionario, «12» va antes que «5» porque el «1» va antes que el «5». Para ordenar números hay que pasarle una **función de comparación**: `(a, b) => a - b` ordena de menor a mayor y `(a, b) => b - a`, de mayor a menor.

Trampa 2: modifica el original

`sort()` ordena el array **en el sitio** y devuelve una referencia al mismo array [97]. A diferencia de `map` y `filter`, cambia el original. Por eso ordeno una copia: `[...precios]`. Los tres puntos son la

sintaxis *spread*, que veréis a fondo en el ejercicio 07.



Qué tenéis que observar

- La primera y la última fila de la tabla son idénticas: `map`, `filter`, `reduce` y las copias ordenadas no han tocado `precios`.
- Total: 120.49 €. Total con IVA de los menores de 20 €: 55.04 €.

Ejercicio 05 · La ficha de un producto

> 🎯 **Objetivo:** Crear un objeto literal con propiedades y un método, leer propiedades con punto y con corchetes (también con una clave guardada en una variable), añadir, modificar y borrar propiedades, y recorrerlo con for...in y Object.keys().
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Crear un objeto con varios datos y un método, consultarlo, modificarlo y recorrer sus propiedades.



Arrays frente a objetos

En un array, los datos se identifican por su **posición**: `notas[2]`. En un objeto, por su **nombre**: `producto.precio`. Cuando los datos son distintos entre sí (un nombre, un precio, un stock...) y cada uno tiene un significado, lo natural es un objeto [13].

Un objeto es una ficha con etiquetas

Array: datos numerados por posición · Objeto: datos con nombre

```
const producto = {
   nombre: 'Teclado',
   precio: 39.9,
   stock: 12,
   colores: ['negro', 'blanco'],
   conIva() {
      return this.precio * 1.21;
   }
};
```

Propiedad = clave: valor

La clave es el nombre; el valor, cualquier cosa: número, texto, array... o incluso otro objeto.

Método = propiedad que es una función

Dentro, `this` es el propio objeto.
`producto.conIva() → 48.279`

Dos formas de leer una propiedad

`producto.precio`	punto: lo habitual
`producto['precio']`	corchetes: también
`producto[clave]`	corchetes con variable: solo así

Figura 33 · Un objeto



La solución paso a paso

1. Crear el objeto

```
const producto = {
    nombre: 'Teclado mecánico',
    precio: 59.9,
    stock: 12,
    colores: ['negro', 'blanco'],
    conIva() {
        return this.precio * 1.21;
    },
};
```

Cada línea es una propiedad `clave: valor`, separadas por comas. Los valores pueden ser de cualquier tipo, incluido un array. `conIva` es un **método**: una propiedad cuyo valor es una función [13]. Dentro de un método, `this` se refiere al objeto desde el que se llama [99], así que `this.precio` es el precio de este producto.

2. Punto o corchetes

```
producto.precio        // punto: lo habitual
producto['precio']     // corchetes con el nombre entre comillas
producto[clave]        // corchetes con una variable
```

Las tres leen la misma propiedad. ¿Para qué los corchetes? Para cuando el nombre de la propiedad **está dentro de una variable**, como aquí, donde lo escribe el usuario. `producto.clave` buscaría una propiedad que se llama literalmente «clave» [13].

Si la propiedad no existe, no hay error: se obtiene `undefined`. Por eso lo compruebo.

3. Modificar, añadir y borrar

```
producto.stock -= 2;        // modificar
producto.oferta = true;     // añadir
delete producto.colores;    // borrar
```

Para añadir una propiedad basta con asignarla. Para borrarla está el operador `delete` [107]. Y sí, el objeto se declaró con `const` y aun así lo cambio: `const` impide reasignar la variable, no modificar el contenido del objeto. Os lo explico a fondo en el ejercicio 08.

4. Recorrer el objeto

```
for (const prop in producto) {
    ficha += `${prop}: ${producto[prop]}\n`;
}
```

`for...in` recorre los **nombres** de las propiedades de un objeto [100]. En cada vuelta, `prop` vale `'nombre'`, `'precio'`... y con `producto[prop]` (corchetes, porque el nombre está en una variable) obtengo el valor.

```
Object.keys(producto)   // ['nombre', 'precio', 'stock', 'conIva', 'oferta']
```

`Object.keys()` devuelve un **array** con los nombres de las propiedades [101]. Al ser un array, podéis usar `.length` para contarlas o cualquier método del ejercicio 04.



💡 Consejo

`for...in` para objetos, `for...of` para arrays. Se parecen mucho en el nombre, pero no los mezcléis: `for...of` sobre un objeto normal da error, y `for...in` sobre un array os da los índices como texto.



Qué tenéis que observar

- La ficha ya no tiene `colores`, tiene `oferta: true` y el stock es 10.
- El precio con IVA es 72.48 €.

Ejercicio 06 · Catálogo de una tienda

🎯 **Objetivo:** Trabajar con un array de objetos combinando todo lo anterior: filtrar por una propiedad, buscar un objeto concreto, transformar a un array de textos, calcular un total y ordenar por una propiedad.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Consultar un catálogo guardado como array de objetos: filtrar, buscar, transformar, totalizar y ordenar.



El array de objetos

```
const productos = [
    { nombre: 'Teclado mecánico', categoria: 'periféricos', precio: 59.9, stock: 12 },
    …
];
```

Cada elemento del array es un objeto con las mismas propiedades. Es como una tabla: cada objeto es una **fila** y cada propiedad, una **columna**. Esta estructura es la que os devolverán casi todas las API del bloque 11, así que conviene que os resulte familiar.

¿Os acordáis de los arrays paralelos del ejercicio 02? Aquí ese problema desaparece: el nombre, el precio y el stock de cada producto viajan juntos en el mismo objeto. No se pueden descuadrar.



Los métodos de siempre, con objetos

La única diferencia con el ejercicio 04 es que, dentro del *callback*, cada elemento es un objeto, así que accedéis a sus propiedades con el punto: `p.categoria`, `p.precio`...

Necesito	Código	Devuelve
Los de una categoría	`filter((p) => p.categoria === categoria)`	Array de objetos [94]
Uno concreto	`find((p) => p.nombre === nombre)`	Un objeto o `undefined` [88]
Solo los nombres	`map((p) => p.nombre)`	Array de textos [93]
Valor del inventario	`reduce((acc, p) => acc + p.precio * p.stock, 0)`	Un número [95]
Ordenar por precio	`sort((a, b) => a.precio - b.precio)`	El mismo array, ordenado [96]

Ordenar por una propiedad

```
[...productos].sort((a, b) => a.precio - b.precio)
```

El comparador recibe dos **objetos** y compara la propiedad que os interesa. Como en el ejercicio 04, ordeno una copia para no alterar el catálogo original.

`slice`: quedarse con un trozo

```
.slice(0, 3)
```

`slice(inicio, fin)` devuelve una **copia** de una parte del array, desde `inicio` hasta `fin` sin incluirlo [98]. No confundir con `splice`, que **modifica** el original.



Dos atajos con `||`

```
listado || 'No hay productos en esa categoría.'
agotados.join(', ') || 'ninguno'
```

Si el texto está vacío, es *falsy* y se usa el de la derecha. Lo visteis en el bloque 04.



Con «periféricos» y «Monitor 27"»

```
• Teclado mecánico — 59.90 € (12 uds.)
• Ratón inalámbrico — 24.50 € (0 uds.)
• Alfombrilla XL — 15.95 € (30 uds.)
```

Pregunta	Respuesta
Producto buscado	Monitor 27": 219.00 €
Agotados	Ratón inalámbrico, Monitor 24"
Valor del inventario	3204.23 €
Los tres más baratos	Alfombrilla XL, Ratón inalámbrico, Altavoces

Ejercicio 07 · Desestructuración y spread

> 🎯 **Objetivo:** Extraer valores de objetos y arrays con desestructuración (con valores por defecto, renombrado y resto) y crear arrays y objetos nuevos combinando otros con la sintaxis spread.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Sacar datos de objetos y arrays con desestructuración, y construir arrays y objetos nuevos con la sintaxis *spread*.



Desestructuración: sacar

La desestructuración permite extraer valores de arrays o propiedades de objetos y guardarlos en variables, todo en una línea [102].

De un objeto: por nombre

```
const { nombre, ciudad, edad: edadUsuario, pais = 'España' } = usuario;
```

Es lo mismo que escribir:

```
const nombre = usuario.nombre;
const ciudad = usuario.ciudad;
const edadUsuario = usuario.edad;
const pais = usuario.pais === undefined ? 'España' : usuario.pais;
```

Sintaxis	Qué hace
`nombre`	Saca la propiedad `nombre` a una constante `nombre`
`edad: edadUsuario`	Saca `edad`, pero la llama `edadUsuario`
`pais = 'España'`	Si no existe o vale `undefined`, usa el valor por defecto [102]

Fijaos en que el **orden no importa**: con objetos, manda el nombre de la propiedad.

De un array: por posición

```
const [oro, plata, bronce, ...resto] = participantes;
// oro = 'Ana', plata = 'Luis', bronce = 'Marta', resto = ['Pablo', 'Hugo']
```

Con arrays, manda la **posición**. Y `...resto` recoge todo lo que sobra en un array nuevo. Tiene que ir siempre al final.

Un truco: intercambiar variables

```
[a, b] = [b, a];
```

A la derecha se crea un array con los valores intercambiados, y a la izquierda se desestructura sobre las mismas variables. Sin variable auxiliar.

En los parámetros de una función

```
function presentar({ nombre, ciudad }) { … }
presentar(usuario);
```

La función recibe un objeto y se queda directamente con las propiedades que le interesan. Lo veréis muchísimo en código moderno.



Spread: meter

Los tres puntos `...` delante de un array o un objeto lo **despliegan** en sus elementos [108].

```
const cesta = [...frutas, ...verduras, 'pan'];
// ['manzana', 'pera', 'tomate', 'lechuga', 'pan']

Math.max(...numeros);   // Math.max(14, 3, 27, 8) → 27
```

`Math.max` espera los números sueltos, no un array. El *spread* los saca del array.

Combinar objetos

```
const config = { ...configPorDefecto, ...configUsuario };
// { tema: 'oscuro', idioma: 'es', tamanoLetra: 18 }
```

Se copian las propiedades del primero y luego las del segundo. Si una clave se repite, **gana la última**. Es el patrón perfecto para «valores por defecto que el usuario puede cambiar».



📝 Nota

Los mismos tres puntos hacen cosas opuestas según dónde estén. **A la izquierda del `=`** (o en los parámetros), recogen: son el *resto*. **A la derecha**, despliegan: son el *spread*.

Ejercicio 08 · Guardar y cargar la partida

🎯 **Objetivo:** Entender la diferencia entre copiar un valor y copiar una referencia, hacer copias superficiales con spread y profundas con structuredClone(), y convertir objetos a texto JSON y de vuelta con JSON.stringify() y JSON.parse().

📂 Enunciado · Solución completa



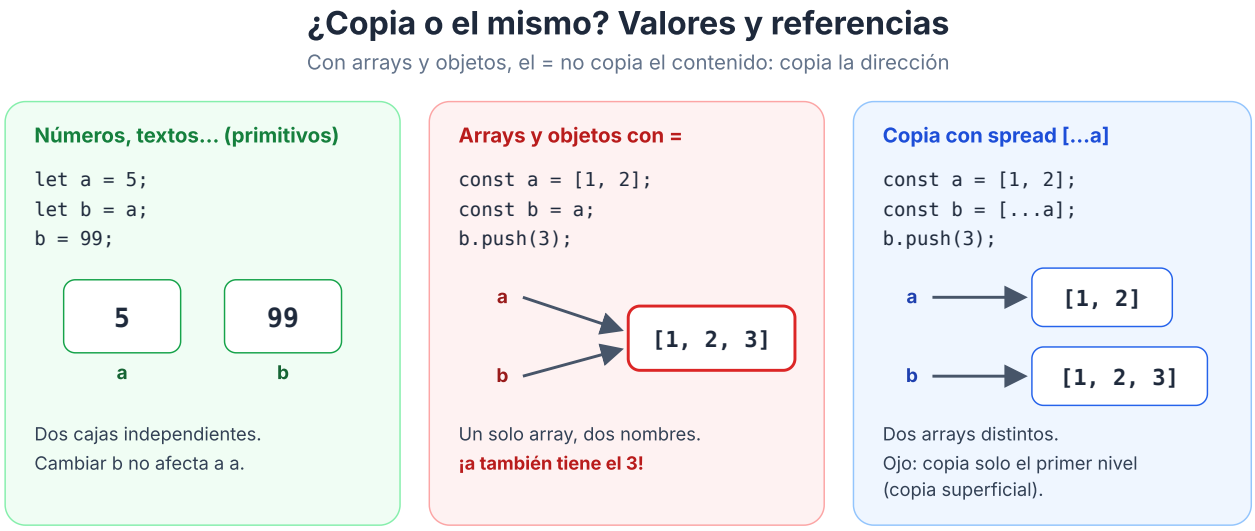
Qué pedía el ejercicio

Comprobar con seis experimentos cuándo copiar un array o un objeto crea algo nuevo y cuándo no, y guardar y cargar un objeto como texto JSON.



Valores y referencias

Con los tipos primitivos (números, textos, booleanos), `=` copia el valor: tenéis dos cajas independientes. Con arrays y objetos no: la variable no guarda el contenido, sino una **referencia**, una especie de dirección que indica dónde está. Al hacer `=`, lo que se copia es esa dirección.



📊 Figura 34 · Valores y referencias



Los seis experimentos

Nº	Código	Resultado	Por qué
1	`alias = inventario`	El original **cambia**	Dos nombres para el mismo array
2	`copia = [...inventario]`	El original no cambia	*Spread* crea un array nuevo
3	`{ ...partida }` y cambio `nivel`	El original no cambia	El primer nivel se copia
4	...y cambio `inventario`	El original **cambia**	El array de dentro se comparte
5	`structuredClone(partida)`	El original no cambia	Copia profunda
6	`[1, 2] === [1, 2]`	`false`	Son dos arrays distintos

Copia superficial frente a profunda

El *spread* hace una **copia superficial**: copia el primer nivel, pero si una propiedad es un array u objeto, copia solo su referencia, y los dos acaban compartiéndolo [103]. Por eso el experimento 3 funciona y el 4 no.

`structuredClone()` hace una **copia profunda**: copia también todo lo que hay dentro, nivel a nivel [104]. Usadla cuando el objeto tenga arrays u objetos dentro y necesitéis una copia totalmente independiente.

`const` no significa «no cambia»

Ahora entendéis por qué `const inventario` admite un `push`. `const` impide que la variable apunte a **otro** array (`inventario = []` da error), pero no impide modificar el array al que apunta.

Comparar arrays y objetos

`===` con arrays y objetos compara **referencias**, no contenido. Dos arrays con los mismos elementos son dos arrays distintos, así que la comparación da `false`. Para saber si tienen el mismo contenido hay que comparar elemento a elemento.



JSON: guardar y cargar

JSON es un formato de **texto** para representar datos. Es el idioma en el que las aplicaciones web se intercambian información, y lo veréis en los bloques 09 y 11.

```
const guardado = JSON.stringify(partida, null, 2);   // objeto → texto
const cargada = JSON.parse(guardado);                 // texto → objeto
```

- `JSON.stringify()` convierte un valor de JavaScript en un texto JSON [105]. Los argumentos `null, 2` piden que lo formatee con sangría de 2 espacios, para que se lea bien.
- `JSON.parse()` analiza un texto JSON y construye el valor que describe [106].

```
{
  "jugador": "Aria",
  "nivel": 3,
  "inventario": [
    "espada",
    "poción",
    "llave"
  ]
}
```

Fijaos en las diferencias con un objeto de JavaScript: en JSON las claves van **siempre entre comillas dobles**, y no hay métodos, solo datos.

¿Y por qué `cargada === partida` es `false`? Porque `JSON.parse` construye un objeto **nuevo**. Tiene el mismo contenido, pero es otro objeto, igual que en el experimento 6.



💡 Consejo

`JSON.parse(JSON.stringify(objeto))` también sirve para hacer una copia profunda, y lo veréis en mucho código antiguo. `structuredClone()` es la forma moderna y más completa de hacerlo.



Bloque 06

Bloque 06 · El DOM

Empieza la fase 2. Hasta ahora, JavaScript y la página se hablaban poco: un `textContent` por aquí, un `querySelector` por allá. En este bloque os voy a enseñar a **controlar la página entera desde JavaScript**: encontrar cualquier elemento, cambiar su texto, sus atributos y su aspecto, y sobre todo **crear elementos nuevos a partir de datos**. Lo que en el bloque 05 mostrabais con un `join`, a partir de hoy serán tarjetas, listas y tablas de verdad.

Al terminar este bloque sabréis:

- Explicar qué es el DOM y cómo se relaciona con vuestro HTML.
- Seleccionar uno o varios elementos con selectores CSS.
- Moveros por el árbol: padres e hijos.
- Cambiar textos, HTML interno y atributos, incluidos los `data-*`.
- Cambiar el aspecto con `classList` y saber cuándo usar `style`.
- Crear, añadir, mover y eliminar elementos.
- Generar HTML a partir de un array de datos con funciones reutilizables.



1. Qué es el DOM

Cuando el navegador carga una página, lee el HTML y construye en memoria una representación de ella: un **árbol de objetos** llamado DOM, *Document Object Model* [14]. Cada etiqueta se convierte en un nodo, y los nodos quedan unos dentro de otros, igual que en vuestro código.

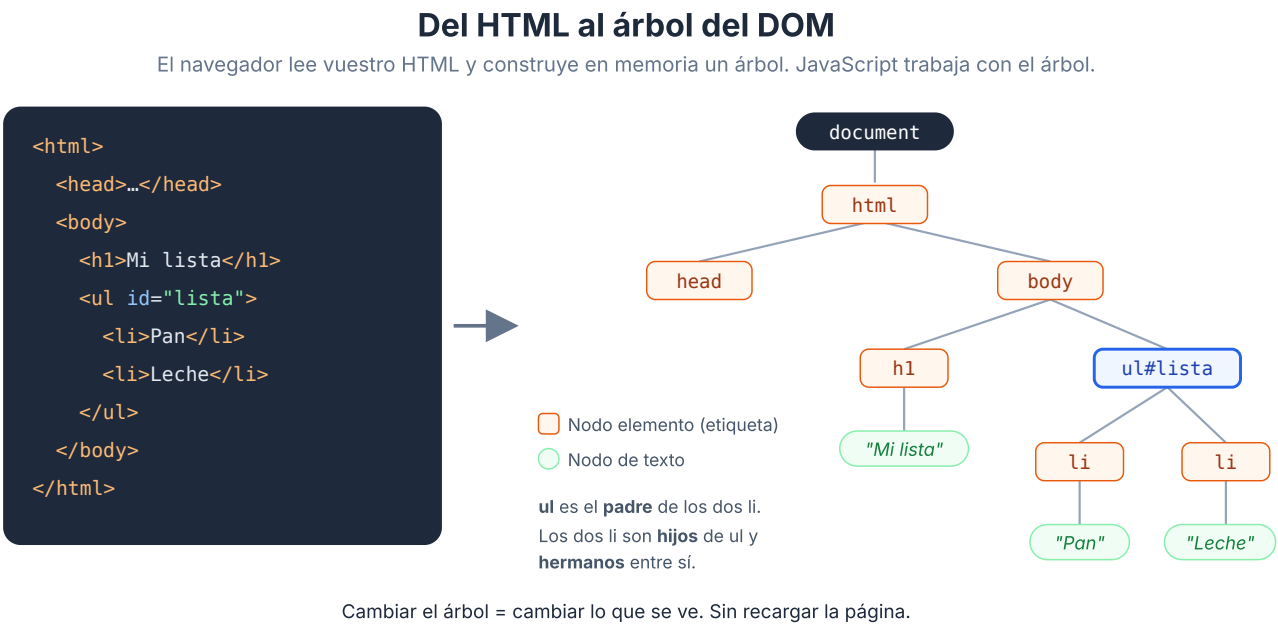


Figura 35 · Del HTML al árbol del DOM

La idea clave: **JavaScript no modifica el fichero HTML. Modifica el árbol.** Y como el navegador dibuja la página a partir del árbol, cualquier cambio se ve al instante, sin recargar [109].

Para hablar del árbol se usan palabras de familia:

Término	Significado	En el dibujo
Padre	El elemento que contiene a otro	`ul` es padre de los `li`
Hijo	Un elemento contenido directamente en otro	Cada `li` es hijo de `ul`
Hermanos	Elementos con el mismo padre	Los dos `li`



💡 Consejo

La pestaña **Elementos** de las herramientas de desarrollo (`F12`) muestra el DOM **en vivo**, no el HTML original. Si JavaScript añade o cambia algo, lo veréis ahí. Usadla en todos los ejercicios de este bloque.



2. Seleccionar elementos



Figura 36 · Seleccionar uno o todos

```
const titulo = document.querySelector('h1');          // el PRIMERO que encaja, o null
const tareas = document.querySelectorAll('.tarea');   // TODOS, en una NodeList
```

`querySelector()` devuelve el primer elemento que encaja con el selector, o `null` si no hay ninguno [36]. `querySelectorAll()` devuelve todos en una `NodeList`, que estará vacía si no encuentra nada [110].

Los dos aceptan **cualquier selector CSS**:

Selector	Encuentra
`'p'`	Elementos `<p>`
`'#salida'`	El elemento con `id="salida"`
`'.tarea'`	Elementos con la clase `tarea`
`'.tarea.hecha'`	Con las dos clases a la vez
`'#lista li'`	Los `li` que están dentro de `#lista`
`'li:last-child'`	Los `li` que son el último hijo
`'.tarea:not(.hecha)'`	Con `tarea` pero sin `hecha`
`'[data-tiempo]'`	Con el atributo `data-tiempo`

También podéis buscar **dentro** de un elemento, en lugar de en todo el documento:

```js
const lista = document.querySelector('#lista');
const items = lista.querySelectorAll('li');   // solo los li de esta lista
```

2.1. NodeList no es un array

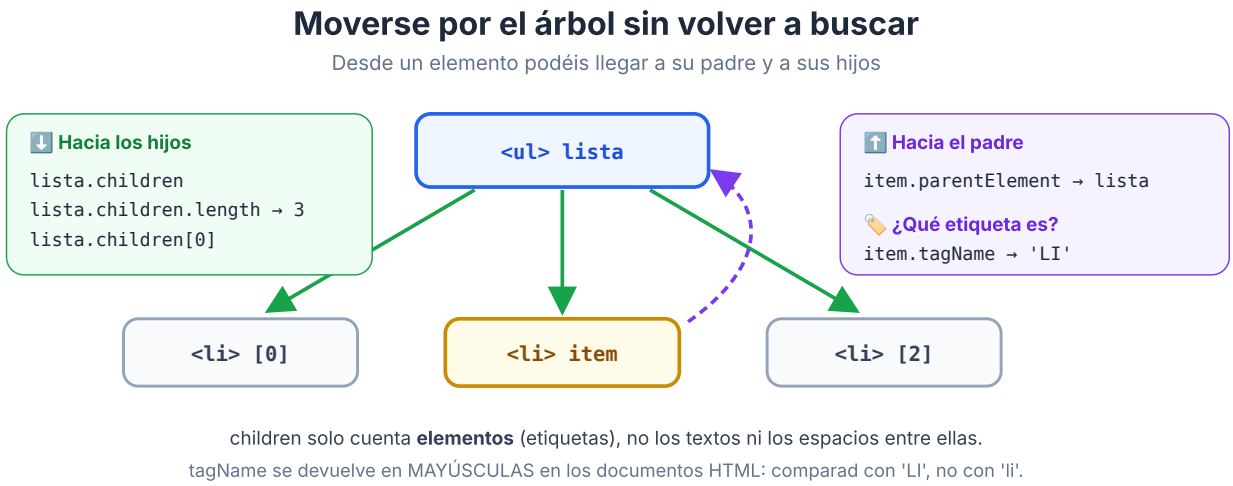
Una `NodeList` tiene `length`, se puede leer por índice y se puede recorrer con `forEach` o `for...of`. Pero **no tiene** `map`, `filter` ni `reduce`. Para usarlos, convertidla con `Array.from()` [5]:

```js
Array.from(tareas).map((li) => li.textContent);
```

Además, la `NodeList` de `querySelectorAll` es **estática**: una foto del momento en que buscasteis [110]. Si después añadís elementos, no aparecen en ella.



3. Moverse por el árbol



📊 Figura 37 · Moverse por el árbol del DOM

Desde un elemento podéis llegar a sus vecinos sin volver a buscar:

```js
lista.children              // sus elementos hijos [6]
lista.children.length       // cuántos tiene
lista.children[0]           // el primero
item.parentElement          // su elemento padre [7]
item.tagName                // 'LI' (en mayúsculas en documentos HTML) [8]
```



4. Cambiar el contenido

```
nombre.textContent = nombreUsuario;                              // texto
bio.innerHTML = 'Desarrolladora <strong>front-end</strong>';      // HTML
```

Propiedad	Interpreta etiquetas	Úsala para
`textContent` [34]	No: las muestra como texto	Cualquier texto, **siempre** si viene del usuario
`innerHTML` [15]	Sí: crea los elementos	HTML escrito por vosotros, sin datos de fuera



⚠️ Importante

Ya lo visteis en el bloque 01, pero aquí cobra todo su sentido: nunca metáis en `innerHTML` texto que venga del usuario o de fuera. MDN lo señala como una vía para ataques de *cross-site scripting* (XSS) [15]. Para construir HTML con datos, usad `createElement` + `textContent` (apartado 7).



5. Atributos y datos propios

5.1. Cualquier atributo

```
avatar.getAttribute('src');                  // leer: el valor, o null si no existe [11]
avatar.setAttribute('src', 'nueva.svg');     // cambiar o crear [12]
celda.setAttribute('colspan', '3');
```

5.2. Atributos `data-*`

Los atributos que empiezan por `data-` sirven para guardar vuestros propios datos en el HTML. JavaScript los ve en `dataset` [13]:

```
<div id="perfil" data-usuario-id="42" data-plan="gratis">
```

```
perfil.dataset.usuarioId       // '42'
perfil.dataset.plan = 'premium';
perfil.dataset.ultimaVisita = 'hoy';   // crea data-ultima-visita="hoy"
```

Dos reglas [13]:

- **Cambio de nombre:** se quita `data-`, desaparecen los guiones y la letra siguiente pasa a mayúscula. `data-usuario-id` ↔ `usuarioId`.
- **Siempre texto:** `'42'`, no `42`. Convertid con `Number()` si vais a operar.

En el bloque 07 os serán imprescindibles: guardaréis el `id` de cada producto en su tarjeta para saber cuál ha pulsado el usuario.



6. Cambiar el aspecto



Figura 38 · style frente a classList

```
// classList: añadir y quitar clases definidas en el CSS  ★ por defecto
tarjeta.classList.add('destacada');
tarjeta.classList.remove('borde');
tarjeta.classList.toggle('oculta');       // pone o quita; devuelve true si al final la tiene [14]
tarjeta.classList.contains('oculta');     // true / false

// style: estilos en línea, para valores calculados al momento
barra.style.width = `${porcentaje}%`;
```

`classList` gestiona la lista de clases del elemento [111]; `style` da acceso a sus estilos **en línea**, con las propiedades CSS escritas en *camelCase*: `backgroundColor`, `fontSize` [112].

Mi regla: **`classList` por defecto**. El diseño se queda en el CSS y JavaScript solo decide qué clase toca. `style`, solo cuando el valor no se puede saber de antemano, como la anchura de una barra de progreso.



7. Crear, mover y eliminar elementos

7.1. Crear → configurar → colgar

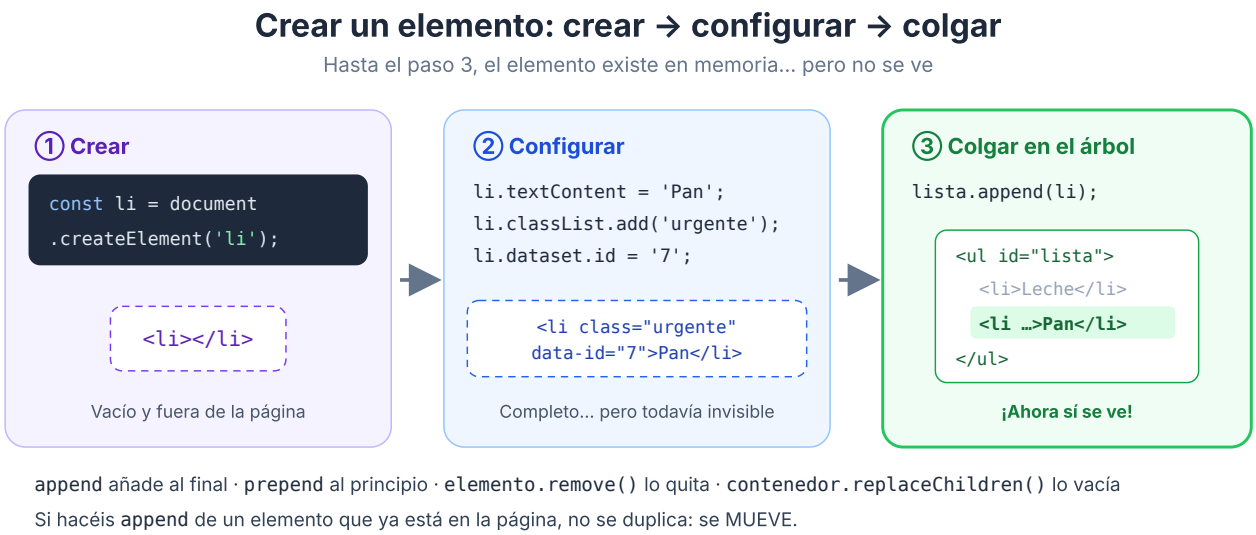


Figura 39 · Crear un elemento

```
const li = document.createElement('li');   // 1. crear [17]
li.textContent = 'Pan';                     // 2. configurar
li.classList.add('urgente');
lista.append(li);                           // 3. colgar: al final [18]
```

Hasta el paso 3, el elemento existe en memoria pero **no se ve**. Si un elemento creado no aparece, comprobad lo primero que lo habéis colgado.

7.2. Añadir, quitar y vaciar

Código	Qué hace
`padre.append(a, b, …)`	Añade uno o varios al final [113]
`padre.prepend(a)`	Añade al principio [114]
`elemento.remove()`	Quita el elemento de la página [115]
`padre.replaceChildren()`	Vacía el elemento: quita todos sus hijos [116]

7.3. `append` también mueve

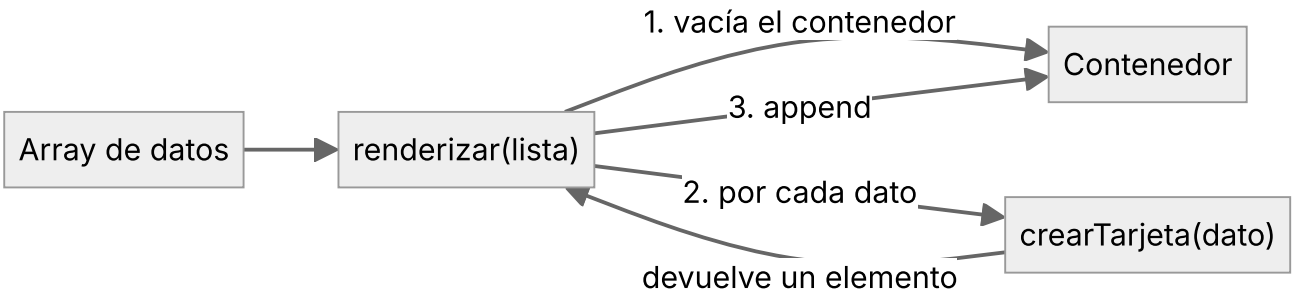
Si hacéis `append` de un elemento que **ya está** en el documento, no se duplica: **se mueve** a la nueva posición [117]. Es la forma de reordenar una lista sin crear nada:

```
ordenados.forEach((li) => lista.append(li));   // cada uno se va al final, en orden
```



8. El patrón «de datos a HTML»

Este es el patrón más importante del bloque, y lo vais a usar en todos los bloques que quedan. Los **datos** viven en un array; el **HTML** se genera a partir de ellos con dos funciones:




```
// recibe UN dato y DEVUELVE su elemento (sin colgarlo)
function crearTarjeta(producto) {
    const tarjeta = document.createElement('article');
    tarjeta.classList.add('tarjeta-producto');
    tarjeta.dataset.id = producto.id;

    const titulo = document.createElement('h3');
    titulo.textContent = producto.nombre;

    tarjeta.append(titulo);
    return tarjeta;
}

// recibe una LISTA y la pinta entera, desde cero
function renderizar(lista) {
    catalogo.replaceChildren();
    lista.forEach((producto) => {
        catalogo.append(crearTarjeta(producto));
    });
}

renderizar(productos.filter((p) => p.precio < 100));
```

¿Por qué así?

- **Seguro:** `textContent` muestra los datos como texto, sin riesgo de XSS.
- **Reutilizable:** `crearTarjeta` no sabe dónde va a acabar su tarjeta; quien la llama lo decide.

- **Fácil de actualizar:** si los datos cambian, se vuelve a llamar a `renderizar` y listo. En el bloque 07 lo haréis cada vez que el usuario pulse un botón.



9. La plantilla de trabajo en este bloque

La sección de **constantes al DOM** crece: ahora guarda los contenedores donde se van a pintar los datos. Y las **funciones** de crear y renderizar se colocan al principio, con las demás.

```
// estructura del programa principal
const tareas = [
    { texto: 'Estudiar el DOM', hecha: false },
    { texto: 'Hacer los ejercicios', hecha: true },
];

// funciones
function crearTarea(tarea) {
    const li = document.createElement('li');
    li.textContent = tarea.texto;
    if (tarea.hecha) {
        li.classList.add('hecha');
    }
    return li;
}

function renderizar(lista) {
    contenedor.replaceChildren();
    lista.forEach((t) => contenedor.append(crearTarea(t)));
}

// constantes al DOM que queremos controlar
const contenedor = document.querySelector('#tareas');

// procesamiento y salidas
renderizar(tareas);
```

Todavía falta una pieza: que el programa reaccione cuando el usuario hace algo. Eso son los eventos, y llegan en el bloque 07. Cuando lleguen, la función `renderizar` estará esperándolos.



10. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Explorador del árbol DOM	Selectores CSS en `querySelector`, `null`, `children`, `parentElement`	Ver
02	Muchos elementos a la vez	`querySelectorAll`, `NodeList`, `forEach`, `Array.from`	Ver
03	Retocar una tarjeta de perfil	`textContent`, `innerHTML`, `getAttribute`, `setAttribute`, `dataset`	Ver
04	Estilos: style frente a classList	`classList.add/remove/toggle/contains`, `style` para valores calculados	Ver
05	La lista de la compra, ahora en HTML	`createElement`, `append`, `prepend`	Ver
06	Tarjetas de producto desde datos	Patrón `crearTarjeta` + `renderizar`, `replaceChildren`, caso vacío	Ver
07	Clasificación: mover y eliminar	`remove()`, `append` que mueve, ordenar elementos existentes	Ver
08	Tabla de notas dinámica	Generar filas de tabla, clases según el valor, `colspan`, fila de resumen	Ver

El ejercicio 06 es el que más vais a reutilizar: su patrón aparecerá en casi todo lo que queda de curso.



🧪 Los ejercicios, explicados

Ejercicio 01 · Explorador del árbol DOM

> 🎯 **Objetivo:** Localizar elementos del DOM con querySelector usando distintos tipos de selector CSS (etiqueta, id, clase, descendiente, pseudoclase y atributo), comprobar qué devuelve cuando no encuentra nada y moverse por el árbol con children y parentElement.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Localizar elementos de una página con `querySelector` y distintos selectores CSS, y moverse por el árbol del DOM.



El DOM en una frase

Cuando el navegador carga vuestro HTML, construye en memoria un **árbol de objetos** que representa la página: el DOM [14]. Cada etiqueta es un nodo, y los nodos están unos dentro de otros como en vuestro código. JavaScript no toca el fichero HTML: trabaja con ese árbol [109]. Lo tenéis dibujado en la infografía de la teoría.



La solución paso a paso

1. `querySelector` acepta cualquier selector CSS

`querySelector()` devuelve el **primer** elemento que encaja con el selector, o `null` si no encuentra ninguno [36]. Y el selector es CSS normal y corriente, el mismo que usaríais en `style.css`:

Selector	Tipo	Encuentra
`'h2'`	Etiqueta	`<H2>` «Tortilla de patatas» (el primero de la página)
`'#ingredientes'`	Id	`<UL>` con los cuatro ingredientes
`'.importante'`	Clase	`<LI>` «Huevos»
`'#ingredientes li'`	Descendiente	`<LI>` «Patatas»: el primer `li` dentro de la lista
`'li:last-child'`	Pseudoclase	`<LI>` «Aceite de oliva»
`'[data-tiempo]'`	Atributo	`<P>` «Tiempo: 40 minutos»
`'#noExiste'`	Id	`null`

Fijaos en el caso de `'h2'`: la página tiene **dos** `h2`, el de la receta y el de «Informe del explorador». `querySelector` se queda con el primero que aparece en el documento.

2. Comprobar el `null`

```
if (elemento === null) {
    return `${selector} → null`;
}
```

Si intentáis leer `null.textContent`, tenéis el `TypeError` del bloque 01. Cuando no estéis seguros de que un elemento exista, comprobadlo antes.

3. `tagName`

`tagName` devuelve el nombre de la etiqueta en **mayúsculas** en los documentos HTML [118]. Por eso el informe muestra `<UL>` y no `<ul>`.

4. Navegar por el árbol

```
lista.children.length              // 4
lista.children[0].textContent      // 'Patatas'
importante.parentElement.id        // 'ingredientes'
```

- `children` es la colección de **elementos hijos** de un elemento [119].
- `parentElement` es su **elemento padre** [120].

Una vez que tenéis un elemento, podéis llegar a sus vecinos sin volver a buscar desde `document`.

Dos extras de formato

- `selector.padEnd(18)` rellena el texto con espacios hasta 18 caracteres, para que las flechas del informe salgan alineadas.
- El `textContent` de la lista incluye los saltos de línea y la sangría del HTML. `replace(/\s+/g, ' ')` usa una expresión regular para convertir cada grupo de espacios en uno solo. No hace falta que la memoricéis: solo deja el informe más limpio.

Ejercicio 02 · Muchos elementos a la vez: NodeList

> 🎯 **Objetivo:** Seleccionar varios elementos a la vez con querySelectorAll, recorrer la NodeList resultante con forEach y convertirla en un array con Array.from para poder usar map.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Trabajar con varios elementos a la vez: seleccionarlos, contarlos, recorrerlos y convertirlos en un array.



`querySelectorAll`

```
const todas = document.querySelectorAll('.tarea');
const hechas = document.querySelectorAll('.tarea.hecha');
const pendientes = document.querySelectorAll('.tarea:not(.hecha)');
```

`querySelectorAll()` devuelve **todos** los elementos que encajan con el selector, en una `NodeList` [110]. Si no hay ninguno, la lista está vacía: `length` vale 0. No hay `null` que vigilar.

Los selectores son CSS de nuevo:

- `.tarea.hecha` (sin espacio): elementos con **las dos** clases.
- `.tarea:not(.hecha)`: elementos con la clase `tarea` que **no** tienen `hecha`.



Una NodeList se parece a un array... pero no lo es

Puedo usar	✔ / ✘
`lista.length`, `lista[0]`	✔
`lista.forEach(…)`	✔
`for (const el of lista)`	✔
`lista.map(…)`, `lista.filter(…)`, `lista.reduce(…)`	✘ `TypeError`

`NodeList` no es un `Array`, pero se puede recorrer con `forEach()` y se puede convertir en un array de verdad con `Array.from()` [2]:

```
Array.from(pendientes).map((li) => li.textContent);
```

`Array.from()` crea un array nuevo a partir de cualquier cosa que se pueda recorrer [121]. A partir de ahí, tenéis todos los métodos del bloque 05.



Recorrer y modificar

```
todas.forEach((li, indice) => {
    li.textContent = `${indice + 1}. ${li.textContent}`;
});
```

Igual que el `forEach` de los arrays: un *callback* con el elemento y su índice. Aquí lo uso para modificar cada `li` de la página.



📝 Nota

La `NodeList` de `querySelectorAll` es **estática**: es una foto de cómo estaba la página en el momento de buscar [110]. Si después añadís otra tarea, la lista no se actualiza sola. Habría que volver a buscar.



Qué tenéis que observar

- Resumen: 3 de 6 tareas hechas (50 %).
- Las seis tareas aparecen numeradas, y las hechas siguen tachadas, porque solo he cambiado el texto, no las clases.
- Pendientes: 2. Llamar al taller · 4. Estudiar el bloque 06 · 5. Regar las plantas. Salen ya numeradas porque el `forEach` de arriba se ejecutó antes.

Ejercicio 03 · Retocar una tarjeta de perfil

🎯 **Objetivo:** Modificar el contenido de los elementos con textContent e innerHTML, leer y cambiar atributos con getAttribute y setAttribute, y trabajar con los atributos data-* a través de dataset.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Cambiar el texto, el HTML interno, los atributos y los datos `data-*` de una tarjeta de perfil.



La solución paso a paso

1. Texto: `textContent`

```
nombre.textContent = nombreUsuario;
```

`textContent` lee y escribe el texto de un elemento [34]. Si el texto contiene etiquetas, se muestran tal cual, sin interpretarlas. Por eso es la opción segura para cualquier dato que escriba el usuario.

2. HTML: `innerHTML`

```
bio.innerHTML = 'Desarrolladora <strong>front-end</strong> en formación.';
```

`innerHTML` interpreta el texto como HTML y crea los elementos correspondientes [15]. Aquí es aceptable porque **el HTML lo he escrito yo**. Recordad lo que visteis en el bloque 01: si ese texto viniera del usuario, alguien podría colar código, y MDN avisa de que es una vía para ataques XSS [15].

Propiedad	Interpreta etiquetas	Úsala para
`textContent`	No	Cualquier texto, sobre todo si viene del usuario
`innerHTML`	Sí	HTML que escribís vosotros

3. Atributos: `getAttribute` y `setAttribute`

```
avatar.getAttribute('src');                  // 'avatar-gris.svg'
avatar.setAttribute('src', 'avatar-color.svg');
```

`getAttribute()` devuelve el valor de un atributo [122] y `setAttribute()` lo cambia, o lo crea si no existía [123]. Funcionan con cualquier atributo: `src`, `alt`, `href`, `title`, `disabled`...

4. Datos propios: `dataset`

```
<div id="perfil" data-usuario-id="42" data-plan="gratis">
```

Los atributos que empiezan por `data-` sirven para guardar datos propios en el HTML. JavaScript los ve todos en `dataset` [5]:

```
perfil.dataset.usuarioId        // '42'
perfil.dataset.plan = 'premium';
perfil.dataset.ultimaVisita = 'hoy';
```

Dos detalles que MDN explica [5]:

- **El nombre cambia de formato.** Se quita el prefijo `data-`, desaparecen los guiones y la letra que va detrás pasa a mayúscula: `data-usuario-id` → `usuarioId`. Al revés también: `dataset.ultimaVisita` crea el atributo `data-ultima-visita`.
- **Los valores son siempre texto.** `dataset.usuarioId` es `'42'`, no `42`. Si vais a hacer cuentas, convertid con `Number()`.

Lo usaréis mucho en el bloque 07: guardaréis el `id` de un producto en un `data-id` y, cuando el usuario pulse su botón, sabréis de qué producto se trata.



Qué tenéis que observar

- El avatar cambia de gris a color: el `src` ahora apunta a otra imagen.
- Inspeccionad la tarjeta con las herramientas de desarrollo (pestaña **Elementos**): veréis `data-plan="premium"` y el nuevo `data-ultima-visita="hoy"` en el HTML vivo.

Ejercicio 04 · Estilos: style frente a classList

> 🎯 **Objetivo:** Cambiar el aspecto de los elementos de dos formas, con estilos en línea (style) y con clases CSS (classList: add, remove, toggle y contains), y saber cuándo conviene cada una.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Cambiar el aspecto de varios elementos con estilos en línea y con clases, y comparar los dos enfoques.



Opción 1: `style`

```
tarjeta1.style.backgroundColor = 'gold';
```

La propiedad `style` da acceso a los estilos **en línea** del elemento, los del atributo `style` del HTML [112]. Si inspeccionáis la tarjeta 1, veréis que ahora tiene `style="background-color: gold;"`.

Fijaos en el nombre: en CSS es `background-color`; en JavaScript, `backgroundColor`. Las propiedades con guion se escriben en *camelCase*.



Opción 2: `classList`

```
tarjeta2.classList.add('destacada');
tarjeta3.classList.toggle('oculta');
tarjeta4.classList.remove('borde');
tarjeta3.classList.contains('oculta');   // true
```

`classList` es la lista de clases del elemento, con métodos para añadir, quitar y comprobar [2]:

Método	Qué hace
`add('x')`	Añade la clase
`remove('x')`	La quita
`toggle('x')`	La añade si no está y la quita si está. Devuelve `true` si al final la tiene [124]
`contains('x')`	¿La tiene? `true` o `false`

Las clases `destacada`, `oculta` y `borde` ya estaban definidas en `style.css`. JavaScript no sabe nada de colores ni de bordes: solo decide **qué clase toca**.



¿Cuál uso?

Cambiar el aspecto: style frente a classList

El mismo resultado, pero ¿dónde vive el diseño?

elemento.style

```
tarjeta.style.backgroundColor = 'gold';
tarjeta.style.fontWeight = 'bold';
```

Escribe un estilo en línea en el HTML:
`<div style="background-color: gold; …">`

✔ Útil para valores calculados al momento (una anchura en %, una posición…)

✘ El diseño acaba repartido por el JavaScript
✘ Propiedades en camelCase: backgroundColor

elemento.classList

style.css

```
.destacada {
  background: gold;
}
```

app.js

```
tarjeta.classList
.add('destacada');
```

`.add('x')`	pone la clase
`.remove('x')`	la quita
`.toggle('x')`	la pone si no está y al revés
`.contains('x')`	¿la tiene? true / false

★ **La que usamos por defecto: el diseño se queda en el CSS y JavaScript solo decide qué clase toca**

Figura 40 · style frente a classList

Por defecto, `classList`. Las ventajas son claras:

- El diseño se queda en el CSS, donde lo buscará cualquiera que quiera cambiarlo.
- Una clase puede cambiar diez propiedades de golpe; con `style` serían diez líneas de JavaScript.
- Quitar una clase devuelve el elemento a su estado anterior. Con `style` tendríais que recordar el valor de antes.

`style` solo cuando el valor se calcula al momento, como la anchura de la barra:

```
relleno.style.width = `${porcentaje}%`;
```

No podéis tener una clase para cada porcentaje posible, así que aquí `style` es la herramienta adecuada.

Un detalle de la barra

```
Math.min(100, Math.max(0, terminados / TOTAL_BLOQUES * 100)) || 0
```

`Math.max(0, …)` impide valores negativos, `Math.min(100, …)` impide pasar del 100 % y `|| 0` convierte un posible `NaN` en 0. Tres protecciones en una línea para que la barra nunca se salga de su sitio.



Qué tenéis que observar

- La tarjeta 3 ha desaparecido: la clase `oculta` le pone `display: none`.
- La tarjeta 4 ya no tiene el borde discontinuo.

- En el informe, la tarjeta 2 muestra `style="null"`: no tiene atributo `style`, y en ese caso `getAttribute()` devuelve `null` [122].
- Con 6 bloques, la barra se llena hasta el 46 %.

Ejercicio 05 · La lista de la compra, ahora en HTML

> 🎯 **Objetivo:** Crear elementos HTML desde JavaScript a partir de un array con createElement, configurarlos con textContent y classList, y añadirlos a la página con append y prepend.
> 📂 Enunciado · Solución completa

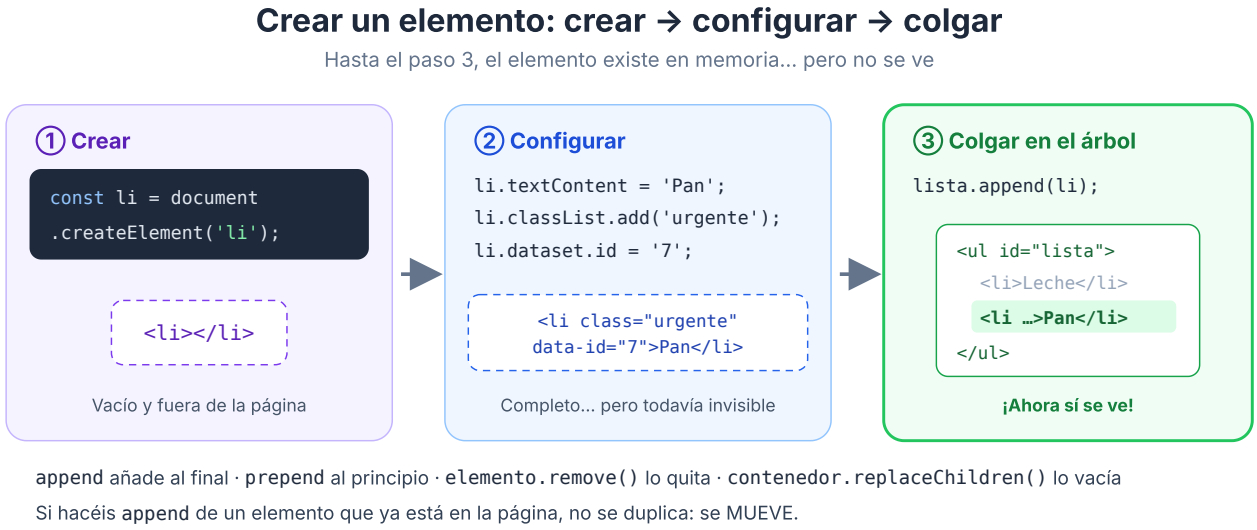


Qué pedía el ejercicio

Convertir un array de productos en elementos `<li>` reales, creados desde JavaScript.



Crear → configurar → colgar



📊 Figura 41 · Crear un elemento

```
const li = document.createElement('li');   // 1. crear
li.textContent = producto.nombre;          // 2. configurar
li.classList.add('urgente');
lista.append(li);                          // 3. colgar
```

1. `document.createElement('li')` crea un elemento nuevo del tipo indicado [125]. Existe en memoria, pero todavía **no está en la página**.
2. Lo configuráis con todo lo que visteis en los ejercicios anteriores: `textContent`, `classList`, `setAttribute`, `dataset`...

3. `append()` lo añade como **último hijo** del elemento padre [113]. En ese momento aparece en pantalla.

Si os saltáis el paso 3, no veréis nada y no habrá ningún error. Es un despiste muy habitual: si un elemento creado no aparece, comprobad lo primero que lo habéis colgado.



`append` y `prepend`

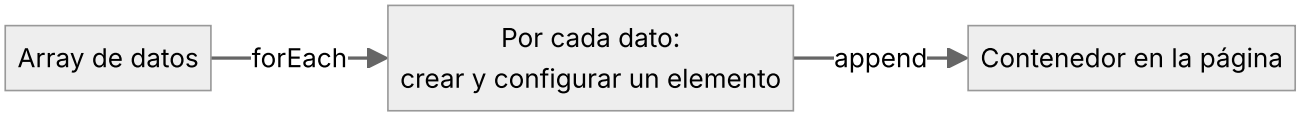
```
lista.append(li);          // al final
lista.prepend(cabecera);   // al principio
```

`prepend()` hace lo mismo que `append()`, pero coloca el elemento como **primer** hijo [114]. Por eso la cabecera queda arriba aunque la haya creado la última.



El patrón «de datos a HTML»

Fijaos en el esquema general, porque lo vais a repetir constantemente:



Los **datos** viven en el array. El **HTML** se genera a partir de ellos. Si mañana la lista tiene 50 productos, el código es exactamente el mismo.



¿Y por qué no `innerHTML`?

Se podría construir un texto con todas las etiquetas y asignarlo con `innerHTML`. Pero el nombre del producto lo ha escrito el usuario, y ya sabéis lo que eso significa: riesgo de XSS. Con `createElement` + `textContent`, cualquier cosa que escriba el usuario se muestra como texto, sin peligro.



Qué tenéis que observar

- Seis productos (cinco del array y «tomates»), con «leche» y «café» en rojo.
- La cabecera arriba: 📝 6 productos (2 urgentes).
- En la pestaña **Elementos**, el `<ul>` tiene siete `<li>` aunque en `index.html` está vacío.

Ejercicio 06 · Tarjetas de producto desde datos

> 🎯 **Objetivo:** Organizar la generación de HTML en dos funciones: una que crea el elemento de un dato (crearTarjeta) y otra que pinta una lista entera vaciando antes el contenedor (renderizar), y usarlas para mostrar un catálogo filtrado.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Generar un catálogo de tarjetas a partir de un array de objetos, repartiendo el trabajo en dos funciones.



Dos funciones, dos responsabilidades

Función	Recibe	Hace	Devuelve
`crearTarjeta(producto)`	Un objeto	Construye el HTML de **un** producto	El elemento, sin colgar
`renderizar(lista)`	Un array	Vacía el contenedor y pinta **todos**	Nada

Esta separación es la del bloque 04, «una función, una tarea», aplicada al DOM. Y tiene premio: en el bloque 07, cuando el usuario cambie un filtro con un botón, os bastará con volver a llamar a `renderizar` con otra lista. Nada más.



`crearTarjeta`: construir sin colgar

```
function crearTarjeta(producto) {
    const tarjeta = document.createElement('article');
    …
    tarjeta.append(titulo, precio, chip);
    return tarjeta;
}
```

La función crea cuatro elementos y los ensambla: el `h3`, el `p` y el `span` dentro del `article`. Fijaos en que `append()` admite **varios** elementos a la vez [113].

Y fijaos en que **no la cuelga en la página**: la devuelve. Así, quien la llama decide dónde ponerla. Hoy va en el catálogo; mañana podría ir en una lista de favoritos, y la función no cambiaría.

El `dataset.id` no se ve, pero queda guardado en el HTML (`data-id="3"`). En el bloque 07 os servirá para saber sobre qué producto ha hecho clic el usuario.



`renderizar`: vaciar y pintar

```
function renderizar(lista) {
    catalogo.replaceChildren();
    lista.forEach((producto) => {
        catalogo.append(crearTarjeta(producto));
    });
}
```

`replaceChildren()` sin argumentos **vacía** el elemento [116]. Es imprescindible: si llamáis a `renderizar` dos veces sin vaciar, tendríais las tarjetas duplicadas. La regla es «borrar y pintar

desde cero».



El caso vacío

```
if (lista.length === 0) {
    … 'No hay productos en ese rango de precio.'
    return;
}
```

Un buen detalle de interfaz: si no hay nada que mostrar, decidlo. Un hueco en blanco hace pensar al usuario que la página se ha roto.



Con distintos precios

Máximo	Se muestran
100	5 de 6 (todos menos el monitor)
300	6 de 6
5	Ninguno: aparece el aviso
Cancelar	6 de 6, sin límite de precio

Ejercicio 07 · Clasificación: mover y eliminar

> 🎯 **Objetivo:** Modificar elementos que ya existen en la página: leer sus datos con dataset, eliminarlos con remove() y reordenarlos aprovechando que append() mueve un elemento cuando ya está en el documento.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Limpiar y ordenar una lista que ya existe en la página sin crear ningún elemento nuevo.



La solución paso a paso

1. De NodeList a array

```
const equipos = Array.from(clasificacion.querySelectorAll('li'));
```

Necesito `filter` y `sort`, así que convierto la `NodeList` en array, como en el ejercicio 02. Fijaos también en que busco con `clasificacion.querySelectorAll`, no con `document.querySelectorAll`: así solo busco **dentro** de la lista.

2. Leer los datos de cada elemento

```
const puntosDe = (li) => Number(li.dataset.puntos);
```

Cada `li` guarda sus puntos en `data-puntos`. Como `dataset` devuelve siempre texto, convierto con `Number()`. Lo meto en una pequeña función flecha porque lo uso en cuatro sitios.

3. Eliminar: `remove()`

```
li.remove();
```

`remove()` quita el elemento del árbol del DOM [115]. Desaparece de la página al momento. Es la operación contraria a `append`.

4. Mover: `append` de algo que ya existe

```
siguen.forEach((li) => {
    clasificacion.append(li);
});
```

Aquí está la clave del ejercicio. Si hacéis `append` de un elemento que **ya está en el documento**, no se crea una copia: el elemento **se mueve** de su posición actual a la nueva [117].

Así que recorro el array ya ordenado y voy mandando cada equipo al final de la lista: primero Tiburones, después Águilas, después Leones... Cuando termino, la lista está en orden, y sigue teniendo exactamente los mismos cuatro `li`.



📝 Nota

MDN lo documenta en `appendChild()`, la versión clásica de `append` [117]. Las dos se comportan igual en este aspecto.

5. Retoques

```
siguen[0].classList.add('primero');
```

Como el array está ordenado, el primero es el líder. Le pongo una clase y CSS hace el resto.



El resultado

```
1. Tiburones · 21 pts   ← resaltado
2. Águilas · 15 pts
3. Leones · 12 pts
4. Lobos · 7 pts
```

Halcones y Búhos han desaparecido. Y si inspeccionáis la lista con las herramientas de desarrollo, veréis cuatro `li`, ni uno más.

Ejercicio 08 · Tabla de notas dinámica

> 🎯 **Objetivo:** Generar las filas de una tabla a partir de un array de objetos, aplicando una clase CSS a cada celda según su valor, añadir una fila de resumen en el pie y mostrar un mensaje cuando no hay datos.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Generar una tabla de notas a partir de un array, con una clase de color por calificación, una fila de media en el pie y un mensaje si no hay datos. Junta todo lo del bloque.



Tres funciones que se apoyan entre sí

Función	Devuelve
`calificar(nota)`	Un objeto `{ texto, clase }`
`crearCelda(texto, clase)`	Un `<td>`
`crearFila(alumno)`	Un `<tr>` con tres celdas

`crearFila` usa `crearCelda` y `calificar`. Cada pieza es pequeña y se entiende sola. Fijaos también en que `crearCelda` aparece porque crear un `td`, ponerle texto y quizá una clase se repetía seis veces. Cuando veáis código repetido, convertidlo en función.



Devolver dos datos a la vez

```
function calificar(nota) {
    if (nota < 5) {
        return { texto: 'Suspenso', clase: 'suspenso' };
    }
    …
}

const { texto, clase } = calificar(alumno.nota);
```

Una función solo puede devolver un valor... pero ese valor puede ser un objeto con varios datos dentro. Y con la desestructuración del bloque 05 los saco en una sola línea.



Las clases hacen el trabajo visual

La celda de calificación recibe la clase `suspenso`, `aprobado` o `sobresaliente`, y el color está definido en `style.css`. JavaScript decide **qué** es cada nota; CSS decide **cómo** se ve. Es la regla de `classList` del ejercicio 04.



La tabla ya existe; las filas, no

```
<tbody id="cuerpo"></tbody>
<tfoot id="pie"></tfoot>
```

La estructura fija (la tabla y la cabecera) está en el HTML. JavaScript solo genera la parte que depende de los datos: las filas. Y, como en el ejercicio 06, primero vacío con `replaceChildren()` [116] y después relleno con `append` [113].



El caso vacío con `colspan`

```
celda.setAttribute('colspan', '3');
```

Si no hay alumnos, el pie muestra un mensaje en una sola celda que ocupa las tres columnas. `colspan` es un atributo HTML normal, así que se pone con `setAttribute` [123].



Con nota mínima 5

Alumno	Nota	Calificación
Sara	10.0	Sobresaliente
Marta	9.2	Sobresaliente
Hugo	8.8	Aprobado
Ana	7.5	Aprobado
Lucía	6.0	Aprobado
Media	**8.30**	**Aprobado**

«Mostrando 5 de 8 alumnos con nota ≥ 5».



Bloque 07

Bloque 07 · Eventos

Llegamos al bloque que cambia la forma de programar. Hasta ahora, vuestros programas empezaban, hacían su trabajo de arriba abajo y terminaban. A partir de hoy, el programa **prepara la página y se queda esperando** a que el usuario haga algo: un clic, una tecla, un cambio en un campo. Cuando ocurre, responde. Y vuelve a esperar. Así funcionan todas las aplicaciones web que usáis a diario. Y, por fin, nos despedimos de `prompt()`.

Al terminar este bloque sabréis:

- Escuchar eventos con `addEventListener` y dejar de escucharlos con `removeEventListener`.
- Reaccionar al ratón, al teclado y a los campos de formulario.
- Usar el objeto evento: qué ha pasado, dónde y con qué tecla.
- Anular el comportamiento por defecto del navegador con `preventDefault`.
- Entender cómo se propaga un evento por el árbol del DOM.
- Usar la delegación de eventos para listas que cambian.
- Organizar una aplicación con el patrón **estado → renderizar**.



1. Programar con eventos

Un evento es algo que ocurre en la página y que el navegador os avisa de que ha ocurrido: un clic, una tecla pulsada, un campo que cambia… [126]. Vuestro trabajo es decir **qué función** se ejecuta cuando pase **qué cosa** en **qué elemento**.



Figura 42 · El ciclo de un evento

Fijaos en los dos momentos del dibujo:

- **Al cargar** (una sola vez): se buscan los elementos y se colocan los escuchadores.
- **Cada vez que ocurre el evento**: se ejecuta la función, con su esquema de siempre: entradas → procesamiento → salidas.



2. `addEventListener`

```
const btn = document.querySelector('#btnSaludar');

btn.addEventListener('click', saludar);          // con una función con nombre

btn.addEventListener('click', () => {            // con una función flecha anónima
    salida.textContent = '¡Hola!';
});
```

`addEventListener()` registra una función que se ejecutará cada vez que ocurra el evento indicado en ese elemento [127]. Recibe el nombre del evento, como texto, y la función, que es un *callback* como los del bloque 04.



🚨 Atención

La función se pasa **sin paréntesis**: `saludar`, no `saludar()`. Con paréntesis, se ejecuta una vez al cargar la página y lo que se registra es su resultado. El botón no hará nada y no veréis ningún error. Es el fallo número uno de este bloque.

Función	Cuándo usarla
Con nombre	Si la reutilizáis en varios eventos o la queréis retirar después
Flecha anónima	Si es corta y solo la usa ese elemento

Si registráis la misma función dos veces en el mismo elemento y para el mismo evento, no se duplica [127].

2.1. Dejar de escuchar

```
btn.removeEventListener('click', saludar);
```

`removeEventListener()` retira un escuchador, y para ello necesita **el mismo evento y la misma función** que se registraron [128]. Por eso solo se pueden retirar las funciones con nombre.



3. Los eventos más habituales



Figura 43 · Los eventos más habituales

Evento	Se dispara cuando…
`click`	Se pulsa y se suelta el botón del ratón sobre el elemento [129]
`dblclick`	Se hace doble clic [130]
`mouseover` / `mouseout`	El puntero entra en el elemento / sale de él [131] [132]
`keydown`	Se pulsa una tecla [133]
`input`	El valor de un campo cambia por una acción del usuario: con cada letra [134]
`change`	El usuario confirma un cambio: al elegir en un `<select>` o al salir de un campo que ha cambiado [135]
`focus` / `blur`	Un elemento recibe el foco / lo pierde [136] [137]

`input` y `change` se confunden mucho. Mi regla: **para reaccionar mientras se escribe, `input`**; para un `<select>`, una casilla o un cambio ya confirmado, `change`.



4. El objeto evento

Cuando ocurre un evento, el navegador llama a vuestra función y le pasa un objeto con los detalles. Por convenio se llama `e`:

```
campo.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' && e.ctrlKey) {
        publicar();
    }
});
```

Propiedad	Contiene
`e.type`	El nombre del evento: `'click'`, `'keydown'`… [138]
`e.target`	El elemento donde **se originó** el evento [139]
`e.currentTarget`	El elemento cuyo escuchador **se está ejecutando** [140]
`e.key`	La tecla pulsada: `'a'`, `'Enter'`, `'Escape'`, `'ArrowUp'`… [141]
`e.ctrlKey`	`true` si Ctrl estaba pulsada [142]. También existen `shiftKey` y `altKey`

La diferencia entre `target` y `currentTarget` la veréis en el apartado 7. De momento, quedaos con que **`currentTarget` es siempre el elemento que tiene el escuchador**.



5. Leer los campos de la página

Adiós, `prompt()`. A partir de ahora los datos se leen de los campos:

```
const campo = document.querySelector('#celsius');
campo.addEventListener('input', () => {
    const texto = campo.value;          // SIEMPRE texto [18]
    if (texto.trim() === '') {
        return;                         // Number('') sería 0
    }
    const celsius = Number(texto);
    // …
});
```

`value` es siempre un texto, incluso en `<input type="number">` [5]. Las reglas que aprendisteis con `prompt()` siguen valiendo: comprobad si está vacío y convertid antes de calcular.



6. `preventDefault`: «de esto me encargo yo»

Muchos eventos tienen un comportamiento por defecto: una tecla escribe un carácter, un enlace navega, un formulario se envía y recarga la página. `e.preventDefault()` le indica al navegador que no haga esa acción [19]:

```
// el carácter no llega a escribirse
telefono.addEventListener('keydown', (e) => {
    if (!esDigito(e.key)) {
        e.preventDefault();
    }
});

// el enlace no navega
enlace.addEventListener('click', (e) => {
    if (!confirm('¿Seguro que quieres salir?')) {
        e.preventDefault();
    }
});
```

Su uso más importante, evitar que un formulario recargue la página, lo veréis en el bloque 08.



7. Propagación: la burbuja

Cuando hacéis clic en un botón que está dentro de un `li` que está dentro de un `ul`, el evento ocurre en el botón... y después **sube** por todos sus antepasados. Cada uno que escuche ese evento ejecuta su función. Es la propagación en fase de burbuja [143].

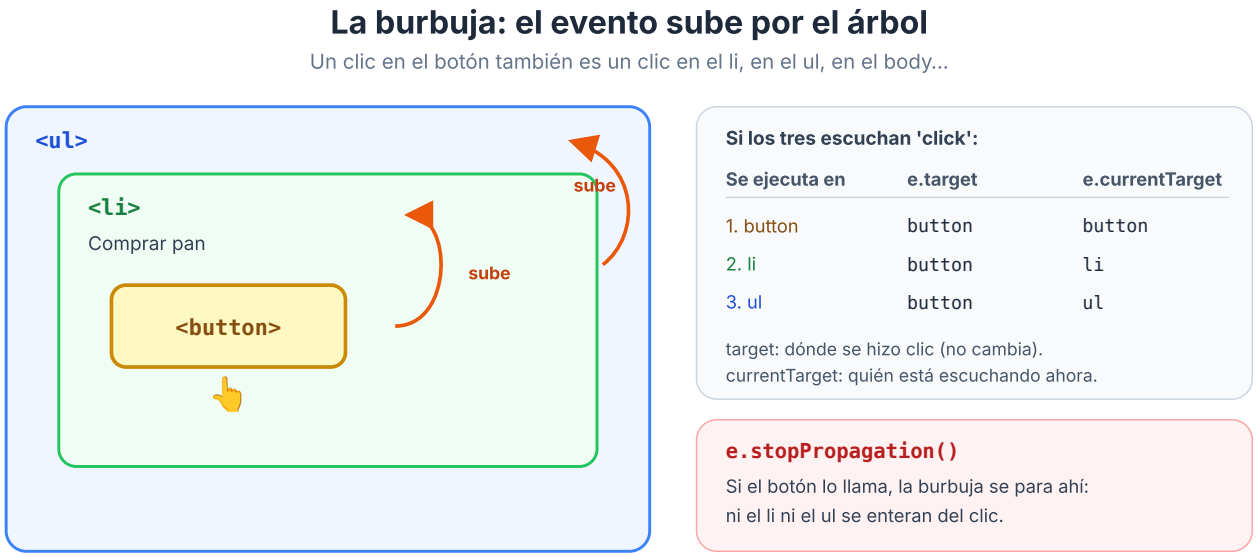


Figura 44 · La burbuja

Por eso existen dos propiedades distintas:

- `e.target` no cambia mientras el evento sube: es el botón donde se hizo clic [139].
- `e.currentTarget` va cambiando: el botón, luego el `li`, luego el `ul` [140].

Con `e.stopPropagation()`, el evento deja de subir [144]. Usadlo con mucha prudencia: cualquier código que escuche más arriba dejará de enterarse, y esos errores cuestan mucho de encontrar.



8. Delegación de eventos

La burbuja tiene una aplicación estupenda. En lugar de poner un escuchador en cada elemento de una lista, se pone **uno solo en el contenedor**: todos los clics de los hijos acaban llegando a él [143].



📊 Figura 45 · Delegación

```
lista.addEventListener('click', (e) => {
    const li = e.target.closest('li');     // el li que contiene el clic
    if (!li) {
        return;                            // clic en la lista, pero fuera de un li
    }
    if (e.target.closest('.borrar')) {
        // se ha pulsado el botón de borrar de ese li
    }
});
```

`closest(selector)` devuelve el propio elemento o su antepasado más cercano que encaje con el selector, o `null` si no hay ninguno [145]. Así llegáis al `li` aunque el clic haya caído en un `span` o en un botón de dentro.

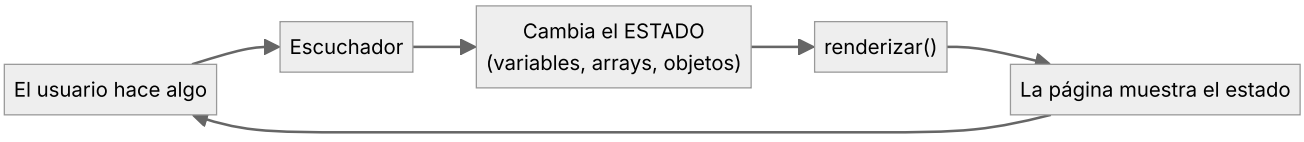
Ventajas de la delegación:

- **Un escuchador** en lugar de cien.
- **Funciona con elementos creados después.** Si `renderizar()` borra y recrea los `li`, los escuchadores puestos en ellos desaparecerían. El del contenedor, no.



9. El patrón estado → renderizar

Juntad el bloque 06 con este y tendréis la forma de organizar cualquier aplicación:



```
// el estado: todo lo que puede cambiar
const estado = { categoria: 'todas', busqueda: '', carrito: [] };

// los escuchadores solo cambian el estado y repintan
buscar.addEventListener('input', () => {
    estado.busqueda = buscar.value.trim();
    renderizar();
});
```

Los escuchadores **nunca tocan las tarjetas directamente**: cambian los datos y llaman a `renderizar()`, que pinta la página a partir del estado. Así la página y los datos nunca dicen cosas distintas. El ejercicio 08 es una tienda entera construida así.



10. La plantilla de trabajo, completa

Por fin la plantilla del curso aparece entera. Es la que vais a usar a partir de ahora en todos los ejercicios:

```
// estructura del programa principal
const IVA = 0.21;
let total = 0;                        // estado

// funciones
function pintar() {
    salida.textContent = `Total: ${total.toFixed(2)} €`;
}

// constantes al DOM que queremos escuchar
const inputPrecio = document.querySelector('#inputPrecio');
const btnAnadir = document.querySelector('#btnAnadir');
const salida = document.querySelector('#salida');

// colocar escuchadores de eventos
btnAnadir.addEventListener('click', () => {
    // entradas
    const precio = Number(inputPrecio.value);

    // procesamiento
    total += precio + precio * IVA;

    // salidas
    pintar();
});
```

Lo que está fuera de los escuchadores se ejecuta **una vez**, al cargar. Lo que está dentro, **cada vez que ocurre el evento**. Tenedlo siempre presente cuando decidáis dónde declarar una variable: si tiene que recordar algo entre un clic y el siguiente, va fuera.



11. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Contador de clics	`addEventListener`, funciones con nombre y anónimas, estado, `removeEventListener`	Ver
02	Conversor en tiempo real	`input`, `change`, `value`, una función para dos eventos	Ver
03	Mini-tuit con atajos de teclado	`keydown`, `e.key`, `e.ctrlKey`, contador de caracteres	Ver
04	Galería con eventos de ratón	`click`, `dblclick`, `mouseover`, `mouseout`, `e.type`, `e.currentTarget`	Ver
05	Campos que ayudan	`focus`, `blur`, `preventDefault` en teclas y enlaces	Ver
06	¿Quién se entera del clic?	Burbuja, `target` frente a `currentTarget`, `stopPropagation`	Ver
07	Lista de tareas con delegación	Delegación, `closest`, estado → renderizar	Ver
08	Catálogo interactivo	Aplicación completa: filtros, búsqueda, orden y carrito sobre un estado	Ver

Los ejercicios 07 y 08 son los más importantes del curso hasta ahora. Si los entendéis bien, el resto de bloques serán añadir piezas a una estructura que ya dominái.



🧪 Los ejercicios, explicados

Ejercicio 01 · Contador de clics

> 🎯 **Objetivo:** Reaccionar a los clics del usuario con addEventListener usando funciones con nombre y funciones flecha, mantener un estado entre clic y clic, y retirar un escuchador con removeEventListener.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Construir un contador que responde a tres botones y que deja de escuchar uno de ellos al llegar a un límite.



El cambio de mentalidad

Hasta ahora, vuestros programas se ejecutaban de arriba abajo y **terminaban**. Este no. Al cargar, prepara los botones y se queda esperando. El código de dentro de cada escuchador se ejecuta **cada vez** que el usuario hace clic, y ni una vez más [126]. Lo tenéis dibujado en la infografía del ciclo de un evento.



La solución paso a paso

1. `addEventListener`

```
btnMas.addEventListener('click', sumar);
```

`addEventListener()` registra una función que se ejecutará cuando ocurra el evento indicado en ese elemento [127]. Dos argumentos: el **nombre del evento**, como texto, y la **función** que se ejecutará, que es un *callback* como los del bloque 04.



> 🚨 **Atención**
>
> `sumar`, sin paréntesis. Si escribís `sumar()`, la función se ejecuta **una vez al cargar la página** y lo que se registra como escuchador es su resultado, `undefined`. El botón no hará nada. Es el error más típico de este bloque.

2. Con nombre o anónima

```
btnMas.addEventListener('click', sumar);   // función con nombre
btnMenos.addEventListener('click', () => { contador--; pintar(); });   // anónima
```

Las dos formas funcionan. ¿Cuándo usar cada una?

- **Con nombre**, cuando la vais a reutilizar o a retirar después.
- **Anónima**, cuando es corta y solo se usa en ese botón.

3. El estado

```
let contador = 0;
```

`contador` se declara **fuera** de los escuchadores, en el ámbito global. Si lo declarara dentro, se crearía de nuevo con valor 0 en cada clic y nunca pasaría de 1. Las variables que deben recordar algo entre un evento y el siguiente son el **estado** del programa.

4. Una función que pinta

Los tres botones cambian el contador y los tres necesitan mostrarlo. En lugar de repetir el código, todos llaman a `pintar()`. Es el mismo principio que `renderizar()` en el bloque 06: **cambia el estado, después pinta**.

```
salidaContador.classList.toggle('negativo', contador < 0);
```

`toggle` con un segundo argumento deja de ser un interruptor: si es `true`, solo añade la clase; si es `false`, solo la quita [124]. Perfecto para decir «pon esta clase si se cumple esta condición».

5. `removeEventListener`

```
btnMas.removeEventListener('click', sumar);
```

`removeEventListener()` retira un escuchador. Hay que indicarle **el mismo evento y la misma función** que se registraron [128]. Por eso `sumar` tiene nombre: una función flecha anónima no se puede retirar, porque no tenéis forma de volver a nombrarla.

Al reiniciar, vuelvo a añadir `sumar`. Y si ya estaba añadido, no pasa nada: si la función ya está en la lista de escuchadores de ese elemento, no se añade una segunda vez [127].



Qué tenéis que observar

- El número se pone rojo al bajar de 0.
- Al llegar a 10, el botón +1 se desactiva.
- Tras reiniciar, vuelve a funcionar.

Ejercicio 02 · Conversor en tiempo real

> 🎯 **Objetivo:** Reaccionar a lo que el usuario escribe en un campo con el evento input, y a una elección en un desplegable con el evento change, reutilizando la misma función para los dos.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Convertir la temperatura mientras el usuario escribe, sin botón, y recalcular también al cambiar el número de decimales.



Adiós a `prompt()`

A partir de este bloque, los datos se leen de los **campos de la página**:

```
const texto = campoCelsius.value;
```

La propiedad `value` de un `<input>` contiene lo que hay escrito, y es **siempre un texto** [5], incluso en un `<input type="number">`. La regla del bloque 01 sigue valiendo: convertid antes de calcular.

¿Y por qué compruebo primero si está vacío? Porque `Number('')` es `0`, como visteis en el bloque 04. Sin esa comprobación, al borrar el campo aparecería «0 °C = 32 °F».



`input`: con cada cambio

```
campoCelsius.addEventListener('input', actualizar);
```

El evento `input` se dispara cada vez que el valor del campo cambia por una acción del usuario [2]: con cada letra que escribe, borra o pega. Es el evento perfecto para los resultados en tiempo real y las búsquedas mientras se escribe.



`change`: cambio confirmado

```
selectDecimales.addEventListener('change', actualizar);
```

`change` se dispara cuando el usuario **confirma** un cambio [135]. En un `<select>`, al elegir una opción. En un campo de texto, en cambio, no salta con cada letra, sino cuando el campo pierde el foco después de haber cambiado [135].

Evento	En un campo de texto	En un `<select>`
`input`	Con cada letra	Al elegir una opción
`change`	Al salir del campo, si cambió	Al elegir una opción

Para un `<select>`, los dos funcionan igual. Para un campo donde queréis reaccionar mientras se escribe, `input`.



Una función, dos eventos

Los dos escuchadores llaman a la misma función, `actualizar`, porque hacen lo mismo: leer los dos controles y recalcular. Si escribierais dos funciones flecha distintas con el mismo código dentro, cualquier cambio habría que hacerlo dos veces.

Fijaos en que `actualizar` lee **los dos** controles cada vez, sin importar cuál ha cambiado. Así el resultado siempre es coherente con lo que se ve en pantalla.

Ejercicio 03 · Mini-tuit con atajos de teclado

> 🎯 **Objetivo:** Reaccionar al teclado con keydown, distinguir teclas con e.key y combinaciones con e.ctrlKey, y combinarlo con el evento input para mantener un contador de caracteres que cambia de color.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un campo para escribir mensajes cortos con contador de caracteres, botón de publicar y dos atajos de teclado.



El objeto evento

```
texto.addEventListener('keydown', (e) => {
    if (e.key === 'Enter' && e.ctrlKey) { … }
});
```

Cuando ocurre un evento, el navegador llama a vuestra función y le pasa un **objeto con los detalles** de lo que ha pasado. Por convenio se llama `e` o `evento`. Hasta ahora no lo necesitabais: un clic es un clic. Pero con el teclado sí importa **qué** tecla se ha pulsado.

Propiedad	Qué contiene	Ejemplo
`e.key`	La tecla pulsada [141]	`'a'`, `'Enter'`, `'Escape'`, `'ArrowUp'`
`e.ctrlKey`	Si la tecla Ctrl estaba pulsada [142]	`true` / `false`

Existen también `e.shiftKey` y `e.altKey`, que funcionan igual.



`keydown`

`keydown` se dispara al pulsar una tecla [133]. Lo escucho en el propio `textarea`, así que los atajos solo funcionan mientras se está escribiendo en él. Si quisierais atajos para toda la página, lo escucharíais en `document`.



Dos eventos que colaboran

Evento	Para qué
`input`	Mantener el contador al día con cada cambio del texto
`keydown`	Detectar los atajos

¿Por qué no usar `keydown` también para el contador? Porque `keydown` no se entera de todo: si el usuario pega texto con el ratón, no hay tecla, pero sí hay `input`. Cada evento para lo que mejor sabe hacer.



Detalles que marcan la diferencia

- **El botón se desactiva** cuando no se puede publicar (`btnPublicar.disabled`). El usuario ve de un vistazo que algo no está bien.
- `publicar()` **vuelve a comprobar** el texto, aunque el botón esté desactivado. El atajo de teclado no pasa por el botón, así que la función se protege sola.
- `texto.focus()` devuelve el cursor al campo después de publicar, para seguir escribiendo sin tocar el ratón.
- `textContent` para el mensaje: lo escribe el usuario. Ya conocéis el motivo.

Ejercicio 04 · Galería con eventos de ratón

> 🎯 **Objetivo:** Escuchar varios eventos de ratón (click, dblclick, mouseover y mouseout) en un grupo de elementos, y usar las propiedades type y currentTarget del objeto evento para saber qué ha pasado y en qué elemento.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Hacer que una galería reaccione al ratón: información al pasar por encima, favoritas con doble clic y un registro de los últimos eventos.



Un escuchador por elemento

```
fotos.forEach((foto) => {
    foto.addEventListener('mouseover', …);
    …
});
```

`querySelectorAll` + `forEach` + `addEventListener` es la forma directa de poner el mismo comportamiento en varios elementos. Dentro del `forEach`, la variable `foto` es cada una de las fotos, así que cada escuchador sabe a qué foto pertenece. En el ejercicio 07 veréis una alternativa con un solo escuchador para todas.



Los cuatro eventos

Evento	Cuándo se dispara	Aquí
`click`	Al pulsar y soltar el botón del ratón [129]	Solo se registra
`dblclick`	Al hacer doble clic [130]	Marca o desmarca favorita
`mouseover`	Cuando el puntero entra en el elemento [131]	Muestra la información
`mouseout`	Cuando el puntero sale [132]	Restaura el mensaje

Fijaos en el registro al hacer doble clic: aparecen **dos `click` antes del `dblclick`**. Un doble clic son, al fin y al cabo, dos clics. Si alguna vez usáis los dos eventos en el mismo elemento, tenedlo en cuenta.



`e.type` y `e.currentTarget`

```
li.textContent = `${e.type} · ${e.currentTarget.dataset.nombre}`;
```

- `e.type` es el nombre del evento que ha ocurrido: `'click'`, `'mouseover'`… [138]. Gracias a él, una sola función `registrar` sirve para los cuatro.
- `e.currentTarget` es el elemento **que tiene el escuchador** [6]: la foto.

¿Por qué no `e.target`? Porque `target` es el elemento exacto donde ha ocurrido el evento, y dentro de cada foto hay un `<span>` con el nombre. Si pasáis el ratón por el texto, `target` sería el `span`, que no tiene `data-nombre`. `currentTarget`, en cambio, es siempre la foto. En el ejercicio 06 veréis esta diferencia con todo detalle.

Por el mismo motivo, al mover el ratón entre el emoji y el texto de una misma foto veréis algún `mouseout` y `mouseover` de más en el registro: entrar o salir del `span` también cuenta.



Un registro con tope

```
registro.prepend(li);
if (registro.children.length > MAX_REGISTRO) {
    registro.lastElementChild.remove();
}
```

Lo nuevo entra por arriba y, si hay más de cinco, sale el más antiguo por abajo. Una cola con tamaño fijo, hecha con lo que visteis en el bloque 06.

Ejercicio 05 · Campos que ayudan: focus, blur y preventDefault

> 🎯 **Objetivo:** Reaccionar cuando un campo gana o pierde el foco (focus y blur) y anular el comportamiento por defecto del navegador con preventDefault, tanto en una tecla como en un enlace.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Mostrar la ayuda de cada campo al entrar en él, bloquear las teclas que no son números en el teléfono y pedir confirmación antes de seguir un enlace.



`focus` y `blur`

```
campo.addEventListener('focus', () => { … });
campo.addEventListener('blur', () => { … });
```

- `focus` se dispara cuando un elemento **recibe el foco** [1]: al hacer clic en él o al llegar con la tecla Tab.
- `blur`, cuando lo **pierde** [137].

Los uso para añadir y quitar la clase `activo` al contenedor del campo. El CSS hace el resto: cambia el borde y muestra el párrafo de ayuda. JavaScript solo dice «este campo está activo».

```
campo.parentElement.classList.add('activo');
```

`parentElement`, del bloque 06: el escuchador está en el `input`, pero la clase va en su padre, el `div.campo`.



`preventDefault`: decirle al navegador que no haga lo de siempre

Muchos eventos tienen un **comportamiento por defecto**: al pulsar una tecla en un campo, se escribe; al hacer clic en un enlace, se navega; al enviar un formulario, se recarga la página. `e.preventDefault()` le dice al navegador que vosotros os encargáis y que no haga esa acción [146].

En una tecla

```
telefono.addEventListener('keydown', (e) => {
    if (!esDigito(e.key) && !TECLAS_PERMITIDAS.includes(e.key)) {
        e.preventDefault();
    }
});
```

Si la tecla no es un dígito ni una de las permitidas, la anulo: el carácter nunca llega al campo. ¿Por qué permitir Backspace, las flechas y Tab? Porque sin ellas el usuario no podría corregir ni pasar al siguiente campo. Bloquear de más es tan malo como bloquear de menos.

La función `esDigito` comprueba dos cosas: que `e.key` tenga **un solo carácter**, para descartar nombres como `'Enter'`, y que esté entre `'0'` y `'9'`.

En un enlace

```
enlaceSalir.addEventListener('click', (e) => {
    if (!confirm('…')) {
        e.preventDefault();
    }
});
```

Si el usuario cancela, el navegador no sigue el enlace. Si acepta, no hago nada y el navegador navega como siempre.



📝 Nota

El uso estrella de `preventDefault` es el evento `submit` de los formularios, para que no recarguen la página. Lo veréis en el bloque 08.

Ejercicio 06 · Propagación: ¿quién se entera del clic?

🎯 **Objetivo:** Comprobar cómo un evento se propaga desde el elemento donde ocurre hacia sus antepasados (burbuja), distinguir e.target de e.currentTarget y detener la propagación con stopPropagation.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Observar por dónde viaja un evento cuando hay varios elementos anidados escuchándolo, y detenerlo.



Lo que veis en el registro

Clic en el botón, con la casilla desmarcada:

```
escucha: hijo · target: hijo
escucha: padre · target: hijo
escucha: abuelo · target: hijo
```

Clic en la zona verde, fuera del botón:

```
escucha: padre · target: padre
escucha: abuelo · target: padre
```

Clic en el botón, con la casilla marcada:

```
escucha: hijo · target: hijo
✋ stopPropagation(): la burbuja se para aquí
```



La burbuja

Cuando hacéis clic en el botón, el evento ocurre en el botón… y después **sube** por todos sus antepasados: el padre, el abuelo, el `body`… Cada uno que tenga un escuchador de ese evento lo ejecuta. A esto se le llama propagación en fase de burbuja (*event bubbling*) [143].

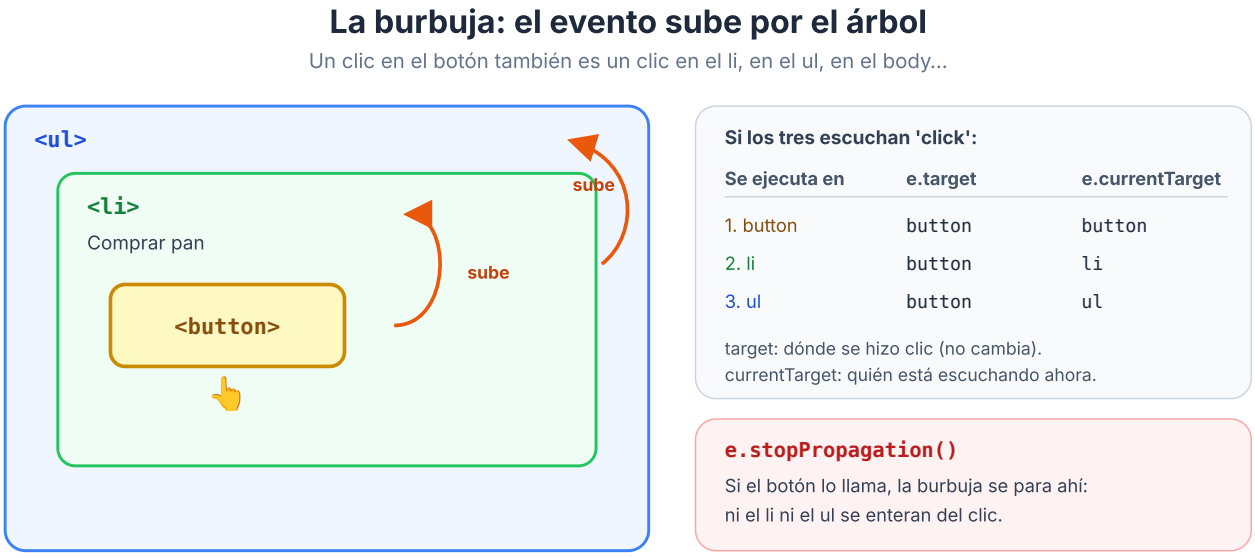


Figura 46 · La burbuja

Tiene lógica: si hacéis clic en el botón, también estáis haciendo clic dentro del padre y dentro del abuelo.



`target` frente a `currentTarget`

Propiedad	Es...	¿Cambia mientras sube?
`e.target`	El elemento donde **se originó** el evento [139]	No
`e.currentTarget`	El elemento cuyo escuchador **se está ejecutando ahora** [140]	Sí

En el primer caso, `target` es siempre `hijo` y `currentTarget` va pasando por `hijo`, `padre` y `abuelo`. Esta diferencia es la base de la delegación de eventos, que veréis en el ejercicio 07.



`stopPropagation`

```
if (parar.checked) {
    e.stopPropagation();
}
```

`stopPropagation()` impide que el evento siga propagándose [144]. Los escuchadores del propio botón se ejecutan todos, pero el padre y el abuelo ya no se enteran.

`parar.checked` es `true` si la casilla está marcada [147].



🚨 Atención

No uséis `stopPropagation` sin un buen motivo. Si cortáis la burbuja, cualquier otro código que esté escuchando más arriba dejará de funcionar sin que nadie entienda por qué. Es una herramienta para casos concretos, no un hábito.

Ejercicio 07 · Lista de tareas con delegación

🎯 **Objetivo:** Gestionar una lista que cambia con un único escuchador en el contenedor (delegación de eventos), usando e.target.closest para saber qué elemento se ha pulsado y un array como estado del que se renderiza la lista.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una lista de tareas completa en la que se puede añadir, marcar y borrar, con un único escuchador para todas las tareas.



El problema

Con lo del ejercicio 04 pondríais un escuchador en cada tarea. Pero las tareas **se crean y se destruyen** constantemente: cada `renderizar()` borra todos los `li` y los crea de nuevo. Tendríais que volver a poner los escuchadores después de cada repintado. Un lío.



La solución: delegación

```
lista.addEventListener('click', (e) => { … });
```

Un solo escuchador, en el **contenedor**. Gracias a la burbuja del ejercicio 06, cualquier clic en cualquier tarea acaba llegando a la lista. Y como el escuchador está en la lista, que nunca se borra, funciona con las tareas que existen y con las que se creen después [143].



Figura 47 · Delegación



¿En qué tarea se ha hecho clic? `closest`

```
const li = e.target.closest('li');
```

`e.target` puede ser el `span` del texto, el botón ✕ o el propio `li`, según dónde caiga el clic exactamente. `closest('li')` sube desde ese elemento y devuelve **el primer antepasado que encaja con el selector, o el propio elemento** si encaja; si no encuentra ninguno, `null` [145]. Así siempre llego al `li`, haya pinchado donde haya pinchado.

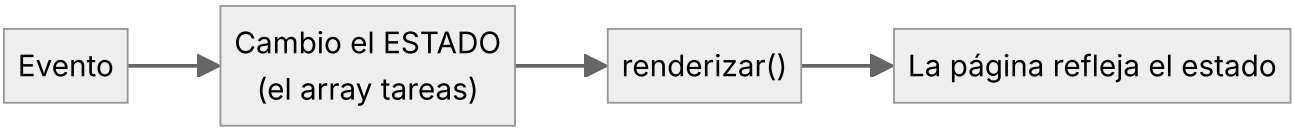
```
if (e.target.closest('.borrar')) { … }
```

El mismo truco para saber si el clic ha caído en el botón de borrar, o dentro de él.



Los datos mandan

Fijaos en que **nunca toco el HTML directamente** para marcar o borrar. Siempre hago lo mismo:



1. El evento **cambia el array** (añade, borra o cambia `hecha`).
2. `renderizar()` **repinta** todo a partir del array.

¿Por qué no hacer `li.remove()` directamente? Porque entonces el array y la página dirían cosas distintas, y el resumen de pendientes sería incorrecto. Con este patrón, la página siempre es un reflejo fiel de los datos. Y cuando en el bloque 09 guardéis las tareas en el navegador, guardaréis el array, no el HTML.



El `data-id` conecta las dos cosas

```
li.dataset.id = tarea.id;              // al crear
const id = Number(li.dataset.id);      // al hacer clic
```

Cada `li` lleva el `id` de su tarea. Al hacer clic, lo leo y busco la tarea en el array. Es el puente entre la página y los datos que os anuncié en el bloque 06.

Ejercicio 08 · Catálogo interactivo

> 🎯 **Objetivo:** Combinar todo el bloque en una aplicación: un estado con varios filtros, botones de categoría con delegación, búsqueda con input, orden con change y botones de añadir al carrito dentro de tarjetas generadas, todo repintado desde el estado.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una tienda con filtro por categoría, búsqueda, orden y carrito. Junta todo lo que habéis visto en los bloques 05, 06 y 07.

 La arquitectura: estado → renderizar

```
const estado = {
    categoria: 'todas',
    busqueda: '',
    orden: 'nombre',
    carrito: [],
};
```

Todo lo que puede cambiar en la aplicación está en **un solo objeto**. Los cuatro escuchadores hacen siempre lo mismo: **cambiar el estado y llamar a `renderizar()`**. Ninguno toca las tarjetas directamente.

Evento	Cambia	Después
Clic en un filtro	`estado.categoria`	`renderizar()`
`input` en la búsqueda	`estado.busqueda`	`renderizar()`
`change` en el orden	`estado.orden`	`renderizar()`
Clic en «Añadir»	`estado.carrito`	`renderizar()`

¿La ventaja? `renderizar()` siempre tiene en cuenta **los tres filtros a la vez**. Si el usuario elige «Periféricos», escribe «ra» y ordena por precio, todo encaja sin código extra.

 `productosVisibles()`: los datos a la carta

```
productos
    .filter((p) => estado.categoria === 'todas' || p.categoria === estado.categoria)
    .filter((p) => p.nombre.toLowerCase().includes(texto));
```

Dos `filter` encadenados, uno por criterio. Fijaos en el primero: si la categoría es `'todas'`, la condición es `true` para cualquier producto y no se descarta ninguno.

La búsqueda convierte los dos textos a minúsculas antes de comparar, para que «MONITOR» encuentre «Monitor 27"». Era el reto que os dejé en el bloque 05.

Para ordenar por nombre uso `localeCompare`, que compara textos respetando el orden alfabético del idioma. Con el `a - b` de los números no funcionaría.



Dos delegaciones

```
filtros.addEventListener('click', (e) => {
    const boton = e.target.closest('button');
    …
});

catalogo.addEventListener('click', (e) => {
    const boton = e.target.closest('button[data-id]');
    …
});
```

- **En los filtros**, por comodidad: un escuchador en lugar de cuatro.
- **En el catálogo**, por **necesidad**: las tarjetas se borran y se recrean en cada `renderizar()`. Un escuchador puesto en un botón desaparecería con él. El del catálogo, que nunca se borra, sigue funcionando [143].

`closest` con un selector de atributo, `button[data-id]`, me asegura que el clic ha sido en un botón de producto [145].



El botón activo

```
filtros.querySelectorAll('button').forEach((b) => {
    b.classList.toggle('activo', b === boton);
});
```

Recorro los cuatro botones y le pongo la clase `activo` solo al pulsado, quitándosela al resto. Una línea gracias al `toggle` con condición del ejercicio 01.



Prueba completa

1. Pulsad «Periféricos»: 3 productos.
2. Escribid «r» en el buscador: quedan «Alfombrilla XL» y «Ratón inalámbrico».
3. Ordenad por precio de menor a mayor: la alfombrilla primero.
4. Añadid la alfombrilla dos veces: 🛒 2 productos · 31.90 €.

Ese recorrido son cuatro tipos de evento distintos trabajando sobre un mismo estado. Si lo entendéis, tenéis la base de cualquier aplicación web.



Bloque 08

Bloque 08 · Formularios

Casi todo lo que el usuario os cuenta en una página web pasa por un formulario: un registro, un pedido, un mensaje de contacto, un buscador. En este bloque os voy a enseñar a **recoger esos datos y a comprobarlos antes de darlos por buenos**, y a hacerlo de forma que cualquier persona entienda qué ha fallado y cómo arreglarlo. Un formulario que rechaza datos sin explicar por qué es un formulario que la gente abandona.

Al terminar este bloque sabréis:

- Escuchar el envío de un formulario y evitar que recargue la página.
- Leer cualquier control: texto, número, select, casillas, radios y textarea.
- Recoger el formulario entero de una vez con `FormData`.
- Validar con atributos HTML y personalizar los mensajes con la API de validación.
- Programar reglas propias con `setCustomValidity`.
- Mostrar los errores de forma clara y accesible.
- Crear formularios que se adaptan a lo que elige el usuario.



1. Un formulario bien construido

Antes del JavaScript, el HTML. Un buen formulario se apoya en cuatro piezas:

```
<form id="registro">
    <label for="email">Email</label>
    <input type="email" id="email" name="email" required>

    <button type="submit">Registrarme</button>
</form>
```

Pieza	Para qué
`<form>`	Agrupa los campos y dispara el evento `submit`
`<label for="…">`	Nombra el campo; al pulsarla, el foco va al campo. Un lector de pantalla la lee
`name`	Identifica el dato. Sin `name`, el campo no existe para `FormData`
`<button type="submit">`	Envía el formulario, igual que pulsar Enter en un campo



2. El envío: `submit` y `preventDefault`



Figura 48 · Del botón «Enviar» al resultado

```
formulario.addEventListener('submit', (e) => {
    e.preventDefault();   // ¡lo primero!
    // leer, validar, procesar…
});
```

Dos reglas que os ahorrarán muchos disgustos:

1. **Escuchad el `submit` del formulario, no el `click` del botón.** El evento `submit` se dispara en el propio `<form>` [148], tanto si se pulsa el botón como si se pulsa Enter en un campo.
2. **`e.preventDefault()`, lo primero.** El comportamiento por defecto de un formulario es enviar los datos y recargar la página [146]. Sin esta línea, vuestro resultado aparecería un instante y se esfumaría.



3. Leer los datos

3.1. Control a control

¿Cómo leo cada control?

Casi todos se leen con .value... y casi todo lo que devuelve es TEXTO

Control	Cómo se lee	Devuelve	Ejemplo
✏️ Texto, email, textarea	`campo.value`	texto	`'Lucía'`
🔢 Número	`Number(campo.value)`	número (si convertís)	`28`
🔽 Select	`select.value`	texto (su value)	`'madrid'`
☑️ Checkbox (una)	`casilla.checked`	booleano	`true`
🔘 Radio (grupo)	`form.querySelector('[name="envio"]:checked').value`	texto, si hay alguno marcado (si no: null)	`'urgente'`
📦 Todo el formulario	`new FormData(form)`	pares name → valor	`.get('email')`

💡 Para FormData y los grupos de radio, cada control necesita su atributo **name**. Sin name, el control no existe para el formulario.

📊 Figura 49 · Cómo leer cada control

```
campoNombre.value                                    // texto [3]
Number(campoCantidad.value)                          // número (convertido)
selectTamano.value                                   // value de la opción elegida
casillaQueso.checked                                 // true / false [4]
formulario.querySelector('[name="entrega"]:checked').value   // radio marcado [5]
```

Recordad la regla de siempre: `value` es **texto**, incluso en un `<input type="number">`.

3.2. Todo el formulario: `FormData`

```
const datos = new FormData(formulario);

datos.get('nombre');          // el valor de un campo, o null [6]
datos.get('modalidad');       // el radio marcado, sin querySelector
datos.getAll('intereses');    // array con TODAS las casillas marcadas con ese name [7]

const objeto = Object.fromEntries(datos);   // un objeto normal [8]
```

`FormData` recoge todos los campos como pares **name → valor** [149]. Es la forma más cómoda de leer un formulario completo. Un aviso con `Object.fromEntries`: si varios campos comparten `name`, como un grupo de casillas, en el objeto solo queda uno. Para esos, usad `getAll`.



4. Validación nativa: atributos HTML

El navegador sabe validar sin que escribáis JavaScript:

Atributo	Regla
`required`	Obligatorio; en una casilla, tiene que estar marcada [150]
`minlength` / `maxlength`	Mínimo / máximo de caracteres [151]
`type="email"`	Formato de email [152]
`min` / `max`	Valor mínimo / máximo de un número [153]
`pattern`	Debe encajar con una expresión regular, como `[0-9]{5}` [154]

Si algún campo no cumple su regla, **el navegador no envía el formulario**: muestra un aviso junto al campo y el evento `submit` ni se dispara [18]. Para colorear los campos, la pseudoclase `:user-invalid` selecciona los inválidos, pero solo después de que el usuario haya interactuado con ellos [16]:

```
input:user-invalid {
    border-color: #dc2626;
}
```



⚠️ **Importante**

Validar en el navegador es una **comodidad** para el usuario, no una medida de seguridad: cualquiera puede saltársela. Si los datos llegan a un servidor, el servidor debe volver a validarlos siempre [18].



5. La API de validación

La validación nativa tiene dos límites: los mensajes los escribe el navegador, y no admite reglas propias. La API de validación resuelve los dos sin renunciar a los atributos.

5.1. `novalidate`: las reglas siguen, los avisos no

```
<form id="registro" novalidate>
```

Con `novalidate`, el formulario no se valida al enviarse [17]: el `submit` se dispara siempre y los avisos automáticos desaparecen. Pero **las reglas siguen ahí**, y cada campo sigue sabiendo si las cumple.

5.2. `validity`: qué regla falla

Cada regla HTML tiene su chivato en validity

El atributo pone la regla · campo.validity dice cuál se ha incumplido

Atributo en el HTML	Si no se cumple…	Mensaje que podéis mostrar
`required`	`validity.valueMissing`	Este campo es obligatorio
`type="email"`	`validity.typeMismatch`	Escribe un email válido
`minlength="3"`	`validity.tooShort`	Mínimo 3 caracteres
`min="18" max="99"`	`rangeUnderflow / rangeOverflow`	Debe estar entre 18 y 99
`pattern="[0-9]{5}"`	`validity.patternMismatch`	5 cifras, sin letras
`setCustomValidity('…')`	`validity.customError`	Vuestra regla: «no coinciden»…

`validity.valid` es true si no falla ninguna.
`form.checkValidity()` lo comprueba en todos.

`<form novalidate>` apaga los avisos del navegador, pero las reglas siguen funcionando.

Figura 50 · Cada regla tiene su chivato

Cada campo tiene una propiedad `validity`, un objeto `ValidityState` con un booleano por cada tipo de problema: `valueMissing`, `typeMismatch`, `tooShort`, `rangeUnderflow`, `patternMismatch`, `customError`… y `valid`, que es `true` si no falla ninguno [155].

```
function mensajeError(campo) {
    if (campo.validity.valueMissing) return 'Este campo es obligatorio.';
    if (campo.validity.typeMismatch) return 'Escribe un email válido.';
    if (campo.validity.tooShort) return `Mínimo ${campo.minLength} caracteres.`;
    return '';
}
```

5.3. Comprobar el formulario entero

Método	Qué hace
`formulario.checkValidity()`	Devuelve `true` si ningún control tiene problemas [156]
`formulario.reportValidity()`	Lo mismo y, además, muestra los avisos del navegador [157]

5.4. Reglas propias: `setCustomValidity`

```
const coinciden = repetir.value === clave.value;
repetir.setCustomValidity(coinciden ? '' : 'Las contraseñas no coinciden.');
```

`setCustomValidity(mensaje)` marca el campo como inválido con vuestro mensaje [158]. Desde ese momento se comporta como si incumpliera cualquier otra regla: `validity.customError` es `true` y

`checkValidity()` devuelve `false`.



🚨 Atención

Para que vuelva a ser válido, hay que llamar a `setCustomValidity('')`, con un texto vacío. Si se os olvida, el campo queda marcado como inválido para siempre. Es lo que hacen todos los ejemplos de la guía de MDN [19].



6. Errores que todo el mundo entiende



📊 *Figura 51 · Errores accesibles*

Un error accesible tiene cuatro ingredientes:

1. **Un texto que explica qué falla y cómo arreglarlo.** No solo un borde rojo: no todo el mundo distingue los colores.
2. `aria-invalid="true"` en el campo, que anuncia a la tecnología de apoyo que su valor no es válido [159]. Ponedlo solo después de validar, nunca al cargar la página.
3. `aria-describedby`, que enlaza el campo con el párrafo del error [160]. El lector de pantalla lo leerá al llegar al campo.
4. **El foco donde está el problema.** Tras un envío fallido, llevad el foco al primer error o a un resumen de errores con `focus()` [161].

```
<input type="email" id="email" aria-describedby="error-email">
<p id="error-email" class="error-campo"></p>
```

```
parrafoError.textContent = `⚠ ${mensaje}`;
campo.setAttribute('aria-invalid', 'true');
```

Para los mensajes que aparecen en otra parte de la página, como el de «enviado correctamente», usad una región viva con `aria-live="polite"`: así un lector de pantalla también anuncia el cambio [162].

6.1. ¿Cuándo validar?

Momento	Qué hacer
Mientras se escribe por primera vez (`input`)	Nada: ver un error desde la primera letra es muy molesto
Al salir del campo (`blur`)	Validar ese campo
Mientras se corrige un campo que ya tenía error (`input`)	Volver a validarlo, para que el aviso desaparezca en cuanto esté bien
Al enviar (`submit`)	Validar todos y llevar el foco al primer error



7. Formularios que se adaptan

Algunos campos solo tienen sentido según lo que se haya elegido: los datos de factura, la provincia si el país es España... El patrón es el mismo que el `renderizar()` del bloque 07:

```
function actualizar() {
    const conFactura = quieroFactura.checked;
    datosFactura.hidden = !conFactura;   // mostrar u ocultar
    nif.required = conFactura;           // obligatorio SOLO si se ve
}

formulario.addEventListener('input', actualizar);    // los eventos de los campos
formulario.addEventListener('change', actualizar);   // suben hasta el formulario
```

Dos reglas:

- **Un campo oculto nunca debe ser obligatorio.** Si no, el formulario bloquea el envío por un campo que el usuario ni ve.
- **`formulario.reset()` solo reinicia los valores** [163]. Lo que hayáis cambiado con JavaScript, como un grupo que se ha mostrado, no vuelve solo: llamad a `actualizar()` justo después.



8. La plantilla de trabajo con un formulario

```
// estructura del programa principal
const inscritos = [];

// funciones
function mensajeError(campo) { … }
function validarCampo(campo) { … }
function renderizar() { … }

// constantes al DOM que queremos escuchar
const formulario = document.querySelector('#inscripcion');
const campos = formulario.querySelectorAll('input');

// colocar escuchadores de eventos
campos.forEach((campo) => {
    campo.addEventListener('blur', () => validarCampo(campo));
});

formulario.addEventListener('submit', (e) => {
    e.preventDefault();

    // entradas
    const datos = new FormData(formulario);

    // procesamiento: validar y, si todo va bien, cambiar el estado
    campos.forEach(validarCampo);
    if (!formulario.checkValidity()) {
        formulario.querySelector('[aria-invalid="true"]').focus();
        return;
    }
    inscritos.push(Object.fromEntries(datos));

    // salidas
    renderizar();
    formulario.reset();
});
```



9. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Pedido en la pizzería	`submit`, `preventDefault`, leer los seis tipos de control	Ver
02	FormData	`get`, `getAll`, `Object.fromEntries`	Ver
03	Validación nativa del navegador	`required`, `minlength`, `type="email"`, `min` / `max`, `pattern`, `:user-invalid`	Ver
04	Mensajes de error propios	`novalidate`, `validity`, `checkValidity`, `aria-invalid`	Ver
05	Contraseñas con reglas propias	`setCustomValidity`, requisitos en vivo, `reportValidity`	Ver
06	Formulario de contacto accesible	`aria-describedby`, resumen de errores, `aria-live`, cuándo validar	Ver
07	Pedido con factura	Campos que aparecen, `hidden` + `required`, total en vivo, `reset`	Ver
08	Inscripciones a un taller	Aplicación completa: regla de email repetido, tabla, plazas y borrado	Ver

Un consejo para el 06: hacedlo una vez con el ratón y otra **solo con el teclado** (Tab, Enter y las flechas). Es la mejor forma de comprobar si vuestro formulario es de verdad accesible.



🧪 Los ejercicios, explicados

Ejercicio 01 · Pedido en la pizzería: leer cada control

> 🎯 **Objetivo:** Escuchar el envío de un formulario con el evento submit, evitar que recargue la página con preventDefault y leer los valores de seis tipos de control distintos: texto, número, select, checkbox, radio y textarea.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Recoger un pedido con seis tipos de control distintos y calcular el total al enviar el formulario.



El evento `submit`

```
formulario.addEventListener('submit', (e) => {
    e.preventDefault();
    …
});
```

Dos ideas clave:

1. **El escuchador va en el `<form>`, no en el botón.** El evento `submit` se dispara en el propio formulario [148], tanto si el usuario pulsa el botón como si pulsa Enter dentro de un campo. Si escucharais el `click` del botón, el Enter se os escaparía.
2. **`e.preventDefault()`, lo primero.** El comportamiento por defecto de un formulario es enviar los datos y **recargar la página** [146]. Sin esta línea, veríais el resultado un instante y desaparecería. Ponedla siempre la primera, antes de cualquier código que pueda fallar.

Fijaos también en que el botón es `type="submit"`: es el que envía el formulario.



Cómo se lee cada control

¿Cómo leo cada control?

Casi todos se leen con .value... y casi todo lo que devuelve es TEXTO

Control	Cómo se lee	Devuelve	Ejemplo
✏️ Texto, email, textarea	`campo.value`	texto	`'Lucía'`
🔢 Número	`Number(campo.value)`	número (si convertís)	`28`
🔽 Select	`select.value`	texto (su value)	`'madrid'`
☑️ Checkbox (una)	`casilla.checked`	booleano	`true`
🔘 Radio (grupo)	`form.querySelector('[name="envio"]:checked').value`	texto, si hay alguno marcado (si no: null)	`'urgente'`
📦 Todo el formulario	`new FormData(form)`	pares name → valor	`.get('email')`

💡 Para FormData y los grupos de radio, cada control necesita su atributo **name**. Sin name, el control no existe para el formulario.

📊 Figura 52 · Cómo leer cada control

Control	Código	Devuelve
Texto y textarea	`campo.value`	Texto [5]
Número	`Number(campo.value)`	Texto, que convertís [5]
Select	`select.value`	El `value` de la opción elegida
Checkbox	`casilla.checked`	`true` o `false` [147]
Radio	`querySelector('[name="entrega"]:checked').value`	El `value` del marcado

El caso del radio

Los botones de radio de un mismo grupo comparten el atributo `name`, y solo uno puede estar marcado [164]. Para saber cuál, busco el que tiene ese `name` y está marcado (`:checked`). Si ninguno estuviera marcado, `querySelector` devolvería `null` y `.value` daría error. Aquí no pasa porque en el HTML uno empieza marcado con `checked`. Es una buena costumbre.

`typeof campoCantidad.value`

La última línea del ticket lo confirma: `"string"`. Aunque el campo sea `type="number"`, su `value` es texto. Por eso convierto con `Number()` antes de multiplicar.



Un detalle: precios en un objeto

```
const PRECIOS = { mediana: 9, familiar: 14 };
PRECIOS[tamano]
```

El `value` de cada opción del `select` coincide con una clave del objeto. Así, `PRECIOS[tamano]` me da el precio directamente, sin `if` ni `switch`. Corchetes con variable, como en el bloque 05.



Con los valores iniciales

2 pizzas familiares, sin queso y para recoger: **28.00 €**. Con extra de queso y a domicilio: (14 + 1.5) × 2 + 3 = **34.00 €**.

Ejercicio 02 · FormData: el formulario entero de una vez

> 🎯 **Objetivo:** Leer todos los datos de un formulario de una sola vez con FormData, obtener valores sueltos con get, valores múltiples con getAll y convertir el formulario en un objeto con Object.fromEntries.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Leer todos los datos de un formulario con `FormData` y comparar dos formas de obtenerlos.



`new FormData(formulario)`

```
const datos = new FormData(formulario);
```

`FormData` recoge en un solo objeto todos los campos del formulario como pares **nombre → valor** [149]. Y el nombre es el del atributo **`name`**, no el `id`. Un campo sin `name` no aparece en `FormData`. Es el error más habitual con esta herramienta.

Lo bueno: da igual cuántos campos tenga el formulario, o de qué tipo sean. No hace falta seleccionarlos uno a uno.



`get` y `getAll`

```
datos.get('nombre');          // 'Lucía'
datos.get('modalidad');       // 'presencial': el radio marcado
datos.getAll('intereses');    // ['frontend', 'diseño']: todas las casillas marcadas
```

- `get()` devuelve **el primer valor** asociado a ese nombre, o `null` si no hay ninguno [165].

- `getAll()` devuelve **un array con todos** los valores asociados a ese nombre [166].

Fijaos en dos ventajas frente al ejercicio 01:

- **Radio:** `get('modalidad')` ya devuelve el marcado. Adiós al `querySelector('[name=…]:checked')`.
- **Casillas con el mismo `name`:** solo aparecen las **marcadas**, y `getAll` las devuelve en un array.



`Object.fromEntries`: de FormData a objeto

```
const objeto = Object.fromEntries(datos);
```

`Object.fromEntries()` construye un objeto a partir de una lista de pares clave-valor [167], y un `FormData` lo es. El resultado es un objeto normal, perfecto para guardarlo o enviarlo como JSON.

```
{
  "nombre": "Lucía",
  "email": "lucia@ejemplo.com",
  "taller": "javascript",
  "intereses": "diseño",
  "modalidad": "presencial"
}
```

¿Y los intereses? En un objeto, cada clave solo puede aparecer **una vez**. Como hay dos casillas `intereses` marcadas, la segunda sobrescribe a la primera y solo queda `"diseño"`. Para los campos con varios valores, usad siempre `getAll`.



¿Cuándo uso cada forma?

Necesito	Uso
Uno o dos campos concretos	`campo.value` / `campo.checked`
Todo el formulario	`new FormData(form)` + `get`
Casillas con el mismo `name`	`getAll`
Un objeto para guardar o enviar	`Object.fromEntries(datos)`, con cuidado con los campos múltiples

Ejercicio 03 · Validación nativa del navegador

🎯 **Objetivo:** Delegar la validación en el navegador con los atributos HTML required, minlength, type="email", min, max y pattern, dar estilo a los campos con :user-invalid y comprobar que el evento submit solo se dispara si todo es válido.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Validar un formulario solo con atributos HTML y comprobar que el navegador se encarga del resto.



Los atributos de validación

```
<input type="text" name="nombre" required minlength="2">
<input type="email" name="email" required>
<input type="number" name="edad" required min="18" max="99">
<input type="text" name="cp" required pattern="[0-9]{5}">
<input type="checkbox" name="acepto" required>
```

Atributo	Regla
`required`	El campo no puede quedar vacío; en una casilla, tiene que estar marcada [150]
`minlength`	Número mínimo de caracteres [151]
`type="email"`	El texto debe tener formato de email [152]
`min` / `max`	Valor mínimo y máximo de un número [153]
`pattern`	El valor debe encajar con una expresión regular [154]

`[0-9]{5}` significa «exactamente cinco caracteres del 0 al 9». Es una expresión regular, un minilenguaje para describir patrones de texto. No hace falta que la dominéis: con unos pocos patrones típicos tenéis para casi todo.



El navegador valida antes de `submit`

Haced la prueba: con el formulario vacío, pulsad «Registrarme». El navegador muestra un aviso junto al primer campo incorrecto… y **vuestro escuchador ni se ejecuta**. MDN lo explica: si los datos no cumplen las reglas, el navegador no envía el formulario [18].

Eso significa que, cuando vuestro código de `submit` se ejecuta, **los datos ya son válidos**. No hace falta comprobarlo otra vez.



Estilos con `:user-invalid`

```
input:user-invalid {
    border-color: #dc2626;
}
```

`:user-invalid` selecciona los campos inválidos, pero **solo después de que el usuario haya interactuado con ellos** o haya intentado enviar el formulario [168]. Así no aparece todo en rojo nada

más cargar la página, cuando el usuario aún no ha escrito nada. Existe también `:invalid`, que se aplica desde el principio y suele resultar agresivo.



`formulario.reset()`

Devuelve todos los campos a su valor inicial [163]. Lo uso tras el registro para dejar el formulario listo para otra persona.



¿Es suficiente con esto?

Para muchos formularios, sí. Pero tiene limitaciones:

- Los mensajes los escribe el navegador, en su idioma y con su estilo.
- No sirve para reglas propias, como «las dos contraseñas deben coincidir».

Para eso está la API de validación, que veréis en los ejercicios 04 y 05.



⚠️ Importante

La validación en el navegador es una **comodidad para el usuario**, no una medida de seguridad. Cualquiera puede saltársela. Si los datos se envían a un servidor, el servidor tiene que volver a validarlos siempre [18].

Ejercicio 04 · Mensajes de error propios con la API de validación

🎯 **Objetivo:** Desactivar los avisos del navegador con novalidate y usar la API de validación (checkValidity y la propiedad validity) para mostrar mensajes de error propios bajo cada campo, marcarlos con aria-invalid y llevar el foco al primer error.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Mantener las reglas de validación del HTML, pero mostrar los mensajes con nuestro propio texto y diseño.



`novalidate`: el navegador valida, pero no avisa

```
<form id="registro" novalidate>
```

El atributo `novalidate` indica que el formulario no debe validarse al enviarse [169]. Ojo con lo que eso significa: las reglas (`required`, `min`, `pattern`…) **siguen existiendo** y el navegador sigue sabiendo si cada campo las cumple. Lo único que desaparece son sus avisos automáticos, y el `submit` se dispara siempre. Ahora sois vosotros quienes decidís qué hacer.



`validity`: qué regla falla

```
campo.validity.valueMissing    // true si es obligatorio y está vacío
campo.validity.typeMismatch    // true si no tiene formato de email
campo.validity.tooShort        // true si no llega a minlength
…
campo.validity.valid           // true si no falla ninguna
```

Cada campo tiene una propiedad `validity`, un objeto `ValidityState` con una propiedad booleana por cada tipo de problema [155]. Tenéis la tabla completa en la infografía de la teoría. `mensajeError` las consulta una a una y devuelve el mensaje que corresponde.

Fijaos en que los mensajes usan los propios atributos: `campo.minLength`, `campo.min` y `campo.max`. Si mañana cambiáis el mínimo en el HTML, el mensaje se actualiza solo.



`checkValidity()`: ¿todo bien?

```
if (!formulario.checkValidity()) { … }
```

`checkValidity()` devuelve `true` si ningún control del formulario tiene problemas de validez y `false` en caso contrario [156].



Mostrar el error donde toca

```
parrafoError.textContent = `⚠ ${mensaje}`;
campo.setAttribute('aria-invalid', 'true');
```

Cada campo tiene debajo su propio párrafo de error, enlazado mediante `aria-describedby`:

```
<input id="email" aria-describedby="error-email">
<p id="error-email" class="error-campo"></p>
```

- `aria-describedby` le dice a un lector de pantalla que ese párrafo describe el campo, así que leerá el error al llegar a él [160].
- `aria-invalid="true"` anuncia que el valor no es válido [159]. Además, lo aprovecho en el CSS para pintar el borde en rojo.

El ejercicio 06 profundiza en la accesibilidad de los errores.



Llevar al usuario al error

```
formulario.querySelector('[aria-invalid="true"]').focus();
```

Si hay errores, pongo el foco en el primero [161]. El usuario no tiene que buscar qué ha fallado.



Dos momentos de validación

Cuándo	Por qué
`blur`, al salir del campo	El usuario se entera del error enseguida, sin esperar al final
`submit`	Comprobación final de **todos** los campos, incluidos los que no ha tocado

¿Por qué no validar con `input`, en cada letra? Porque mientras alguien está escribiendo su email, durante un buen rato «no es válido», y verlo en rojo desde la primera letra es muy molesto.

Ejercicio 05 · Contraseñas: reglas propias con setCustomValidity

> 🎯 **Objetivo:** Añadir reglas de validación propias a un campo con setCustomValidity, mostrar en tiempo real qué requisitos se cumplen y usar reportValidity para que el navegador muestre los avisos al enviar.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Validar una contraseña con reglas que no existen como atributo HTML y comprobar que las dos contraseñas coinciden.



`setCustomValidity`: vuestras propias reglas

```
campo.setCustomValidity('Las contraseñas no coinciden.');   // inválido, con este mensaje
campo.setCustomValidity('');                                // válido otra vez
```

`setCustomValidity()` asigna un mensaje de error propio a un campo [158]. Mientras ese mensaje no esté vacío, el campo se considera **inválido**: `validity.customError` es `true` y `checkValidity()` devuelve `false`, exactamente igual que con un `required` incumplido [19].



> 🚨 **Atención**
>
> Hay que **borrar el error** con `setCustomValidity('')` cuando el valor vuelva a ser correcto. Si no, el campo seguirá marcado como inválido para siempre. La guía de MDN lo hace así en todos sus ejemplos [19].

Por eso las dos funciones terminan siempre en un ternario: o mensaje, o texto vacío.



Comprobar caracteres

```
const caracteres = [...texto];
caracteres.some((c) => c >= 'A' && c <= 'Z');   // ¿alguna mayúscula?
caracteres.some((c) => c >= '0' && c <= '9');   // ¿algún número?
```

El *spread* del bloque 05 convierte el texto en un array de caracteres. Y con `some` pregunto si **alguno** cumple la condición. Comparar letras con `>=` y `<=` funciona porque se comparan por su código: las mayúsculas A–Z van seguidas, igual que los dígitos 0–9. Ojo: con este método la «Ñ» no cuenta como mayúscula, porque no está en ese rango.



Requisitos en vivo

```
requisitos.forEach((li) => {
    li.classList.toggle('cumplido', cumple[li.dataset.regla]);
});
```

Cada `li` dice en su `data-regla` qué regla representa (`longitud`, `mayuscula`, `numero`), y esa palabra coincide con una clave del objeto que devuelve `comprobarRequisitos`. Así, con `cumple[li.dataset.regla]`, cada `li` sabe si debe marcarse. El CSS cambia la ✘ por una ✔.



Dos campos que dependen uno del otro

```
clave.addEventListener('input', () => {
    actualizarClave();
    actualizarRepetir();
});
```

Si el usuario escribe las dos contraseñas iguales y después cambia la primera, ya no coinciden. Por eso, al cambiar la clave, también reviso la repetición.



`reportValidity()`

```
if (!formulario.reportValidity()) {
    return;
}
```

`reportValidity()` hace lo mismo que `checkValidity()`, pero además **muestra los avisos** del navegador junto a los campos con problemas [157]. Con un `setCustomValidity`, el aviso muestra vuestro mensaje. Es la forma más sencilla de tener reglas propias sin construir todo el sistema de mensajes del ejercicio 04.

Un detalle de acceso: `formulario.usuario`

`formulario.usuario.value` accede al campo cuyo `name` es `usuario`. Los formularios exponen sus controles por nombre, igual que `FormData`.

Ejercicio 06 · Formulario de contacto accesible

> 🎯 **Objetivo:** Mostrar los errores de un formulario de forma accesible: mensaje junto a cada campo enlazado con aria-describedby, estado con aria-invalid, validación al salir del campo y en vivo una vez marcado, y un resumen de errores al enviar con enlaces a cada campo.
>
> 📂 Enunciado · Solución completa

Qué pedía el ejercicio

Un formulario de contacto cuyos errores se entiendan bien tanto mirando la pantalla como con un lector de pantalla o solo con el teclado.



📊 Figura 53 · Errores accesibles

Las cuatro piezas de un error accesible

1. Texto, no solo color

Cada error tiene un mensaje escrito que dice **qué está mal y cómo arreglarlo**: «Escribe al menos 20 caracteres (llevas 12)». El borde rojo ayuda, pero no basta: no todo el mundo distingue los colores. La guía de validación de MDN insiste en que los mensajes sean claros y digan cómo corregir el problema [18].

2. `aria-invalid`

```
campo.setAttribute('aria-invalid', 'true');
```

Le dice a la tecnología de apoyo que el valor del campo no es válido [159]. MDN recomienda no ponerlo a `true` antes de que se haya validado, solo como respuesta a una validación [159]. Por eso lo pongo al salir del campo o al enviar, nunca al cargar la página.

3. `aria-describedby`

```
<input id="email" aria-describedby="error-email">
<p id="error-email" class="error-campo"></p>
```

Enlaza el campo con el párrafo que lo describe [160]. Cuando el usuario de un lector de pantalla llega al campo, escucha la etiqueta... y a continuación el error.

4. El resumen de errores

```
<div id="resumenErrores" class="resumen-errores" role="alert" tabindex="-1"></div>
```

Al enviar con errores, aparece arriba un resumen con **un enlace por error** que lleva directamente al campo. Le pongo el foco con `focus()` [161], para que sea lo primero que se lea. `tabindex="-1"` permite darle el foco desde JavaScript aunque un `div` no sea normalmente enfocable, y `role="alert"` hace que se anuncie al aparecer.



El mensaje de éxito: `aria-live`

```
<p id="exito" class="exito" aria-live="polite"></p>
```

Un lector de pantalla lee lo que tiene delante. Si un texto cambia en otra parte de la página, el usuario no se entera... salvo que esa zona sea una **región viva**. `aria-live` indica que el elemento se actualizará y que esos cambios deben anunciarse; con `polite`, se anuncian cuando el lector termina lo que estaba diciendo [162].



Cuándo validar: el equilibrio

Momento	Qué hago	Por qué
`blur` (salir del campo)	Validar siempre [137]	El usuario ya ha terminado de escribir
`input` (cada letra)	Solo si el campo **ya estaba en error** [134]	El aviso desaparece en cuanto se corrige, pero no molesta mientras se escribe por primera vez
`submit`	Validar todo y mostrar el resumen	Comprobación final, incluidos los campos no tocados

Probad la diferencia: escribid «hola» en el mensaje y salid. Aparece el error. Volved y seguid escribiendo: en cuanto lleguéis a 20 caracteres, el error se va solo.



Probadlo solo con el teclado

Enviad el formulario vacío con Enter. El foco salta al resumen. Con Tab llegáis al primer enlace; con Enter, saltáis a ese campo. Si un formulario se puede usar así, habéis hecho un buen trabajo.

Ejercicio 07 · Formulario dinámico: campos que aparecen

> 🎯 **Objetivo:** Adaptar un formulario a lo que elige el usuario: recalcular el total con input y change, mostrar u ocultar un grupo de campos con el atributo hidden, hacerlos obligatorios solo cuando son visibles y reiniciar el formulario devolviendo la interfaz a su estado inicial.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un formulario que se adapta a lo que elige el usuario: total en vivo, campos de factura que aparecen y desaparecen y un botón para empezar de cero.



Un escuchador para todo el formulario

```
formulario.addEventListener('input', actualizar);
formulario.addEventListener('change', actualizar);
```

Es la delegación del bloque 07: los eventos `input` y `change` de cada campo suben hasta el formulario, así que con dos escuchadores cubro la cantidad, los tres radios y la casilla. ¿Por qué los dos eventos? `input` reacciona mientras se escribe la cantidad [134]; `change`, al marcar un radio o una casilla [135]. Con los dos, la interfaz siempre está al día.



`actualizar()`: la interfaz sigue al formulario

Es el mismo patrón que `renderizar()` en el bloque 07: una función que lee el estado (aquí, lo que hay elegido en el formulario) y pone la página a juego. No importa qué haya cambiado: `actualizar` lo revisa todo.

Mostrar y ocultar con `hidden`

```
datosFactura.hidden = !conFactura;
```

La propiedad `hidden` se corresponde con el atributo HTML del mismo nombre: un elemento con `hidden` no se muestra. Es más claro que añadir una clase solo para ocultar.

`required` solo si el campo está visible

```
nif.required = conFactura;
```

Esto es importante. Si el NIF fuera siempre obligatorio, nadie podría hacer un pedido **sin** factura: el formulario exigiría un campo que ni siquiera se ve. Por eso pongo o quito `required` a la vez que muestro u oculto el grupo. Un campo oculto nunca debe bloquear el envío.



Reiniciar bien

```
btnLimpiar.addEventListener('click', () => {
    formulario.reset();
    actualizar();
});
```

`reset()` devuelve todos los campos a su valor inicial [3]: la casilla de factura se desmarca, la cantidad vuelve a 1… pero **el grupo de factura seguiría visible**, porque `reset` solo toca los valores, no lo que habéis cambiado con JavaScript. Por eso llamo a `actualizar()` justo después.

¿Por qué un botón `type="button"` y no `type="reset"`? Porque con un botón normal controlo el orden: primero `reset()` y **después** `actualizar()`. Así me aseguro de que la interfaz se calcula con los valores ya reiniciados.



Un `pattern` para el NIF

```
<input id="nif" pattern="[0-9]{8}[A-Za-z]">
```

Ocho cifras y una letra [154]. Comprueba el formato, no si el NIF existe de verdad: eso requeriría calcular la letra de control.



Algunos totales

Cajas	Envío	Total
1	Estándar	23.85 €
3	Urgente	67.65 €
2	Recogida	39.80 €

Ejercicio 08 · Inscripciones a un taller

> 🎯 **Objetivo:** Construir un formulario completo que combina todo el bloque: validación nativa y una regla propia (email repetido), mensajes accesibles, FormData, un array de inscritos como estado,

una tabla generada a partir de él con borrado por delegación y un límite de plazas.

📂 Enunciado · Solución completa

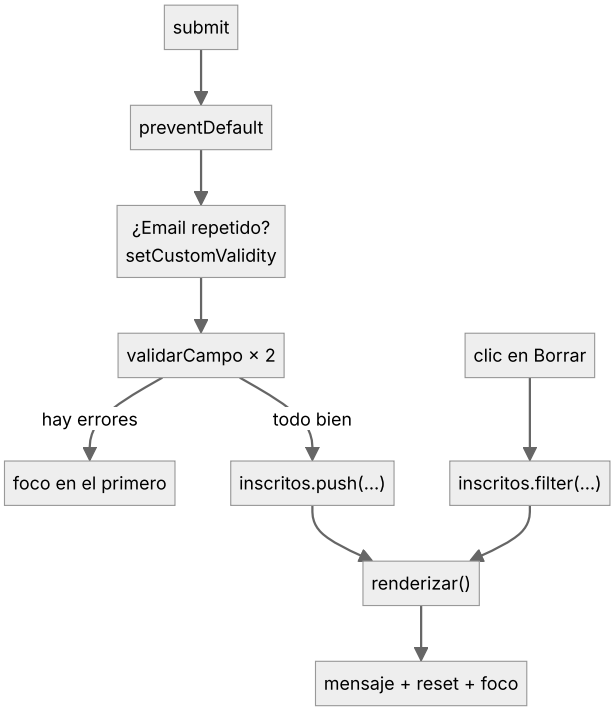


Qué pedía el ejercicio

Una aplicación completa de inscripciones con validación, regla propia, tabla de inscritos y límite de plazas. Junta todo lo del bloque y buena parte de los bloques 06 y 07.



El mapa de la aplicación





Una regla que HTML no puede saber

```
const repetido = inscritos.some((p) => p.email === email);
campoEmail.setCustomValidity(repetido ? 'Este email ya está inscrito.' : '');
```

Ningún atributo HTML sabe quién se ha inscrito ya: eso solo lo sabe vuestro array. Así que compruebo con `some` y lo comunico con `setCustomValidity` [158]. A partir de ahí, el campo se comporta como si incumpliera cualquier otra regla: `validity.customError` es `true` [155] y el mensaje que escribí está en `campo.validationMessage`. Por eso `mensajeError` solo necesita un caso más.

Y recordad la advertencia del ejercicio 05: hay que **borrar** el error. Lo hago en dos sitios: al volver a enviar (el ternario pone `''` si ya no está repetido) y en cuanto el usuario escribe en el campo.

Comparo en minúsculas para que `Lucia@Ejemplo.com` y `lucia@ejemplo.com` cuenten como el mismo email.



Validar sin cortocircuito

```
const nombreOk = validarCampo(campoNombre);
const emailOk = validarCampo(campoEmail);
if (!nombreOk || !emailOk) { … }
```

¿Por qué no `if (!validarCampo(campoNombre) || !validarCampo(campoEmail))` ? Por el cortocircuito del bloque 03: si el nombre fallara, `||` ya sabría el resultado y **no llegaría a validar el email**. El usuario vería un error, lo corregiría y entonces descubriría el segundo. Validando antes los dos, ve todos los errores a la vez.



Estado → renderizar, otra vez

`inscritos` es el estado. El `submit` lo amplía, el botón Borrar lo reduce, y `renderizar()` repinta la tabla, las plazas y el botón. Es exactamente la arquitectura del catálogo del bloque 07. Los botones Borrar se recrean en cada renderizado, así que uso **delegación** en el `tbody`, con `closest` para encontrar el botón [145].



El límite de plazas

```
btnInscribir.disabled = libres === 0;
```

Con el taller completo, el botón se desactiva. Y como `renderizar()` se ejecuta también al borrar, en cuanto alguien se da de baja vuelve a activarse. No hay que acordarse de hacerlo en dos sitios.



Después de inscribir

```
formulario.reset();
campoNombre.focus();
```

El formulario queda limpio [163] y el cursor vuelve al primer campo, listo para la siguiente persona [161]. Y el mensaje de éxito está en una región `aria-live` [162], así que también se anuncia a quien usa lector de pantalla.



Bloque 09

Bloque 09 · Almacenamiento en el navegador

Hasta ahora, todo lo que hacían vuestras aplicaciones se esfumaba al recargar la página: las tareas, el carrito, las preferencias. Volvíais y estaba todo como al principio. En este bloque os voy a enseñar a **darle memoria a la página**: guardar datos en el navegador del usuario y recuperarlos en su próxima visita, sin servidor y sin base de datos. Y también os voy a contar qué cosas no debéis guardar nunca ahí.

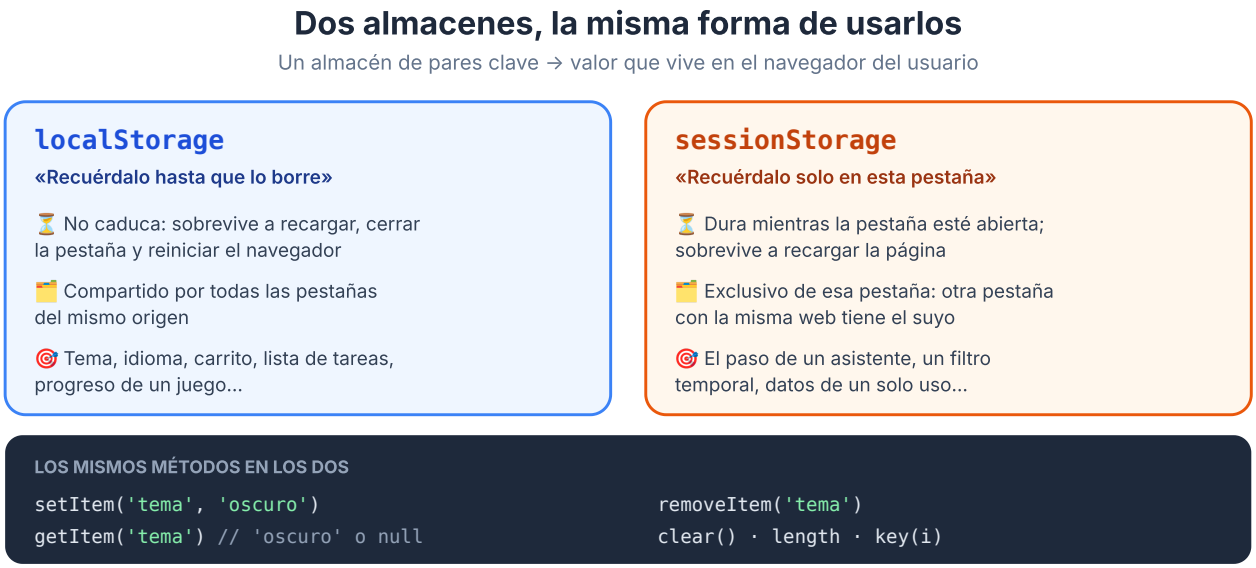
Al terminar este bloque sabréis:

- Guardar, leer y borrar datos con `localStorage` y `sessionStorage`.
- Elegir entre los dos según cuánto deban durar los datos.
- Guardar arrays y objetos con JSON sin perder nada por el camino.
- Aplicar el patrón **cargar al empezar, guardar en cada cambio**.
- Cargar datos guardados de forma segura, aunque estén corruptos o manipulados.
- Sincronizar varias pestañas con el evento `storage`.
- Decidir qué datos se pueden guardar en el navegador y cuáles no.



1. Dos almacenes en el navegador

La Web Storage API ofrece dos almacenes de pares **clave → valor**: `localStorage` y `sessionStorage` [170].



📊 Figura 54 · localStorage frente a sessionStorage

	localStorage	sessionStorage
Cuánto dura	No caduca: sobrevive a recargar y a cerrar el navegador [21]	Mientras la pestaña esté abierta; sobrevive a recargar [171]
Quién lo ve	Todas las pestañas del mismo origen [21]	Solo esa pestaña [171]
Para qué	Tema, idioma, carrito, tareas, progreso de un juego	El paso de un asistente, un filtro temporal

1.1. Los métodos

Los dos almacenes tienen exactamente los mismos métodos [4]:

```
localStorage.setItem('tema', 'oscuro');   // guardar o sobrescribir [5]
localStorage.getItem('tema');             // leer: 'oscuro', o null si no existe [6]
localStorage.removeItem('tema');          // borrar una clave [7]
localStorage.clear();                     // borrar TODO el almacén [8]
localStorage.length;                      // cuántas claves hay [9]
localStorage.key(0);                      // el nombre de la clave en la posición 0 [10]
```

Grabaos el `null` de `getItem`: es vuestra forma de saber si había algo guardado.

1.2. El almacén es del origen, no de la página

El almacén pertenece a un **origen** (protocolo + dominio + puerto), no a una página concreta [21]. Todas las páginas de `https://mitienda.com` comparten el mismo `localStorage`. En el curso pasa lo mismo: todos los ejercicios abiertos con Live Server, o con doble clic desde el disco, comparten almacén.

Por eso, a partir del ejercicio 02, cada ejercicio usa claves con **un prefijo propio**:

```
const CLAVE = 'curso-js:b09e02:nombre';
```

Y por eso `clear()` es peligroso: borra también lo que hayan guardado las demás páginas del origen. Usad `removeItem` con vuestras claves.



📝 Nota

MDN advierte de que el almacén puede no estar disponible con orígenes `file:` [21]. Chrome y Edge lo permiten, pero si algún ejercicio no guarda nada al abrirlo con doble clic, usad **Live Server**.

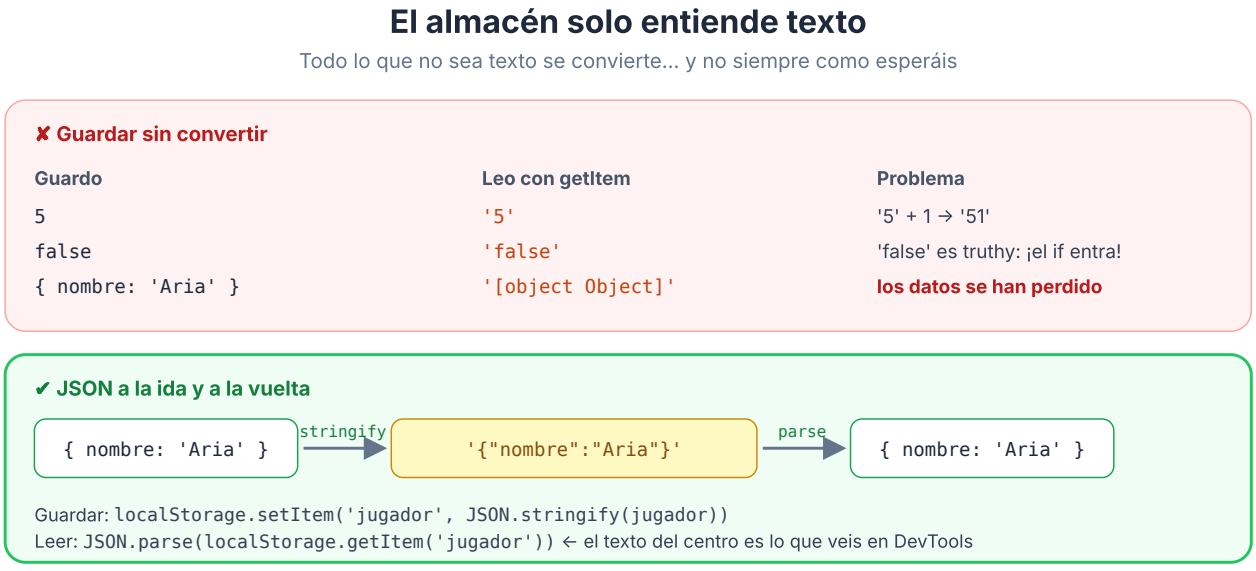
1.3. Miradlo con vuestros ojos

Herramientas de desarrollo → pestaña **Aplicación** (Chrome, Edge) o **Almacenamiento** (Firefox) → *Local Storage*. Veréis todas las claves, podréis editarlas y borrarlas. Tenedlo abierto durante todo el bloque.



2. Solo guarda texto

Este es el punto en el que más gente tropieza. `setItem` guarda **texto**; todo lo demás se convierte en texto por el camino [172].



📊 Figura 55 · El almacén solo entiende texto

Guardo	Leo	Problema
`5`	`'5'`	`'5' + 1` → `'51'`
`false`	`'false'`	Es un texto no vacío: *truthy*. El `if` entra
`{ nombre: 'Aria' }`	`'[object Object]'`	Los datos se han perdido
`['a', 'b']`	`'a,b'`	Ya no es un array

2.1. La solución: JSON

```
// guardar: valor → texto JSON
localStorage.setItem('jugador', JSON.stringify(jugador));

// leer: texto JSON → valor
const jugador = JSON.parse(localStorage.getItem('jugador'));
```

`JSON.stringify()` convierte el valor en un texto JSON [105] y `JSON.parse()` lo reconstruye [106]. JSON conserva los tipos: los números vuelven como números y los booleanos como booleanos.

Mi regla: si no es un texto sencillo, JSON a la ida y a la vuelta.

2.2. Leer con valor por defecto

La primera vez que alguien visita la página no hay nada guardado. Preparaos siempre para ese caso:

```
const tema = localStorage.getItem('tema') ?? 'claro';     // texto
const texto = localStorage.getItem('tareas');
const tareas = texto === null ? [] : JSON.parse(texto);    // JSON
```

El operador `??` devuelve el valor de la derecha solo si el de la izquierda es `null` o `undefined`. Es como `||`, pero no salta con un texto vacío.



3. El patrón: cargar al empezar, guardar en cada cambio



📊 Figura 56 · Cargar al empezar, guardar en cada cambio

Si vuestra aplicación ya sigue el patrón **estado → renderizar** del bloque 07, hacerla persistente es cuestión de dos funciones:

```js
const CLAVE = 'curso-js:tareas';

function cargar() {
    const texto = localStorage.getItem(CLAVE);
    return texto === null ? [] : JSON.parse(texto);
}

function guardar() {
    localStorage.setItem(CLAVE, JSON.stringify(tareas));
}

// 1. al empezar: el estado se carga, no se escribe a mano
let tareas = cargar();
renderizar();

// 2. en cada evento: cambiar el estado → guardar → renderizar
formulario.addEventListener('submit', (e) => {
    e.preventDefault();
    tareas.push({ texto: campo.value, hecha: false });
    guardar();
    renderizar();
});
```

Guardad **el estado completo** cada vez, no solo lo que ha cambiado: es más sencillo y el almacén nunca se desincroniza de lo que hay en memoria.



4. Cargar a la defensiva

Lo que hay en el almacén **no es de fiar**. El usuario puede editarlo desde las herramientas de desarrollo, una versión antigua de vuestra web puede haber guardado otro formato o los datos pueden corromperse. Si `JSON.parse` recibe un texto que no es JSON válido, lanza un `SyntaxError` [106] y vuestro script se detiene.

```
function cargarCarrito() {
    const texto = localStorage.getItem(CLAVE);
    if (texto === null) {
        return [];
    }
    let datos;
    try {
        datos = JSON.parse(texto);     // puede fallar…
    } catch (error) {
        return [];                     // …y si falla, empiezo de cero
    }
    if (!Array.isArray(datos)) {       // ¿es de verdad un array?
        return [];
    }
    return datos.filter((linea) => …); // ¿cada elemento tiene sentido?
}
```

`try...catch` intenta ejecutar un código y, si lanza un error, lo captura en lugar de detener el programa [173]. Lo veréis a fondo en el bloque 10; este es su uso más habitual. Y aunque el texto sea JSON válido, puede no ser lo que esperáis: `Array.isArray()` comprueba que lo leído es de verdad un array [174], y el `filter` final descarta los elementos sin sentido.



5. Sincronizar pestañas: el evento `storage`



Figura 57 · El evento storage entre pestañas

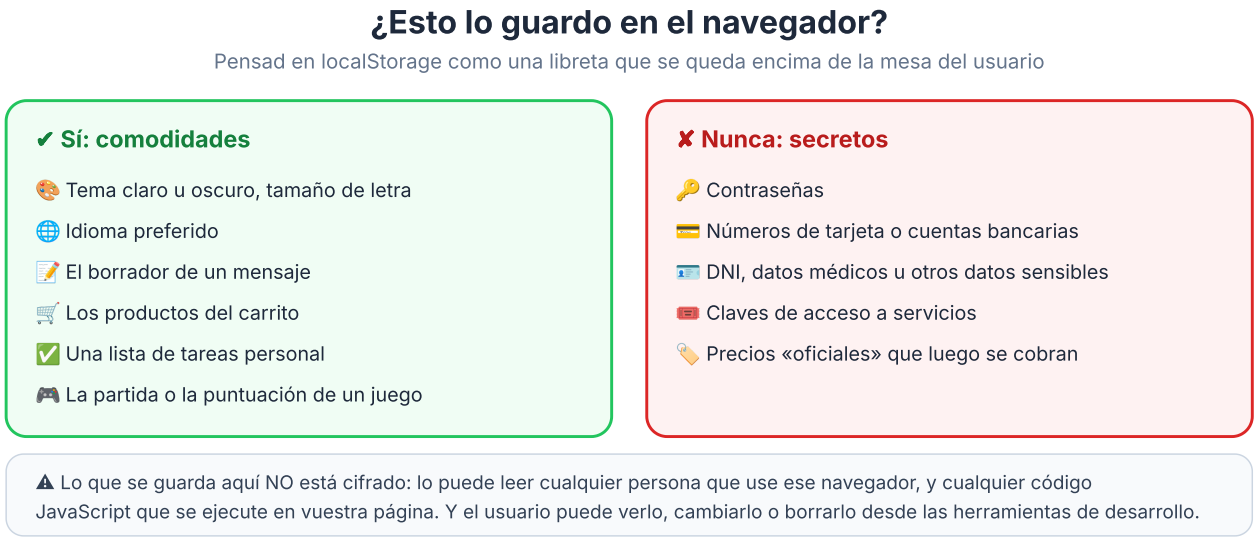
Si el usuario tiene vuestra tienda abierta en dos pestañas y añade algo al carrito en una, la otra no se entera... salvo que escuche el evento `storage`:

```
window.addEventListener('storage', (e) => {
    if (e.key === CLAVE) {
        carrito = cargarCarrito();
        renderizar();
    }
});
```

`storage` se dispara cuando **otro documento** del mismo origen modifica el almacén [175]. No salta en la pestaña que hace el cambio, solo en las demás. `e.key` indica qué clave ha cambiado.



6. Qué guardar y qué no



📊 Figura 58 · ¿Esto lo guardo en el navegador?

El almacenamiento del navegador **no está cifrado** ni protegido. Cualquier persona que use ese navegador puede leerlo, cualquier código JavaScript que se ejecute en vuestra página tiene acceso a él, y el usuario puede verlo y cambiarlo con las herramientas de desarrollo.

Así que:

- **Sí:** preferencias, borradores, listas personales, el contenido de un carrito (como identificadores y cantidades), el progreso de un juego.
- **Nunca:** contraseñas, datos bancarios, documentos de identidad, datos médicos ni claves de acceso.
- **Nunca os fiéis de lo que leéis:** guardad el **identificador** de un producto, no su precio. El precio sale siempre de vuestro código (y, en una tienda real, del servidor). Si no, bastaría con editar el almacén para comprar a céntimo.

Y tened en cuenta que el usuario puede borrarlo en cualquier momento, y que en el modo de navegación privada los navegadores tratan `localStorage` como si fuera `sessionStorage` [170]. Vuestra aplicación tiene que funcionar también cuando el almacén está vacío.



7. La plantilla de trabajo con persistencia

```
// estructura del programa principal
const CLAVE = 'curso-js:b09:tareas';

// funciones
function cargar() {
    const texto = localStorage.getItem(CLAVE);
    return texto === null ? [] : JSON.parse(texto);
}
function guardar() {
    localStorage.setItem(CLAVE, JSON.stringify(tareas));
}
function renderizar() { … }

// el estado, CARGADO del almacén
let tareas = cargar();

// constantes al DOM que queremos escuchar
const formulario = document.querySelector('#formTarea');

// colocar escuchadores de eventos
formulario.addEventListener('submit', (e) => {
    e.preventDefault();
    // entradas → procesamiento (cambiar el estado) → guardar → salidas
    tareas.push({ texto: formulario.tarea.value });
    guardar();
    renderizar();
});

renderizar();
```



8. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Mi primer almacén	`setItem`, `getItem`, `removeItem`, `clear`, `length`, `key`, el origen compartido	Ver
02	Recuerda mi nombre	Cargar al empezar, el `null` de `getItem`, claves con prefijo	Ver
03	Preferencias de tema y tamaño	Preferencias en `data-*` del `body`, valor por defecto con `??`	Ver
04	localStorage frente a sessionStorage	Los dos almacenes con la misma función, números en el almacén	Ver
05	El almacén solo guarda texto	Siete experimentos con tipos, JSON a la ida y a la vuelta	Ver
06	Un borrador que no se pierde	Autoguardado de varios campos como un objeto JSON	Ver
07	Lista de tareas que persiste	El proyecto guiado: estado → guardar → renderizar	Ver
08	Un carrito que sobrevive	Guardar solo ids, carga segura con `try...catch`, evento `storage`	Ver

En el 08, haced el experimento de la ayuda: escribid `hola` en el valor guardado desde las herramientas de desarrollo y recargad. Si vuestra tienda sigue en pie, lo habéis hecho bien.



🧪 Los ejercicios, explicados

Ejercicio 01 · Mi primer almacén

> 🎯 **Objetivo:** Usar los métodos básicos de localStorage (setItem, getItem, removeItem y clear) y recorrer su contenido con length y key(), comprobando que los datos siguen ahí después de recargar la página.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Guardar, leer y borrar datos en `localStorage` y comprobar que sobreviven a una recarga.



Los cuatro métodos

```
localStorage.setItem('color', 'azul');   // guardar (o sobrescribir)
localStorage.getItem('color');           // leer → 'azul'
localStorage.removeItem('color');        // borrar una clave
localStorage.clear();                    // borrar TODO
```

`localStorage` es un almacén de pares **clave → valor** que el navegador guarda para vuestro origen y que no caduca: sobrevive a recargar la página y a cerrar el navegador [21].

Método	Qué hace
`setItem(clave, valor)`	Crea la clave o, si ya existía, cambia su valor [172]
`getItem(clave)`	Devuelve el valor, o **`null`** si esa clave no existe [176]
`removeItem(clave)`	Borra esa clave [177]
`clear()`	Borra todas las claves del almacén [178]

Fijaos en el `null` de `getItem`: es vuestra forma de saber si había algo guardado. Lo usaréis en casi todos los ejercicios.



Recorrer el almacén

```
for (let i = 0; i < localStorage.length; i++) {
    const clave = localStorage.key(i);
    …
}
```

`length` dice cuántos elementos hay [179] y `key(i)` devuelve el nombre de la clave que ocupa la posición `i` [180]. No contéis con ningún orden concreto: depende del navegador.



Una sorpresa: claves que no son vuestras

Si ya habéis hecho otros ejercicios de este bloque, en la lista aparecen claves como `curso-js:b09e02:nombre`. ¿Por qué? Porque `localStorage` se comparte **por origen** [1]: todas las páginas del mismo sitio ven el mismo almacén. Con Live Server, todas las páginas de `http://127.0.0.1:5500` comparten almacén; abriendo los ficheros con doble clic, Chrome comparte uno para todos los ficheros locales.

Dos consecuencias prácticas:

- Por eso, en el resto del bloque, cada ejercicio usa claves con **un prefijo propio**: `curso-js:b09e03:tema`. Así no se pisan.
- **`clear()` es un martillo:** borra también lo de los demás ejercicios. Por eso pido confirmación. En una aplicación real, casi siempre preferiréis `removeItem` con las claves que son vuestras.



Comprobadlo en las herramientas de desarrollo

Pestaña **Aplicación** en Chrome y Edge, o **Almacenamiento** en Firefox, apartado *Local Storage*. Veréis vuestras claves, podéis editarlas a mano y borrarlas. Recordadlo para el ejercicio 08: si vosotros podéis tocarlo, el usuario también.

Ejercicio 02 · Recuerda mi nombre

> 🎯 **Objetivo:** Aplicar el patrón básico de persistencia: al cargar, comprobar si hay un dato guardado con getItem y adaptar la página; al cambiarlo, guardarlo con setItem; y borrarlo con removeItem.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Que la página recuerde el nombre del usuario entre visitas y lo salude al volver.



El patrón: cargar al empezar

```
renderizar();   // última línea del programa
```

Lo primero que hace la página al cargar es preguntar al almacén si ya nos conocemos:

```
const nombre = localStorage.getItem(CLAVE);
```

`getItem` devuelve el valor guardado o `null` si esa clave no existe [176]. Con ese dato, `renderizar` decide qué zona mostrar. La primera vez aparece la pregunta; las siguientes, el saludo.



Guardar al cambiar

```
localStorage.setItem(CLAVE, nombre);
renderizar();
```

Cuando el usuario envía su nombre, lo guardo [172] y vuelvo a pintar. Fijaos en que `renderizar` **lee siempre del almacén**: no le paso el nombre como parámetro. Así hay una única fuente de la verdad, y la página se ve igual tanto si acabáis de guardar como si venís de otra visita.



Olvidar

```
localStorage.removeItem(CLAVE);
```

Borra solo esa clave [177]. Tras renderizar, `getItem` vuelve a dar `null` y reaparece la pregunta.



La clave con prefijo

```
const CLAVE = 'curso-js:b09e02:nombre';
```

Como os conté en el ejercicio 01, todas las páginas del mismo origen comparten el almacén [21]. Si dos ejercicios usaran la clave `nombre`, uno sobrescribiría el del otro. El prefijo lo evita. Y al guardarla en una constante, solo la escribo una vez: imposible equivocarse con una letra en uno de los tres sitios donde se usa.



`hidden` para mostrar y ocultar

```
zonaPregunta.hidden = hayNombre;
zonaSaludo.hidden = !hayNombre;
```

Lo visteis en el bloque 08: una línea por zona, y siempre una es lo contrario de la otra.

Ejercicio 03 · Preferencias de tema y tamaño de letra

> 🎯 **Objetivo:** Guardar las preferencias del usuario en localStorage, aplicarlas a la página mediante atributos data-* en el body y recuperarlas al cargar, usando un valor por defecto cuando no hay nada guardado.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Guardar el tema y el tamaño de letra elegidos y aplicarlos automáticamente en cada visita.



Las preferencias viven en el `body`

```
document.body.dataset.tema = tema;       // <body data-tema="oscuro">
document.body.dataset.tamano = tamano;   // <body data-tamano="grande">
```

Con `dataset` escribo atributos `data-*` en el `body` [181]. Y el `style.css` ya tiene las reglas:

```
body[data-tema="oscuro"] {
    background-color: #0f172a;
    color: #e2e8f0;
}
```

Es la regla del bloque 06 aplicada a toda la página: **JavaScript decide el estado, CSS decide el aspecto**. Si mañana queréis un tercer tema, solo tocáis el CSS.



Leer con valor por defecto

```
const temaGuardado = localStorage.getItem(CLAVE_TEMA) ?? 'claro';
```

La primera vez no hay nada guardado y `getItem` devuelve `null` [176]. El operador `??` devuelve lo de la derecha **solo** si lo de la izquierda es `null` o `undefined`. Es muy parecido al `||` que conocéis, con una diferencia: `||` también saltaría con un texto vacío `''`, y `??` no. Para leer del almacén, `??` es el más preciso.



Guardar al cambiar

```
selectTema.addEventListener('change', () => {
    localStorage.setItem(CLAVE_TEMA, selectTema.value);
    aplicar(selectTema.value, selectTamano.value);
});
```

El evento `change` de un `select` salta al elegir otra opción. Guardo el valor [172] y aplico.



`aplicar` también mueve los controles

```
selectTema.value = tema;
```

Al cargar, los `select` muestran la opción que tienen marcada en el HTML. Si el usuario había elegido «Oscuro», el desplegable debe decir «Oscuro», no «Claro». Por eso `aplicar` también coloca los controles: la página y los controles cuentan siempre la misma historia.



Dos claves o un objeto

Aquí uso una clave por preferencia, porque cada una es un simple texto. Si tuvierais diez preferencias, sería más cómodo guardarlas juntas en un objeto. Para eso hace falta JSON, y es justo lo que veréis en el ejercicio 05.

Ejercicio 04 · localStorage frente a sessionStorage

> 🎯 **Objetivo:** Comparar en la práctica localStorage y sessionStorage con dos contadores de visitas, convirtiendo a número lo que se lee del almacén.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Comprobar con dos contadores de visitas en qué se diferencian los dos almacenes del navegador.



Una función para los dos almacenes

```
function contarVisita(almacen, clave) {
    const anteriores = Number(almacen.getItem(clave));
    const total = anteriores + 1;
    almacen.setItem(clave, total);
    return total;
}

contarVisita(localStorage, CLAVE_LOCAL);
contarVisita(sessionStorage, CLAVE_SESION);
```

`localStorage` y `sessionStorage` son dos objetos del mismo tipo, `Storage`, con los mismos métodos [182]. Por eso una sola función sirve para los dos: recibe el almacén como parámetro. Es el bloque 04 en estado puro.



Leer números del almacén

El almacén guarda **texto**. Lo que leo es `'3'`, no `3`, así que convierto con `Number()` antes de sumar. Si no, `'3' + 1` daría `'31'`.

¿Y la primera vez, cuando no hay nada? `getItem` devuelve `null`, y `Number(null)` vale `0` [78]. Aquí esa rareza que tanto os advertí en el bloque 04 juega a nuestro favor: la primera visita cuenta desde 0. Pero sabed que lo estáis aprovechando a propósito.



El experimento

Acción	`localStorage`	`sessionStorage`
Recargar la página	Sigue sumando	Sigue sumando
Abrir otra pestaña con la misma página	Sigue sumando	**Empieza en 1**
Cerrar todo y volver	Sigue sumando	**Empieza en 1**

- `localStorage` no tiene fecha de caducidad y es **el mismo para todo el origen** [21].
- `sessionStorage` dura lo que dura la sesión de la pestaña: sobrevive a las recargas, pero cada pestaña tiene **la suya**, y se borra al cerrarla [171].

Dos almacenes, la misma forma de usarlos

Un almacén de pares clave → valor que vive en el navegador del usuario

`localStorage`

«Recuérdalo hasta que lo borre»

⏳ No caduca: sobrevive a recargar, cerrar la pestaña y reiniciar el navegador

🗂️ Compartido por todas las pestañas del mismo origen

🎯 Tema, idioma, carrito, lista de tareas, progreso de un juego...

`sessionStorage`

«Recuérdalo solo en esta pestaña»

⏳ Dura mientras la pestaña esté abierta; sobrevive a recargar la página

🗂️ Exclusivo de esa pestaña: otra pestaña con la misma web tiene el suyo

🎯 El paso de un asistente, un filtro temporal, datos de un solo uso...

LOS MISMOS MÉTODOS EN LOS DOS

```
setItem('tema', 'oscuro')
getItem('tema') // 'oscuro' o null
removeItem('tema')
clear() · length · key(i)
```

Figura 59 · localStorage frente a sessionStorage



¿Cuándo uso cada uno?

Necesito que se recuerde...	Uso
Entre visitas, días después	`localStorage`
Solo mientras dura esta pestaña	`sessionStorage`

Un ejemplo de `sessionStorage`: un formulario de varios pasos. Si el usuario recarga en el paso 3, no pierde lo que ha escrito; pero si cierra la pestaña y vuelve mañana, empieza de cero, que es lo esperable.

Ejercicio 05 · El almacén solo guarda texto: JSON al rescate

> 🎯 **Objetivo:** Comprobar que localStorage convierte en texto todo lo que se guarda (números, booleanos, objetos y arrays), con los errores que eso provoca, y resolverlo guardando con JSON.stringify y leyendo con JSON.parse.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Descubrir qué le pasa a cada tipo de dato al guardarlo en `localStorage` y aprender a guardar arrays y objetos sin perder nada.



Los resultados

Experimento	Resultado
1. Número `5`, más 1	`'51'` (typeof: `string`)
2. `false` en un `if`	**Entra**: es el texto `'false'`
3. Objeto sin convertir	`[object Object]`
4. Array sin convertir	`espada,escudo`
5. Objeto con JSON	nombre: Aria · nivel: 3 (typeof nivel: `number`)
6. Texto guardado en el caso 5	`{"nombre":"Aria","nivel":3}`
7. `JSON.parse` de algo que no existe	`null`

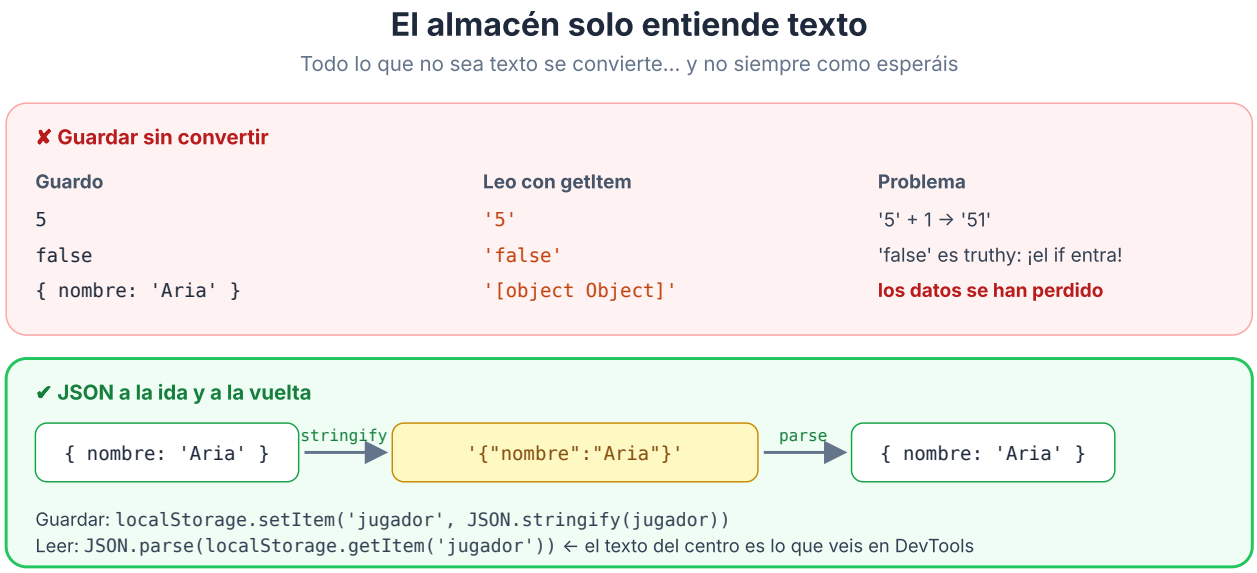


Por qué pasa

`setItem` solo guarda **texto**. Si le dais otra cosa, la convierte en texto antes de guardarla [172]. Y esa conversión no siempre os conviene:

- **Números:** `5` → `'5'`. Al sumar, se concatena. Lo visteis en el bloque 01 con `prompt`: mismo problema, otro sitio.
- **Booleanos:** `false` → `'false'`. Y como `'false'` es un texto no vacío, es *truthy*: el `if` entra. Este error es especialmente traicionero porque no da ningún aviso.
- **Objetos:** se convierten en el texto `'[object Object]'`. Los datos **se han perdido** y no hay forma de recuperarlos.

- **Arrays:** se convierten en sus elementos separados por comas. Parece que funciona, pero al leer tenéis un texto, no un array.



📊 Figura 60 · El almacén solo entiende texto



La solución: JSON

```
localStorage.setItem(clave, JSON.stringify(jugador));     // objeto → texto JSON
const jugador = JSON.parse(localStorage.getItem(clave));  // texto JSON → objeto
```

Lo visteis en el bloque 05: `JSON.stringify()` convierte un valor en un texto JSON [105] y `JSON.parse()` reconstruye el valor a partir de ese texto [106]. JSON conserva los tipos: el nivel vuelve como `number`, y un `false` volvería como `false`.

Mi regla para este bloque: **todo lo que no sea un texto sencillo, con JSON a la ida y a la vuelta**.



Cuidado con el caso 7

```
JSON.parse(localStorage.getItem('no-existe'))   // null
```

Si la clave no existe, `getItem` devuelve `null`, y `JSON.parse(null)` devuelve `null` sin protestar. Si a continuación hacéis `datos.nombre`, tendréis el `TypeError` de siempre. Por eso, al cargar, comprobad siempre si había algo guardado y, si no, usad un valor inicial. En los ejercicios 07 y 08 lo meteréis en una función `cargar()`.

Ejercicio 06 · Un borrador que no se pierde

> 🎯 **Objetivo:** Guardar automáticamente un borrador de varios campos como un único objeto JSON mientras el usuario escribe, recuperarlo al cargar la página y borrarlo al enviar o descartar.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Guardar automáticamente lo que el usuario va escribiendo para que no lo pierda si recarga o cierra la página.



Guardar mientras se escribe

```
formMensaje.addEventListener('input', guardarBorrador);
```

Con delegación (el evento `input` de cada campo sube hasta el formulario), una sola línea guarda el borrador con cada letra, se escriba en el asunto o en el mensaje.

¿Guardar con cada tecla no es demasiado? Para un texto de este tamaño, no: `setItem` es una operación rápida. Si guardarais algo muy grande, o lo enviarais a un servidor, convendría esperar a que el usuario dejara de escribir. Lo veréis en el bloque 10.



Varios campos, una clave

```
const borrador = { asunto: campoAsunto.value, mensaje: campoMensaje.value };
localStorage.setItem(CLAVE, JSON.stringify(borrador));
```

En lugar de una clave por campo, junto todo en un objeto y lo guardo como un único texto JSON [105]. Al leer, `JSON.parse` [106] me devuelve el objeto con los dos campos. Si mañana el formulario tiene cinco campos, solo añado tres propiedades al objeto: la clave y el resto del código no cambian.



Recuperar al cargar

```
const texto = localStorage.getItem(CLAVE);
if (texto === null) {
    return;
}
const borrador = JSON.parse(texto);
```

Primero compruebo si hay algo guardado (el caso 7 del ejercicio 05): `getItem` devuelve `null` si no existe la clave [176]. Solo si hay texto lo convierto y relleno los campos.

El ciclo de vida del borrador

Momento	Qué pasa con el borrador
El usuario escribe	Se guarda
Recarga o vuelve otro día	Se recupera
Envía el mensaje	Se borra: ya no hace falta
Pulsa «Descartar» y confirma	Se borra

Borrarlo al enviar es tan importante como guardarlo: si no, la próxima vez que el usuario abriera la página, encontraría un mensaje que ya envió.

Un detalle de accesibilidad

`#estado` tiene `aria-live="polite"`, como en el bloque 08. Así, quien use un lector de pantalla también se entera de que su borrador se ha guardado o recuperado.

Ejercicio 07 · Lista de tareas que persiste

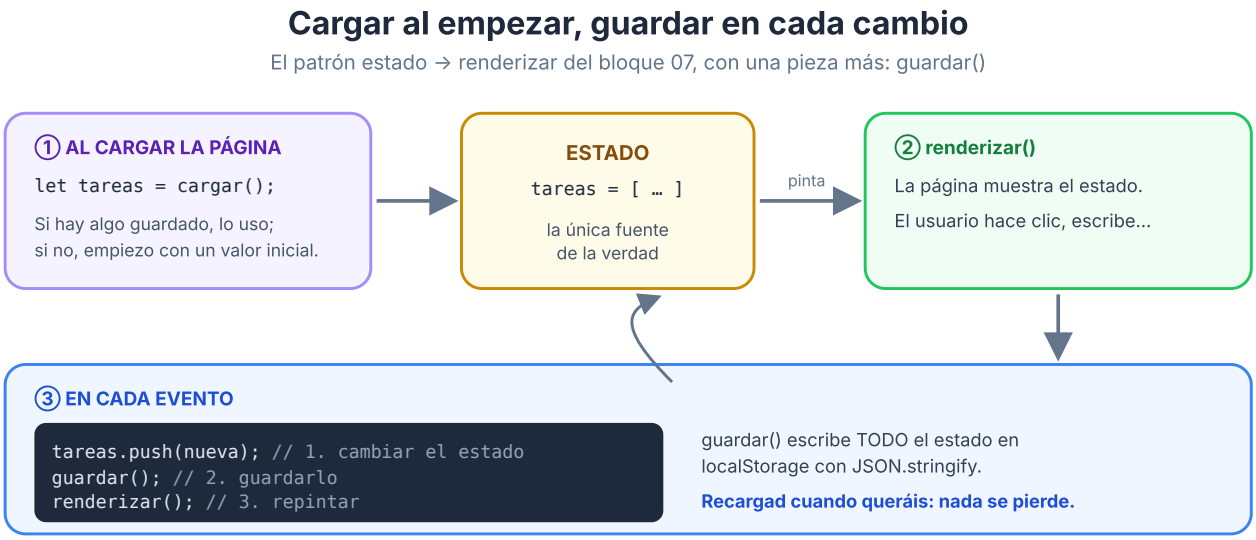
> 🎯 **Objetivo:** Convertir una aplicación con estado en persistente: cargar el estado guardado al empezar (con un valor inicial si no hay nada), guardarlo en cada cambio y repintar, guardando también una preferencia de la interfaz (el filtro activo).
>
> 📂 Enunciado · Solución completa

Qué pedía el ejercicio

Darle memoria a la lista de tareas del bloque 07: que las tareas y el filtro elegido sobrevivan a recargar o cerrar la página.

Lo que cambia respecto al bloque 07: casi nada

Esta es la buena noticia. Como la aplicación ya seguía el patrón **estado → renderizar**, añadir persistencia es cuestión de tres piezas:



📊 Figura 61 · Cargar al empezar, guardar en cada cambio

1. **Cargar** el estado al empezar, en lugar de escribirlo a mano.
2. **Guardar** el estado después de cada cambio.
3. Todo lo demás, igual.



1. Cargar

```
function cargarTareas() {
    const texto = localStorage.getItem(CLAVE_TAREAS);
    return texto === null ? structuredClone(TAREAS_INICIALES) : JSON.parse(texto);
}

let tareas = cargarTareas();
let filtro = localStorage.getItem(CLAVE_FILTRO) ?? 'todas';
```

Si no hay nada guardado (`null`) [176], empiezo con las tareas de ejemplo. ¿Por qué una **copia** con `structuredClone` ? Por las referencias del bloque 05: si `tareas` fuera el mismo array que `TAREAS_INICIALES` , al marcar una tarea estaría cambiando también los «datos iniciales».

Si hay algo, lo convierto de JSON a array [106].

El siguiente `id`

```
let siguienteId = Math.max(0, ...tareas.map((t) => t.id)) + 1;
```

En el bloque 07, `siguienteId` empezaba en 3 a mano. Ahora no sé cuántas tareas hay guardadas, así que lo calculo: el mayor `id` existente más uno. El `0` del principio cubre el caso de la lista vacía, porque `Math.max()` sin argumentos da `-Infinity` .



2. Guardar en cada cambio

```
function guardar() {
    localStorage.setItem(CLAVE_TAREAS, JSON.stringify(tareas));
    localStorage.setItem(CLAVE_FILTRO, filtro);
}

function actualizar() {
    guardar();
    renderizar();
}
```

Todos los escuchadores terminan igual: cambian el estado y llaman a `actualizar()`. Guardo **el estado completo** cada vez [172], no «el cambio». Es más sencillo y evita que el almacén y la memoria se desincronicen.



También se guardan las preferencias de la interfaz

El filtro no es un dato de las tareas, pero sí parte de la experiencia. Si el usuario estaba viendo «Pendientes», al volver espera seguir viendo «Pendientes». Lo guardo en su propia clave, como un texto sencillo.



Probadlo con las herramientas de desarrollo

Abrid la pestaña **Aplicación** y mirad la clave `curso-js:b09e07:tareas` mientras marcáis y borráis tareas. Veréis el JSON cambiar en directo. Y probad a borrar la clave a mano y recargar: vuelven las tareas de ejemplo.

Ejercicio 08 · Un carrito que sobrevive

> 🎯 **Objetivo:** Guardar un carrito de forma robusta: almacenar solo identificadores y cantidades (nunca precios), cargar con try…catch y validar los datos leídos, y sincronizar varias pestañas con el evento storage.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un carrito de la compra persistente, preparado para lo que puede salir mal con los datos guardados y sincronizado entre pestañas.



Regla 1: guardad lo mínimo, nunca precios

```
carrito = [{ id: 3, cantidad: 2 }, { id: 1, cantidad: 1 }];
```

El almacén solo guarda **qué** productos y **cuántos**. El nombre y el precio salen siempre del array `productos`, que está en el código.

¿Por qué? Porque el usuario puede abrir las herramientas de desarrollo y **cambiar lo que hay guardado**. Si guardarais `{ id: 3, precio: 45 }` y usarais ese precio, bastaría con editarlo a `0.01` para comprar unos auriculares a céntimo. Lo que hay en el navegador es del usuario: tratadlo como información que os da él, no como una verdad.



📝 Nota

En una tienda de verdad, ni siquiera el precio del código sería definitivo: el servidor volvería a calcular el total. Pero el principio es el mismo: no os fiéis de lo que llega del navegador.



Regla 2: cargad a la defensiva

```
let datos;
try {
    datos = JSON.parse(texto);
} catch (error) {
    return [];
}
```

Si alguien cambia el valor guardado por `hola`, `JSON.parse` no puede interpretarlo y **lanza un error** (`SyntaxError`) [106]. Sin protección, el script se detendría y la tienda dejaría de funcionar.

`try...catch` permite **intentar** un código y, si falla, **capturar** el error y decidir qué hacer [2]: aquí, empezar con el carrito vacío. Lo veréis con calma en el bloque 10, pero este es su uso más típico.

Y aunque el texto sea JSON válido, puede no ser lo que esperáis:

```
if (!Array.isArray(datos)) {
    return [];
}
return datos.filter((linea) => buscarProducto(linea.id) !== undefined && …);
```

- `Array.isArray()` comprueba que lo leído es de verdad un array [174]. Un `JSON.parse('42')` es válido, pero no es un carrito.
- El `filter` descarta líneas de productos que ya no existen (quizá los retirasteis del catálogo) o con cantidades sin sentido.

Probadlo: escribid `hola`, `42` o `[{"id":99,"cantidad":1}]` en la clave desde las herramientas de desarrollo y recargad. La tienda sigue funcionando.



Regla 3: sincronizad las pestañas

```
window.addEventListener('storage', (e) => {
    if (e.key === CLAVE) {
        carrito = cargarCarrito();
        renderizar();
    }
});
```

El evento `storage` se dispara cuando **otro documento** del mismo origen cambia el almacén [175]. Ojo: no salta en la pestaña que hace el cambio, sino en las demás. `e.key` dice qué clave ha cambiado.

Abrid la tienda en dos pestañas, añadid algo en una y mirad la otra: se actualiza sola. Sin esto, el usuario tendría dos carritos distintos en pantalla y, según la pestaña desde la que siguiera comprando, perdería productos.



Lo de siempre, que ya dominái

- Delegación: un escuchador por zona, porque los botones se recrean en cada renderizado. De hecho, **la misma función** `manejarClic` sirve para el catálogo y para el carrito, porque cada botón dice en `data-accion` lo que quiere.
- Estado → guardar → renderizar, como en el ejercicio 07.



Bloque 10

Bloque 10 · Asincronía

Hay tareas que no terminan al instante: esperar unos segundos, pedir datos a un servidor, leer un fichero. Si JavaScript se quedara parado esperando cada una, la página se congelaría. En este bloque os voy a enseñar cómo **encargar una tarea y seguir trabajando**, y cómo recoger el resultado cuando llegue: con temporizadores, con promesas y con `async` / `await`. Es la última pieza que os falta antes de pedir datos de verdad a un servidor en el bloque 11.

Al terminar este bloque sabréis:

- Distinguir el código síncrono del asíncrono, y por qué importa.
- Programar tareas con `setTimeout` y `setInterval`, y cancelarlas.
- Explicar el bucle de eventos y por qué no hay que bloquear la página.
- Crear y consumir promesas con `then`, `catch` y `finally`.
- Escribir código asíncrono legible con `async`, `await` y `try...catch`.
- Lanzar varias tareas a la vez con `Promise.all`.



1. Síncrono y asíncrono

JavaScript ejecuta vuestro código en **un solo hilo**: hace una cosa cada vez [22]. Si una tarea tarda y el programa se queda esperándola, todo lo demás espera también: ni los clics, ni el scroll, ni las animaciones.

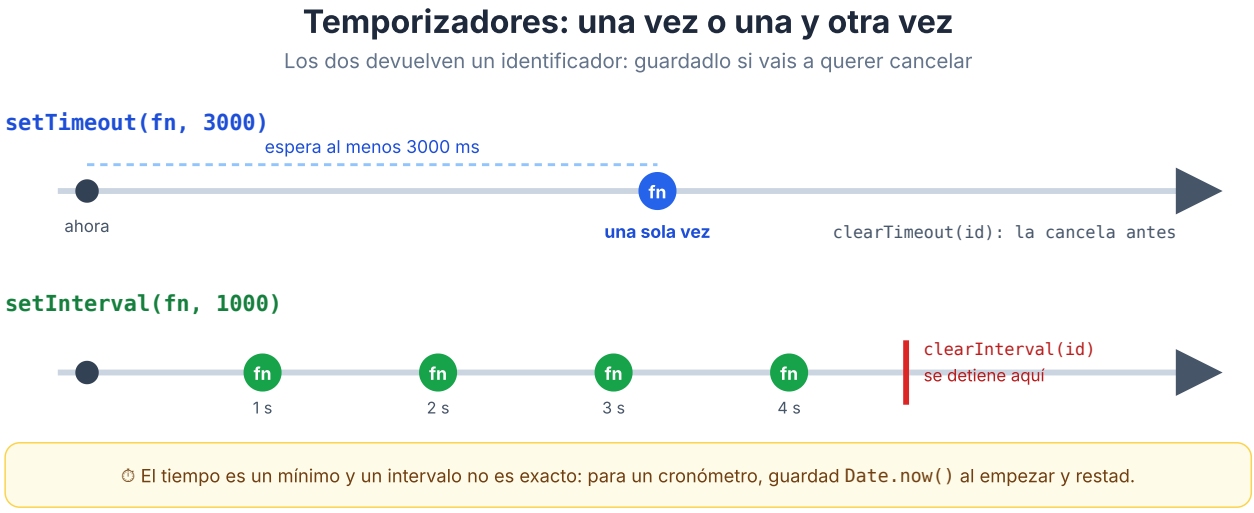


📊 Figura 62 · Síncrono frente a asíncrono

La solución es la **asincronía**: en lugar de esperar, se encarga la tarea al navegador (que sabe contar tiempo, hacer peticiones de red...) y se le dice qué hacer cuando termine. Mientras tanto, el programa sigue con lo suyo [183].



2. Temporizadores



📊 Figura 63 · setTimeout y setInterval

2.1. `setTimeout`: una vez, más tarde

```
const id = setTimeout(() => {
    salida.textContent = '¡Tiempo!';
}, 3000);   // milisegundos

clearTimeout(id);   // cancelarlo antes de que ocurra
```

`setTimeout` ejecuta la función cuando pasa el tiempo indicado y devuelve un identificador [184]. Con ese identificador, `clearTimeout` lo cancela [185]. El retraso es un **mínimo**: el tiempo real puede ser algo mayor [184].

2.2. `setInterval`: una y otra vez

```
const id = setInterval(actualizarReloj, 1000);   // cada segundo

clearInterval(id);   // detenerlo
```

`setInterval` repite la función cada vez que pasa el intervalo hasta que se detiene con `clearInterval` [186] [187].

2.3. Tres reglas con temporizadores

1. **Guardad siempre el identificador** si vais a necesitar cancelar.
2. **Nunca dos a la vez.** Antes de arrancar uno, comprobad si ya hay otro en marcha (por ejemplo, `if (id !== null)`), o desactivad el botón mientras corre.
3. **Medid el tiempo, no lo contéis.** Un intervalo no garantiza una cadencia exacta [186]. Para un cronómetro, guardad el instante de inicio con `Date.now()` [188] y restad: `Date.now() - inicio`.

2.4. Esperar a que el usuario deje de escribir

Un uso clásico de `setTimeout` + `clearTimeout`: no reaccionar con cada letra, sino cuando el usuario **se detiene** un momento.

```
let idEspera = null;

buscador.addEventListener('input', () => {
    clearTimeout(idEspera);                 // cada letra cancela la espera anterior…
    idEspera = setTimeout(() => {
        buscar(buscador.value);             // …y solo se busca tras 400 ms sin escribir
    }, 400);
});
```

Es perfecto para guardar un borrador o para un buscador que pide datos a un servidor: en lugar de una petición por letra, una sola cuando el usuario termina.



3. El bucle de eventos



📊 Figura 64 · El bucle de eventos

¿Cómo encaja todo esto con un solo hilo? Con tres piezas [1]:

Pieza	Qué hace
Pila de llamadas	Donde se ejecuta el código, una cosa cada vez
Cola de tareas	Donde esperan los callbacks listos para ejecutarse (un temporizador cumplido, un clic...)
Bucle de eventos	Cuando la pila se vacía, pasa a ella la siguiente tarea de la cola

La consecuencia más importante: **un callback nunca interrumpe el código que se está ejecutando**. Espera a que termine todo el código síncrono. Por eso esto escribe 1, 3, 2:

```
console.log('1');
setTimeout(() => console.log('2'), 0);
console.log('3');
```



🚨 Atención

Si la pila no se vacía nunca, la cola no avanza. Un bucle que tarde varios segundos **congela la página entera**: no se puede hacer clic ni escribir mientras se ejecuta [183]. Todo lo que tarde, de forma asíncrona.



4. Callbacks y la pirámide

La forma original de encadenar tareas asíncronas era pasar un callback: «cuando termines, haz esto».

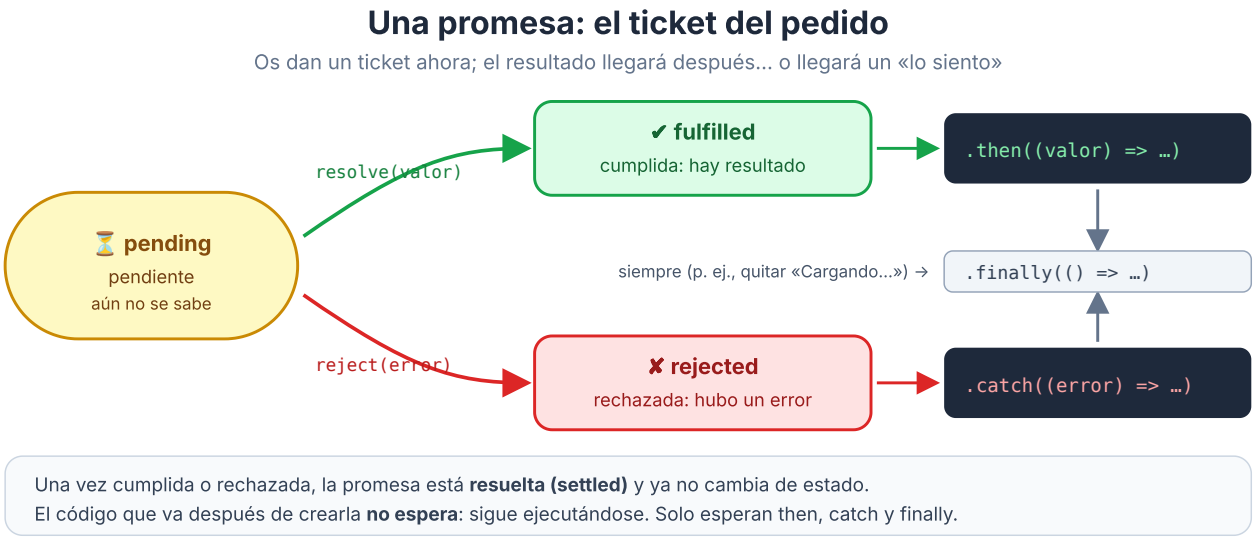
```
prepararMasa(() => {
    anadirIngredientes(() => {
        hornear(() => {
            servir(() => {
                console.log('¡Lista!');
            });
        });
    });
});
```

Funciona, pero cada paso va **dentro** del anterior. Con unos pocos pasos se forma una pirámide difícil de leer, y gestionar los errores en cada nivel se complica mucho [183]. Es el famoso *callback hell*.



5. Promesas

Una **promesa** es un objeto que representa el resultado futuro de una operación asíncrona [189]. Pensad en el ticket que os dan en una cafetería: aún no tenéis el café, pero tenéis algo que os lo garantiza... o que os dirá que se ha acabado la leche.



📊 Figura 65 · Los estados de una promesa

Una promesa está siempre en uno de tres estados [8]:

- **Pendiente** (*pending*): todavía no se sabe.

- **Cumplida** (*fulfilled*): la operación ha ido bien y hay un valor.
- **Rechazada** (*rejected*): ha fallado y hay un error.

5.1. Crear una promesa

```
function consultarStock(producto) {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (existe(producto)) {
                resolve(stock);                    // ✔ cumplida
            } else {
                reject(new Error('No existe'));    // ✘ rechazada
            }
        }, 700);
    });
}
```

`new Promise` recibe una función con dos parámetros: `resolve` para cumplirla y `reject` para rechazarla [190]. Lo más habitual, en cualquier caso, no será crearlas, sino **usarlas**: `fetch`, en el bloque 11, os devolverá promesas.

5.2. Usar una promesa

```
consultarStock('teclado')
    .then((stock) => mostrar(stock))          // si se cumple
    .catch((error) => mostrarError(error))    // si se rechaza
    .finally(() => ocultarCargando());        // siempre
```

`then` recibe la función que se ejecuta si la promesa se cumple [191]; `catch`, la que se ejecuta si se rechaza [192]; y `finally`, la que se ejecuta en los dos casos, ideal para limpiar: quitar un «Cargando…», reactivar un botón [193].

Y como `then` devuelve una promesa nueva [191], las tareas se **encadenan en vertical**, sin pirámide:

```
prepararMasa()
    .then(() => anadirIngredientes())
    .then(() => hornear())
    .then(() => servir())
    .catch((error) => console.error(error));   // un solo catch para toda la cadena
```



6. `async` y `await`

`async` y `await` son otra forma de usar las promesas, que permite escribir el código asíncrono **como si fuera síncrono**, de arriba abajo [194].

Mismas promesas, dos formas de escribirlas

async/await no sustituye a las promesas: es otra forma, más lineal, de usarlas

Con then / catch / finally

```
salida.textContent = 'Cargando…';
consultarStock('teclado')
  .then((stock) => {
    salida.textContent = stock;
  })
  .catch((error) => mostrarError(error))
  .finally(() => ocultarCargando());
```

El resultado llega «dentro» de un callback.

Con async / await

```
async function mostrarStock() {
  salida.textContent = 'Cargando…';
  try {
    const stock = await consultarStock(…);
    salida.textContent = stock;
  } catch (error) {
    mostrarError(error);
  } finally {
    ocultarCargando();
  } }
```

`await` pausa SOLO esta función; la página sigue viva.

Figura 66 · De then a async/await

```
async function mostrarStock(producto) {
    salida.textContent = 'Cargando…';
    try {
        const stock = await consultarStock(producto);
        salida.textContent = `Quedan ${stock}`;
    } catch (error) {
        salida.textContent = error.message;
    } finally {
        boton.disabled = false;
    }
}
```

- `async` delante de una función permite usar `await` dentro. Y una función `async` **siempre devuelve una promesa** [194].
- `await` pausa la función hasta que la promesa se resuelve y devuelve su valor [195]. Si se rechaza, lanza el error.
- El error se captura con `try...catch` [173], y `finally` se ejecuta siempre.



⚠️ Importante

`await` pausa **solo la función** `async` en la que está. El resto de la página sigue funcionando mientras tanto [195]. No es como el bucle que congela todo del apartado 3.

6.1. Una pausa reutilizable

```
const esperar = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function cuentaAtras() {
    for (const n of [3, 2, 1]) {
        salida.textContent = n;
        await esperar(1000);
    }
    salida.textContent = '¡Ya!';
}
```

Convertir `setTimeout` en una promesa permite escribir secuencias con pausas sin un solo callback a la vista.



7. Una detrás de otra, o todas a la vez



Usad el bucle solo si cada paso necesita el resultado del anterior. Si una tarea de Promise.all falla, falla el conjunto.

Figura 67 · Secuencial frente a Promise.all

```
// SECUENCIAL: cada una empieza cuando termina la anterior → suma de los tiempos
for (const producto of productos) {
    stocks.push(await consultarStock(producto));
}

// PARALELO: todas empiezan a la vez → lo que tarde la más lenta
const stocks = await Promise.all(productos.map((p) => consultarStock(p)));
```

`Promise.all` recibe un array de promesas y devuelve una que se cumple cuando se cumplen **todas**, con los resultados en el mismo orden. Si una se rechaza, la conjunta se rechaza también [196].

Forma	Tiempo con 4 tareas de 700 ms	Cuándo
`await` en un bucle	~2800 ms	Cada paso necesita el resultado del anterior
`Promise.all`	~700 ms	Las tareas son independientes



8. La plantilla de trabajo con asincronía

Los escuchadores pueden llamar a funciones `async`. La plantilla es la de siempre, con dos detalles: avisar al usuario mientras espera y recoger los errores.

```
// estructura del programa principal
// funciones
async function buscar(texto) {
    // salidas inmediatas: el usuario sabe que estamos en ello
    boton.disabled = true;
    salida.textContent = '⏳ Buscando…';
    try {
        // procesamiento asíncrono
        const resultados = await consultar(texto);
        // salidas
        renderizar(resultados);
    } catch (error) {
        salida.textContent = `✘ ${error.message}`;
    } finally {
        boton.disabled = false;
    }
}

// constantes al DOM que queremos escuchar
const formulario = document.querySelector('#buscador');
const boton = document.querySelector('#btnBuscar');
const salida = document.querySelector('#salida');

// colocar escuchadores de eventos
formulario.addEventListener('submit', (e) => {
    e.preventDefault();
    // entradas
    buscar(formulario.texto.value.trim());
});
```

En el bloque 11, `consultar` será una petición real con `fetch`. Todo lo demás ya lo sabéis hacer.



9. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Un aviso con retraso	`setTimeout`, `clearTimeout`, el orden de ejecución	Ver
02	Cronómetro	`setInterval`, `clearInterval`, medir con `Date.now()`	Ver
03	Cuenta atrás con barra	Un intervalo que se para solo, controles bloqueados mientras corre	Ver
04	¿En qué orden?	El bucle de eventos en tres experimentos, incluido un bloqueo	Ver
05	Del callback hell a las promesas	Callbacks anidados frente a `new Promise` y `then`	Ver
06	Promesas que pueden fallar	`resolve`, `reject`, `then`, `catch`, `finally`	Ver
07	async y await	`async`, `await`, `try...catch`, secuencial frente a `Promise.all`	Ver
08	Juego de reflejos	`await esperar()`, temporizador cancelable, `Date.now()`, récord guardado	Ver

En el 04, no os saltéis el experimento C. Ver la página congelada dos segundos explica mejor que cualquier párrafo por qué existe todo este bloque.



🧪 Los ejercicios, explicados

Ejercicio 01 · Un aviso con retraso: setTimeout

> 🎯 **Objetivo:** Ejecutar código más tarde con setTimeout, cancelarlo con clearTimeout usando su identificador, y comprobar que el programa no se detiene a esperar: el código posterior se ejecuta antes que el aviso.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Programar un aviso para dentro de unos segundos, poder cancelarlo y observar en qué orden se ejecuta el código.



`setTimeout`: «hazlo dentro de un rato»

```
idAviso = setTimeout(() => {
    anotar('3. ¡Aviso!');
}, segundos * 1000);
```

`setTimeout(función, retraso)` programa la función para que se ejecute cuando pase el tiempo indicado, en **milisegundos** [184]. Por eso multiplico por 1000: 3 segundos son 3000 ms.



Lo que veis en el registro

```
[+0 ms]    1. Programo el aviso para dentro de 3 s
[+1 ms]    2. El programa sigue: no espera
[+3002 ms] 3. ¡Aviso! Han pasado 3 s
```

¡La línea 2 sale **antes** que la 3, aunque esté escrita después en el código! Esta es la idea central de todo el bloque. `setTimeout` no detiene el programa: entrega la función al navegador, que se encarga de contar el tiempo, y **el programa sigue inmediatamente** con la siguiente línea [183]. Cuando pasa el tiempo, el navegador llama a la función.

Fijaos también en que el aviso llega a los 3002 ms, no a los 3000 exactos. El retraso indicado es un mínimo: el tiempo real puede ser algo mayor [184].



Cancelar: `clearTimeout`

```
clearTimeout(idAviso);
```

`setTimeout` devuelve un número que identifica ese temporizador [184]. Si lo guardáis, podéis cancelarlo antes de que se cumpla con `clearTimeout` [185]. Es como el número del ticket de un pedido: sin él, no podéis anularlo.

Nunca dos a la vez

```
if (idAviso !== null) {
    clearTimeout(idAviso);
}
```

Si el usuario pulsa «Programar» tres veces seguidas, sin esta comprobación tendríais **tres** avisos en marcha, y solo podríais cancelar el último, porque los otros ids se habrían perdido. Por eso cancelo el anterior antes de crear uno nuevo, y pongo `idAviso` a `null` cuando el aviso se cumple o se cancela. Así siempre sé si hay uno pendiente.



`Date.now()`

Devuelve los milisegundos transcurridos desde el 1 de enero de 1970 [188]. Por sí solo no es muy útil, pero **restando dos** obtenéis el tiempo que ha pasado entre ellos. Lo usaréis en los cronómetros del ejercicio 02 y en el juego del 08.

Ejercicio 02 · Cronómetro con setInterval

> 🎯 **Objetivo:** Repetir una tarea cada cierto tiempo con setInterval y detenerla con clearInterval, evitando arrancar dos intervalos a la vez y midiendo el tiempo real con Date.now() en lugar de contar ticks.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un cronómetro con inicio, pausa, vueltas y reinicio.



`setInterval`: «hazlo una y otra vez»

```
idIntervalo = setInterval(pintar, 100);
```

`setInterval(función, intervalo)` ejecuta la función **repetidamente**, cada vez que pasa el intervalo indicado en milisegundos, hasta que se detenga [186]. Como `setTimeout`, devuelve un

identificador [186], y con él se detiene:

```
clearInterval(idIntervalo);
```

`clearInterval` para la repetición [187].

	`setTimeout`	`setInterval`
Se ejecuta	Una vez	Una y otra vez
Se cancela con	`clearTimeout(id)`	`clearInterval(id)`
Ejemplo	Un aviso, una pausa	Un reloj, un cronómetro, un carrusel



Medir, no contar

Podríais pensar en sumar 100 ms en cada tick del intervalo. Funcionaría... más o menos. El intervalo indicado no está garantizado: si el navegador está ocupado, un tick puede llegar tarde [186], y esos retrasos se acumulan. Al cabo de unos minutos, vuestro cronómetro iría atrasado.

La solución es **medir** con el reloj del sistema:

```
function tiempoActual() {
    return acumulado + (enMarcha ? Date.now() - inicio : 0);
}
```

`Date.now()` da el instante actual en milisegundos [188]. Restándole el momento en que arrancó, sé exactamente cuánto ha pasado, llegue el tick cuando llegue. El intervalo ya no cuenta el tiempo: **solo repinta la pantalla**.

¿Y el `acumulado`? Guarda el tiempo de los tramos anteriores a cada pausa. Al reanudar, empiezo un tramo nuevo con otro `inicio`.



Nunca dos intervalos a la vez

```
if (idIntervalo === null) {
    // arrancar
} else {
    pausar();
}
```

Si el botón arrancara un intervalo cada vez que se pulsa, dos clics crearían dos intervalos repintando a la vez, y el `id` del primero se perdería: no habría forma de pararlo. Usar `idIntervalo === null` como «está parado» resuelve los dos problemas. Es la misma lección que con `setTimeout` en el ejercicio 01.



Formatear el tiempo

```
String(5).padStart(2, '0')   // '05'
```

`padStart` rellena un texto por la izquierda hasta la longitud indicada. Lo uso para que los minutos y los segundos tengan siempre dos cifras. Y `%`, el operador resto del bloque 02, me da lo que sobra al quitar los minutos enteros y los segundos enteros.

Ejercicio 03 · Cuenta atrás con barra de progreso

> 🎯 **Objetivo:** Construir una cuenta atrás con setInterval que se detiene sola al llegar a cero, actualizar una barra de progreso con style.width y bloquear los controles que no tienen sentido mientras corre.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una cuenta atrás que se para sola en cero, con una barra que se va llenando.



La estructura: un intervalo que sabe pararse

```
idIntervalo = setInterval(tic, 1000);

function tic() {
    restante--;
    pintar();
    if (restante <= 0) {
        terminar('🎉 ¡Tiempo!');
    }
}
```

Cada segundo, `tic` resta uno y repinta [186]. Y es el propio `tic` el que comprueba si ha llegado a cero y, en ese caso, detiene el intervalo con `clearInterval` [187]. Si os olvidarais de detenerlo, seguiría ejecutándose cada segundo para siempre, con números negativos.

¿`restante <= 0` y no `=== 0`? Es una pequeña precaución: si por cualquier motivo se saltara el cero, `<=` lo para igualmente. Las condiciones de parada, cuanto más amplias, más seguras.



Una función para terminar

`terminar` se llama desde dos sitios: al llegar a cero y al pulsar «Parar». Las dos situaciones hacen lo mismo (detener el intervalo y dejar los controles listos), así que lo meto en una función. Si

mañana añadís un tercer motivo para parar, ya tenéis la función.



Los controles cuentan la historia

```
btnEmpezar.disabled = true;
campoSegundos.disabled = true;
btnParar.disabled = false;
```

Mientras la cuenta corre, «Empezar» y el campo se desactivan. Así:

- es **imposible** arrancar dos intervalos a la vez, el error que visteis en los ejercicios 01 y 02;
- el usuario entiende de un vistazo qué puede hacer en cada momento.

Prevenir un error desde la interfaz es mejor que gestionarlo desde el código.



La barra

```
relleno.style.width = `${porcentaje}%`;
```

Es el caso que os anuncié en el bloque 06: `style` tiene sentido para un valor que se calcula al momento. No podéis tener una clase CSS para cada porcentaje.



Validar la entrada

`Number.isInteger(segundos)` descarta a la vez los textos (`NaN`), los decimales y los campos vacíos. Y el rango evita cuentas atrás de cero segundos o de varias horas.

Ejercicio 04 · ¿En qué orden? El bucle de eventos

> 🎯 **Objetivo:** Comprobar experimentalmente cómo funciona el bucle de eventos: el código síncrono termina siempre antes que cualquier callback de temporizador, los temporizadores salen por orden de vencimiento, y un bucle largo bloquea toda la página.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Descubrir con tres experimentos cómo decide JavaScript qué se ejecuta y cuándo.



Lo que pasa en cada experimento

Experimento	Orden escrito	Orden en el registro
A	A1, A2 (0 ms), A3	**A1, A3, A2**
B	B1 (300 ms), B2 (0 ms), B3	**B3, B2, B1**
C	C1, bucle de 2 s, C2	C1, C2… con el reloj **congelado** 2 segundos



La explicación: la pila, la cola y el bucle



Figura 68 · El bucle de eventos

JavaScript ejecuta el código en una **pila de llamadas** (*call stack*), y solo puede hacer una cosa cada vez [22]. Cuando llamáis a `setTimeout`, el callback no va a la pila: se lo entregáis al navegador. Cuando el tiempo se cumple, el navegador lo deja en una **cola de tareas**. Y el **bucle de eventos** (*event loop*) va pasando tareas de la cola a la pila, pero **solo cuando la pila está vacía** [22].

A: ¿por qué A2 sale el último si su retraso es 0?

Un temporizador de 0 ms no significa «ahora mismo», sino «en cuanto sea posible». El callback entra en la cola enseguida, pero tiene que esperar a que termine **todo** el código síncrono del clic, incluido A3. Solo entonces la pila queda vacía y A2 puede ejecutarse.

B: ¿por qué B2 sale antes que B1?

B3 es síncrono, así que sale primero, como en A. Después, los temporizadores salen **por orden de vencimiento**: el de 0 ms vence antes que el de 300 ms, aunque esté escrito después.

C: ¿por qué se congela todo?

El bucle `while` mantiene la pila ocupada durante 2 segundos. Mientras tanto, el bucle de eventos no puede pasar nada a la pila: ni los ticks del reloj, ni el clic en «Limpiar». La página está

congelada. MDN lo muestra con un ejemplo parecido: mientras una función larga se ejecuta, el programa no responde y no se puede escribir, hacer clic ni nada más [183].

Si pulsasteis «Limpiar» durante el bloqueo, quizá visteis que el registro se vaciaba **después** de los 2 segundos: el clic esperaba en la cola su turno.



La moraleja



> ⚠️ **Importante**
>
> Nunca bloqueéis la pila con tareas largas. Todo lo que tarda (esperar un tiempo, pedir datos a un servidor, leer un fichero) debe hacerse de forma **asíncrona**: se encarga al navegador y se recoge el resultado con un callback o una promesa. Así la página sigue viva mientras tanto.

El experimento C es lo que **no** hay que hacer. Los ejercicios 05 a 08 os enseñan cómo hacerlo bien.

Ejercicio 05 · Del callback hell a las promesas

> 🎯 **Objetivo:** Encadenar tareas asíncronas una detrás de otra, primero anidando callbacks (y viendo por qué se vuelve ilegible) y después con funciones que devuelven promesas encadenadas con then.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Ejecutar cuatro tareas que tardan, una detrás de otra, primero con callbacks anidados y después con promesas.



Versión 1: callbacks anidados

```
pasoConCallback('Preparar la masa', 800, () => {
    pasoConCallback('Añadir ingredientes', 500, () => {
        pasoConCallback('Hornear', 1000, () => {
            pasoConCallback('Servir', 300, () => {
                anotar('🍕 ¡Lista!');
            });
        });
    });
});
```

Para que el paso 2 empiece cuando termine el 1, tiene que ir **dentro** del callback del 1. Y el 3, dentro del 2. Con cuatro pasos ya tenéis una pirámide. MDN lo describe así: al llamar callbacks dentro de callbacks se obtiene un código profundamente anidado, mucho más difícil de leer y de depurar, y en el que gestionar los errores se complica en cada nivel de la pirámide [183]. A esto se le llama, con cariño, *callback hell*.



Versión 2: promesas

```
function paso(nombre, ms) {
    return new Promise((resolve) => {
        setTimeout(() => {
            anotar(nombre);
            resolve();
        }, ms);
    });
}
```

Una **promesa** es un objeto que representa el resultado futuro de una operación asíncrona [189]. Es como el ticket que os dan al pedir en una cafetería: todavía no tenéis el café, pero tenéis algo que os lo garantiza.

`new Promise(…)` recibe una función con un parámetro `resolve`. Cuando la tarea termina, llamáis a `resolve()` y la promesa pasa de **pendiente** a **cumplida** [190].

Encadenar con `then`

```
paso('Preparar la masa', 800)
    .then(() => paso('Añadir ingredientes', 500))
    .then(() => paso('Hornear', 1000))
    .then(() => paso('Servir', 300))
    .then(() => anotar('🍕 ¡Lista!'))
    .finally(() => bloquearBotones(false));
```

`then` recibe la función que se ejecutará cuando la promesa se cumpla, y **devuelve una promesa nueva** [191]. Y si esa función devuelve otra promesa, como `paso(…)`, el siguiente `then` espera a que se cumpla. Por eso la cadena va paso a paso, y es **plana**: se lee de arriba abajo, como una receta.

`finally` se ejecuta cuando la cadena termina, vaya bien o mal [193]. Es el sitio ideal para reactivar los botones.



Comparad las dos

	Callbacks	Promesas
Forma	Pirámide hacia la derecha	Lista hacia abajo
Añadir un paso en medio	Reindentar todo lo de dentro	Añadir una línea `.then`
Gestionar errores	En cada nivel	Un solo `.catch` al final (ejercicio 06)

El registro sale idéntico en las dos: los pasos terminan a los 800, 1300, 2300 y 2600 ms. Las promesas no hacen el código más rápido: lo hacen **más legible**. Y en el ejercicio 07 veréis una forma aún más clara de escribir esta misma cadena.

Ejercicio 06 · Promesas que pueden fallar: then, catch y finally

> 🎯 **Objetivo:** Consumir una función que devuelve una promesa que puede cumplirse o rechazarse, tratar los dos casos con then y catch, limpiar con finally, y comprobar que el código posterior no espera al resultado.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Consultar el stock de un producto con una función que tarda y que puede fallar, y tratar cada caso.



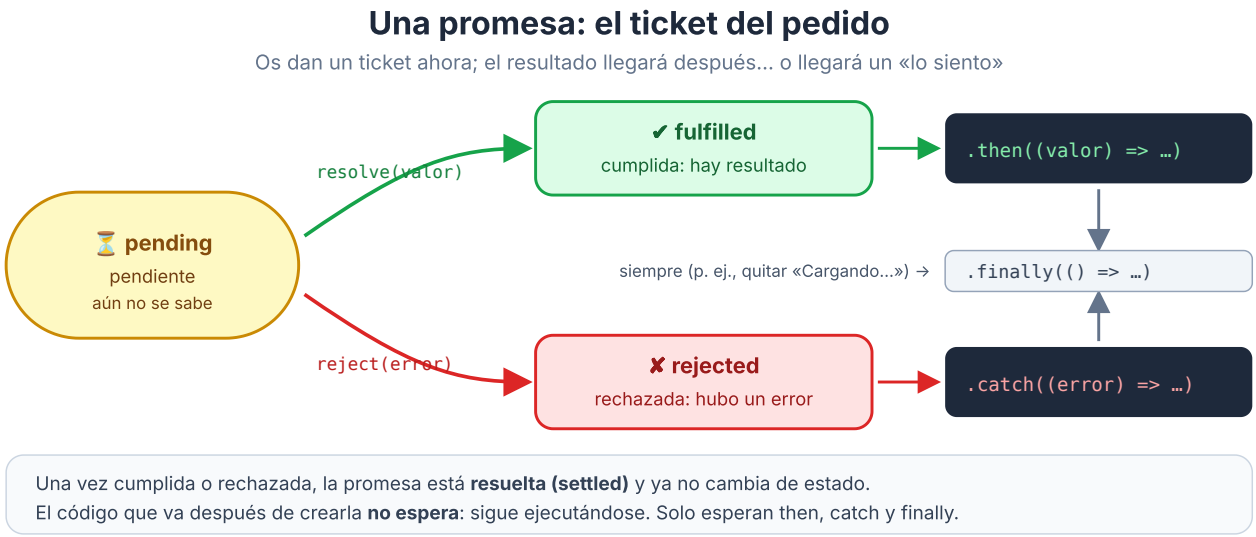
Una promesa que puede fallar

```
return new Promise((resolve, reject) => {
    setTimeout(() => {
        if (Object.keys(STOCK).includes(producto)) {
            resolve(STOCK[producto]);
        } else {
            reject(new Error(`No existe el producto «${producto}».`));
        }
    }, 700);
});
```

La función que recibe `new Promise` tiene **dos** parámetros [1]:

- `resolve(valor)`: la operación ha ido bien. La promesa pasa a **cumplida** (*fulfilled*) con ese valor.
- `reject(error)`: algo ha fallado. La promesa pasa a **rechazada** (*rejected*).

Mientras no se llama a ninguno de los dos, está **pendiente** (*pending*) [189]. Es exactamente lo que hará `fetch` en el bloque 11 cuando pidáis datos a un servidor: tardará, y podrá ir bien o mal.



📊 Figura 69 · Los estados de una promesa



Tratar los dos casos

```
consultarStock(producto)
    .then((stock) => { … })      // si se cumple: recibe el valor
    .catch((error) => { … })     // si se rechaza: recibe el error
    .finally(() => { … });       // siempre
```

Método	Se ejecuta si...	Recibe
`then`	la promesa se cumple [191]	el valor de `resolve`
`catch`	la promesa se rechaza [192]	el error de `reject`
`finally`	la promesa termina, de una forma u otra [193]	nada

`finally` es perfecto para el código de **limpieza** que tiene que ejecutarse pase lo que pase: reactivar el botón, quitar el «Cargando...» [193]. Sin él, tendríais que repetir esas líneas en el `then` y en el `catch`.



El orden del registro

```
[+0 ms]   1. Pido el stock
[+1 ms]   2. Mientras tanto, la página sigue viva
[+702 ms] 3. Ha llegado la respuesta
[+703 ms] 4. Fin (finally)
```

La línea 2 está escrita **después** de toda la cadena, y sin embargo sale antes que la respuesta. Lo visteis en el ejercicio 01 con `setTimeout`: crear una promesa no detiene el programa. El código sigue, y `then`, `catch` y `finally` se ejecutan **cuando llegue** el resultado.



Lo que ve el usuario

1. Pulsa «Consultar»: el botón se desactiva y aparece «⏳ Consultando...». Sabe que algo está pasando.
2. Llega la respuesta: ve el resultado o un error claro.
3. El botón se reactiva.

Nunca dejéis al usuario sin respuesta mientras esperáis. Un botón que no hace nada visible durante un segundo invita a pulsarlo cinco veces.

Ejercicio 07 · async y await: el mismo código, más claro

> 🎯 **Objetivo:** Reescribir código con promesas usando funciones async, await y try...catch...finally, y comparar el tiempo de esperar varias promesas una detrás de otra frente a esperarlas a la vez con Promise.all.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Reescribir la consulta del ejercicio 06 con `async` / `await` y comparar dos formas de esperar varias promesas.



`async` y `await`

```
async function mostrarStock(producto) {
    try {
        const stock = await consultarStock(producto);
        resultado.textContent = `✔ ${producto}: ${stock} unidades.`;
    } catch (error) {
        resultado.textContent = `✘ ${error.message}`;
    } finally {
        console.log('Consulta terminada');
    }
}
```

- `async` delante de una función permite usar `await` dentro de ella [194]. Además, una función `async` **siempre devuelve una promesa** [194].
- `await` delante de una promesa **pausa la función** hasta que la promesa se resuelve, y entonces devuelve su valor [195]. La línea siguiente ya tiene el `stock`, como si fuera código normal y síncrono.
- Si la promesa se **rechaza**, `await` lanza el error, y lo capturáis con el `try...catch` que ya visteis en el bloque 09 [173]. El `finally` se ejecuta en los dos casos, igual que el `.finally()` de las

promesas.



Figura 70 · De then a async/await

Comparadlo con el ejercicio 06: hace exactamente lo mismo, pero se lee **de arriba abajo**, sin callbacks.



⚠️ Importante

`await` solo pausa **la función** `async` en la que está, no la página. Mientras espera, el resto del programa sigue funcionando: los botones responden y los temporizadores avanzan [195]. No es el bloqueo del ejercicio 04.



La carrera

Uno detrás de otro: ~2800 ms

```
for (const producto of PRODUCTOS) {
    stocks.push(await consultarStock(producto));
}
```

Cada `await` espera a que termine una consulta antes de **empezar** la siguiente. Cuatro consultas de 700 ms: 4 × 700 = 2800 ms.

Todos a la vez: ~700 ms

```
const promesas = PRODUCTOS.map((producto) => consultarStock(producto));
const stocks = await Promise.all(promesas);
```

El `map` **lanza las cuatro consultas de golpe**: cada `consultarStock` empieza en el acto y devuelve su promesa. `Promise.all` recibe un array de promesas y devuelve una sola, que se cumple cuando

se cumplen **todas**, con un array de resultados en el mismo orden [196]. Las cuatro esperas transcurren a la vez: unos 700 ms.

Forma	Tiempo	Úsala cuando...
`await` en un bucle	Suma de todas	Cada paso necesita el resultado del anterior
`Promise.all`	La más lenta	Las tareas son independientes entre sí

Un aviso: si **una** de las promesas de `Promise.all` se rechaza, la promesa conjunta se rechaza también [196]. Para esos casos necesitaríais un `try...catch` alrededor.

Ejercicio 08 · Juego de reflejos

> 🎯 **Objetivo:** Combinar todo el bloque en un juego: una pausa reutilizable con una promesa, una cuenta atrás con async/await, un temporizador aleatorio que se cancela con clearTimeout, medición precisa con Date.now() y un récord guardado en localStorage.
>
> 📂 Enunciado · Solución completa

 Qué pedía el ejercicio

Un juego que mide vuestro tiempo de reacción. Junta temporizadores, promesas, `async` / `await`, medición del tiempo y almacenamiento.

 `esperar` : una pausa que se puede esperar

```
const esperar = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
```

Una línea con mucha miga. Devuelve una promesa que `setTimeout` cumple pasados `ms` milisegundos [190] [184]. Como `resolve` es una función, se la paso directamente a `setTimeout` como callback.

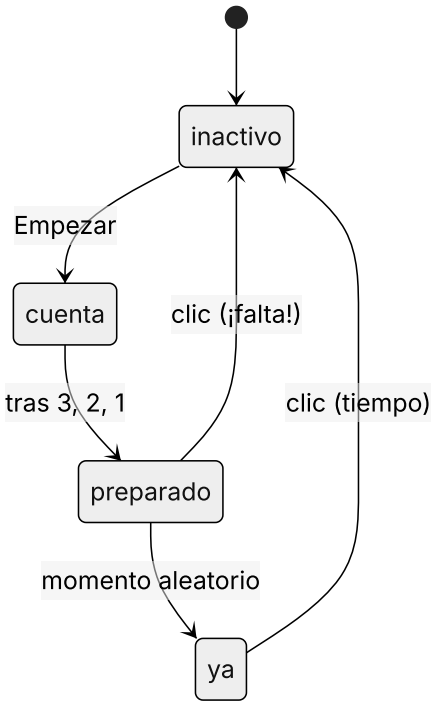
Con ella, la cuenta atrás se escribe como una lista de pasos:

```
for (const numero of [3, 2, 1]) {
    ponerZona('', numero);
    await esperar(700);
}
```

Compárala con la pirámide del ejercicio 05: aquí ni siquiera hay callbacks a la vista. `await` pausa `empezar()` 700 ms en cada vuelta [195], pero la página sigue viva: podéis hacer clic en la zona durante la cuenta, y `pulsar()` se ejecuta, aunque decida ignorarlo.



Las fases del juego



La variable `fase` es el estado del juego, y `pulsar()` decide qué hacer según ella. Es más fiable que mirar el color de la zona: el color es la **consecuencia** del estado, no el estado.



Cancelar lo que no ha pasado

```
if (fase === 'preparado') {
    clearTimeout(idTemporizador);
}
```

Si el jugador pulsa en rojo, el paso a verde todavía está programado. Sin `clearTimeout` [185], la zona se pondría verde un segundo después, en mitad del mensaje de falta. Siempre que un temporizador pueda quedar obsoleto, guardad su `id` y cancelad.



¿Por qué `setTimeout` y no `await esperar()` para el verde?

Porque hay que poder **cancelarlo**. Una promesa no se puede «des-cumplir» desde fuera. Para la cuenta atrás, que nunca se cancela, `await esperar()` es cómodo; para el verde, que sí, `setTimeout` con su `id`. Cada herramienta para lo suyo.



Medir el tiempo

```
momentoVerde = Date.now();
…
const tiempo = Date.now() - momentoVerde;
```

Restar dos instantes de `Date.now()` [188] da los milisegundos entre que apareció el verde y vuestro clic. Lo visteis en el cronómetro del ejercicio 02.



El récord persiste

```
let record = Number(localStorage.getItem(CLAVE_RECORD)) || null;
```

Del bloque 09: guardo el mejor tiempo con `setItem` [21] y lo leo al cargar. Si no hay nada guardado, `Number(null)` da `0`, y `0 || null` da `null`: «todavía no hay récord».



Accesibilidad

La zona tiene `role="button"` y `tabindex="0"`, así que se puede alcanzar con Tab y pulsar con Enter o con la barra espaciadora. Con `e.preventDefault()` evito que el espacio desplace la página.



Bloque 11

Bloque 11 · Fetch y APIs

Hasta ahora, todos los datos de vuestras páginas los habéis escrito vosotros: un array en el código, un formulario que rellena el usuario, algo guardado en `localStorage`. Las páginas de verdad no funcionan así. El tiempo, las noticias, el catálogo de una tienda o los mensajes de un chat viven en un **servidor**, y la página los pide cuando los necesita. En este bloque os voy a enseñar a hacer esas peticiones con `fetch`, a comprobar si han ido bien y a contarle al usuario qué está pasando mientras tanto. Todo lo que aprendisteis de `async` y `await` en el bloque 10 va a trabajar aquí.

Al terminar este bloque sabréis:

- Explicar qué es una API y qué viaja en una petición y en una respuesta HTTP.
- Pedir datos con `fetch` y convertirlos a JavaScript con `respuesta.json()`.
- Comprobar `respuesta.ok` y distinguir los tres tipos de error que pueden ocurrir.
- Mostrar al usuario los estados de carga, datos, vacío y error.
- Entender qué es el origen de una página, por qué necesitáis Live Server y qué es CORS.
- Enviar datos a un servidor con `POST`.
- Lanzar varias peticiones a la vez con `Promise.all`.



⚠️ Importante

En este bloque, abrid siempre los ejercicios con Live Server. Si hacéis doble clic en el `index.html`, la página se abre como fichero local y todas las peticiones fallarán. En el apartado 5 os explico por qué.



1. Qué es una API

Una **API** es la puerta que un programa abre para que otros programas le pidan cosas [197]. En la web, normalmente es un servidor que responde a ciertas URL con datos en lugar de con páginas. Vuestra página no ve la base de datos del servidor: solo ve lo que la API decide contestar.

La conversación siempre tiene la misma forma: el cliente (vuestra página) envía una **petición** y el servidor devuelve una **respuesta** [198].

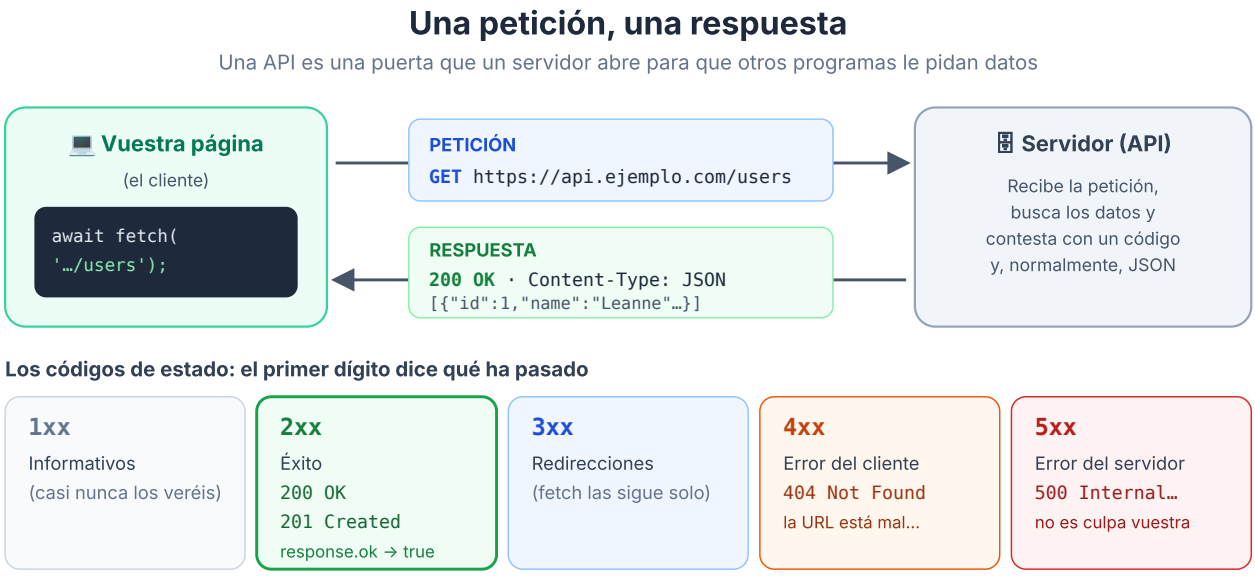
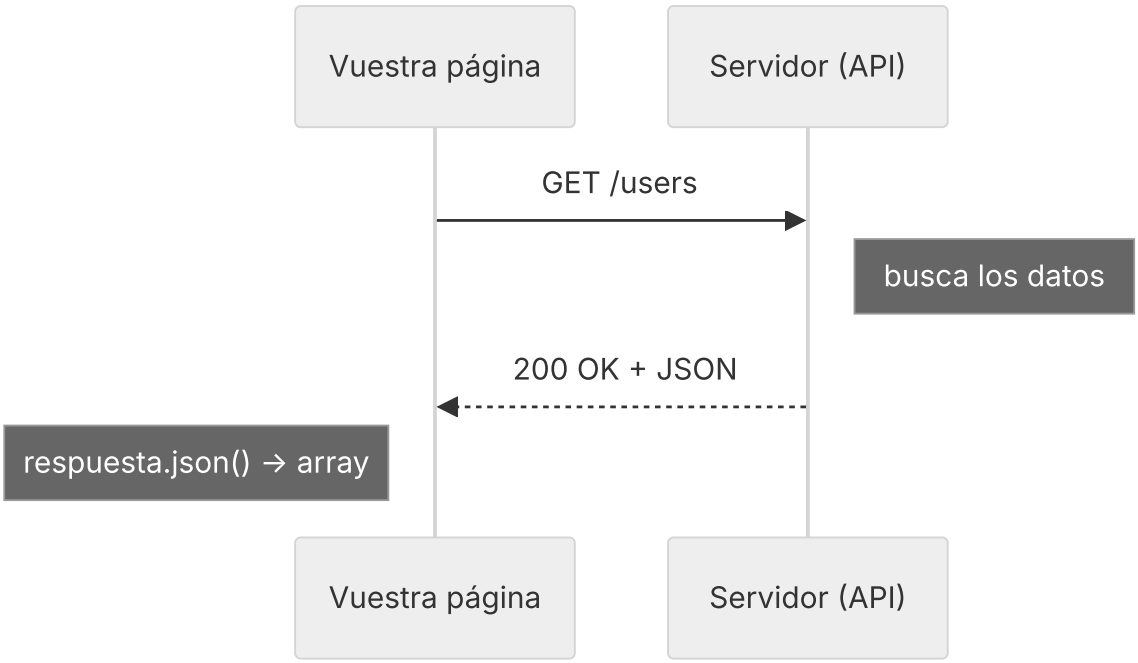


Figura 71 · Una petición HTTP a una API



Viaja en la petición	Viaja en la respuesta
Un **método**: `GET` para pedir, `POST` para enviar…	Un **código de estado**: `200`, `404`, `500`…
La **URL** del recurso que queréis	Unas **cabeceras** que describen el contenido
Cabeceras y, a veces, un **cuerpo** con datos	El **cuerpo**: los datos, casi siempre en JSON

1.1. Los códigos de estado

El código de estado es un número de tres cifras, y la primera dice qué ha pasado [199]. Las dos familias que más vais a ver son los `2xx` (todo bien) y los `4xx` (la petición tiene un problema, como pedir una URL que no existe: el famoso `404`). Si veis un `5xx`, el que ha fallado es el servidor.

1.2. JSON, otra vez

Los datos llegan como **texto** en formato JSON [200]. Ya lo conocéis del bloque 09: es el mismo formato que usabais con `JSON.stringify` y `JSON.parse` para guardar en `localStorage`. La diferencia es que ahora no lo escribís vosotros: lo escribe el servidor.



2. `fetch` en cuatro pasos

`fetch` es la función del navegador para hacer peticiones HTTP [201]. Recibe una URL y devuelve una **promesa**, así que se usa con `await` dentro de una función `async` [202].



Figura 72 · Los cuatro pasos de fetch

```
async function cargarConsejos() {
    // ① pedir: llega la respuesta (el «sobre»)
    const respuesta = await fetch('consejos.json');

    // ② comprobar: ¿ha ido bien?
    if (!respuesta.ok) {
        throw new Error(`Error ${respuesta.status}`);
    }

    // ③ leer el contenido: JSON → JavaScript
    const consejos = await respuesta.json();

    // ④ usar los datos
    renderizar(consejos);
}
```

Fijaos en que hay **dos** `await`. El primero espera a que lleguen el código de estado y las cabeceras; en ese momento tenéis un objeto `Response` [203], pero el contenido puede no haber terminado de descargarse. El segundo, `respuesta.json()`, espera a que llegue todo el cuerpo y lo convierte en un array u objeto normal de JavaScript.

Un objeto `Response` os da, entre otras cosas:

Propiedad o método	Qué contiene
`respuesta.ok`	`true` si el código está entre 200 y 299 [204]
`respuesta.status`	El código de estado: `200`, `404`… [205]
`respuesta.json()`	Una promesa con el contenido ya convertido a JavaScript [206]

2.1. Mi función de cabecera: `pedirJSON`

Los pasos ①, ② y ③ son siempre los mismos, así que en los ejercicios los meto en una función que reutilizo:

```
async function pedirJSON(url) {
    const respuesta = await fetch(url);
    if (!respuesta.ok) {
        throw new Error(`Error ${respuesta.status} al pedir ${url}`);
    }
    return respuesta.json();
}

// y luego, en una línea:
const productos = await pedirJSON('productos.json');
```



3. Tres formas de fallar

Esta es la parte que más se olvida, y la que más problemas da en clase. Una petición puede salir mal de tres maneras distintas, y no todas se comportan igual:

Qué pasa	Qué hace JavaScript
No hay conexión, la URL es imposible o el navegador bloquea la petición	`fetch` **rechaza** la promesa con un `TypeError` [202]
El servidor contesta, pero con un `404`, un `500`…	`fetch` **no rechaza**: `respuesta.ok` es `false` [202]
El contenido no es JSON válido	`respuesta.json()` rechaza con un `SyntaxError` [206]

La segunda fila es la trampa: para `fetch`, un `404` es una respuesta como otra cualquiera. Por eso el paso ② no es opcional. Si no comprobáis `respuesta.ok` y lanzáis vosotros el error, vuestro código seguirá adelante intentando leer datos que no existen.

Lo bueno es que, con el `throw` del paso ②, los tres casos acaban en el mismo sitio: el `catch` [173].

```
try {
    const datos = await pedirJSON(url);
    renderizar(datos);
} catch (error) {
    estado.textContent = `✘ No se han podido cargar los datos (${error.message}).`;
    btnReintentar.hidden = false;
}
```

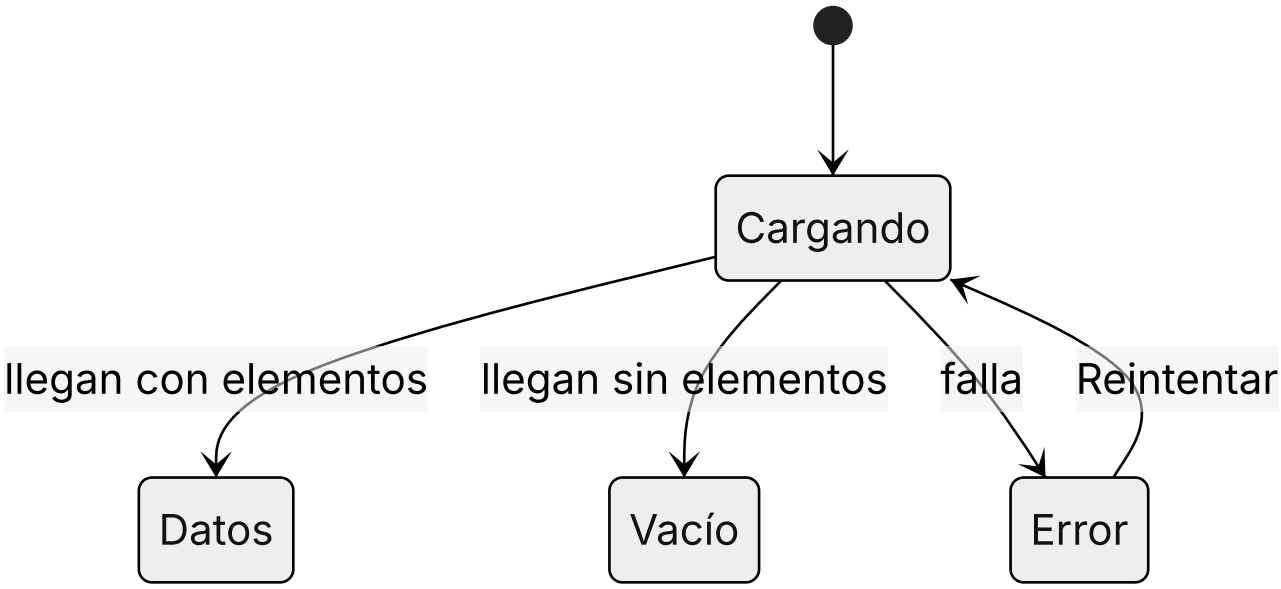


4. Los cuatro estados de la interfaz

Una petición puede tardar un instante o varios segundos. Mientras tanto, una pantalla en blanco no le dice nada al usuario: ¿está cargando?, ¿se ha roto?, ¿no hay nada? Cada situación necesita su propio mensaje.



📊 Figura 73 · Los cuatro estados de una interfaz que carga datos



Mis reglas para los cuatro estados:

1. **Cargando.** Escribid un mensaje (`⏳ Cargando…`) y desactivad el botón que lanza la petición, para que nadie la repita diez veces.
2. **Datos.** Pintadlos y decid cuántos hay: «Mostrando 10 usuarios».
3. **Vacío.** Que no haya resultados no es un error. Decidlo con claridad: «No hay noticias publicadas todavía».
4. **Error.** Explicad lo que ha pasado en lenguaje humano y ofreced un botón de **Reintentar**.

Poned el mensaje de estado en un elemento con `aria-live="polite"`. Así, los lectores de pantalla anuncian cada cambio sin que el usuario tenga que buscarlo [162]. Ya lo usasteis en los formularios del bloque 08.



5. El origen, Live Server y CORS

El **origen** de una página es la combinación de su protocolo, su dominio y su puerto [207]. Si cambia cualquiera de los tres, es otro origen. El navegador usa el origen para proteger al usuario: no deja que una página cualquiera lea datos de cualquier sitio.



📊 Figura 74 · El origen y CORS

Esto tiene tres consecuencias prácticas:

- **Abrir el fichero con doble clic no sirve.** La página se carga con `file:///`, y los navegadores no aceptan esas peticiones como peticiones HTTP normales [208]. `fetch('datos.json')` falla con `TypeError: Failed to fetch`. Lo he comprobado: con el mismo código y la misma carpeta, la petición funciona en cuanto la página se sirve por HTTP.
- **Con Live Server, pedir ficheros de vuestra carpeta funciona siempre.** La página y el JSON vienen del mismo servidor (`http://127.0.0.1:5500`), así que son del mismo origen. Es el caso de los ejercicios 01 a 05.
- **Pedir a otro origen solo funciona si el servidor lo autoriza.** Lo hace con la cabecera `Access-Control-Allow-Origin`; ese mecanismo se llama **CORS** [209]. Las API pensadas para usarse desde el navegador la incluyen. Si una API no la envía, veréis un error de CORS en la consola, y vuestro código no lo puede arreglar: la decisión es del servidor. Es el caso de los ejercicios 06 a 08.



6. Pedir y enviar: `GET` y `POST`



Figura 75 · GET frente a POST

Si no le decís otra cosa, `fetch` hace una petición `GET`, el método para **pedir** datos sin cambiar nada en el servidor [210]. Muchas API aceptan parámetros en la URL para filtrar:

```
// solo las publicaciones del usuario 3
const publicaciones = await pedirJSON(`${API}/posts?userId=3`);
```

Para **enviar** datos, se usa `POST` [211], y a `fetch` se le pasa un segundo argumento con las opciones de la petición [5]:

```
const respuesta = await fetch(`${API}/posts`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title: 'Hola', body: 'Mi primer POST', userId: 1 }),
});
const creada = await respuesta.json();   // el servidor devuelve lo que ha creado
```

Opción	Para qué sirve
`method`	El método HTTP: `'POST'`
`headers`	Las cabeceras. `Content-Type` le dice al servidor en qué formato va el cuerpo [212]
`body`	Los datos. Un objeto no se puede enviar tal cual: hay que convertirlo con `JSON.stringify`



7. Varias peticiones a la vez

Si una página necesita dos cosas independientes (por ejemplo, la lista de usuarios y las tareas del primero), no tiene sentido esperar a una para pedir la otra. `Promise.all` lanza todas a la vez y espera a que terminen [19]:

```
const [usuarios, tareas] = await Promise.all([
    pedirJSON(`${API}/users`),
    pedirJSON(`${API}/todos?userId=1`),
]);
```

Es lo mismo que visteis en el bloque 10, ahora con peticiones de verdad. Si una sola falla, `Promise.all` rechaza y saltáis al `catch`.



8. La plantilla de trabajo con `fetch`

La plantilla de siempre, con la carga de datos dentro de una función `async` que gestiona los cuatro estados:

```js
// estructura del programa principal
const URL_DATOS = 'productos.json';

// funciones
async function pedirJSON(url) {
    const respuesta = await fetch(url);
    if (!respuesta.ok) {
        throw new Error(`Error ${respuesta.status} al pedir ${url}`);
    }
    return respuesta.json();
}

async function cargar() {
    // salidas inmediatas: CARGANDO
    boton.disabled = true;
    estado.textContent = '⏳ Cargando…';
    try {
        // entradas y procesamiento
        const datos = await pedirJSON(URL_DATOS);
        // salidas: VACÍO o DATOS
        if (datos.length === 0) {
            estado.textContent = '📭 No hay resultados.';
            return;
        }
        renderizar(datos);
        estado.textContent = `Mostrando ${datos.length} elementos`;
    } catch (error) {
        // salidas: ERROR
        estado.textContent = `✘ ${error.message}`;
    } finally {
        boton.disabled = false;
    }
}

// constantes al DOM que queremos escuchar
const boton = document.querySelector('#btnCargar');
const estado = document.querySelector('#estado');

// colocar escuchadores de eventos
boton.addEventListener('click', cargar);
```



9. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Mi primer fetch: leer un JSON	`fetch`, `respuesta.json()`, los dos `await`	Ver
02	respuesta.ok y los códigos de estado	`200`, `404` y un fallo de red, uno al lado del otro	Ver
03	El catálogo llega de un fichero	`pedirJSON`, pintar tarjetas, botón Reintentar	Ver
04	Buscador de países	Cargar una vez y filtrar en memoria, búsqueda sin tildes con espera	Ver
05	Estados de carga, errores y reintentos	Cargando, datos, vacío y error con cinco escenarios simulados	Ver
06	Una API pública: usuarios y publicaciones	Pedir a otro origen, datos anidados, parámetros en la URL	Ver
07	Enviar datos con POST	`method`, `headers`, `body` y `JSON.stringify`	Ver
08	Tareas de la API con favoritos	`Promise.all`, filtros, búsqueda y favoritos en `localStorage`	Ver

Los ejercicios 01 a 05 piden ficheros JSON de su propia carpeta. Del 06 al 08 se conectan a JSONPlaceholder, una API pública y gratuita para practicar, así que necesitáis conexión a Internet. Y una prueba que os recomiendo: en el 02, abrid la pestaña **Red** de las herramientas de desarrollo y mirad cada petición. Ver el `404` en rojo con vuestros propios ojos se olvida mucho más tarde que leerlo aquí.



🧪 Los ejercicios, explicados

Ejercicio 01 · Mi primer fetch: leer un JSON

> 🎯 **Objetivo:** Pedir un fichero JSON con fetch, convertir la respuesta en datos de JavaScript con response.json() y usar esos datos en la página, gestionando el error que aparece si la página no se sirve desde un servidor.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Leer los consejos de un fichero JSON con `fetch` en lugar de tenerlos escritos en el código.



`fetch` en dos pasos

```
const respuesta = await fetch('consejos.json');
const consejos = await respuesta.json();
```

`fetch()` hace una petición y devuelve una **promesa** que se cumple con un objeto `Response`, la respuesta del servidor [202]. Es como recibir un sobre: todavía no lo habéis abierto.

Para leer el contenido hay que «abrir el sobre» con `respuesta.json()`, que lee el cuerpo de la respuesta y lo convierte de JSON a datos de JavaScript. Y también devuelve una promesa [206]. Por eso hay **dos** `await`.

Paso	Código	Obtenéis
1	`await fetch(url)`	La respuesta: código de estado, cabeceras…
2	`await respuesta.json()`	Los datos: aquí, un array de objetos

A partir de ahí, `consejos` es un array normal, como los del bloque 05.



¿Por qué hace falta Live Server?

Si abrís la página con doble clic, la consola muestra:

```
TypeError: Failed to fetch
```

Al abrir un fichero local, la dirección empieza por `file:///`, y los navegadores no permiten hacer peticiones a ficheros locales como si fueran peticiones HTTP [208]. Con Live Server, la página se

sirve desde `http://127.0.0.1:5500`, un servidor de verdad, y `fetch('consejos.json')` le pide el fichero al mismo servidor que entregó la página.

Por eso, en este bloque, cada ejercicio empieza con el aviso azul. Y por eso el `catch` da un mensaje que **ayuda**: no se limita a decir «error», sino que apunta a la causa más probable.



La estructura de siempre

El escuchador es una función `async`, como en el bloque 10. Lo nuevo es solo de dónde salen los datos. El resto (mensaje de espera, botón desactivado, `try` / `catch` / `finally`) ya lo conocíais.



Miradlo en la pestaña Red

Herramientas de desarrollo → **Red** (*Network*). Cada vez que pulsáis el botón aparece una línea con `consejos.json`, su código de estado (`200`) y lo que ha tardado. Pinchad en ella y veréis el contenido del fichero en la pestaña **Respuesta**. Es la herramienta que más vais a usar en este bloque.

Ejercicio 02 · respuesta.ok y los códigos de estado

> 🎯 **Objetivo:** Comprobar que fetch solo rechaza la promesa cuando falla la red, no cuando el servidor responde con un error como 404, y aprender a comprobar respuesta.ok y respuesta.status.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Descubrir en qué casos «falla» `fetch` y cómo saber si una respuesta es buena.



Los resultados

Petición	¿Rechaza?	`status`	`ok`
`consejos.json`	No	`200`	`true`
`no-existe.json`	**No**	`404`	`false`
Servidor inexistente	Sí (`TypeError`)	—	—



La sorpresa: un 404 no es un error para `fetch`

MDN lo dice sin rodeos: la promesa de `fetch()` **no** se rechaza cuando el servidor responde con un código de error como 404 o 504; solo se rechaza cuando la petición falla, por ejemplo por un error de red [202].

Tiene su lógica: con un 404, el servidor **ha respondido**. Os ha dicho «eso no existe», que es una respuesta perfectamente válida. Quien decide si eso es un error para vuestra aplicación sois

vosotros.

 `respuesta.ok` y `respuesta.status`

```
respuesta.status       // 404
respuesta.statusText   // 'Not Found'
respuesta.ok           // false
```

- `status` es el código de estado HTTP [205].
- `statusText` es el mensaje que lo acompaña [213]. Cada servidor puede escribirlo a su manera, así que no lo uséis para tomar decisiones.
- `ok` es `true` si el código está entre 200 y 299, y `false` en cualquier otro caso [204].

Los códigos se agrupan por su primera cifra: los 2xx indican éxito, los 4xx errores del cliente y los 5xx errores del servidor [199]. Lo tenéis en la infografía de la teoría.



La regla para el resto del curso

```
const respuesta = await fetch(url);
if (!respuesta.ok) {
    throw new Error(`Error ${respuesta.status} al pedir ${url}`);
}
const datos = await respuesta.json();
```

Comprobad **siempre** `respuesta.ok` antes de leer los datos. Y si no lo es, lanzad vosotros un error con `throw`: así acaba en el mismo `catch` que los fallos de red, y lo tratáis todo en un único sitio.

Sin esa comprobación, con un 404 intentaríais leer como JSON una página de error en HTML, y obtendríais un error confuso que no dice qué ha pasado de verdad.

Ejercicio 03 · El catálogo llega de un fichero

🎯 **Objetivo:** Cargar un array de objetos desde un fichero JSON al abrir la página, comprobar la respuesta, pintarlo con el patrón crear + renderizar del bloque 06 y ofrecer un botón de reintento si falla.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Cargar el catálogo de productos desde un fichero JSON al abrir la página y pintarlo como en el bloque 06.



Los datos salen del código

En el bloque 06, el array `productos` estaba escrito dentro de `app.js`. Ahora vive en `productos.json`, un fichero de **datos** separado del código. Es el paso previo a pedirlos a un servidor de verdad: para vuestro programa, la diferencia será solo la URL.



`pedirJSON`: la función que usaréis en todo el bloque

```
async function pedirJSON(url) {
    const respuesta = await fetch(url);
    if (!respuesta.ok) {
        throw new Error(`Error ${respuesta.status} al pedir ${url}`);
    }
    return respuesta.json();
}
```

Empaqueta lo que aprendisteis en los ejercicios 01 y 02:

1. Pide la URL [202].
2. Comprueba `respuesta.ok` y, si no es correcta, **lanza un error** con `throw` [204].
3. Devuelve los datos leídos con `json()` [206].

Así, quien la llama solo tiene que preocuparse de una cosa: o recibe datos, o salta al `catch`. Da igual si el problema fue de red, un 404 o un JSON roto: todos los errores acaban en el mismo sitio [173].

Un detalle: `return respuesta.json()` devuelve la promesa sin `await`. Como `pedirJSON` es `async`, quien haga `await pedirJSON(…)` esperará a esa promesa igualmente.



Cargar al abrir

```
cargarCatalogo();   // última línea
```

No hay que esperar a ningún clic: la página pide sus datos en cuanto arranca. Mientras llegan, el usuario ve «⏳ Cargando catálogo…».



El botón de reintentar

```
btnReintentar.addEventListener('click', cargarCatalogo);
```

Si falla la carga, aparece un botón que vuelve a llamar a la misma función. Los fallos de red suelen ser pasajeros, así que ofrecer un segundo intento es lo más útil que podéis hacer por el usuario. Fijaos en que `cargarCatalogo` oculta el botón al empezar: así no se queda visible si el segundo intento sale bien.

Ejercicio 04 · Buscador de países

> 🎯 **Objetivo:** Cargar los datos una sola vez al abrir la página y filtrarlos en memoria mientras el usuario escribe, esperando a que deje de teclear (setTimeout + clearTimeout) y comparando los textos sin tener en cuenta mayúsculas ni tildes.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un buscador de países que carga los datos una vez y filtra mientras el usuario escribe.



Cargar una vez, filtrar muchas

```
let paises = [];
…
paises = await pedirJSON('paises.json');
```

Los datos se piden **una sola vez**, al abrir la página, y se guardan en el estado. A partir de ahí, cada búsqueda filtra en memoria con `filter`, como en el bloque 05. No tiene sentido volver a pedir el fichero con cada letra: los datos no han cambiado.

Mientras llegan, los controles están desactivados. Así nadie puede buscar en un array todavía vacío y pensar que no hay resultados.



> 📝 **Nota**
>
> Con 22 países, filtrar en memoria es lo ideal. Con un millón de registros, no os los descargaríais todos: le pediríais al servidor que buscara por vosotros, con una URL como `/paises?buscar=esp`. Lo veréis en los ejercicios 06 a 08, donde la API filtra por usuario.



Esperar a que el usuario deje de escribir

```
buscar.addEventListener('input', () => {
    clearTimeout(idEspera);
    idEspera = setTimeout(filtrar, 300);
});
```

Es la técnica que os conté en la teoría del bloque 10. Cada letra **cancela** la espera anterior [185] y programa una nueva de 300 ms [184]. Solo cuando pasan 300 ms sin teclear se ejecuta `filtrar`. Si escribís «portugal» deprisa, se filtra una vez, no ocho.

Aquí filtrar es tan rápido que no se notaría la diferencia. Pero cuando cada búsqueda sea una petición a un servidor, esta técnica es la diferencia entre una petición y ocho.



Buscar sin tildes

```
texto.toLowerCase().normalize('NFD').replace(/[\u0300-\u036f]/g, '');
```

Nadie escribe «Japón» con tilde en un buscador. Esta línea convierte cualquier texto a minúsculas y le quita las tildes:

- `normalize('NFD')` separa cada letra acentuada en la letra y su tilde (`ó` → `o` + `´`).
- `replace(/[\u0300-\u036f]/g, '')` borra todas las tildes sueltas. Es una expresión regular; no hace falta que la memoricéis, solo que sepáis para qué sirve.

Lo aplico a los **dos** lados de la comparación: a lo que escribe el usuario y a los datos.



Dos filtros a la vez

```
return coincideTexto && coincideContinente;
```

Un país se muestra si cumple los dos criterios. Con `'todos'`, `coincideContinente` es siempre `true`, igual que con la categoría `'todas'` del catálogo del bloque 07.

Ejercicio 05 · Estados de carga, errores y reintentos

> 🎯 **Objetivo:** Gestionar los cuatro estados de una interfaz que carga datos (cargando, datos, vacío y error), distinguir los tipos de error que pueden ocurrir con fetch y permitir reintentar la última carga.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Un tablón de noticias que muestre bien cada situación posible al cargar datos: mientras carga, cuando llegan, cuando no hay nada y cuando algo falla.



Los cuatro estados



Con `aria-live="polite"` en la zona de estado, los lectores de pantalla también se enteran de cada cambio.

📊 Figura 76 · Los cuatro estados de la interfaz

Escenario	Estado que veis
Todo bien	⏳ Cargando… → 4 noticias
Conexión lenta	⏳ Cargando… durante 2 s → 4 noticias
Sin noticias	📭 No hay noticias publicadas todavía
JSON roto	✘ El fichero ha llegado, pero no es un JSON válido
No existe	✘ Error 404 al pedir no-existe.json

El escenario lento es el más instructivo: sin el mensaje de carga, el usuario vería dos segundos de pantalla vacía y pensaría que no funciona.



Vacío no es error

```
if (noticias.length === 0) {
    estado.textContent = '📭 No hay noticias publicadas todavía.';
    return;
}
```

Si el servidor devuelve un array vacío, **todo ha ido bien**: simplemente no hay datos. Tratarlo como un error confundiría al usuario («¿se ha roto algo?»). Merece su propio mensaje.

Y fijaos en el `return` dentro del `try`: el `finally` se ejecuta igualmente, así que el botón se reactiva.



Tres tipos de error

```
if (error.name === 'SyntaxError') { … }   // json() no ha podido leerlo
if (error.name === 'TypeError') { … }     // fetch no ha podido conectar
return error.message;                     // el que lanzamos al ver !respuesta.ok
```

Error	Quién lo lanza	Cuándo
`TypeError`	`fetch`	No hay red, el servidor no existe, CORS... [202]
`Error` propio	Vuestro `throw`	El servidor responde con un código que no es 2xx [204]
`SyntaxError`	`respuesta.json()`	El contenido no se puede interpretar como JSON [206]

Cada uno pide un mensaje distinto. «No hay conexión» invita a comprobar el wifi; «el fichero no es válido» dice que el problema no es del usuario. Un mensaje útil explica qué ha pasado y, si se puede, qué hacer.



Reintentar

El botón «Reintentar» llama a la misma función `cargar`. Solo aparece en el estado de error: en los demás no tendría sentido. Los estados se anuncian en una región `aria-live` [162] para que también lleguen a quien usa lector de pantalla.

Ejercicio 06 · Una API pública: usuarios y publicaciones

> 🎯 **Objetivo:** Consumir una API REST pública desde otro origen: pedir un listado, pintar los datos que interesan de cada objeto y, al pulsar en un elemento, hacer una segunda petición con un parámetro en la URL.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Pedir datos a una API real de Internet: primero un listado de usuarios y, al pulsar uno, sus publicaciones.



Qué es una API REST

Una API es la puerta que un servidor abre para que otros programas le pidan datos [197]. Las API REST organizan esos datos como **recursos** con su propia URL:

URL	Devuelve
`/users`	Todos los usuarios
`/users/1`	El usuario con `id` 1
`/posts?userId=1`	Las publicaciones del usuario 1

JSONPlaceholder es una API pública y gratuita pensada para practicar, sin registro, que permite peticiones desde cualquier origen [214]. Sus datos son inventados, pero su estructura es la de una API real.



Pedir a otro origen: CORS

Hasta el ejercicio 05, la página pedía ficheros a su propio servidor. Ahora pide a `https://jsonplaceholder.typicode.com`, que es **otro origen**. El navegador solo os deja leer la respuesta si el servidor lo autoriza con unas cabeceras especiales: es el mecanismo CORS [209]. JSONPlaceholder lo autoriza, así que funciona. Si probáis con un servidor que no lo hace, veréis un error de CORS en la consola, y vuestro código no puede hacer nada para saltárselo: es el servidor quien decide.

Lo tenéis explicado en la infografía de la teoría.



Coger solo lo que hace falta

Un usuario de JSONPlaceholder tiene esta forma (resumida):

```
{
  "id": 1,
  "name": "Leanne Graham",
  "email": "Sincere@april.biz",
  "address": { "street": "Kulas Light", "city": "Gwenborough", … },
  "company": { "name": "Romaguera-Crona", … }
}
```

No hay que pintarlo todo. Elegís lo que interesa y llegáis a lo que está anidado con puntos: `usuario.address.city`, `usuario.company.name`. Es el bloque 05 con datos que no habéis escrito vosotros.



Un parámetro en la URL

```
pedirJSON(`${API}/posts?userId=${id}`)
```

Lo que va detrás de `?` son los **parámetros** de la petición. En lugar de descargar las 100 publicaciones y filtrarlas, le pido a la API solo las del usuario elegido, y ella filtra por mí [214]. Menos datos viajando y menos trabajo en el navegador.

Todas estas peticiones son de tipo **GET**, el método para pedir datos sin modificar nada en el servidor [210]. Es el que usa `fetch` si no le decís otra cosa.



La tarjeta seleccionada

```
t.classList.toggle('seleccionado', t.contains(boton));
```

`contains` dice si un elemento está dentro de otro. Cada tarjeta se marca solo si contiene el botón pulsado. El usuario ve en todo momento de quién son las publicaciones que está leyendo.



📝 Nota

En el momento de preparar este ejercicio no he podido comprobar el acceso en directo a JSONPlaceholder desde mi entorno. He verificado el formato de sus datos en su guía y en su repositorio oficial, y he probado el código con una copia de esos datos. Si el día de clase la API no responde, veréis el mensaje de error del `catch`: aprovechadlo para comprobar que funciona.

Ejercicio 07 · Enviar datos con POST

🎯 Objetivo: Enviar datos a un servidor con fetch usando el método POST, un cuerpo en JSON y la cabecera Content-Type, y procesar la respuesta, validando antes el formulario y bloqueando el botón mientras se envía.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Enviar una publicación nueva a una API con el método POST y mostrar lo que responde el servidor.



GET y POST

Hasta ahora todas las peticiones eran **GET**: pedir datos. Para **enviar** datos que el servidor debe procesar o guardar, se usa **POST** [211].

Método	Para qué	Lleva cuerpo
`GET`	Pedir datos	No
`POST`	Enviar datos para crear algo	Sí



`fetch` con opciones

```
fetch(`${API}/posts`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json; charset=UTF-8' },
    body: JSON.stringify(datos),
});
```

`fetch` admite un segundo argumento con las opciones de la petición [2]:

- **`method`**: el método HTTP. Sin él, `fetch` hace un GET.
- **`body`**: el contenido que se envía. Tiene que ser **texto**, así que el objeto se convierte con `JSON.stringify`, igual que al guardarlo en `localStorage` en el bloque 09.
- **`headers`**: información sobre la petición. `Content-Type` le dice al servidor en qué formato va el cuerpo [212]. Sin ella, el servidor podría no entender que lo que le llega es JSON.

Es exactamente el ejemplo de la guía de JSONPlaceholder [214].



La respuesta

```
{
  "title": "Mi primera publicación",
  "body": "Enviada desde el ejercicio 07",
  "userId": 1,
  "id": 101
}
```

El servidor devuelve lo que habéis enviado más el `id` que le ha asignado. Con JSONPlaceholder siempre es el 101, porque hay 100 publicaciones de ejemplo.



📝 **Nota**

La guía de JSONPlaceholder avisa: el recurso **no se crea de verdad** en el servidor, solo se simula. Si después pedís `/posts/101`, obtendréis un 404 [214]. Es justo lo que queremos para practicar: podéis enviar lo que queráis sin ensuciar nada.



Detalles de un buen envío

- **Validar antes de enviar.** `reportValidity()` del bloque 08 [5]: si el formulario no es válido, ni siquiera se hace la petición.
- **Botón desactivado mientras se envía.** Si el usuario hace doble clic, sin esta línea enviaríais dos publicaciones.
- **El formulario se vacía solo si ha ido bien.** Si falla, el usuario no pierde lo que ha escrito y puede volver a intentarlo.

- `finally` **reactiva el botón**, vaya bien o mal.

Ejercicio 08 · Tareas de la API con favoritos

> 🎯 **Objetivo:** Construir una aplicación completa con datos remotos: cargar en paralelo con Promise.all, recargar al cambiar de usuario, filtrar y buscar en memoria con espera, y combinar los datos de la API con preferencias del usuario guardadas en localStorage.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una aplicación que muestra las tareas de cada usuario de una API, con filtros, búsqueda y favoritas que se recuerdan. Junta este bloque con los anteriores.



Dos fuentes de datos

Dato	De dónde sale	¿Se puede cambiar?
Usuarios y tareas	La API	No: son del servidor
Favoritas	`localStorage` [21]	Sí: son del usuario

Las tareas no tienen una propiedad «favorita»: esa información es **vuestra**, no del servidor. Por eso la guardo aparte, como un array de ids, y la **combino** con los datos de la API al pintar: `favoritas.includes(t.id)`. Es la misma idea que el carrito del bloque 09, que guardaba ids y no productos.



Arrancar en paralelo

```
const [usuarios, tareasIniciales] = await Promise.all([
    pedirJSON(`${API}/users`),
    pedirJSON(`${API}/todos?userId=1`),
]);
```

Para arrancar necesito los usuarios (para el desplegable) y las tareas del primero. Son independientes, así que las pido **a la vez** con `Promise.all` [196], como en la carrera del bloque 10. Y con la desestructuración del bloque 05 saco los dos resultados en una línea, en el mismo orden en que los pedí.

Si cualquiera de las dos falla, `Promise.all` falla entero [196] y salta el `catch`: sin usuarios ni tareas, no hay aplicación que mostrar.



Qué vuelve a pedir y qué no

Acción del usuario	¿Petición a la API?
Cambiar de usuario	**Sí**: son otras tareas
Escribir en el buscador	No: se filtra en memoria
Cambiar de filtro	No: se filtra en memoria
Marcar una favorita	No: solo `localStorage`

Esta tabla es la decisión de diseño más importante del ejercicio. Pedid al servidor solo lo que de verdad cambia; todo lo demás, con los datos que ya tenéis.



Todo lo demás ya lo conocéis

- **Estado → renderizar** (bloque 07): los cuatro escuchadores cambian el estado y llaman a `renderizar()`.
- **Delegación** (bloque 07): un escuchador para los filtros y otro para todas las estrellas, que se recrean en cada renderizado.
- **Espera al escribir** (bloque 10): el buscador espera 300 ms con `setTimeout` [184] y `clearTimeout` [185].
- **Carga segura** (bloque 09): las favoritas se leen dentro de un `try`, validando que sea un array de enteros.

Si este ejercicio os sale, tenéis en la mano todo lo necesario para el proyecto final.



📝 **Nota**

Como en el ejercicio 06, he verificado el formato de `/users` y `/todos?userId=…` en la guía y el repositorio oficiales de JSONPlaceholder [214] y he probado el código con una copia de esos datos, pero no he podido comprobar el acceso en directo desde mi entorno.



Bloque 12

Bloque 12 · Módulos, clases y buenas prácticas

Vuestros programas ya hacen muchas cosas: leen formularios, guardan datos, piden información a un servidor... y empiezan a tener cientos de líneas en un único `app.js`. Encontrar algo ahí dentro cuesta cada vez más, y tocar una parte sin romper otra, también. En este bloque no vais a aprender a hacer cosas nuevas, sino a **ordenar** las que ya sabéis hacer: repartir el código en ficheros con **módulos**, agrupar datos y comportamiento con **clases**, separar responsabilidades y encontrar fallos con las herramientas de **depuración**. Es lo que distingue un programa que funciona de un programa que se puede mantener.

Al terminar este bloque sabréis:

- Repartir una aplicación en módulos con `export`, `import` y `<script type="module">`.
- Explicar por qué los módulos necesitan un servidor.
- Declarar clases con `constructor`, propiedades y métodos, y crear instancias con `new`.
- Reutilizar código con herencia: `extends` y `super`.
- Proteger datos con elementos privados `#`, *getters*, *setters* y miembros `static`.
- Separar una aplicación en modelo, vista y controlador.
- Depurar con puntos de interrupción y la instrucción `debugger`.



⚠️ Importante

Los ejercicios 01, 02, 06 y 08 usan módulos: **abridlos con Live Server**. Los ejercicios 03, 04, 05 y 07 funcionan también con doble clic.



1. Módulos: un fichero, una responsabilidad

Un **módulo** es un fichero JavaScript con su propio ámbito: lo que declara se queda dentro, salvo lo que decida compartir [215].



Figura 77 · Módulos: export e import

1.1. `export` e `import`

```
// utilidades.js
export const PORCENTAJE_IVA = 21;
export function formatearPrecio(cantidad) { … }
function privada() { … }                    // sin export: nadie más la ve
```

```
// app.js
import { PORCENTAJE_IVA, formatearPrecio } from './utilidades.js';
```

`export` marca lo que el módulo comparte [216]; `import` lo trae a otro fichero por su nombre exacto [217]. La ruta empieza por `./` (misma carpeta) y lleva la extensión `.js`.

1.2. Cuatro formas de importar

Forma	Código	Detalle
Con nombre	`import { formatearPrecio } from './utilidades.js'`	Con llaves y el nombre exacto
Con alias	`import { formatearPrecio as precio } from …`	Cambia el nombre en este fichero
Por defecto	`import convertir from './conversor.js'`	Sin llaves; el nombre lo elegís vosotros [217]
Entero	`import * as util from './utilidades.js'`	Todo lo exportado, dentro de un objeto

Un módulo puede tener muchas exportaciones con nombre, pero solo **una** por defecto [216]. Lo importado es de solo lectura: si intentáis reasignarlo, obtenéis un `TypeError` [217].

1.3. `<script type="module">`

```
<script type="module" src="app.js"></script>
```

En el HTML solo se carga el módulo principal, y él importa los demás. El atributo `type="module"` es imprescindible: le dice al navegador que el fichero es un módulo [6]. Sin él, `import` provoca un `SyntaxError` [217].

1.4. Lo que cambia con los módulos

Diferencia	Qué significa para vosotros
Necesitan un servidor	Con doble clic (`file://`) el navegador bloquea los módulos por seguridad y no se ejecuta nada [215]. Siempre Live Server
Modo estricto	Siempre activo, sin escribir `"use strict"` [215]. Algunos descuidos que antes se ignoraban ahora dan error [218]
Diferidos	Se ejecutan cuando el HTML ya está leído, aunque el `<script>` esté en el `<head>` [215]
Ámbito propio	Sus variables no son globales: no aparecen en `window` ni en la consola [215]
Una sola vez	Aunque los importen varios ficheros, se ejecutan una vez y todos comparten el mismo [215]

Lo de necesitar servidor ya os sonará: es el mismo bloqueo que vivisteis con `fetch` en el bloque 11 [208].



2. Clases: el molde de los objetos

En el bloque 05 creabais objetos con llaves, uno a uno. Cuando necesitáis muchos objetos con la misma forma y el mismo comportamiento, se escribe una **clase**: una plantilla para crearlos [219].



Figura 78 · Una clase y sus instancias

```
class Producto {
    constructor(nombre, precio) {
        this.nombre = nombre;
        this.precio = precio;
        this.unidades = 1;
    }

    total() {
        return this.precio * this.unidades;
    }
}

const teclado = new Producto('Teclado', 59.9);
teclado.total();   // 59.9
```

Pieza	Qué es
`class Producto`	El nombre de la clase. Por costumbre, empieza por mayúscula
`constructor`	Se ejecuta al crear cada objeto y prepara sus datos [220]
`this`	Dentro de la clase, el objeto concreto con el que se trabaja [221]
Métodos (`total()`)	Lo que cada objeto sabe hacer. Se escriben sin `function`
`new`	Crea un objeto nuevo a partir de la clase [222]. Sin `new`, una clase no se puede usar: da `TypeError` [222]

Cada **instancia** (cada objeto creado con `new`) guarda sus propios datos, pero todas comparten los mismos métodos: `total` se escribe una vez y sirve para todas [223]. Y la gran ventaja: las reglas viven **dentro** del objeto. Si `quitar()` no deja bajar de 0, nadie que use la clase puede olvidarlo.



3. Herencia: `extends` y `super`

Cuando varias clases comparten una parte, esa parte se escribe una vez en una **clase madre**, y las demás la **heredan** con `extends` [224].

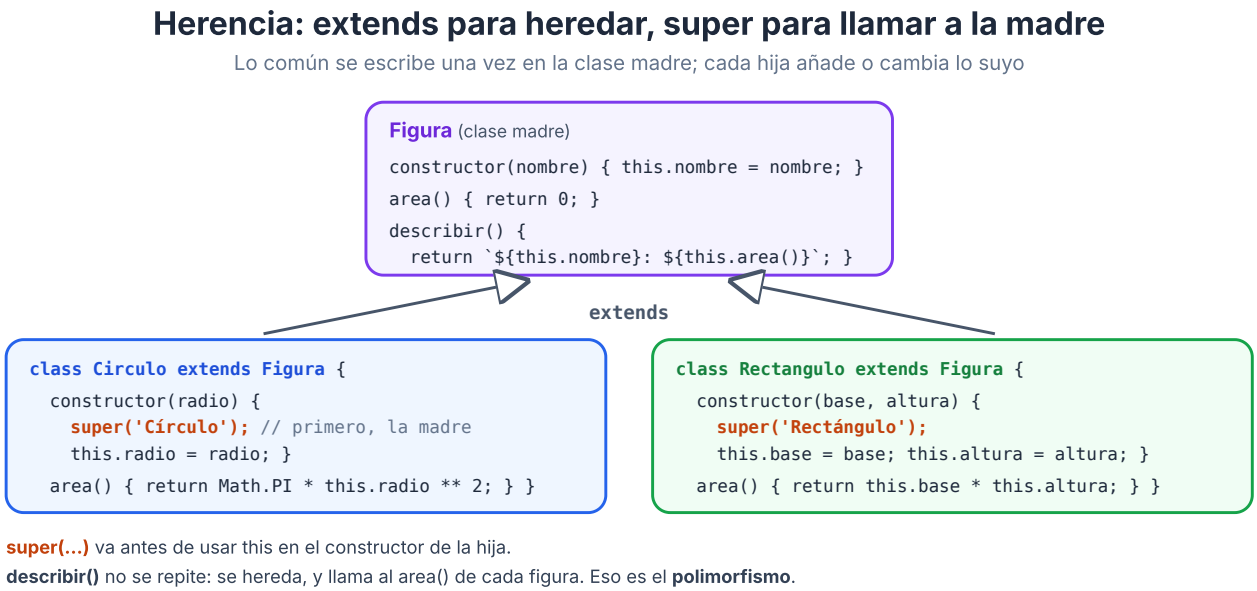


Figura 79 · Herencia con extends y super

```
class Figura {
    constructor(nombre) { this.nombre = nombre; }
    area() { return 0; }
    describir() { return `${this.nombre}: ${this.area().toFixed(2)}`; }
}

class Circulo extends Figura {
    constructor(radio) {
        super('Círculo');         // primero, el constructor de la madre
        this.radio = radio;
    }
    area() { return Math.PI * this.radio ** 2; }    // sobrescribe el de la madre
}
```

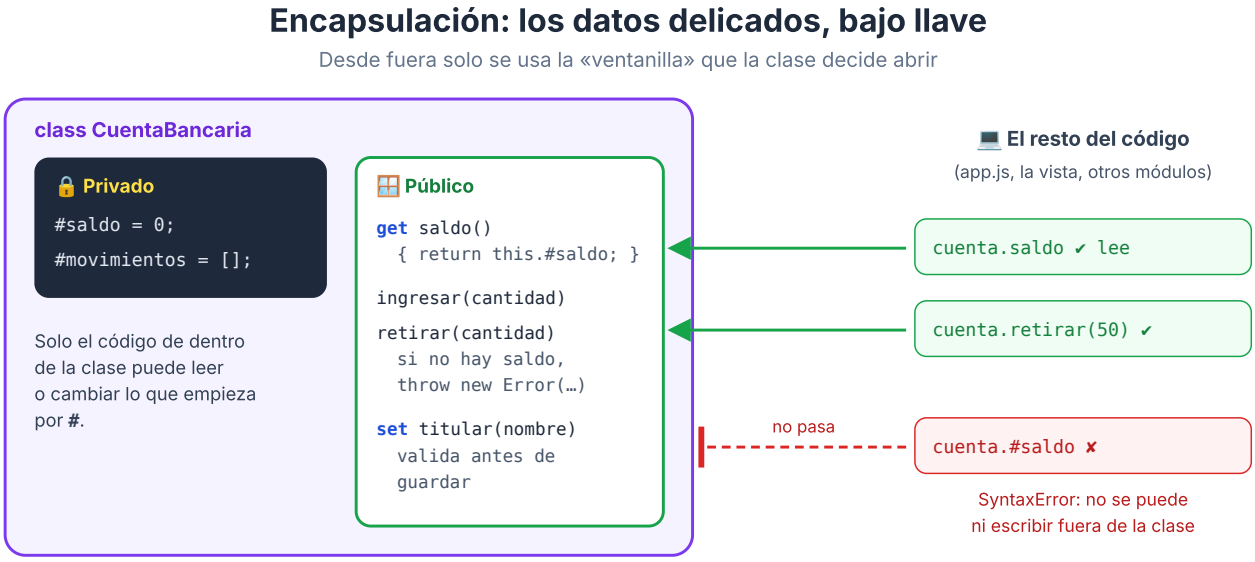
- `super(…)` llama al constructor de la madre [225]. En una clase hija hay que llamarlo **antes** de usar `this`; si no, la consola lanza un `ReferenceError` [224].

- `super.metodo()` llama a un método de la madre desde la hija [13]: útil para ampliar lo que hace en lugar de reescribirlo.
- **Sobrescribir:** si la hija declara un método con el mismo nombre, se usa el suyo.
- **Polimorfismo:** `describir()` solo está en `Figura`, pero llama al `area()` de cada figura. El mismo código funciona con todas, y añadir una figura nueva no obliga a cambiarlo.
- `instanceof` dice si un objeto pertenece a una clase, mirando toda la cadena de herencia [14]: un círculo es `instanceof Circulo` y también `instanceof Figura`.



4. Encapsulación: privados, *getters*, *setters* y `static`

Si cualquiera puede escribir `cuenta.saldo = 1000000`, las reglas de la clase no sirven de nada. La **encapsulación** guarda los datos delicados bajo llave y deja una ventanilla de métodos que ponen condiciones.



📊 Figura 80 · Encapsulación: privados, getters y setters

```
class CuentaBancaria {
    static LIMITE_RETIRADA = 600;        // de la clase, no de cada cuenta
    #saldo = 0;                          // privado

    get saldo() {                        // se lee: cuenta.saldo
        return this.#saldo;
    }

    retirar(cantidad) {
        if (cantidad > this.#saldo) {
            throw new Error('Saldo insuficiente.');
        }
        this.#saldo -= cantidad;
    }
}
```

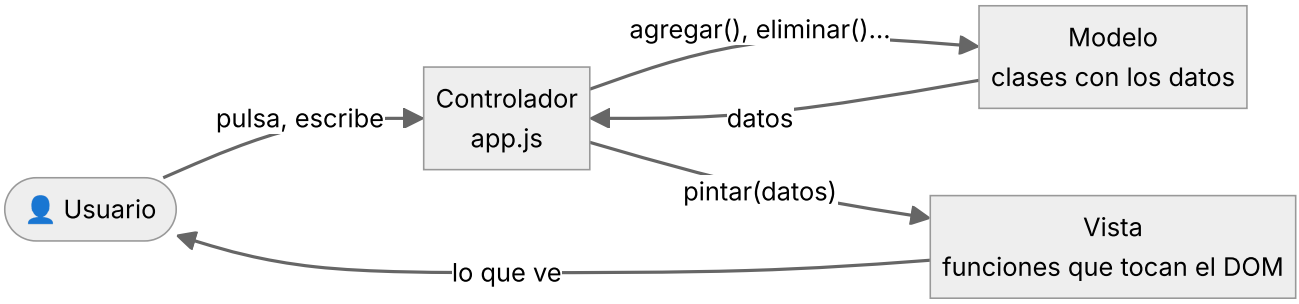
Herramienta	Qué hace
`#nombre`	Campo o método **privado**: solo lo puede usar el código de la clase. Escribirlo fuera es un error de sintaxis [226]
`get`	Una propiedad que se lee como un dato pero ejecuta una función [227]
`set`	Una propiedad que se asigna como un dato pero ejecuta una función, normalmente para validar [228]
`static`	Un miembro de la **clase**, no de cada objeto: `CuentaBancaria.LIMITE_RETIRADA` [229]
`throw new Error(…)`	La clase avisa de que algo no se puede hacer [230]; quien la usa lo recoge con `try...catch`

Una regla de oro: una clase **no escribe en el DOM**. Lanza errores con mensajes claros y deja que la página decida cómo mostrarlos.



5. Separar responsabilidades: modelo, vista y controlador

Con módulos y clases ya tenéis las piezas para ordenar una aplicación entera. El reparto más habitual tiene tres papeles:



Papel	Sabe...	No sabe...
Modelo	Qué datos hay y qué reglas cumplen	Que existe una página
Vista	Cómo se pinta cada cosa	De dónde vienen los datos ni cómo se cambian
Controlador	Qué hacer cuando el usuario actúa	Cómo se valida ni cómo se pinta

Una prueba rápida de que el reparto está bien hecho: en el modelo no aparece `document`, y en la vista no aparece el modelo. Así, si cambia el aspecto, solo se toca la vista; si cambia una regla, solo el modelo.



6. Depuración: detener el programa y mirar



Figura 81 · Depurar con un punto de interrupción

Los fallos más difíciles no dan ningún error: el programa funciona, pero el resultado está mal. Para encontrarlos, `console.log` se queda corto. Las herramientas de desarrollo permiten **detener la ejecución** en una línea y examinarlo todo en ese momento [231].

Herramienta	Qué hace
Punto de interrupción	En **Fuentes** (*Sources*), pulsad el número de una línea: la ejecución se detendrá ahí [231]
Pasar a la siguiente (F10)	Ejecuta la línea y se detiene en la siguiente
Entrar (F11)	Entra en la función que se llama en esa línea
Ámbito (*Scope*)	Todas las variables con su valor en ese instante [231]
`debugger;`	Escrito en el código, detiene la ejecución si las herramientas están abiertas [232]

Y el método, que vale más que cualquier herramienta: **sabed qué debería salir**, detened el programa antes del fallo, **mirad los valores en lugar de suponerlos** y corregid de uno en uno.



7. La plantilla de trabajo con módulos

A partir de ahora, una aplicación mediana queda así:

```
mi-aplicacion/
├── index.html      ← <script type="module" src="app.js">
├── style.css
├── app.js          ← controlador: importa, escucha y coordina
├── modelo.js       ← clases con los datos y sus reglas
├── vista.js        ← funciones que pintan en el DOM
└── almacen.js      ← guardar y cargar (localStorage o fetch)
```

Y el módulo principal mantiene la plantilla de siempre:

```
// estructura del programa principal
import { ListaCompra } from './modelo.js';
import { pintarLista, mostrarMensaje } from './vista.js';

const lista = new ListaCompra();

// funciones
function actualizar() {
    pintarLista(contenedor, lista.articulos);
}

// constantes al DOM que queremos escuchar
const formulario = document.querySelector('#formArticulo');
const contenedor = document.querySelector('#articulos');
const mensaje = document.querySelector('#mensaje');

// colocar escuchadores de eventos
formulario.addEventListener('submit', (e) => {
    e.preventDefault();
    // entradas
    const nombre = formulario.nombre.value;
    // procesamiento: el modelo valida
    try {
        lista.agregar(nombre, 1);
        mostrarMensaje(mensaje, '');
    } catch (error) {
        mostrarMensaje(mensaje, `✘ ${error.message}`);
    }
    // salidas
    actualizar();
});

actualizar();
```



8. Ejercicios

Nº	Ejercicio	Qué practicáis	Solución
01	Mi primer módulo	`export`, `import`, `type="module"`, lo que no se exporta	Ver
02	Formas de exportar e importar	Alias, por defecto, `* as`, y cuatro diferencias de los módulos	Ver
03	Mi primera clase: el carrito	`class`, `constructor`, métodos, `new`, `instanceof`	Ver
04	Herencia: figuras geométricas	`extends`, `super`, sobrescribir, polimorfismo	Ver
05	Encapsulación: la cuenta bancaria	`#privados`, `get`, `set`, `static`, `throw`	Ver
06	Modelo, vista y controlador	Separar responsabilidades en tres módulos	Ver
07	Depuración: cuatro fallos escondidos	Puntos de interrupción, paso a paso, `debugger`	Ver
08	Proyecto: mi biblioteca con clases y módulos	Todo el bloque, con `toJSON` y `localStorage`	Ver

El 07 no os lo saltéis aunque parezca el más sencillo. Aprender a depurar os ahorrará más horas que cualquier otra cosa de este curso, y el proyecto final del bloque 13 os lo va a pedir.



🧪 Los ejercicios, explicados

Ejercicio 01 · Mi primer módulo

> 🎯 **Objetivo:** Repartir el código en dos ficheros: un módulo que exporta funciones y constantes, y un módulo principal que las importa. Comprobar que lo que no se exporta no es visible desde fuera.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Sacar las funciones de cálculo a un fichero propio, `utilidades.js`, y usarlas desde `app.js` con `import`.



Exportar: decidir qué se comparte

```
export const PORCENTAJE_IVA = 21;
const IVA = PORCENTAJE_IVA / 100;      // sin export: privada del módulo
export function conIVA(precio) { … }
export function formatearPrecio(cantidad) { … }
```

Un módulo es un fichero con su propio ámbito. Lo que lleva `export` delante queda disponible para otros ficheros; lo demás se queda dentro [215]. Pueden exportarse constantes, funciones y clases, y puede haber tantas exportaciones con nombre como queráis [216].

Fijaos en que `conIVA` usa `IVA` sin problema: está en su mismo fichero. Lo que no puede hacer nadie más es tocarla.



Importar: pedir lo que se necesita

```
import { PORCENTAJE_IVA, conIVA, formatearPrecio } from './utilidades.js';
```

Dentro de las llaves van los nombres **exactos** que exporta el otro módulo [217]. Y la ruta:

Detalle	Por qué
Empieza por `./`	Indica «en esta misma carpeta». Sin ella, el navegador no la reconoce como ruta de un fichero
Lleva `.js`	El navegador pide exactamente esa URL: no adivina extensiones
Va al principio del fichero	Los `import` se resuelven antes de ejecutar el código [217]

Si importáis un nombre que no existe (por ejemplo, porque olvidasteis el `export`), el módulo **no llega a ejecutarse** y la consola os dice qué nombre falta. Probadlo quitando un `export`.



`type="module"`

```
<script type="module" src="app.js"></script>
```

El atributo `type="module"` le dice al navegador que el fichero es un módulo [6]. Solo `app.js` aparece en el HTML: es el módulo principal, él importa `utilidades.js` y el navegador lo descarga solo. Sin `type="module"`, el navegador trataría `app.js` como un script clásico y la palabra `import` provocaría un `SyntaxError` [217].



El experimento: `typeof IVA`

```
typeof IVA desde app.js → undefined
```

`IVA` existe, pero dentro de `utilidades.js`. Desde `app.js` es como si no existiera. Uso `typeof` porque es la única forma de preguntar por una variable inexistente sin provocar un `ReferenceError`. Esto es lo que hace útiles a los módulos: cada fichero esconde sus detalles y enseña solo su «escaparate».



¿Y con doble clic?

Si abrís la página con doble clic, la consola muestra un error de CORS y **no se ejecuta nada**: ni el cálculo ni el experimento. La documentación lo avisa expresamente: los módulos cargados desde `file://` chocan con los requisitos de seguridad, y hay que probarlos a través de un servidor [215]. Es el mismo motivo que en el bloque 11 [208]. Por eso, Live Server.

Ejercicio 02 · Formas de exportar e importar

> 🎯 **Objetivo:** Conocer las formas de exportar e importar (con nombre, con alias, por defecto y el módulo entero) y comprobar en directo cuatro diferencias de los módulos con los scripts clásicos: modo estricto, ámbito propio, carga diferida y ejecución única.

📂 Enunciado · Solución completa



Qué pedía el ejercicio

Probar las cuatro formas de importar y comprobar en directo qué cambia cuando un fichero es un módulo.



Cuatro formas de importar

Forma	Código	Cuándo
Con nombre	`import { areaCirculo } from './geometria.js'`	Lo normal: sabéis qué queréis
Con alias	`import { areaRectangulo as areaRect } from …`	El nombre choca con otro o es muy largo
Por defecto	`import convertir from './conversor.js'`	El módulo tiene una cosa principal
Entero	`import * as geo from './geometria.js'`	Queréis todo, agrupado bajo un nombre

Cada módulo puede tener muchas exportaciones con nombre, pero **solo una por defecto** [216]. La exportación por defecto se importa **sin llaves** y con el nombre que elijáis [217]. Las otras se importan con llaves y con su nombre exacto, salvo que uséis `as`.

Con `* as geo`, `Object.keys(geo)` muestra `areaCirculo, areaRectangulo, perimetroCirculo`: todo lo exportado, convertido en propiedades de un objeto.



Lo que cambia en un módulo

Estas cuatro filas son las que más sorprenden la primera vez:

Experimento	Resultado	Explicación
Se ejecuta una vez	`contador.js se ha ejecutado 1 vez…`	Un módulo se ejecuta una sola vez aunque lo importen varios ficheros [215]. `app.js` e `informe.js` comparten el mismo `contador.js`
`this`	`undefined`	Los módulos funcionan siempre en modo estricto [215], y en su nivel superior `this` vale siempre `undefined` [221]
`window.secreto`	`undefined`	Lo declarado en un módulo no se vuelve global [215]
`readyState`	`interactive`	Los módulos se cargan **diferidos**: aunque el script esté en el `<head>`, se ejecuta cuando el HTML ya está leído [215]. Por eso `querySelector` encuentra la tabla



Un `import` es de solo lectura... y está vivo

```
visitas = 99;    // TypeError: Assignment to constant variable.
```

Lo que importáis no se puede reasignar: el valor es del módulo que lo exporta [217]. Si queréis cambiarlo, tiene que ser con una función de ese módulo, como `sumarVisita()`.

Y al pulsar **Sumar visita**, `visitas` pasa a `1`, `2`, `3`... aunque lo importasteis al principio. Los `import` son **enlaces vivos** [2]: no copian el valor, sino que miran siempre el valor actual dentro de `contador.js`.

Ejercicio 03 · Mi primera clase: el carrito

> 🎯 **Objetivo:** Declarar una clase con constructor y métodos, crear varias instancias con new y comprobar que cada una guarda sus propios datos mientras comparten los mismos métodos.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Crear la clase `Producto` y usarla para construir las líneas de un carrito que se pueden aumentar y reducir.



La clase: el molde

```
class Producto {
    constructor(nombre, precio) {
        this.nombre = nombre;
        this.precio = precio;
        this.unidades = 1;
    }
    total() {
        return this.precio * this.unidades;
    }
}
```

Una clase es una plantilla para crear objetos que tienen los mismos datos y saben hacer las mismas cosas [219]. Dentro hay dos tipos de piezas:

Pieza	Qué es
`constructor`	Un método especial que se ejecuta al crear cada objeto y prepara sus datos [220]
`sumar()`, `quitar()`, `total()`	Los **métodos**: lo que cada objeto sabe hacer

Dentro de los métodos, `this` es **el objeto concreto** con el que se trabaja [221]. `teclado.total()` usa el precio del teclado; `raton.total()`, el del ratón.



`new`: sacar objetos del molde

```
const carrito = CATALOGO.map((p) => new Producto(p.nombre, p.precio));
```

`new` crea un objeto vacío, ejecuta el `constructor` con ese objeto como `this` y lo devuelve [222]. Hago un `map` sobre el catálogo del bloque 05 y obtengo tres **instancias** de `Producto`.

Si olvidáis el `new`, la consola protesta: `TypeError: Class constructor Producto cannot be invoked without 'new'`. Las clases no se pueden llamar como una función normal [222].



Los objetos se cambian a sí mismos

```
if (boton.dataset.accion === 'sumar') {
    producto.sumar();
} else {
    producto.quitar();
}
```

Comparad con los carritos del bloque 09, donde la lógica estaba fuera del objeto: `linea.cantidad++`, comprobaciones sueltas... Ahora la regla «no bajar de 0» vive **dentro** de la clase, en `quitar()`. Quien la use no puede olvidarla.



La línea de información

```
typeof Producto → function · carrito[0] instanceof Producto → true · ¿comparten total()? → true
```

- **`typeof Producto` es `function`.** Por dentro, una clase es un tipo especial de función [219]. Su sintaxis es más clara, pero no es magia.
- **`instanceof`** comprueba si un objeto se creó a partir de una clase [233].
- **`carrito[0].total === carrito[1].total` es `true`.** Cada objeto tiene sus propios `nombre`, `precio` y `unidades`, pero el método `total` es **el mismo** para todos: se escribe una vez en la clase y lo comparten [223]. Es justo lo que muestra la infografía de la teoría.

Este ejercicio funciona con doble clic: las clases no necesitan servidor. Solo los módulos lo necesitan.

Ejercicio 04 · Herencia: figuras geométricas

> 🎯 **Objetivo:** Usar la herencia para no repetir código: una clase madre con lo común y clases hijas que llaman a super y sobrescriben el método area. Comprobar el polimorfismo: el mismo método describir funciona con todas las figuras.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Construir una familia de clases: una madre, `Figura`, y tres hijas que heredan de ella y calculan su área cada una a su manera.



`extends`: heredar

```
class Circulo extends Figura {
    …
}
```

`extends` crea una clase hija a partir de otra [224]. Un `Circulo` tiene todo lo de `Figura` (`nombre`, `describir()`…) sin volver a escribirlo, y añade o cambia lo que necesita. Lo tenéis dibujado en la infografía de la teoría.



`super`: llamar a la madre

```
constructor(radio) {
    super('Círculo');      // primero
    this.radio = radio;    // después
}
```

`super(…)` ejecuta el constructor de la clase madre [225]. Aquí le paso el nombre y `Figura` lo guarda en `this.nombre`. El orden importa: en una clase hija, **no podéis usar `this` hasta haber llamado a `super`** [224]. Si lo intentáis, la consola os detiene con un `ReferenceError` que lo explica con bastante claridad.



Sobrescribir: misma pregunta, otra respuesta

`Figura` tiene un `area()` que devuelve `0`. Cada hija declara **su propio** `area()`, y ese es el que se usa:

Clase	`area()`
`Figura`	`0`
`Circulo`	`Math.PI * this.radio ** 2`
`Rectangulo`	`this.base * this.altura`
`Triangulo`	`(this.base * this.altura) / 2`



Polimorfismo: el método que no se repite

```
describir() {
    return `${this.nombre} · ${this.medidas()} · área ${this.area().toFixed(2)}`;
}
```

`describir()` solo está en `Figura`. Pero cuando lo llama un círculo, `this.area()` es el área **del círculo**; cuando lo llama un triángulo, la del triángulo. El mismo código responde de forma distinta según el objeto: eso es el **polimorfismo**. Gracias a él, `renderizar()` no necesita saber qué figura está pintando:

```
li.textContent = figura.describir();
```

Si mañana añadís un `Pentagono`, `renderizar()` no cambia ni una línea.



`instanceof` también mira a la madre

```
const circulos = figuras.filter((f) => f instanceof Circulo).length;
```

`instanceof` dice si un objeto pertenece a una clase [233]. Y como mira toda la cadena de herencia, un círculo es `instanceof Circulo` **y también** `instanceof Figura`. Probadlo en la consola.

Ejercicio 05 · Encapsulación: la cuenta bancaria

> 🎯 **Objetivo:** Proteger los datos de un objeto con campos privados (#), exponerlos de forma controlada con getters y setters, validar en los métodos lanzando errores y usar una propiedad estática de la clase.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una cuenta bancaria cuyo saldo no se pueda tocar desde fuera: solo con `ingresar()` y `retirar()`, que validan cada operación.



Por qué proteger los datos

Con el `Producto` del ejercicio 03, cualquiera podía escribir `producto.unidades = -7` y saltarse la regla de `quitar()`. En una cuenta bancaria eso es inadmisible. La **encapsulación** consiste en guardar los datos delicados bajo llave y abrir solo una «ventanilla» de métodos que ponen condiciones. Lo tenéis en la infografía de la teoría.



Campos y métodos privados: `#`

```
#saldo = 0;
#movimientos = [];
#validar(cantidad) { … }
```

Lo que empieza por `#` es **privado**: solo el código escrito dentro de la clase puede leerlo o cambiarlo [226]. No es una convención, es una regla del lenguaje: escribir `cuenta.#saldo` fuera de la clase es un **error de sintaxis**, y el programa ni siquiera arranca [226]. Los métodos también pueden ser privados, como `#validar`, `#redondear` y `#registrar`: son herramientas internas que nadie de fuera necesita.



📝 Nota

Si probáis `cuenta.#saldo` en la consola de Chrome, funcionará. Es una excepción de las herramientas de desarrollo para ayudaros a depurar [226], no del lenguaje.



Getters: leer como si fuera una propiedad

```
get saldo() {
    return this.#saldo;
}

salidaSaldo.textContent = formatear(cuenta.saldo);   // sin paréntesis
```

`get` asocia una propiedad a una función que se ejecuta al leerla [227]. Desde fuera, `cuenta.saldo` parece un dato normal, pero no hay forma de cambiarlo, porque no he escrito un `set saldo`.

El getter de `movimientos` devuelve `[...this.#movimientos]`, una **copia** (bloque 05). Si devolviera el array original, cualquiera podría hacer `push` en él aunque fuera privado.



Setters: asignar... con condiciones

```
set titular(nombre) {
    const limpio = String(nombre).trim();
    if (limpio.length < 3) {
        throw new Error('El titular debe tener al menos 3 letras.');
    }
    this.#titular = limpio;
}

cuenta.titular = 'Ana';   // parece una asignación, pero ejecuta el setter
```

`set` asocia una propiedad a una función que se ejecuta al intentar asignarla [228]. Por eso el constructor hace `this.titular = titular`: así el titular inicial también pasa por la validación.



Los errores los lanza la clase; los muestra la página

La clase no sabe nada de la página: no escribe mensajes en el DOM. Cuando algo no se puede hacer, lanza un `Error` con un mensaje claro [230]. Quien la usa decide qué hacer con él:

```
try {
    cuenta.retirar(cantidad);
} catch (error) {
    mensaje.textContent = `✘ ${error.message}`;
}
```

Probad a retirar 700 €, a retirar más de lo que hay o a ingresar `-5`: cada caso tiene su mensaje.



`static`: un dato de la clase

```
static LIMITE_RETIRADA = 600;
…
if (cantidad > CuentaBancaria.LIMITE_RETIRADA) { … }
```

Un miembro `static` pertenece a la **clase**, no a cada cuenta, y se usa con el nombre de la clase [229]. El límite es el mismo para todas, así que no tiene sentido que cada objeto guarde una copia. `cuenta.LIMITE_RETIRADA` es `undefined`.



La trampa que no funciona

```
cuenta.saldo = 1000000;
cuenta.movimientos.push({ … });
```

El saldo sigue igual y no aparece ningún «Regalo»:

- `cuenta.saldo = …` no tiene setter, así que se ignora. En este script clásico, sin avisar. En modo estricto (por ejemplo, dentro de un módulo) lanzaría un `TypeError` [218].
- `cuenta.movimientos.push(…)` modifica la copia que devuelve el getter, no el original.



Un detalle con los céntimos

`#redondear` deja el saldo con dos decimales después de cada operación. Sin él, después de ingresar `0.1` y `0.2` el saldo sería `0.30000000000000004`, el problema de los decimales que visteis en el bloque 02.

Ejercicio 06 · Modelo, vista y controlador

> 🎯 **Objetivo:** Separar responsabilidades: un modelo con los datos y las reglas, que no toca el DOM; una vista que solo pinta; y un controlador que escucha al usuario y los coordina. La vista y casi todo el controlador ya están hechos: os toca escribir el modelo.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Escribir el modelo de una lista de la compra cuya vista y cuyo controlador ya estaban hechos, respetando qué le toca a cada fichero.



Tres ficheros, tres responsabilidades

Fichero	Papel	Sabe…	No sabe…
`modelo.js`	**Modelo**	Qué artículos hay y qué reglas cumplen	Que existe una página
`vista.js`	**Vista**	Cómo se pinta un artículo	De dónde vienen los datos ni cómo se cambian
`app.js`	**Controlador**	Qué hacer cuando el usuario pulsa algo	Cómo se valida ni cómo se pinta

La prueba de que está bien repartido es sencilla: en `modelo.js` no aparece `document`, y en `vista.js` no aparece `lista`. Cada módulo exporta solo lo que los demás necesitan [215] y esconde el resto.



Por qué merece la pena

- **Se puede cambiar una parte sin tocar las otras.** Si mañana la lista se pinta como tabla, solo cambia `vista.js`. Si la cantidad máxima pasa a ser 99, solo cambia `modelo.js`.

- **Los fallos se buscan en un sitio.** ¿Se añade un artículo vacío? Es el modelo. ¿Se ve mal? Es la vista.
- **El modelo se puede reutilizar.** `ListaCompra` funcionaría igual en otra página, o con otra vista, porque no depende del HTML.



El modelo

```
export class ListaCompra {
    #articulos = [];
    #siguienteId = 1;
    agregar(nombre, cantidad) { … throw new Error(…) … }
    get articulos() {
        return this.#articulos.map((a) => ({ ...a }));
    }
    …
}
```

Es todo lo del ejercicio 05 aplicado a un caso real: los datos son **privados** [226], las reglas lanzan un `Error` cuando algo no se puede hacer [230], y el getter devuelve **copias** para que nadie modifique los artículos por la puerta de atrás.

Cada artículo tiene un `id` propio que nunca se repite, porque `#siguienteId` solo crece. Así, eliminar el artículo 2 no confunde al 3, cosa que sí pasaría si usarais su posición en el array.



El controlador: el ciclo de siempre

```
lista.agregar(nombre, cantidad);    // 1. el usuario actúa → se cambia el modelo
actualizar();                        // 2. se repinta la vista con los datos nuevos
```

Es el patrón **estado → renderizar** del bloque 07. La diferencia es que ahora el estado tiene su propio fichero y sus propias reglas, y la vista también.

Ejercicio 07 · Depuración: cuatro fallos escondidos

> 🎯 **Objetivo:** Encontrar fallos que no producen errores usando las herramientas de desarrollo: puntos de interrupción, ejecución paso a paso, inspección de variables y la instrucción debugger.
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Encontrar y corregir cuatro fallos que no dan ningún error en la consola, usando las herramientas de depuración del navegador.



Por qué `console.log` no basta

Hasta ahora, para saber qué valía una variable, escribíais un `console.log`. Funciona, pero hay que adivinar dónde ponerlo, recargar, borrarlo después... Las herramientas de desarrollo permiten **detener el programa** en una línea y mirar todo lo que hay en ese momento [231]. Es como pausar un vídeo para ver qué pasa en cada fotograma.



Las herramientas

Herramienta	Qué hace
Punto de interrupción (*breakpoint*)	Pulsando en el número de una línea, la ejecución se detiene ahí [231]
Pasar a la siguiente (F10)	Ejecuta la línea actual y se detiene en la siguiente
Entrar (F11)	Si la línea llama a una función, entra en ella
Ámbito (*Scope*)	Muestra todas las variables con su valor actual [231]
Inspecciones (*Watch*)	Expresiones que queréis vigilar, como `suma` o `notas[i]`
`debugger;`	Escrito en el código, detiene la ejecución si las herramientas están abiertas [232]



Los cuatro fallos

Con `7, 4.5, 9, 5, 3`, el programa original da `Media: NaN · Aprobados: 2 de 5 · Máxima: NaN`. Esto es lo que se ve con un punto de interrupción en cada función:

Fallo 1 · `leerNotas`: textos en lugar de números

En el panel **Ámbito**, `notas` vale `['7', ' 4.5', ' 9', ' 5', ' 3']`: **textos**, con comillas y espacios. `split` siempre devuelve textos (bloque 02).

```
return texto.split(',').map((parte) => Number(parte));
```

`Number(' 4.5')` ignora los espacios y devuelve `4.5`.

Fallo 2 · `calcularMedia`: una vuelta de más

Vigilando `i` en el bucle, se ve que llega a `5`. Pero `notas` tiene 5 elementos, del `0` al `4`: `notas[5]` es `undefined`, y sumar `undefined` da `NaN`. Es el clásico error de «uno más» del bloque 03.

```
for (let i = 0; i < notas.length; i++) {
```

Con el fallo 1 sin corregir, además, `suma` se convertía en un texto (`'07 4.5 9 5 3undefined'`) porque `+` con textos concatena. Un fallo tapaba a otro: por eso conviene corregirlos de uno en uno y volver a probar.

Fallo 3 · `contarAprobados`: el 5 no contaba

```
return notas.filter((nota) => nota >= 5).length;
```

Este no rompe nada: simplemente da `2` en lugar de `3`. Son los fallos más traicioneros, porque el resultado **parece** razonable. Solo se ven si sabéis qué debería salir. Por eso el enunciado os daba el resultado esperado.

Fallo 4 · `notaMaxima`: un array no es una lista de números

`Math.max(notas)` recibe **un** argumento, el array entero, y no sabe compararlo. Hay que separar sus elementos con el operador de propagación del bloque 05:

```
return Math.max(...notas);
```



El método

1. **Saber qué debería salir.** Sin un resultado esperado no hay forma de saber si algo falla.
2. **Detener el programa antes del fallo** y avanzar paso a paso.
3. **Mirar los valores, no suponerlos.** Los cuatro fallos se ven a simple vista en el panel Ámbito.
4. **Corregir uno, probar, y seguir.**

Y no os olvidéis de quitar los `debugger` al terminar: si alguien abre las herramientas de desarrollo con vuestra página, se le detendrá.

Ejercicio 08 · Proyecto: mi biblioteca con clases y módulos

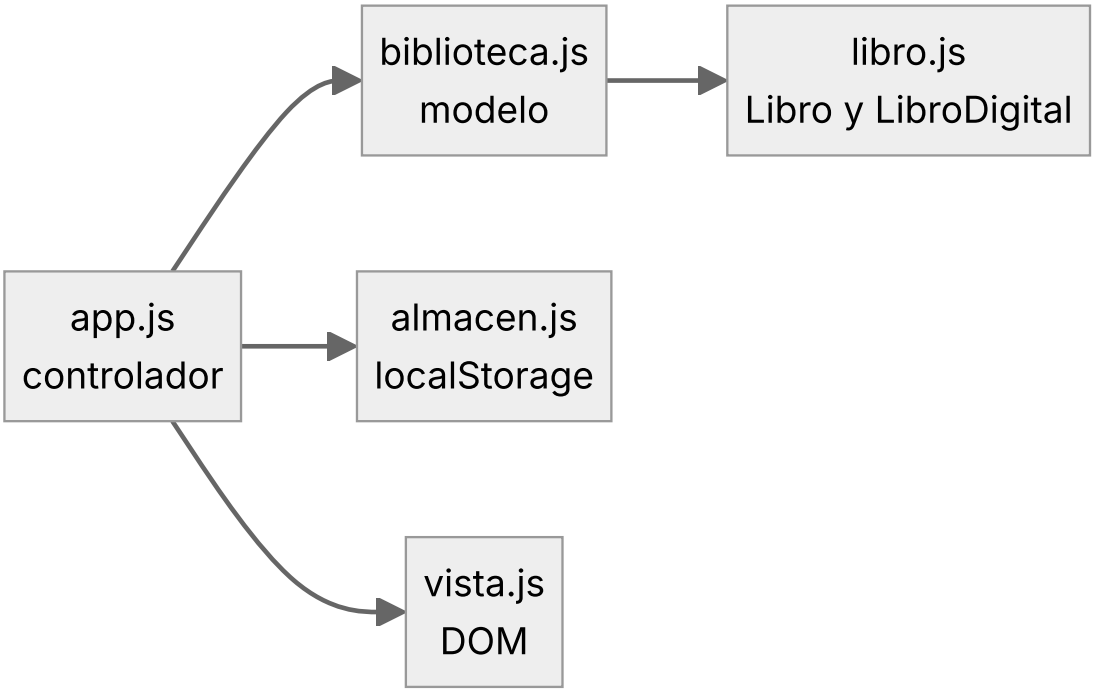
> 🎯 **Objetivo:** Integrar todo el bloque en una aplicación: clases con herencia y validación, un modelo con campos privados y getters, persistencia con toJSON y localStorage, y una arquitectura en módulos con modelo, vista y controlador.
>
> 📂 Enunciado · Solución completa



Qué pedía el ejercicio

Una aplicación de biblioteca personal con todo lo del bloque: clases con herencia, datos privados, módulos con responsabilidades separadas y persistencia en el navegador.

El mapa de la aplicación



Módulo	Exporta	Responsabilidad
`libro.js`	`Libro`, `LibroDigital`, `crearLibro`	Cómo es un libro y qué reglas cumple
`biblioteca.js`	`Biblioteca`	La colección: añadir, quitar, filtrar, estadísticas
`almacen.js`	`cargarDatos`, `guardarDatos`	Guardar y recuperar del navegador
`vista.js`	`pintarLibros`, `pintarEstadisticas`, `mostrarMensaje`	Pintar en el DOM
`app.js`	(nada: es el principal)	Escuchar al usuario y coordinar a los demás

Solo `app.js` conoce a todos. `vista.js` no sabe que existe `localStorage`, y `biblioteca.js` no sabe que existe una página. Cada uno exporta lo justo [215].

La herencia, con `super` en los métodos

```
describir() {
    return `${super.describir()} (${this.formato})`;
}
```

En el ejercicio 04 usabais `super(…)` para llamar al constructor de la madre. Aquí veis su otro uso: `super.describir()` llama al **método** de la madre [225]. `LibroDigital` no repite cómo se describe un libro; reutiliza lo de `Libro` y le añade el formato. Lo mismo hace `toJSON()` con `...super.toJSON()`.

El constructor también reutiliza la validación: `super(datos)` comprueba el título, el autor y las páginas, y `LibroDigital` solo añade la comprobación del formato.



`toJSON`: decidir qué se guarda

```
toJSON() {
    return { tipo: 'papel', id: this.id, titulo: this.titulo, … };
}
```

Cuando `JSON.stringify` encuentra un objeto con un método `toJSON`, guarda lo que ese método devuelva en lugar del objeto [105]. Gracias a eso:

- `JSON.stringify(biblioteca)` funciona aunque `#libros` sea privado: el `toJSON` de `Biblioteca` devuelve el array.
- Cada libro se guarda con un campo `tipo`, que `crearLibro` usa al cargar para reconstruir un `Libro` o un `LibroDigital`.

Esto resuelve un problema que dejamos pendiente en el bloque 09: en `localStorage` solo se guarda texto [21], y al leerlo con `JSON.parse` se obtienen objetos **sin métodos**. Por eso el constructor de `Biblioteca` vuelve a pasar cada dato por `crearLibro`: convierte los datos simples otra vez en instancias de verdad, con su `describir()` y su `alternarLeido()`.



Cargar sin romperse

```
const guardados = cargarDatos();
const biblioteca = new Biblioteca(guardados === null ? EJEMPLOS : guardados);
```

Situación	`cargarDatos()`	Resultado
Primera visita	`null`	Se cargan los tres libros de ejemplo
Habéis borrado todos los libros	`[]`	Biblioteca vacía (no vuelven los ejemplos)
Datos corruptos	`[]`	Biblioteca vacía, sin errores
Un libro guardado no es válido	El array	Se descarta ese libro y se avisa en la consola

Es la carga segura del bloque 09, con una capa más: además de que el JSON sea válido, cada libro tiene que superar la validación de su clase.



`instanceof` para filtrar

```
if (filtro === 'digitales') return l instanceof LibroDigital;
```

No hace falta una propiedad `esDigital`: la clase del objeto ya lo dice [233].



Lo que habéis construido

Este ejercicio es una pequeña aplicación con una arquitectura como la de los proyectos profesionales: si mañana los libros vinieran de una API (bloque 11), solo cambiaría `almacen.js`. Es exactamente la base que necesitáis para el proyecto final.



Bloque 13

Bloque 13 · Proyecto final

Llegamos al final. En doce bloques habéis aprendido a escribir JavaScript, a manejar el DOM, a escuchar eventos, a validar formularios, a guardar datos en el navegador, a pedirlos a un servidor y a ordenar todo eso en módulos y clases. Ahora toca demostrarlo construyendo **una aplicación completa**, de las que se pueden enseñar. En este bloque os doy el enunciado, los criterios con los que se evalúa y una aplicación de referencia, un tablero de tareas, construida en ocho fases y explicada paso a paso. Primero la construimos juntos; después os toca hacer la vuestra.

En este bloque encontraréis:

- El **enunciado** del proyecto y sus requisitos obligatorios.
- Los **criterios de evaluación**, con su puntuación.
- La **aplicación de referencia** en 8 fases, cada una con su ejercicio y su solución explicada.
- Una **lista de comprobación** para repasar antes de entregar.



⚠️ Importante

El proyecto usa módulos y `fetch`: **abrid siempre las páginas con Live Server**.



1. El enunciado

Construid una aplicación web de una sola página, con el tema que queráis, que cumpla **los seis requisitos obligatorios**:

Nº	Requisito obligatorio	Bloques
1	Una interfaz generada y actualizada con el DOM	06
2	Eventos de ratón y de teclado	07
3	Al menos un formulario con validación	08
4	Datos guardados en `localStorage`	09
5	Datos obtenidos de una API con `fetch`	10 y 11
6	Código organizado en módulos	12

Algunas ideas, por si os cuesta arrancar: un gestor de lecturas, un diario de entrenamientos, una agenda de contactos, un cuaderno de recetas, un registro de gastos, una lista de películas pendientes… Lo importante no es el tema, sino que la aplicación resuelva algo **de principio a fin**.

Sobre la API

Necesitáis una API que se pueda usar desde el navegador, es decir, que envíe la cabecera CORS (bloque 11) [209]. Si no encontráis una que encaje con vuestro tema, usad JSONPlaceholder, como la aplicación de referencia: tiene usuarios, publicaciones, comentarios, álbumes, fotos y tareas [214]. Lo que valoro es **cómo** la usáis: traducir sus datos a vuestro formato, mostrar los estados de carga y error, y que la aplicación siga funcionando si la API no responde.



2. Los criterios de evaluación



Figura 82 · Criterios de evaluación

Criterio	Puntos	Qué miro
Requisitos funcionales	3	Los seis obligatorios están y funcionan
Organización del código	2	Módulos con una responsabilidad clara; clases para el modelo; el modelo no toca el DOM y la vista no modifica datos
Robustez	2	Validación en el formulario y en el modelo; mensajes de error comprensibles; estados de carga, vacío y error; carga segura de `localStorage`
Accesibilidad y usabilidad	1,5	Se puede usar solo con el teclado; etiquetas en todos los controles; foco en el primer error; avisos con `aria-live`
Calidad del código	1	Nombres que se entienden; la plantilla de comentarios del curso; ningún error en la consola
Documentación	0,5	Un `README.md` del proyecto (ver apartado 6)



🚨 Atención

Los seis requisitos obligatorios son imprescindibles. Si falta alguno, el proyecto no es apto, aunque el resto esté muy bien.



3. La aplicación de referencia: un tablero de tareas

Para que veáis cómo se hace un proyecto así, he construido uno completo: un tablero donde se crean, completan, editan, filtran y buscan tareas, que se guardan en el navegador y que también se pueden importar de JSONPlaceholder.

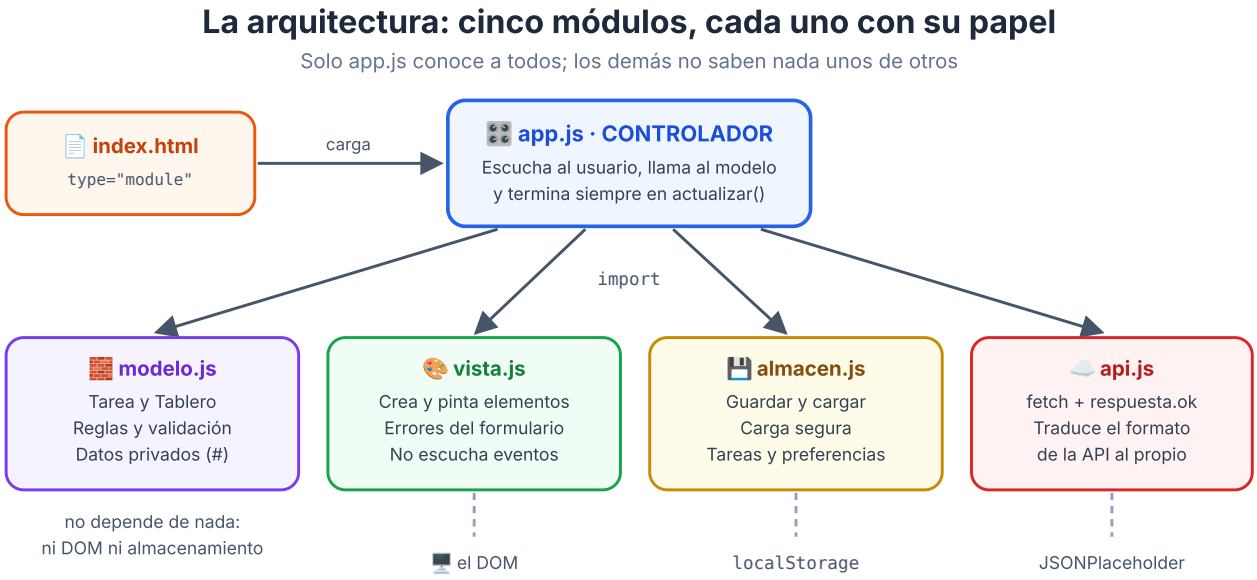


Figura 83 · La arquitectura: cinco módulos

Módulo	Papel	Bloque que lo inspira
`modelo.js`	Las clases `Tarea` y `Tablero`, con datos privados y validación [219] [226]	12
`vista.js`	Funciones que crean y pintan elementos	06
`almacen.js`	Guardar y cargar de `localStorage` [21]	09
`api.js`	Pedir datos con `fetch` [202] y traducirlos	11
`app.js`	El controlador: importa todo lo demás [215], escucha y coordina	07 y 12

3.1. El ciclo de cada acción

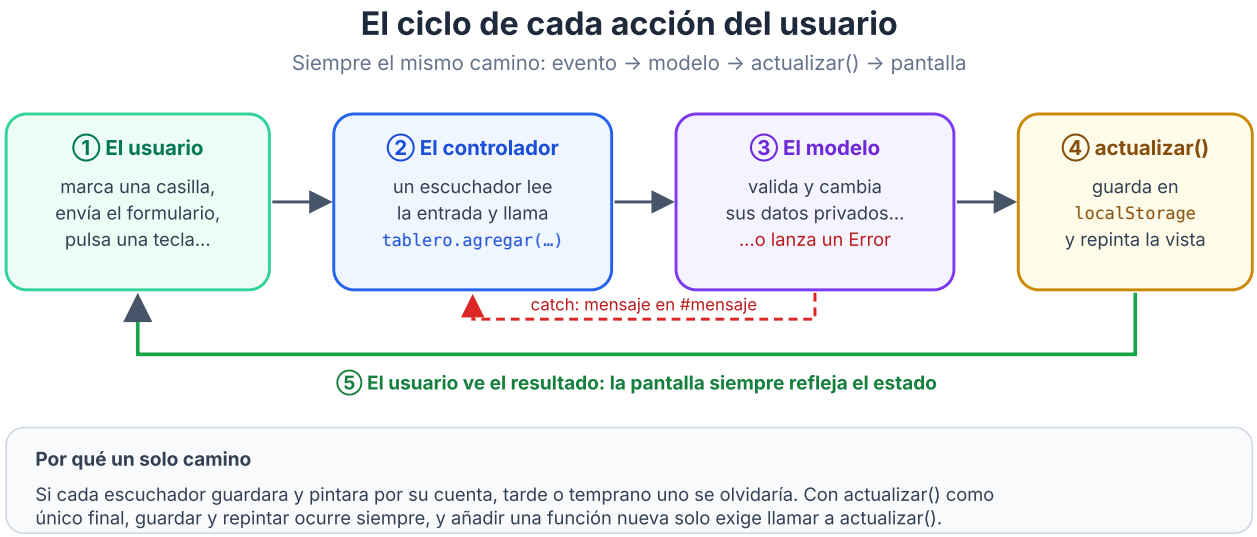


Figura 84 · El ciclo de cada acción del usuario

Todo lo que hace el usuario sigue el mismo camino: un escuchador llama al modelo, el modelo valida y cambia sus datos (o lanza un error), y `actualizar()` guarda y repinta. Es el patrón **estado → renderizar** del bloque 07, con cada pieza en su módulo. Si entendéis este dibujo, entendéis la aplicación.



4. Las ocho fases

Una aplicación así no se escribe de un tirón. Se construye **por fases**, y al final de cada una funciona algo nuevo.



Figura 85 · Las ocho fases del proyecto

Cada ejercicio de este bloque es **una fase**. Su carpeta contiene la aplicación tal como queda al terminar la fase anterior, más los comentarios `// TAREA:` de lo que tenéis que añadir. Su solución es el punto de partida de la siguiente. Así podéis empezar en cualquier fase, aunque no hayáis terminado la anterior.

Fase	Ejercicio	Qué añadís	Ficheros que tocáis	Solución
1	El modelo: Tarea y Tablero	Clases con validación y 9 pruebas automáticas	`modelo.js`	Ver
2	La vista: pintar el tablero	Crear los elementos de cada tarea	`vista.js`	Ver
3	Eventos de ratón	Completar, eliminar, editar con doble clic y filtrar	`app.js`	Ver
4	El formulario con validación	Tareas nuevas con errores accesibles [19]	`app.js`	Ver
5	Persistencia con localStorage	Tareas y filtro que sobreviven a la recarga	`almacen.js`, `app.js`	Ver
6	Importar tareas de una API	Usuarios y tareas de JSONPlaceholder, sin duplicados	`api.js`, `app.js`	Ver
7	Teclado y búsqueda	Atajos globales [133] y buscador con espera	`app.js`	Ver
8	Pulido final	Orden, progreso, borrar completadas y tema oscuro	`app.js`	Ver

Fijaos en el orden: primero el **modelo**, que no depende de nada y se puede probar solo; después la **vista**; y solo entonces la interacción. Las funciones más vistosas (API, atajos, tema) llegan al final, cuando lo básico ya es sólido. Os recomiendo seguir el mismo orden en vuestro proyecto.



📝 Nota

Las fases 5 a 8 guardan en las mismas claves de `localStorage` (`curso-js:b13:…`). Las tareas que creéis en una fase seguirán ahí en la siguiente. Para empezar de cero, borradlas en las herramientas de desarrollo: **Aplicación** → **Local Storage**.



5. Cómo organizar vuestro propio proyecto

1. **Elegid el tema y escribid en un papel qué datos tiene** (como `Tarea`: título, prioridad, fecha…) y qué reglas cumplen. Eso será vuestro modelo.
2. **Dibujad la pantalla** antes de escribir HTML: qué se ve, qué se puede pulsar, dónde salen los mensajes.
3. **Seguid las fases** de la aplicación de referencia, adaptadas a vuestro tema. Al acabar cada una, haced una copia de la carpeta: si algo se rompe, podréis volver atrás.
4. **Probad con datos raros**: un campo vacío, un texto larguísimo, sin conexión, `localStorage` estropeado a mano desde las herramientas de desarrollo. Así lo voy a probar yo.
5. **Usad el teclado** para recorrer toda la aplicación al menos una vez, sin ratón.



6. Qué hay que entregar

Una carpeta con este contenido:

```
mi-proyecto/
├── README.md       ← qué es, cómo se usa, qué API usa y qué ha costado más
├── index.html
├── style.css
├── app.js          ← el controlador
└── …               ← vuestros módulos: modelo, vista, almacén, api…
```

El `README.md` del proyecto no tiene que ser largo, pero sí responder a cuatro preguntas: qué hace la aplicación, cómo se usa (incluidos los atajos de teclado, si los hay), de qué API saca los datos y qué ha sido lo más difícil y cómo lo habéis resuelto.



7. Lista de comprobación antes de entregar

Requisitos

- [ ] La interfaz se genera y se actualiza con el DOM, no está escrita a mano en el HTML.
- [ ] Hay eventos de ratón **y** de teclado.
- [ ] Hay al menos un formulario, y cada error se explica junto a su campo.
- [ ] Los datos se guardan en `localStorage` y sobreviven a una recarga.
- [ ] Hay datos que llegan de una API con `fetch`.
- [ ] El código está repartido en módulos con `import` y `export`.

Robustez

- [ ] Se comprueba `respuesta.ok` y hay mensaje de error y botón de reintentar.
- [ ] Mientras se carga algo, el usuario lo sabe.
- [ ] Con `localStorage` vacío o estropeado, la aplicación arranca sin errores.
- [ ] El modelo valida sus datos aunque el formulario ya lo haga.

Accesibilidad y calidad

- [ ] Todo se puede hacer con el teclado, y los atajos no estorban al escribir.
- [ ] Todos los controles tienen etiqueta; los mensajes de estado usan `aria-live` [162].
- [ ] La consola no muestra ningún error.
- [ ] El código sigue la plantilla de comentarios del curso y los nombres se entienden.
- [ ] Hay un `README.md` del proyecto.

Y hasta aquí el curso. Espero que lo hayáis disfrutado tanto como yo preparándolo. Lo que habéis aprendido es la base sobre la que se construye todo lo demás: los frameworks, las aplicaciones de servidor, las de móvil... Cuando os encontréis con ellos, veréis que por debajo siguen siendo objetos, funciones, eventos y promesas. Ya los conocéis.



🧪 Las fases del proyecto, explicadas

Fase 1 · El modelo: Tarea y Tablero

> 🎯 **Objetivo:** Escribir el modelo de la aplicación en modelo.js: la clase Tarea, que valida sus datos, y la clase Tablero, que guarda la colección con campos privados. Una tabla de pruebas automáticas os dirá si funciona antes de pintar nada.
>
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Escribir el corazón de la aplicación antes de pintar nada: las clases `Tarea` y `Tablero`, en `modelo.js`, comprobadas con una tabla de pruebas automáticas.



Por qué empezar por el modelo

El modelo es lo único que no depende de la página. Si las reglas (qué es una tarea válida, cuándo está vencida, qué id le toca) funcionan bien, todo lo que construyáis encima será más fácil de escribir y de arreglar. Por eso esta fase no tiene botones: tiene **pruebas**.



Las pruebas automáticas

```
['Un título de 2 letras lanza un Error', () => lanzaError(() => new Tarea({ id: 9, titulo:
'ab' }))],
```

Cada prueba es una descripción y una función que devuelve `true` si el modelo se comporta como debe. `ejecutarPruebas()` las ejecuta todas dentro de un `try`, así que una prueba que rompe no impide ver las demás. Es una versión casera de lo que hacen las herramientas profesionales de pruebas, y tiene la misma ventaja: cuando cambiéis el modelo en fases posteriores, podéis volver aquí y comprobar que no habéis roto nada.



`Tarea`: un constructor que no se fía

```
constructor({ id, titulo, prioridad = 'media', fecha = '', completada = false, origen =
'propia', idApi = null }) {
```

El constructor recibe **un objeto** y lo desestructura con valores por defecto (bloques 04 y 05). Así, `new Tarea({ id: 1, titulo: 'Estudiar' })` funciona sin indicar todo lo demás, y el orden de los datos da igual.

Después valida cada dato y lanza un `Error` si algo no cuadra [230]. Una tarea **no puede existir** en un estado inválido: si el constructor termina, la tarea es correcta. Para la fecha uso `Date.parse`, que devuelve `NaN` cuando el texto no es una fecha válida [234].



`vencida`: un getter calculado

```
get vencida() {
    return !this.completada && this.fecha !== '' && this.fecha < fechaDeHoy();
}
```

`vencida` no se guarda: se **calcula** cada vez que se lee [227], así que nunca se queda desfasada. Y la comparación funciona con `<` porque las fechas en formato `AAAA-MM-DD` ordenadas como texto quedan también ordenadas como fechas: `'2025-09-15' < '2026-01-01'`.

`fechaDeHoy()` construye esa fecha con la hora **local**. Con `toISOString()` saldría en hora UTC, y entre la medianoche y la una o las dos de la madrugada (según la época del año) os daría el día anterior.



`Tablero`: la colección, bajo llave

```
#tareas = [];
#siguienteId = 1;
```

Las tareas son privadas [226]. Desde fuera solo se puede usar lo que el tablero ofrece: `agregar`, `eliminar`, `alternar` y dos getters. `get tareas` devuelve una **copia** del array: quien la reciba puede recorrerla, pero no añadir ni quitar tareas sin pasar por las reglas.

El constructor recibe datos y crea una `Tarea` por cada uno **dentro de un `try`**: si alguno no es válido, se descarta y se avisa en la consola. Lo vais a agradecer en la fase 5, cuando los datos lleguen de `localStorage` y podrían estar estropeados.

Fase 2 · La vista: pintar el tablero

> 🎯 **Objetivo:** Escribir vista.js, el módulo que convierte las tareas en elementos del DOM. La vista solo recibe datos y pinta: no modifica el tablero ni escucha eventos.
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Escribir `vista.js`: las funciones que convierten las tareas en elementos de la página.



Las reglas de la vista

La vista sí...	La vista no...
Recibe datos y crea elementos	Importa el modelo ni modifica tareas
Exporta funciones con un nombre claro	Escucha eventos
Decide cómo se ve cada cosa	Decide qué tareas se muestran

Esto es lo que visteis en el bloque 12 con la lista de la compra. `app.js` (el controlador) decide **qué** pintar llamando a `tareasVisibles()`; la vista decide **cómo**.



`crearTarea`: un elemento por tarea

```
li.classList.toggle('hecha', tarea.completada);
li.classList.toggle('vencida', tarea.vencida);
li.dataset.id = tarea.id;
```

Es el patrón crear + renderizar del bloque 06. Tres detalles:

- **`classList.toggle(clase, condición)`** añade o quita la clase según el valor. Una línea en lugar de un `if`.
- **`dataset.id`** guarda el id de la tarea en el propio elemento [181]. En la fase 3, al pulsar una casilla, sabremos de qué tarea se trata.
- **`data-accion`** en la casilla y en el botón dice qué hace cada control, para la delegación de la fase 3.

Todo el texto entra con `textContent`, nunca con `innerHTML`: si un título trae etiquetas HTML (y en la fase 6 vendrán títulos de Internet), se muestran como texto y no se ejecutan.



Un aviso para el estado vacío

```
if (tareas.length === 0) {
    li.textContent = '📭 No hay tareas que mostrar.';
```

Es uno de los cuatro estados del bloque 11: una lista vacía sin explicación parece un error.



`pintarTareas` empieza vaciando

`contenedor.replaceChildren()` sin argumentos elimina todos los hijos [116]. Así, `actualizar()` puede llamar a `pintarTareas` tantas veces como quiera y el resultado siempre refleja el estado actual, sin duplicados.



Las etiquetas de los botones

Cada casilla y cada botón llevan un `aria-label` con el título de su tarea: `Eliminar «Preparar la presentación final»`. Un lector de pantalla que recorre los botones oye así **qué** se va a eliminar, en lugar de un «Eliminar, Eliminar, Eliminar» que no dice nada.

Fase 3 · Eventos de ratón: completar, eliminar, editar y filtrar

> 🎯 **Objetivo:** Dar vida al tablero con eventos de ratón y delegación: marcar tareas con la casilla, eliminarlas, editarlas con doble clic y filtrarlas. El modelo ya trae renombrar y filtrar, y la vista, activarEdicion.
>
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Conectar los controles del tablero con el modelo usando delegación de eventos: casillas, botones de eliminar, edición con doble clic y filtros.



Tres escuchadores para todas las tareas

Evento	En	Qué hace
`change`	`#tareas`	Una casilla cambia → `tablero.alternar(id)` [135]
`click`	`#tareas`	Botón Eliminar → `tablero.eliminar(id)`
`dblclick`	`#tareas`	Doble clic en un título → `editarTarea(li)` [130]

Las tareas se recrean en cada `actualizar()`, así que no tiene sentido poner escuchadores en cada una. Pongo uno en la lista y, con `closest`, averiguo qué control se ha pulsado y a qué `li` pertenece [3]:

```
const casilla = e.target.closest('[data-accion="alternar"]');
tablero.alternar(Number(casilla.closest('li').dataset.id));
```

Es la delegación del bloque 07. Para la casilla uso `change`, no `click`: `change` es el evento que indica que el valor ha cambiado, se cambie con el ratón o con la barra espaciadora.



Editar con doble clic

```
const campo = activarEdicion(li);
campo.addEventListener('keydown', (e) => {
    if (e.key === 'Enter') { terminar(true); } else if (e.key === 'Escape') {
terminar(false); }
});
campo.addEventListener('blur', () => terminar(true));
```

`activarEdicion`, en la vista, cambia el título por un campo con `replaceWith` [235] y le da el foco. Desde ahí, tres salidas: **Enter** guarda, **Escape** cancela y **salir del campo** (`blur`) también guarda [137], como en la mayoría de aplicaciones.

El guardado doble

Pulsar Enter guarda y llama a `actualizar()`, que borra el campo de la página. Según el navegador, eso puede provocar también un `blur`, que intentaría guardar otra vez. La variable `terminada` lo evita:

```
if (terminada) {
    return;
}
terminada = true;
```

Es un problema clásico cuando dos eventos pueden llegar por el mismo motivo. La solución también es clásica: una bandera que dice «esto ya está hecho».

Si el título no es válido

`tablero.renombrar` usa la misma validación que al crear la tarea. Si borráis el título y pulsáis Enter, el modelo lanza un `Error`, el `catch` lo muestra en `#mensaje` y la tarea conserva su título anterior. La vista no tiene que saber cuántos caracteres son válidos.



Los filtros

```
filtro = boton.dataset.filtro;
marcarFiltro();
actualizar();
```

El estado cambia, se marca el botón activo y se repinta. `marcarFiltro` actualiza la clase `activo` y el atributo `aria-pressed`, que indica a los lectores de pantalla qué botón está «pulsado» [236]. El filtrado en sí lo hace el modelo con `tablero.filtrar(filtro)`: la regla de qué es una tarea vencida está en un solo sitio.

Fase 4 · El formulario con validación

> 🎯 **Objetivo:** Añadir tareas nuevas con un formulario accesible: validación propia con mensajes junto a cada campo, aria-invalid, foco en el primer error, y una segunda barrera en el modelo.
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Un formulario para añadir tareas, con validación propia, mensajes junto a cada campo y accesible con el teclado.



`novalidate`: los mensajes los ponemos nosotros

```
<form id="formTarea" novalidate>
```

Con `novalidate`, el navegador no muestra sus burbujas de error al enviar [19] y la validación queda en nuestras manos. Es lo que hicisteis en el bloque 08: así los mensajes salen en español, junto al campo y con el estilo de la aplicación.



Tres campos, tres comprobaciones

Campo	Regla	Cómo se comprueba
Título	Obligatorio, de 3 a 60 caracteres	`value.trim().length`
Prioridad	Obligatoria	`formulario.prioridad.value` es `''` si no hay ninguna marcada
Fecha	Opcional; completa y no anterior a hoy	`validity.badInput` [237] y comparación de textos

El caso de la fecha merece una explicación. Si alguien escribe solo el día en el selector, el valor del campo es `''`, igual que si no hubiera escrito nada. `validity.badInput` es lo que distingue los dos casos: es `true` cuando el usuario ha escrito algo que el navegador no puede convertir en un valor [237]. Y `formulario.fecha.min = fechaDeHoy()` hace que el selector ni siquiera ofrezca días pasados [238]; aun así, lo compruebo, porque la fecha también se puede escribir a mano.



Mostrar los errores de forma accesible

```
mostrarError(campoTitulo, 'error-titulo', errorTitulo);
```

`mostrarError`, en la vista, hace dos cosas:

1. Escribe el mensaje en el párrafo de error del campo. Ese párrafo está enlazado con `aria-describedby`, así que un lector de pantalla lo lee al llegar al campo.
2. Pone `aria-invalid="true"` en el campo (o `false` si se ha corregido) [159].

Además, el foco va al **primer** campo con error. Quien usa el teclado no tiene que buscar qué ha fallado.

Y mientras el usuario corrige, el evento `input` borra el error de ese campo. No hay nada más frustrante que un mensaje de error que sigue ahí después de haber arreglado el problema.



Dos barreras: formulario y modelo

```
if (!validarFormulario()) { … return; }
try {
    tablero.agregar(datos);
} catch (error) {
    mostrarMensaje(mensaje, `✘ ${error.message}`);
}
```

¿Por qué validar dos veces, si el modelo ya comprueba el título? Porque son dos trabajos distintos:

- **El formulario** valida para **ayudar al usuario**: mensajes por campo, foco, explicaciones.
- **El modelo** valida para **proteger los datos**: da igual de dónde vengan (el formulario, la edición con doble clic, `localStorage` o la API), una tarea inválida no entra.

El `try` es la red de seguridad: si algún día el formulario dejara pasar algo, el modelo lo pararía.

Fase 5 · Persistencia con localStorage

> 🎯 **Objetivo:** Conseguir que las tareas y el filtro elegido sobrevivan a una recarga: un módulo almacen.js con carga segura, y toJSON en el modelo para decidir qué se guarda.
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Que las tareas y el filtro elegido se conserven al recargar la página.



Un módulo solo para guardar

`almacen.js` es el único fichero que conoce `localStorage`. Ni el modelo ni la vista saben dónde se guardan los datos. Si mañana se guardaran en un servidor, solo cambiaría este módulo.



`toJSON`: el modelo decide qué se guarda

```
// en Tablero
toJSON() {
    return this.#tareas;
}
```

`JSON.stringify` no puede ver los campos privados, pero si un objeto tiene un método `toJSON`, guarda lo que ese método devuelva [105]. `Tablero` devuelve su array, y cada `Tarea` del array tiene a su vez su `toJSON` con los siete datos que interesan. Por eso esto funciona:

```
localStorage.setItem(CLAVE_TAREAS, JSON.stringify(tablero));
```

Al cargar, `JSON.parse` devuelve objetos **sin métodos**. `new Tablero(guardadas)` los vuelve a pasar por el constructor de `Tarea`, que los valida y los convierte otra vez en tareas de verdad.



Cargar sin romperse

Situación	`cargarTareas()`	El tablero empieza con…
Primera visita	`null`	Las tres tareas de ejemplo
Habéis borrado todas las tareas	`[]`	Nada (no vuelven los ejemplos)
El texto guardado no es JSON	`[]`	Nada, sin errores
Una tarea guardada no es válida	El array	Las válidas; la otra se descarta

Distinguir `null` de `[]` es lo que permite que los ejemplos aparezcan **solo** la primera vez. Recordad que `localStorage` solo guarda texto [21] y que cualquiera puede modificarlo desde las herramientas de desarrollo: por eso todo lo que se lee de ahí se trata como sospechoso.



Las preferencias

```
return { ...PREFERENCIAS_INICIALES, ...guardadas };
```

Mezclar las preferencias iniciales con las guardadas tiene una ventaja: si en la fase 8 añadimos una preferencia nueva (`orden`, `tema`), quien ya tenía preferencias guardadas recibe el valor inicial de las nuevas sin perder las suyas. Y si `guardadas` es `null` (primera visita), el operador de propagación simplemente no añade nada.

Aun así, en `app.js` compruebo que el filtro guardado sea uno de los válidos (`FILTROS.includes(…)`). Un valor raro en `localStorage` no debe dejar la aplicación sin tareas a la vista.



Guardar en un solo sitio

```
function actualizar() {
    guardarTareas(tablero);
    guardarPreferencias({ filtro });
    …
}
```

Cada cambio del usuario termina en `actualizar()`, así que guardar ahí asegura que nunca se olvida ninguno. Es el patrón de persistencia del bloque 09.



📝 **Nota**

Las fases 5 a 8 comparten la misma clave (`curso-js:b13:tareas`), en el ejercicio y en la solución: las tareas que creéis en una fase seguirán ahí en la siguiente. Si queréis empedar de cero, borrad la clave en Aplicación → Local Storage.

Fase 6 · Importar tareas de una API

🎯 **Objetivo:** Conectar la aplicación con JSONPlaceholder: cargar los usuarios al arrancar, importar las tareas del elegido traduciéndolas al formato propio, evitar duplicados y gestionar los estados de carga y error con Reintentar.

📂 Enunciado · Solución completa



Qué pedía esta fase

Traer tareas de JSONPlaceholder: elegir un usuario e importar las suyas, sin duplicados y con los estados de carga y error del bloque 11.



`api.js` traduce

```
return tareas.map((t) => ({
    titulo: recortar(t.title),
    completada: t.completed,
    prioridad: 'media',
    fecha: '',
    origen: 'api',
    idApi: t.id,
}));
```

La API tiene su propio formato (`title`, `completed`, `userId`…) [214]. `api.js` lo traduce al de la aplicación, de forma que el resto del código no sabe nada de JSONPlaceholder. Si mañana cambiarais de API, solo cambiaría este módulo.

Dos decisiones de la traducción:

- **`recortar`**: algunos títulos de la API pasan de 60 caracteres, el máximo que acepta `Tarea`. Sin recortarlos, el modelo los rechazaría.
- **`idApi`**: el id que tenía la tarea en la API. No es el id del tablero (ese lo pone `Tablero`), pero sirve para reconocerla si se vuelve a importar.

La URL `/users/{id}/todos` es una de las rutas anidadas que documenta JSONPlaceholder: las tareas de un usuario concreto [214].



Sin duplicados

```
const repetida = this.#tareas.some((t) => t.idApi !== null && t.idApi === datos.idApi);
```

Si importáis dos veces el mismo usuario, la segunda vez `importar` no añade nada y el mensaje dice `0 tareas importadas (20 ya estaban en el tablero)`. Las tareas propias tienen `idApi: null`, así que nunca se confunden con las de la API.



Los estados

Momento	`#estadoApi`	Controles
Al arrancar	⏳ Conectando con la API…	Select y botón desactivados
Usuarios cargados	Elige un usuario e importa sus tareas.	Activados
Importando	⏳ Importando tareas…	Botón desactivado (`finally` lo reactiva)
Error	✘ … y botón **Reintentar**	

`fetch` no rechaza la promesa con un `404` [202], y por eso `pedirJSON` comprueba `respuesta.ok`, como en el bloque 11. Así, cualquier fallo acaba en el `catch`.



Reintentar lo que falló

```
let ultimaAccion = null;
…
ultimaAccion = cargarUsuarios;     // o importarTareas
…
btnReintentarApi.addEventListener('click', () => ultimaAccion());
```

Puede fallar la carga de usuarios o la importación. En lugar de dos botones, guardo **la función** que hay que repetir en una variable. Las funciones son valores, como visteis con los callbacks del

bloque 04.



📝 Nota

He verificado el formato de `/users` y `/users/{id}/todos` en la guía y el repositorio oficiales de JSONPlaceholder y he probado el código con una copia de esos datos. No he podido comprobar el acceso en directo desde mi entorno. Si el día de clase la API no responde, veréis el mensaje de error y el botón Reintentar.

Fase 7 · Teclado y búsqueda

🎯 Objetivo: Hacer la aplicación cómoda con el teclado: un buscador con espera y atajos globales que no estorban cuando se está escribiendo.

📂 Enunciado · Solución completa



Qué pedía esta fase

Un buscador que filtra mientras se escribe y atajos de teclado para las acciones más comunes.



El buscador, con espera

```
clearTimeout(idEspera);
idEspera = setTimeout(() => {
    texto = buscar.value;
    actualizar();
}, 300);
```

Es la espera del bloque 10: cada tecla cancela la búsqueda anterior [185] y programa una nueva dentro de 300 ms [184]. Solo se repinta cuando el usuario hace una pausa. La búsqueda en sí la hace el modelo: `tablero.filtrar(filtro, texto)` aplica el filtro y el texto a la vez.



Atajos globales

```
document.addEventListener('keydown', (e) => { … });
```

Escucho `keydown` en `document` porque el evento sube desde el elemento que tiene el foco [3]: así llega desde cualquier parte de la página. `e.key` dice qué tecla se ha pulsado, ya traducida al carácter (`'n'` , `'/'` , `'?'`) o a su nombre (`'Escape'` , `'Enter'`) [141].

Tecla	Acción
`n`	Enfocar el título del formulario
`/`	Enfocar el buscador
`?`	Mostrar u ocultar la ayuda
`Escape`	Borrar la búsqueda (si estáis en el buscador) y cerrar la ayuda



Que los atajos no estorben

Un atajo mal hecho es peor que no tenerlo. Tres reglas:

1. **Con Ctrl, Cmd o Alt, no hago nada.** `Ctrl+F` o `Ctrl+R` son del navegador.
2. **Si se está escribiendo, una «n» es una «n».** Si el foco está en un `input`, `textarea` o `select`, salgo antes de mirar los atajos. `Escape` va antes de esta comprobación porque sí tiene sentido dentro del buscador.
3. **`preventDefault` al cambiar el foco.** Sin él, la «n» que enfoca el título se escribiría **dentro** del título, porque el foco cambia antes de que el navegador procese la tecla.



La ayuda

`?` abre un panel con la lista de atajos. Un atajo que nadie conoce no sirve de nada; por eso la página también dice, en un sitio visible, que existe esa ayuda.

Fase 8 · Pulido final: orden, progreso y tema

> 🎯 **Objetivo:** Terminar el proyecto con los detalles que lo hacen agradable: ordenar con un método static, barra de progreso, borrar las completadas con confirmación y un tema oscuro que se recuerda.
>
> 📂 Enunciado · Solución completa



Qué pedía esta fase

Los últimos detalles: ordenar, una barra de progreso, borrar las completadas con confirmación y un tema oscuro que se recuerda. Con esta fase, el proyecto está terminado.



Ordenar con un método `static`

```
static ordenar(tareas, criterio) {
    const copia = [...tareas];
    …
}

Tablero.ordenar(tablero.filtrar(filtro, texto), orden);
```

`ordenar` no necesita los datos de un tablero concreto: recibe una lista y devuelve otra. Por eso es `static`: pertenece a la clase y se llama con su nombre [229].

Empieza haciendo una **copia**, porque `sort` ordena el array original y lo modifica [97]. Para las fechas uso `localeCompare`, que compara textos y devuelve un número negativo, cero o positivo, justo lo que necesita `sort` [239]. Las tareas sin fecha se tratan como `'9999-12-31'` para que queden al final.



La barra de progreso

```
barra.style.width = `${porcentaje}%`;
```

`pintarProgreso` (en la vista) calcula el porcentaje de completadas y lo convierte en el ancho de la barra del bloque 06. Junto a ella, las tareas pendientes de cada prioridad, que el getter `estadisticas` ya calculaba desde la fase 1.



Borrar con confirmación

```
if (confirm(`¿Borrar ${completadas} tareas completadas? No se puede deshacer.`)) {
```

`confirm` muestra un cuadro con Aceptar y Cancelar y devuelve `true` o `false` [61]. Lo uso solo para la acción que **no se puede deshacer**. Pedir confirmación para todo acaba haciendo que nadie lea los avisos.



El tema oscuro

```
function aplicarTema(tema) {
    document.body.dataset.tema = oscuro ? 'oscuro' : 'claro';
    btnTema.setAttribute('aria-pressed', oscuro);
    …
}
```

El CSS ya tenía reglas para `body[data-tema="oscuro"]` desde el bloque 09. El botón funciona como un interruptor, así que lleva `aria-pressed` [236] y cambia su texto. La preferencia se guarda con las

demás en `actualizar()`.



El proyecto completo

Mirad el resultado con los requisitos del enunciado:

Requisito	Dónde está
Interfaz generada y actualizada con el DOM	`vista.js` y `actualizar()` (fase 2)
Eventos de ratón y de teclado	Fases 3 y 7
Formulario con validación	Fase 4
Datos en `localStorage`	`almacen.js` (fase 5)
Datos de una API con `fetch`	`api.js` (fase 6)
Código organizado en módulos	Cinco módulos, cada uno con su papel

Si habéis llegado hasta aquí, tenéis entre manos una aplicación con la estructura de un proyecto profesional. Ahora os toca a vosotros: el enunciado del bloque os propone cómo hacer **la vuestra**.



Fuentes y Documentación Oficial

Todas las explicaciones del manual se apoyan en la documentación oficial. Los números coinciden con las citas del texto.

1. Introducción — Guía de JavaScript (MDN Web Docs)
2. What are browser developer tools? (MDN Web Docs)
3. Módulos JavaScript (MDN Web Docs)
4. Expresiones y operadores — Operadores de comparación (MDN Web Docs)
5. HTMLInputElement: value property (MDN Web Docs)
6. `<script>` HTML script element — atributo `defer` (MDN Web Docs)
7. Gramática y tipos (MDN Web Docs)
8. Control de flujo y manejo de errores (MDN Web Docs)
9. Bucles e iteración (MDN Web Docs)
10. Funciones (MDN Web Docs)
11. Closures (MDN Web Docs)
12. Colecciones indexadas (MDN Web Docs)
13. Trabajando con objetos (MDN Web Docs)
14. Modelo de Objetos del Documento (DOM) (MDN Web Docs)
15. Element: innerHTML property (MDN Web Docs)
16. EventTarget.addEventListener() (MDN Web Docs)
17. Eventos del DOM (MDN Web Docs)
18. Client-side form validation (MDN Web Docs)
19. Using HTML form validation and the Constraint Validation API (MDN Web Docs)
20. API de almacenamiento web (MDN Web Docs)
21. Window: localStorage property (MDN Web Docs)
22. JavaScript execution model (MDN Web Docs)
23. Usar promesas (MDN Web Docs)
24. Uso de Fetch (MDN Web Docs)
25. Clases (MDN Web Docs)
26. What is JavaScript? — Inline JavaScript handlers (MDN Web Docs)
27. What went wrong? Troubleshooting JavaScript (MDN Web Docs)
28. ReferenceError: "x" is not defined (MDN Web Docs)
29. TypeError: "x" is (not) "y" (MDN Web Docs)
30. Window: prompt() method (MDN Web Docs)
31. Number (MDN Web Docs)
32. console: log() static method (MDN Web Docs)
33. Window: alert() method (MDN Web Docs)
34. Node: textContent property (MDN Web Docs)
35. Plantillas literales (plantillas de cadenas) (MDN Web Docs)
36. Document: querySelector() method (MDN Web Docs)
37. Document: title property (MDN Web Docs)
38. let (MDN Web Docs)
39. const (MDN Web Docs)
40. TypeError: invalid assignment to const "x" (MDN Web Docs)

41. var (MDN Web Docs)
42. typeof (MDN Web Docs)
43. NaN (MDN Web Docs)
44. Exponenciación (**) (MDN Web Docs)
45. Incremento (++) (MDN Web Docs)
46. parseInt() (MDN Web Docs)
47. parseFloat() (MDN Web Docs)
48. Comparación (==) (MDN Web Docs)
49. Igualdad Estricta (===) (MDN Web Docs)
50. Math.random() (MDN Web Docs)
51. Number.prototype.toFixed() (MDN Web Docs)
52. Expresiones y operadores (MDN Web Docs)
53. Resto (%) (MDN Web Docs)
54. Addition assignment (+=) (MDN Web Docs)
55. Math (MDN Web Docs)
56. if...else (MDN Web Docs)
57. Number.isNaN() (MDN Web Docs)
58. Operador lógico AND (&&) (MDN Web Docs)
59. Operador lógico OR (||) (MDN Web Docs)
60. Logical NOT (!) (MDN Web Docs)
61. Window: confirm() method (MDN Web Docs)
62. Falsy (MDN Web Docs)
63. Truthy (MDN Web Docs)
64. for (MDN Web Docs)
65. while (MDN Web Docs)
66. do...while (MDN Web Docs)
67. break (MDN Web Docs)
68. continue (MDN Web Docs)
69. String.prototype.trim() (MDN Web Docs)
70. switch (MDN Web Docs)
71. function (MDN Web Docs)
72. Default parameters (MDN Web Docs)
73. return (MDN Web Docs)
74. function — expresión (MDN Web Docs)
75. Arrow function expressions (MDN Web Docs)
76. Scope (MDN Web Docs)
77. Función Callback (MDN Web Docs)
78. Number — Number coercion (MDN Web Docs)
79. Array (MDN Web Docs)
80. Array.prototype.length (MDN Web Docs)
81. Array.prototype.push() (MDN Web Docs)
82. Array.prototype.pop() (MDN Web Docs)
83. Array.prototype.unshift() (MDN Web Docs)
84. Array.prototype.shift() (MDN Web Docs)
85. Array.prototype.splice() (MDN Web Docs)
86. Array.prototype.includes() (MDN Web Docs)
87. Array.prototype.indexOf() (MDN Web Docs)

88. Array.prototype.find() (MDN Web Docs)
89. Array.prototype.findIndex() (MDN Web Docs)
90. Array.prototype.every() (MDN Web Docs)
91. for...of (MDN Web Docs)
92. Array.prototype.forEach() (MDN Web Docs)
93. Array.prototype.map() (MDN Web Docs)
94. Array.prototype.filter() (MDN Web Docs)
95. Array.prototype.reduce() (MDN Web Docs)
96. Array.prototype.sort() (MDN Web Docs)
97. Array.prototype.sort() — versión en inglés (MDN Web Docs)
98. Array.prototype.slice() (MDN Web Docs)
99. this (MDN Web Docs)
100. for...in (MDN Web Docs)
101. Object.keys() (MDN Web Docs)
102. Destructuring (MDN Web Docs)
103. Shallow copy (MDN Web Docs)
104. Window: structuredClone() method (MDN Web Docs)
105. JSON.stringify() (MDN Web Docs)
106. JSON.parse() (MDN Web Docs)
107. Operador delete (MDN Web Docs)
108. Sintaxis Spread (MDN Web Docs)
109. DOM scripting introduction (MDN Web Docs)
110. Document: querySelectorAll() method (MDN Web Docs)
111. Element: classList property (MDN Web Docs)
112. HTMLElement: style property (MDN Web Docs)
113. Element: append() method (MDN Web Docs)
114. Element: prepend() method (MDN Web Docs)
115. Element: remove() method (MDN Web Docs)
116. Element: replaceChildren() method (MDN Web Docs)
117. Node: appendChild() method (MDN Web Docs)
118. Element: tagName property (MDN Web Docs)
119. Element: children property (MDN Web Docs)
120. Node: parentElement property (MDN Web Docs)
121. Array.from() (MDN Web Docs)
122. Element: getAttribute() method (MDN Web Docs)
123. Element: setAttribute() method (MDN Web Docs)
124. DOMTokenList: toggle() method (MDN Web Docs)
125. Document: createElement() method (MDN Web Docs)
126. Introduction to events (MDN Web Docs)
127. EventTarget: addEventListener() method (MDN Web Docs)
128. EventTarget: removeEventListener() method (MDN Web Docs)
129. Element: click event (MDN Web Docs)
130. Element: dblclick event (MDN Web Docs)
131. Element: mouseover event (MDN Web Docs)
132. Element: mouseout event (MDN Web Docs)
133. Element: keydown event (MDN Web Docs)
134. Element: input event (MDN Web Docs)

135. HTMLElement: change event (MDN Web Docs)
136. Element: focus event (MDN Web Docs)
137. Element: blur event (MDN Web Docs)
138. Event: type property (MDN Web Docs)
139. Event: target property (MDN Web Docs)
140. Event: currentTarget property (MDN Web Docs)
141. KeyboardEvent: key property (MDN Web Docs)
142. KeyboardEvent: ctrlKey property (MDN Web Docs)
143. Event bubbling (MDN Web Docs)
144. Event: stopPropagation() method (MDN Web Docs)
145. Element: closest() method (MDN Web Docs)
146. Event: preventDefault() method (MDN Web Docs)
147. HTMLInputElement: checked property (MDN Web Docs)
148. HTMLFormElement: submit event (MDN Web Docs)
149. FormData (MDN Web Docs)
150. `required` HTML attribute (MDN Web Docs)
151. `minlength` HTML attribute (MDN Web Docs)
152. `<input type="email">` HTML attribute value (MDN Web Docs)
153. `min` HTML attribute (MDN Web Docs)
154. `pattern` HTML attribute (MDN Web Docs)
155. ValidityState (MDN Web Docs)
156. HTMLFormElement: checkValidity() method (MDN Web Docs)
157. HTMLFormElement: reportValidity() method (MDN Web Docs)
158. HTMLInputElement: setCustomValidity() method (MDN Web Docs)
159. ARIA: aria-invalid attribute (MDN Web Docs)
160. ARIA: aria-describedby attribute (MDN Web Docs)
161. HTMLElement: focus() method (MDN Web Docs)
162. ARIA: aria-live attribute (MDN Web Docs)
163. HTMLFormElement: reset() method (MDN Web Docs)
164. `<input type="radio">` HTML attribute value (MDN Web Docs)
165. FormData: get() method (MDN Web Docs)
166. FormData: getAll() method (MDN Web Docs)
167. Object.fromEntries() (MDN Web Docs)
168. `:user-invalid` CSS pseudo-class (MDN Web Docs)
169. `<form>` HTML form element (MDN Web Docs)
170. Web Storage API (MDN Web Docs)
171. Window: sessionStorage property (MDN Web Docs)
172. Storage: setItem() method (MDN Web Docs)
173. try...catch (MDN Web Docs)
174. Array.isArray() (MDN Web Docs)
175. Window: storage event (MDN Web Docs)
176. Storage: getItem() method (MDN Web Docs)
177. Storage: removeItem() method (MDN Web Docs)
178. Storage: clear() method (MDN Web Docs)
179. Storage: length property (MDN Web Docs)
180. Storage: key() method (MDN Web Docs)
181. HTMLElement: dataset property (MDN Web Docs)

182. Using the Web Storage API (MDN Web Docs)
183. Introducing asynchronous JavaScript (MDN Web Docs)
184. Window: setTimeout() method (MDN Web Docs)
185. Window: clearTimeout() method (MDN Web Docs)
186. Window: setInterval() method (MDN Web Docs)
187. Window: clearInterval() method (MDN Web Docs)
188. Date.now() (MDN Web Docs)
189. Promise (MDN Web Docs)
190. Promise() constructor (MDN Web Docs)
191. Promise.prototype.then() (MDN Web Docs)
192. Promise.prototype.catch() (MDN Web Docs)
193. Promise.prototype.finally() (MDN Web Docs)
194. async function (MDN Web Docs)
195. await (MDN Web Docs)
196. Promise.all() (MDN Web Docs)
197. API (MDN Web Docs)
198. Overview of HTTP (MDN Web Docs)
199. HTTP response status codes (MDN Web Docs)
200. JSON (MDN Web Docs)
201. Using the Fetch API (MDN Web Docs)
202. Window: fetch() method (MDN Web Docs)
203. Response (MDN Web Docs)
204. Response: ok property (MDN Web Docs)
205. Response: status property (MDN Web Docs)
206. Response: json() method (MDN Web Docs)
207. Origin (MDN Web Docs)
208. Reason: CORS request not HTTP (MDN Web Docs)
209. Cross-Origin Resource Sharing (CORS) (MDN Web Docs)
210. GET request method (MDN Web Docs)
211. POST request method (MDN Web Docs)
212. Content-Type header (MDN Web Docs)
213. Response: statusText property (MDN Web Docs)
214. JSONPlaceholder — Guide
215. JavaScript modules (MDN Web Docs)
216. export (MDN Web Docs)
217. import (MDN Web Docs)
218. Strict mode (MDN Web Docs)
219. Classes (MDN Web Docs)
220. constructor (MDN Web Docs)
221. this (MDN Web Docs)
222. new (MDN Web Docs)
223. Using classes (MDN Web Docs)
224. extends (MDN Web Docs)
225. super (MDN Web Docs)
226. Private elements (MDN Web Docs)
227. get (MDN Web Docs)
228. set (MDN Web Docs)

229. static (MDN Web Docs)
230. Error (MDN Web Docs)
231. JavaScript debugging and error handling (MDN Web Docs)
232. debugger (MDN Web Docs)
233. instanceof (MDN Web Docs)
234. Date.parse() (MDN Web Docs)
235. Element: replaceWith() method (MDN Web Docs)
236. ARIA: aria-pressed attribute (MDN Web Docs)
237. ValidityState: badInput property (MDN Web Docs)
238. <input type="date"> (MDN Web Docs)
239. String.prototype.localeCompare() (MDN Web Docs)